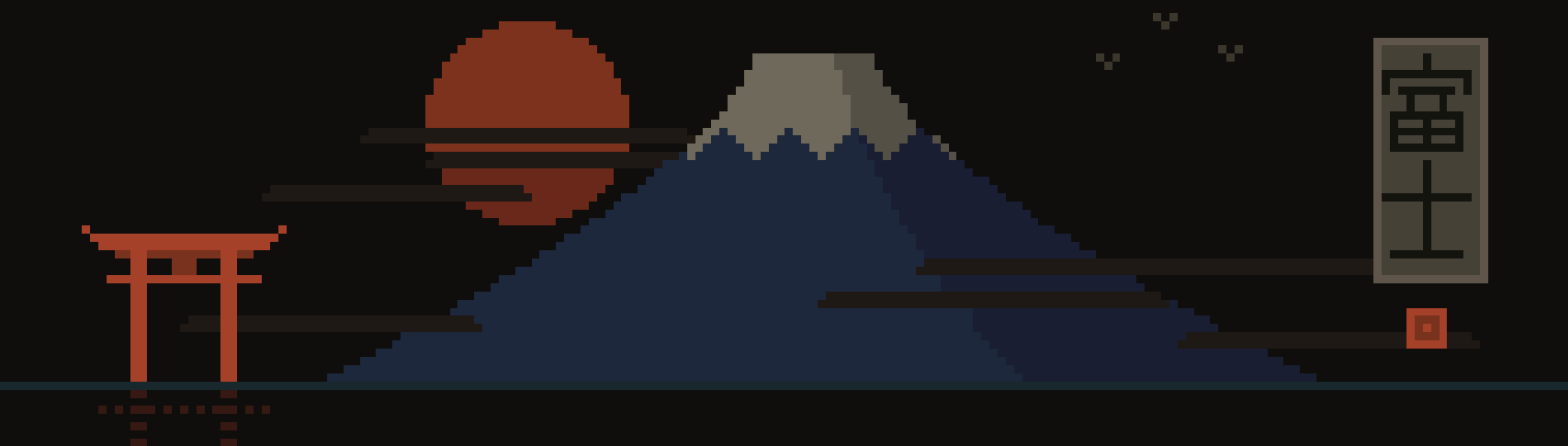
Fedokawaii — the handbook

A Fedora Server install turned into a hand-drawn dwl desktop: how to build it, use it, change it, and keep it healthy

October 2026 · Fedora 44 / 45 · dwl 0.9 · wlroots 0.20



Contents

1. What this is
2. The scripts
3. Installing
   - 3.1 Fedora Server installer
   - 3.2 Run it
   - 3.3 `fedokawaii.conf`
4. Using the desktop
   - 4.1 The bar
   - 4.2 Menus
   - 4.3 Keybindings
   - 4.4 Themes: dark and light
     - Frames
   - 4.5 Wallpapers
   - 4.6 The terminal and the prompt
   - 4.7 Lock, login, boot
   - 4.8 Copy and paste
   - 4.9 Files, file dialogs and default programs
     - File dialogs
   - 4.10 Screen sharing — picture and sound
   - 4.11 Books — Super+R
     - How it keeps track
   - 4.12 Notes — Super+N
   - 4.13 Drives — USB sticks, disks, SD cards
5. How the look is built
   - 5.1 Fonts
   - 5.2 Icons
   - 5.3 Transitions
   - 5.4 Where every app gets its colours
   - 5.5 Pointers
6. Keeping it healthy
   - 6.1 The control — Super+U
   - 6.2 The season's update
   - 6.3 Pinned, and why
   - 6.4 A new Fedora — upgrade
   - 6.5 Built to be left alone
   - 6.6 Kernels, DNF, snapshots
   - 6.7 What verify proves
   - 6.8 Between seasons
     - If an update breaks neovim
   - 6.9 Drivers and firmware
7. Keeping it clean
   - 7.1 Where things go
     - The one rule for configuration
     - Your own files
   - 7.2 Snapshots
     - What makes a snapshot
     - Looking and comparing
     - Undoing
     - Deleting
   - 7.3 Keeping packages lean
     - Adding a package properly
     - Taking one out
     - Kernels
     - The rest of what this desktop installs
   - 7.4 Disk space and logs
     - The journal
     - Caches you can clear
     - Btrfs care
   - 7.5 A seasonal routine
8. Software, and why each piece
9. Troubleshooting
10. Where everything lives
    - 10.1 Your files
    - 10.2 Each program's files
    - 10.3 On the system
11. Using the software
    - Who owns which config
    - 11.1 foot — the terminal
      - A notice when a long command ends
      - Safe clipboard
    - 11.2 The shell
      - By example
      - Fast to start
      - History rules
      - Secrets
    - 11.3 fuzzel — the launcher
    - 11.4 yazi — files
      - Opening
      - Bulk rename
      - Trash
      - Commands on files
    - 11.5 neovim and LazyVim — the editor
      - Modes
      - Never stuck
      - The first hour
      - The grammar: verb, count, motion
      - The dot
      - Text objects
      - Visual mode and block mode
      - Registers and the clipboard
      - Macros
      - Files, buffers and windows
      - Finding
      - The file tree
      - The leader key
      - Search and replace across a project
      - Language servers
      - Completion
      - Formatting
      - Extras and Mason: adding a language
      - Keeping it updated and healthy
      - A daily workflow
      - A learning path
    - 11.6 lazygit — git
      - What git is
      - Once, before the first commit
      - Opening it
      - The screen
      - The colours here
      - A first commit, step by step
      - Staging part of a file
      - Commit, amend, undo
      - Stash
      - Branches
      - Pull and push
      - Resolving a conflict

Rewriting history: interactive rebase
History and diffs
Cherry-pick
Recipes
11.7 bat, ripgrep, fd, fzf, eza, zoxide and tldr
11.8 btop — the system monitor
11.9 Brave Origin — the browser
What this desktop sets up
Worth checking once
11.10 mpv and yt-dlp — video
yt-dlp
11.11 imv — pictures
11.12 zathura — PDF, EPUB and comics
11.13 satty — annotating screenshots
11.14 Sound — pavucontrol and EasyEffects
11.15 spotify_player and cava — music
11.16 presenterm — slides in the terminal
11.17 qalc — the calculator
11.18 Documents — Xournal++, AbiWord, Gnumeric and the PDF tools
11.19 Hardware — OpenRGB, Solaar, liquidctl, LACT
11.20 podman — the Prowlarr container
11.21 snapper — snapshots
11.22 Claude Code — an assistant in the terminal
Signing in
11.23 Steam — games
GE-Proton

12. Torrents: searching and downloading
12.1 The first time: Prowlarr
12.2 Searching and downloading with torrra
Magnet links in Brave
12.3 How it behaves
12.4 When something is wrong

13. Everyday recipes
13.1 Upload a file from Brave
13.2 Share your screen, with sound, in a call
13.3 Take, annotate and paste a screenshot
13.4 Switch the theme and the wallpaper
13.5 Choose which program opens a type of file
13.6 Install a program and keep `verify` happy
13.7 Update everything
13.8 Move to a new Fedora
13.9 Undo a bad update
13.10 Free disk space
13.11 Plug in a second monitor
13.12 Watch or keep a video from the web
13.13 Change a setting for good

14. Do it yourself: what websites charge for
14.1 Before you start
14.2 PDFs: join, split, pick and turn pages
14.3 PDFs and pictures
14.4 PDFs: smaller, locked, unlocked
14.5 PDFs: text, scans and OCR
14.6 Fill in and sign a PDF
14.7 Pictures
14.8 Video and sound
14.9 Download and record
14.10 Files: archives, checksums, trash, renaming, text
14.11 Office documents and spreadsheets
14.12 Quick conversions: units, money, time

15. Changing it yourself
15.1 Two kinds of change
15.2 The working copy
15.3 Where each thing lives
15.4 Rules the scripts count on

16. Learning: languages, ideas, and where to read
16.1 How to learn here
16.2 Languages worth learning now
16.3 Ideas, defined, with an example each
16.4 Learning with an AI assistant
16.5 Where to read
16.6 A ladder of first projects
16.7 Where things stand, October 2026
The Linux kernel
Fedora
The Linux desktop
Gaming
The software industry and AI
Security
How to keep up

17. Linux, from the ground up
17.1 What Linux is
Fedora
The layers of this desktop
17.2 From power button to desktop
1–2. UEFI, shim and Secure Boot
3. GRUB2 and the boot entries
4. The kernel and its arguments
5. The initramfs and early microcode
6. systemd
7–9. greetd, the login, the session
10–11. dwl and the autostart
17.3 Files and the file system
Everything is a file — nearly
The hierarchy
Mounts
Btrfs: subvolumes, snapshots and copy-on-write
Inodes and links
Permissions
SELinux labels
17.4 Users, groups and privilege
sudo
polkit
Why the desktop never runs as root
17.5 Processes, signals and services
How a program starts: fork and exec
Signals
Jobs: the shell's own processes
Priority: nice
systemd and its units
The system manager and yours
Timers, not cron
A container as a unit: the Prowlarr quadlet
The journal
cgroups
17.6 The terminal and the shell
Three streams, redirection and pipes
Environment and PATH
Quoting, globs and regular expressions
Scripts and the shebang
17.7 Packages, and how this desktop's pieces connect

- What an RPM holds
- dnf5
- Weak dependencies, install reasons and groups
- Keys and gpgcheck
- The repositories this desktop uses
- A worked chain: a video, from mpv down to the GPU
- A second chain: the music in the bar
- The manifest, setup and verify

17.8 The graphics stack

- DRM and KMS
- The drivers
- Wayland, wlroots and dwl
- Layer shell: the bar, notifications and menus
- XWayland
- Toolkits
- Fonts

17.9 Sound, devices and network

- Sound: ALSA, PipeWire, WirePlumber
- Devices: the kernel, udev and /dev
- Bluetooth: BlueZ
- Network: NetworkManager
- Names: systemd-resolved
- The firewall: firewalld

17.10 The buses that tie a desktop together

- D-Bus
- The desktop portals
- Notifications
- MPRIS
- polkit and udisks

17.11 The security model, briefly

- SELinux
- Rootless containers

17.12 Commands by job

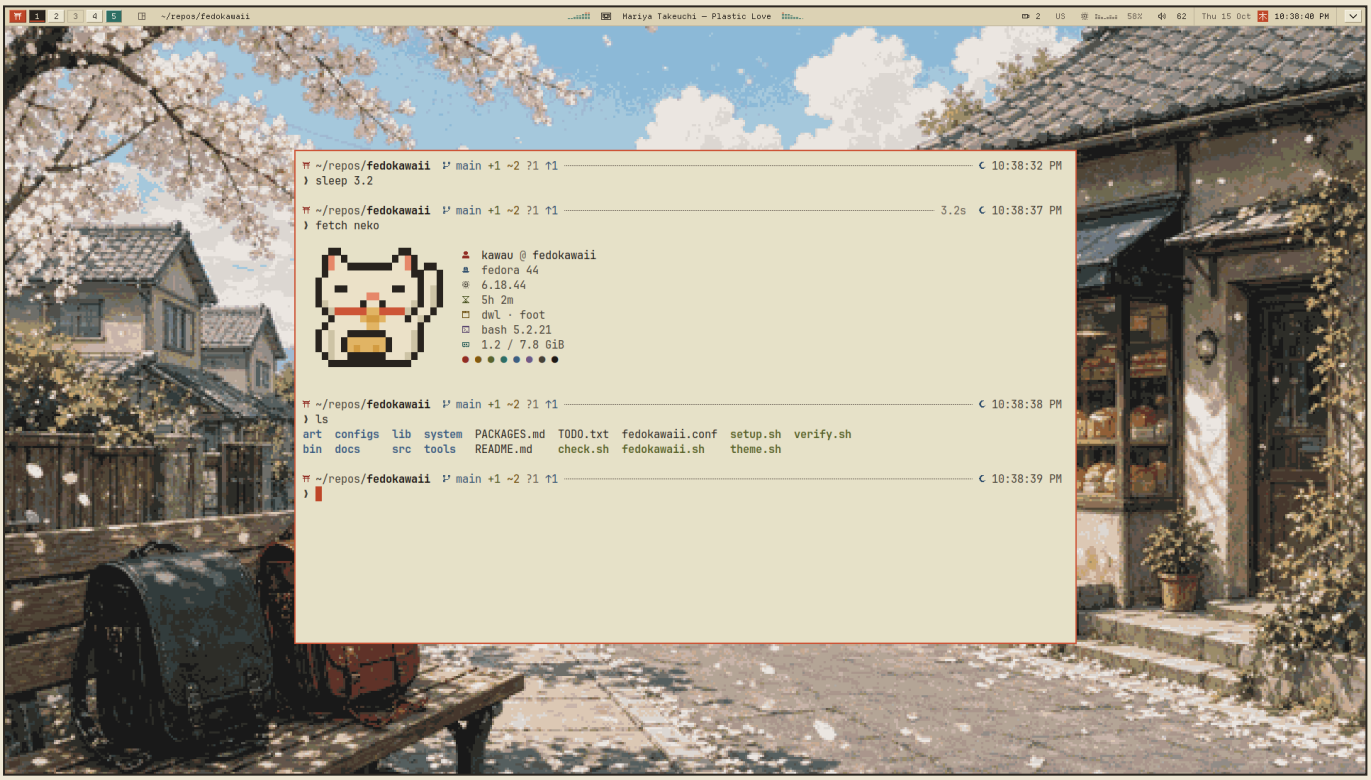
- Files and folders
- Viewing
- Searching
- Text processing
- Archives
- Processes
- Services and logs
- Packages
- Users and permissions
- Disks and file systems
- Network
- Hardware
- Help

17.13 Glossary, A–Z

17.14 Sources



The desktop on the dark theme: the bar on top, a terminal with the prompt, fetch and a listing.



The same desktop on the light theme — switched live with Super+Shift+W.

1. What this is

Fedokawaii takes a **minimal Fedora Server** install and turns it into a small, fast Wayland desktop built around **dwl**, a tiling compositor of about three thousand lines. Everything you see on screen was drawn for it: the icons, the Fuji wallpaper scene, the lock screen, the boot menu, the pictures in `fetch`. Nothing comes from a theme pack. The name is Fedora + *kawaii* (かわいい), the Japanese word for cute.

Terms.

Term	Plain meaning
Wayland	the way programs and the screen talk on modern Linux, the successor of X11: each program draws its own window, and one central program shows them all
compositor	that central program: it places the windows, puts them on the screen and hands them the keyboard and mouse. Under Wayland it is also the display server
tiling	the compositor arranges the windows side by side for you, never overlapping, instead of you dragging them about
dwl	the compositor here: a small one written in C on the *wlroots* library, configured by editing its source (`src/dwl/config.h`) and building it again
package	one program or library in Fedora's format (`.rpm`), installed, updated and removed by `dnf`

Read more: wayland.freedesktop.org · dwl's README, codeberg.org/dwl/dwl.

Four ideas run through every file:

Idea	What it means in practice
One tool per job	one editor (neovim), one browser (Brave Origin), one terminal (foot), one launcher (fuzzel, also every menu). `verify.sh` fails if a second one appears, and update and repair take out what is doubled or fights (§6.2)
Nothing hidden	every package is listed in `lib/manifest.sh`, every release binary in `lib/upstream.sh`, every colour in `tools/build-themes.py`, every icon in `art/icons.txt`
Proof, not hope	`verify.sh` checks the machine against the lists; `theme.sh --check` checks every config against the repository
Pixel-exact	the UI fonts and icons sit on an 11-pixel grid and are drawn at whole multiples of it only (11, 22, 33 — the boot menu 44 on a 5K screen); pixel art is enlarged by whole numbers only, so every pixel stays a sharp square

A first tour. Super is the Windows key. Six keys carry you a long way:

Press	You get
Super+Space	the launcher: type a few letters of a program's name, then **Enter**
Super+Return	a terminal
Super+B · **Super+E**	the browser (Brave Origin) · the files (yazi)
Super+X, or the ⌄ key at the right end of the bar	quick settings — the theme switch is the first line
Super+/	every key, searchable
Super+V	anything you copied, newest first

Everything else in this handbook is detail. Where to read, by what you want:

You want to	Read
install it	chapter 3
use it day to day	chapters 4 and 11, and the recipes in 13 and 14
use a USB stick, a disk or an SD card	§4.13
keep it healthy, update it, move to a new Fedora	chapters 6 and 7

You want to	Read
change anything about it: keys, colours, the bar, packages	chapter 15 (**Super+Shift+U**)
learn programming on it	chapter 16
know where Linux, the desktop and the software trade stand now	§16.7
understand Linux itself: the boot, files, processes, packages, the graphics and sound stacks	chapter 17

2. The scripts

Terms. The table below uses words that recur through the book:

Term	Plain meaning
repository	a server that offers packages; `dnf repolist` lists the ones this machine uses
driver	the part of the system that operates one piece of hardware — here the graphics card
kernel argument	an option handed to the kernel — the core of Linux, which drives the hardware — as it starts; `cat /proc/cmdline` shows the current ones
service	a program that systemd, the system's service manager, starts and looks after, described by a small *unit* file; `systemctl status greetd` shows one
snapshot	a frozen copy of the system's files at one moment, to go back to; on Btrfs it is made in an instant and takes space only for what changes after it
`sudo`	runs one command as root, the administrator, after asking for your password

File	Owns	Touches
`fedokawaii.sh`	the menu for the install	runs the others; nothing of its own
`check.sh`	**readiness**	nothing — is this Fedora, online, on Btrfs, with room to work?
`setup.sh`	**software**	repositories, packages, drivers, kernel arguments, the dwl binary and its session launcher, services, the performance profile, snapshots — and what software needs before it can run: the LazyVim starter, the Prowlarr container file and key, torrra's settings and its key wrapper
`theme.sh`	**look and configuration**	every file it ships under `~/.config` and `~/.local` (`bin/kawa-*`, `share/fedokawaii`, `share/applications`, `share/themes`, fonts), `~/.bashrc.d`, `~/.inputrc`, the pixel art, `/etc/dconf` defaults, the boot menu theme and `/etc/default/grub`, the console font and colours, dwl rebuilt from `src/dwl/config.h`, two C helpers built from `src/`, Brave's colours (set once in its profile) and its launcher entry (so a screen share can carry sound)
`verify.sh`	**proof**	nothing of yours — checks against the manifest and the repository; it only wakes Prowlarr to ask it, and writes its result to `/var/lib/fedokawaii/last-verify`
`system/fedokawaii`	**upkeep** — the system's own control, installed as the `fedokawaii` command (Super+U)	nothing itself: it reads the records and runs the scripts above through `sudo` (§6.1)

`theme.sh` never installs a package, and `setup.sh` writes no desktop look of its own. The menu offers them separately — **Setup** alone gives you dwl with upstream defaults, **Theme** can be run (and re-run) whenever you like, and **Both** runs them in order. To install, the only file you edit is `fedokawaii.conf`; everything else can be changed afterwards, the chapter 15 way.

Each menu entry is also an argument of the same script, for running one part directly:

```
sudo bash fedokawaii.sh check    # the readiness check alone; changes nothing
sudo bash fedokawaii.sh setup    # Setup: the software only, dwl with upstream defaults
sudo bash fedokawaii.sh theme    # Theme: the look and configuration only (after setup)
sudo bash fedokawaii.sh all      # Both: setup, with the theme applied near the end
```

The toolkit. At its first reboot, setup installs the project folder for good as `/usr/local/lib/fedokawaii`, owned by root; from then on everything runs from there, and the folder you installed from can go. Where this book says *the repository* after the install, it means that copy. You never edit it directly: you change your own working copy of it and *apply* that (chapter 15) — so every update keeps your change.

	Until the first reboot	From then on
setup runs from	the folder you unpacked (`~/fedokawaii`)	`/usr/local/lib/fedokawaii`, owned by root
the `fedokawaii` command	not there yet	`/usr/local/bin/fedokawaii`, installed from that copy
`~/fedokawaii`	needed	can be deleted once setup is done

3. Installing

3.1 Fedora Server installer

Screen	Set	Why
Software Selection	**Minimal Install** (or *Fedora Custom Operating System*), nothing extra	the scripts add exactly what is needed, so `verify.sh` can prove nothing else crept in
Installation Destination	**Custom → Btrfs → create automatically**	Server's *Automatic* is LVM + XFS; snapshots need Btrfs. Result: `/boot/efi`, `/boot`, subvolumes `root` and `home`
Network & Host Name	ethernet on, hostname of your choice	NetworkManager is part of `@core`
Time & Date	your zone, Network Time on	
KDUMP	off	a desktop; saves memory
Root / user	root **disabled** (no password); your user **administrator** — setup, Theme and update all run through `sudo`	if the desktop ever fails to start, log in on a text console (Ctrl+Alt+F3) as your user: `fedokawaii` works there; undo a bad update with snapper (§7.2)

Terms. *Btrfs* is a Linux file system that can take instant snapshots. A *subvolume* is a part of a Btrfs file system with its own folder tree, mounted like a partition and snapshotted on its own: the installer makes `root` (mounted at `/`) and `home` (at `/home`). *LVM* and *XFS* are the volume manager and file system of Server's automatic layout; the snapshots here are Btrfs ones, so `check.sh` stops on anything else. *kdump* keeps memory set aside at every boot to record a kernel crash for later study. To check the layout after the install:

```
findmnt -no FSTYPE /                          # → btrfs
findmnt -no OPTIONS / | grep -o 'subvol=[^,]*'    # → subvol=/root
```

Read more: Btrfs subvolumes.

3.2 Run it

The project is yours alone and private: it comes to the new machine on a USB stick, as `fedokawaii.tar.gz` (a tar archive keeps every file's permissions, and the minimal install has `tar`).

```
lsblk -o NAME,SIZE,LABEL,FSTYPE          # find the stick, say sdb1
sudo mount /dev/sdb1 /mnt
tar -xzf /mnt/fedokawaii.tar.gz -C ~     # → ~/fedokawaii
sudo umount /mnt
cd ~/fedokawaii
sudo bash fedokawaii.sh                  # the menu: Check · Setup · Theme · Both · Verify …
```

`lsblk` lists every disk and its partitions. The internal disk (`nvme0n1` or `sda`) carries the `btrfs` and `ext4` partitions the installer just made; the stick is the small disk after it, usually `sdb`, with one `vfat` or `exfat` partition, `sdb1` — the one to mount. In the `tar` line, `-x` extracts, `-z` undoes the gzip compression, `-f` names the archive and `-C ~` puts the result in your home.

Or straight from GitHub, where the project is the private repository `MoeDia/fedokawaii`. The minimal install has no git, so it comes first; git then asks for your GitHub user name, and a *personal access token* as the password (§15.2 shows where to make one):

```
sudo dnf install -y git-core
git clone https://github.com/MoeDia/fedokawaii ~/fedokawaii
cd ~/fedokawaii && sudo bash fedokawaii.sh
```

Choose **Both** for a first install. Setup walks through three phases and reboots twice between them; after each reboot it takes tty1 by itself (`fedokawaii-resume.service`) and carries on, so you can walk away. It stops with the failing line and the log (`/var/log/fedokawaii.log`) if anything goes wrong; fix it and run it again — finished steps are recorded in `/var/lib/fedokawaii` and skipped. When it is done the login screen comes up, and from then on the system looks after itself through one menu: **Super+U** (§6.1).

Phase	Steps	Why this order
1	readiness check → record what is installed → DNF tuning → boot-menu and two-kernel policy → full update → base tools → the performance profile → remove nano, vim, rescue kernel → SSH policy → **reboot**	the record lets `verify.sh` spot anything extra later; tuning first so the big update downloads ten at a time; update before any third-party repository so RPM Fusion matches the base libraries; SSH closed before the first reboot, so a run that stops early never leaves it open
2	RPM Fusion, Brave, Claude Code (key fingerprint checked), COPRs → GPU drivers → full ffmpeg and media tools → **reboot**	drivers before the programs that use them, so nothing pulls the wrong variant; the reboot activates the GPU kernel argument
3	build tools → dwl (pinned commit + patch) → greetd login → desktop → fonts and toolkit themes → apps → release binaries → the yazi file dialog → Prowlarr + torrra → LazyVim → Steam → groups → theme (if chosen) → **verify gate** → enable the login, snapshots, baseline	the login is enabled last, only after verification, so a failure never leaves you at a black screen

tty1, above, is the first text console — the full-screen text login the machine shows before any desktop — and `fedokawaii-resume.service` is a *systemd unit*: a small file that tells systemd what to start at boot. To see how far setup has got, log in on a second console (**Ctrl+Alt+F2**) as yourself:

```
sudo bash ~/fedokawaii/fedokawaii.sh status  # the phase, and every finished step
sudo tail -f /var/log/fedokawaii.log         # the log as it grows; Ctrl+C stops watching
```

Halfway through, *status* prints (with your user name in place of *you*):

```
phase 2 · user you · Fedora 44 · theme with setup
  ✔ 1.0 readiness check
  ✔ 1.1 record user-installed packages
  ✔ 1.2 dnf5 tuning
  ✔ 1.3 boot menu
  ✔ 1.4 full system update
  ✔ 1.5 base packages
  ✔ 1.5b performance: tuned profile, Wi-Fi
  ✔ 1.6 remove nano, vim, rescue kernel
  ✔ 1.7 inbound SSH (no)
```

Terms.

Term	Plain meaning
RPM Fusion	a community repository for software Fedora does not ship itself, such as the full ffmpeg and the *freeworld* video drivers
COPR	Fedora's build service, where anyone can publish a small repository; LACT and lazygit come from two (`dnf copr list` shows them)
pinned commit	one exact version of dwl's source, named by its commit hash; setup refuses to build if the tag `v0.9` points anywhere else
patch	a file of line changes applied to dwl's source before it is built (`src/dwl/dwl.patch` : keys, gaps, frames, centring, the layout for the bar)
greetd	the login manager: it holds tty1, asks for your user name and password through its plain prompt, agreety, then starts the desktop
verify gate	`verify.sh` , run before the login is switched on; any failure stops setup there

3.3 `fedokawaii.conf`

Every value is validated before anything runs; a wrong value stops the script with a message that names it. Set `GPU="nvidia"`, for example, and setup stops at once with:

```
08:52:07 ✘ fedokawaii.conf:
    GPU must be auto, amd or intel (is 'nvidia')
```

The file is read as data, never run: only `KEY="value"` lines (single quotes, or none, also do) and `#` comments. After the install, change it through *settings* in the control (§6.1), which checks every value before it saves.

Setting	Default	Meaning
`USERNAME`	empty	the desktop user; empty = whoever ran `sudo`
`GPU`	`auto`	`auto`, `amd` or `intel` — drivers, video decoding, and mpv's scaling
`STEAM`	`yes`	Steam, gamescope, gamemode, MangoHud, GE-Proton; switched to `no`, the next update hands those packages back to dnf, and its clean-up removes them (your games stay in your home)
`SSH_SERVER`	`no`	`no` disables sshd and closes ssh/cockpit in the firewall
`MONITOR_HZ`	`144`	the highest refresh rate at or below this is used (30–540)
`KB_LAYOUTS`	`us,ara`	keyboard layouts by their XKB names; the first one is always used for shortcuts. `localectl list-x11-keymap-layouts` lists the names
`KB_TOGGLE`	`grp:alt_shift_toggle`	Alt+Shift switches layout; the bar shows US / AR. `localectl list-x11-keymap-options` lists the others (the switches start with `grp:`)
`TIMEZONE`	`Asia/Dubai`	your time zone, as `timedatectl list-timezones` names it
`WEATHER_CITY`	`Sharjah`	the weather line in quick settings (wttr.in, no key)
`NIGHT_LIGHT`	`25.35:55.40`	latitude:longitude for gammastep, which works out sunset and sunrise from it (north and east are positive)
`THEME`	`dark`	the theme a new install starts in; afterwards it is whatever you last switched to
`WALLPAPER_PACK`	`yes`	fetch the wallpaper pack once (70 pictures, about 210 MB to download; kept redrawn as pixel art, about 4 MB)
`BOOT_MENU_SECONDS`	`2`	how long the themed boot menu shows (0–10); `0` = Fedora's hidden menu
`DNF_PARALLEL`	`10`	parallel downloads (1–20)
`DWL_TAG`, `DWL_COMMIT`, `DWL_WLROOTS`	`v0.9`, pinned, `0.20`	dwl is built from exactly this commit, against wlroots 0.20; should the tag point anywhere else, setup refuses to build
`TFC_TAG`, `TFC_COMMIT`	`v1.4.3`, pinned	the yazi file dialog's portal is built from exactly this commit

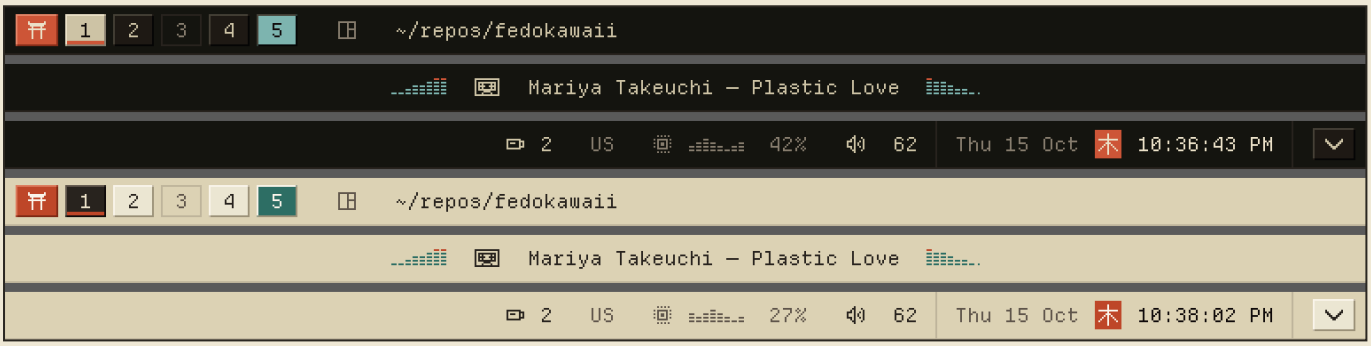
Read more: keyboard layout names and switches — localectl(1).

4. Using the desktop

Terms.

Term	Plain meaning
workspace	a separate set of windows, one set at a time on a screen; dwl calls them *tags*, and a window can be on more than one
tiled · floating · monocle	dwl's three layouts: windows side by side, never overlapping · windows placed freely, by you · one window at a time, at full size
main area	the large left part of a tiled screen; the other windows share the column on the right
launcher	the box that starts a program by its name (fuzzel)
focus	the window that receives what you type; its frame is vermilion

4.1 The bar



The bar, dark and light, at twice its size: the left end, the middle, the right end.

Part	What it shows	Click
torii key	the vermilion key at the far left	opens the launcher
workspaces	1, 2, 3 always; 4–9 only while they hold a window, are the one you are on or want attention. A raised key = windows on it; the lit key with a vermilion stamp = the one you are on; a teal key = something there wants attention	goes to it
layout icon	tiled, floating, or monocle (with the number of stacked windows)	
title	the focused window	
now playing	artist — title, with a cassette whose reels turn while sound plays, between the two halves of a spectrum meter drawn like an old hi-fi's fluorescent display: lowest notes nearest the title, a full column lights its top segment in vermilion	play/pause · right: next · middle: previous
mode seal	a small vermilion seal while something is on that you should not forget: **静** (*quiet*) while do-not-disturb holds notices back, **共有** (*sharing*) while the screen is being shared; nothing otherwise	quick settings
tray	the small icons some programs put there while they run (Steam, Solaar, OpenRGB); nothing otherwise	the program · right: its menu
drive mark	the drive icon while a USB stick, disk or SD card is plugged in: quiet, then lit once one of them is open; a number beside it for two or more	the drives menu: open or eject (§4.13)
US / AR	the keyboard layout (vermilion while it is not the first)	
CPU	the chip icon, the last eight readings (one every two seconds, the newest on the right) in the same segment style, then the load now in per cent; vermilion above 80 %	opens btop
volume	speaker icon and level; vermilion when muted	the audio menu · right: mute · scroll: ±2 %

Part	What it shows	Click
date, day, time	`Wed 14 Oct` 水 `08:52:07 PM`: the date, the day of the week as a kanji seal (月 火 水 木 金 土 日, Monday to Sunday), the time to the second	the calendar
⌄ key	quick settings	

Every player that speaks MPRIS shows in *now playing* and answers the media keys: spotify_player, Brave (a video or a tab playing sound), and mpv (through the mpv-mpris plugin). *MPRIS* is the standard D-Bus interface — D-Bus being the message system programs on the desktop use to talk to each other — through which a player says what it plays and takes play, pause and skip from anyone. `playerctl`, which the bar and the media keys use, does the same from a terminal:

```
playerctl -l            # every player that is up, one per line (mpv, for example)
playerctl status        # Playing, Paused or Stopped
playerctl play-pause    # what the play/pause key does
```

Read more: the MPRIS specification.

The bar costs almost nothing at rest. Nothing in it *polls* — asks again and again whether anything has changed:

Part	Wakes
workspaces, layout, title, keyboard layout	only when dwl reports a change; only that part is redrawn
clock	once a second (waybar's own clock)
CPU	every two seconds, in pure bash reading `/proc/stat` — no program is started
drive mark	only when a drive comes, opens, closes or goes: `kawa-drives` pokes it (`SIGRTMIN+13`)
music meter	its analyser (cava) runs only while a player is playing: paused, it is not running at all; silent, it sleeps

4.2 Menus



Quick settings, the power menu, the calendar and the audio menu, on both themes.

Every menu is the same fuzzel window, so they look and behave alike: type to filter, arrows or mouse to choose, **Esc** or a click outside to close.

Menu	Open with	Inside
quick settings	⌵ key, **Super+X**	**theme** first, as a two-way switch (■ dark □ light) · brightness · night light · wallpaper · sound · network · Bluetooth on/off and pair a device (only when the PC has a Bluetooth adapter) · drives · do not disturb · screenshot · keybindings · **system** (the control, §6.1) · **change** (chapter 15) · power; the weather on top
power	**Super+Shift+E**	lock · sleep · log out · restart · shut down — centred, with your uptime
audio	volume, **Super+A**	the level as a framed bar at the top · louder / quieter / mute (stay open for repeated taps), every output and input — picking one moves playing sound to it — microphone mute, the full mixer
calendar	the clock, **Super+C**	this month (Monday first, today in brackets), previous / next

Menu	Open with	Inside
brightness	quick settings	the monitor's real backlight over DDC/CI, like Twinkle Tray on Windows, each level drawn as a small framed bar; **Super+]** / **Super+[** step by 10 %
share what?	a screen share	the screens and windows you can share (§4.10)
drives	the bar's drive mark, quick settings	every file system on a USB stick, disk or SD card — open it in yazi — and **eject** for each drive (§4.13)
help	**Super+/**	every keybinding, searchable
change	**Super+Shift+U**, quick settings	every part of the desktop by name — keys, bar, bar look, colours, terminal, shell, launcher, notifications, files, drives, autostart, icons, packages, settings — opened in your working copy, then applied; the files that are yours alone; what changed; its history; where to learn each (chapter 15)

DDC/CI (Display Data Channel Command Interface) is the standard way a computer changes a monitor's own settings — brightness, contrast — over the video cable. Most monitors have it, some with it switched off in their own on-screen menu. `ddcutil`, which the brightness menu uses, shows whether yours answers:

```
ddcutil detect --terse    # one "Display 1" block for each monitor that answers
ddcutil getvcp 10         # feature 10 is brightness: its current and maximum value
```

Read more: ddcutil.com.

The windows that show more than a list are small terminal windows of their own rather than menus, all in one look — the name in a rule across the top, a search line where there is something to search, the list, and a pane beside or below it: the wallpaper picker (**Super+W**, the wallpaper itself shown as you move — §4.5), the clipboard history (**Super+V** — §4.8), the books (**Super+R** — §4.11) and the system's control (**Super+U** — §6.1). Each opens once: a second press while it is open does nothing.

The volume, microphone and brightness keys show a **level display** for a moment at the bottom centre — also over fullscreen windows and while *do not disturb* is on. It is drawn like the volume scale of an old hi-fi: a wedge of twenty pixel columns rising left to right, lit in asagi with the topmost lit one vermilion (the peak lamp), the number beside it at twice the pixel size; it reads *off* when muted, and holding a key updates it in place.



The level display, a notice from an app and a critical notice, dark and light.

4.3 Keybindings

Super is the Windows key. Shortcuts always read the US layout, so they work the same while Arabic is active.

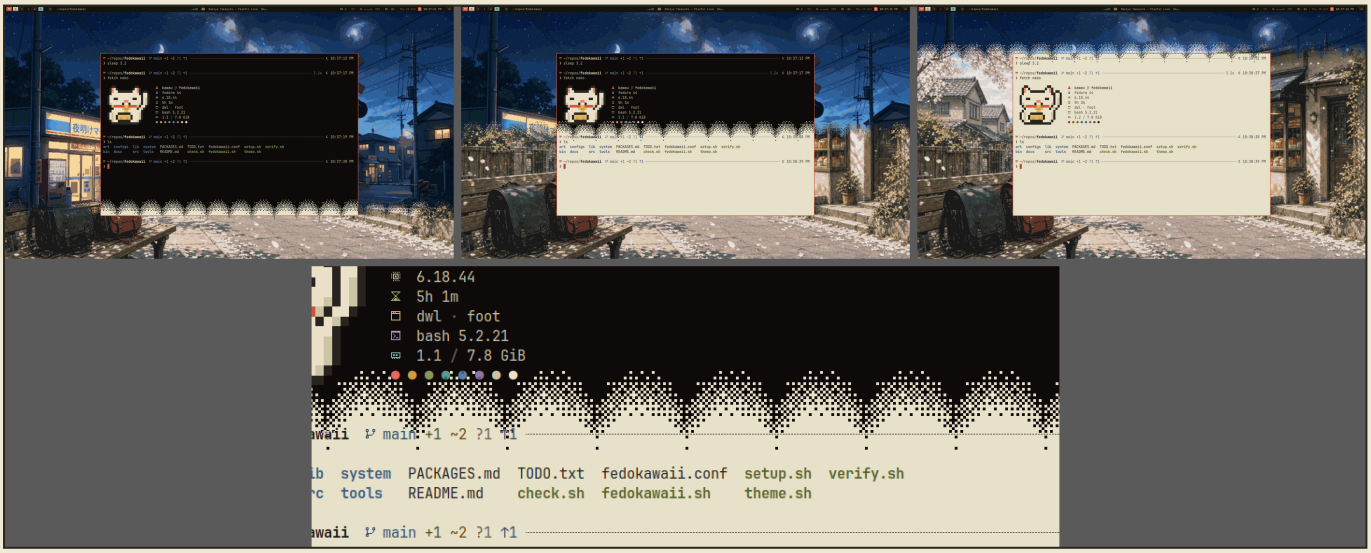
Keys	Does
Super+Return / Space / B / E	terminal · launcher · Brave Origin · files (yazi)
Super+X / A / C / W / V / /	quick settings · audio · calendar · wallpaper · clipboard history · help
Super+Shift+W	**dark ⇄ light theme**
Super+R · Super+N	books (§4.11) · notes, and again to put them away (§4.12)
Super+U	the system: update, a new Fedora, repair (§6.1)

Keys	Does
Super+Shift+U	change the desktop: keys, bar, colours, … (chapter 15)
Super+Shift+N	dismiss notifications
Super+Escape · Super+Shift+E	lock · power menu
Print · Shift+Print · Super+Print	screenshot a region · the whole screen · a region, then annotate it (Enter copies)
Super+Q	close the window
Super+J / K · H / L · I / D	focus next/previous · shrink/grow the main area · more/fewer windows in it
Super+Shift+Return · F · Shift+Space	move to the main area · fullscreen · float/unfloat (also Super+middle-click)
Super+drag / right-drag	move / resize
Super+T / Y / M · Shift+T	tiled · floating · monocle · previous layout
Super+1…9 · Shift+1…9 · Ctrl+1…9 · Ctrl+Shift+1…9	go to · send to · also show a workspace · also show the window there
Super+Tab · Super+0 · Shift+0	previous workspace · all of them · the window on every one
Super+, / . · Shift+, / .	focus · send to the monitor on the left / right
media keys	volume, mute, microphone mute — the level display shows for a moment, also in fullscreen · play/pause, next, previous
Super+scroll	volume ±2 % (as scrolling over the volume in the bar)
Super+] / [· brightness keys	monitor brightness (the same level display)
Alt+Shift	keyboard layout
Right Shift+F12	the game overlay (MangoHud) on / off, in games started with it
Super+Ctrl+Shift+Q · Ctrl+Alt+F2…F6	log out · text consoles (Ctrl+Alt+F1 comes back)

Windows carry no title-bar buttons anywhere — not in GTK apps, not in Brave: dwl tiles them, Super+Q closes them.

4.4 Themes: dark and light

Press **Super+Shift+W** (or quick settings → *theme*). The screen holds still for a moment while the terminals and everything in them, the bar, menus, notifications, GTK and Qt apps, Brave, mpv, btop, zathura and the wallpaper all change underneath; then the new theme arrives as a row of seigaiha wave crests with a dithered fringe — rising from the bottom like dawn for the light theme, falling from the top like nightfall for the dark one. The lock screen and the next menus follow from then on.



A switch to the light theme, three moments of it: the crests rise from the bottom edge.

	dark — *sumi* (ink)	light — *washi* (paper)
ground	`#0f0e0c` ink	`#e9e1cb` paper
text / strongest	`#cfc3a6` / `#ece3cc`	`#2b2721` / `#0f0e0c`
quiet text	`#877f6f`	`#5f594d`
bar / raised key / hairline	`#161411` / `#1f1c18` / `#2b2721`	`#dcd2b8` / `#efe8d5` / `#c2b79b`
vermilion (accent: seals, frames, fills)	`#d0583a`	`#c0462a`
vermilion for words	`#d15b3d`	`#9c3922`
beni · matcha · yamabuki	`#d64b4b` · `#8e9a5b` · `#d29f45`	`#a8342a` · `#5c682d` · `#845b0f`
ai · fuji · asagi	`#5f7fa0` · `#9079a9` · `#7fb8b0`	`#3f6185` · `#6f5a8a` · `#2f6e67`

The accents keep their character on both grounds; on paper they are deepened so they keep their contrast, and on paper the terminal's "bright" colours are the deeper ones. Every colour used for reading — quiet grey and vermilion included — reaches 4.5 : 1 or better against the surface it sits on (the WCAG AA level for body text). *Contrast ratio* compares the lightness of text and ground, from 1 : 1 (the same) to 21 : 1 (black on white). This is why vermilion comes in two shades: the accent is for seals, frames and fills, and a shade of its own is used for words. Measured on the ground:

	dark	light
text	11.0 : 1	11.4 : 1
quiet text	4.9 : 1	5.3 : 1
vermilion for words	4.8 : 1	5.3 : 1
vermilion accent	4.7 : 1	3.9 : 1 — never used for words on paper

Read more: WCAG, Contrast (Minimum).

Decoration that is meant to recede — an empty workspace key, a hairline, the terminal's deliberately faint grey for comments — sits lower on purpose. Terminal programs see the usual 16 colours, arranged the way gruvbox does it, so a program that uses "black" for faint lines and "white" for text reads correctly on paper too.

Terminal colour	usually means	dark	light
0, "black"	faint lines and backgrounds	`#2b2721`	`#d6cdb3` — a shade of paper
7, "white"	ordinary text	`#cfc3a6`	`#4a443a` — ink
15, "bright white"	the strongest text	`#ece3cc`	`#1f1c18`

gruvbox is a well-known retro colour scheme; its light version swaps these roles in the same way.

Frames

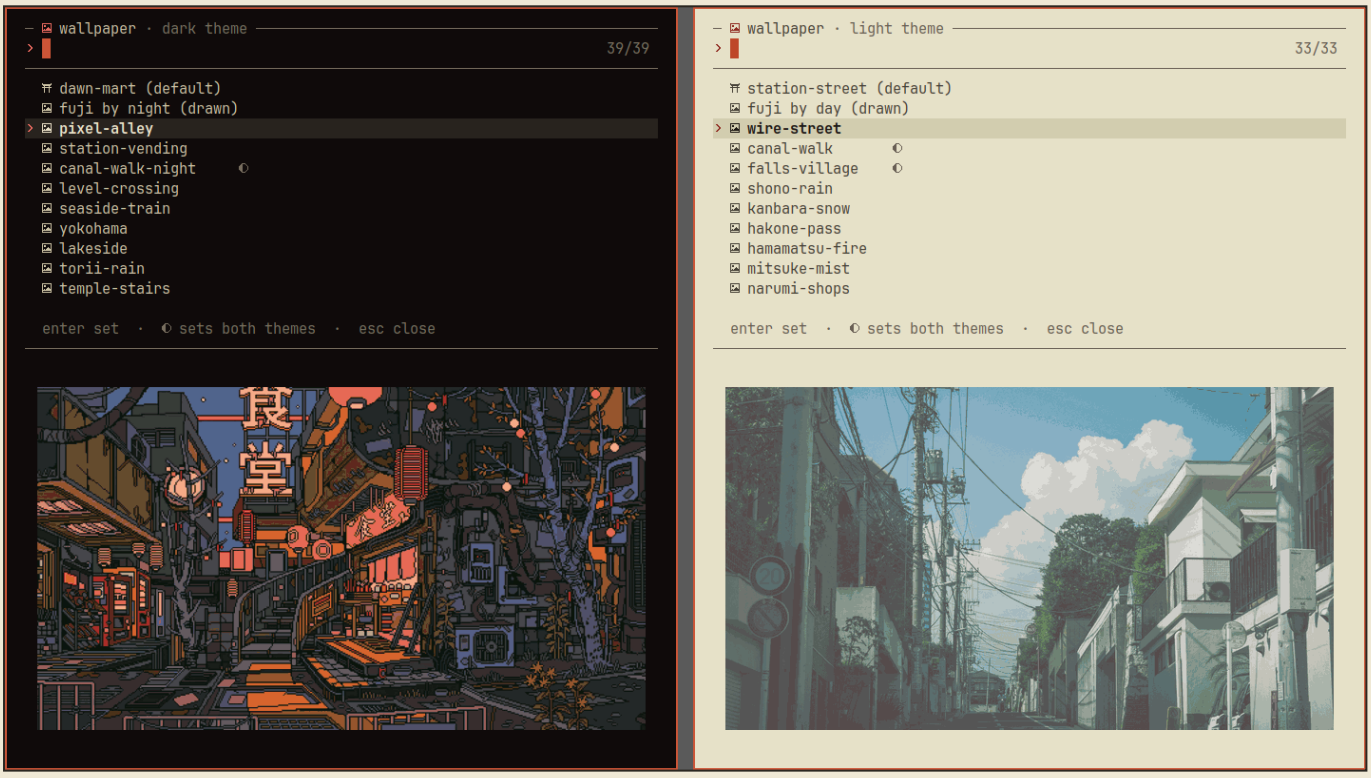
Everything that floats has the same plain, full border, 2 pixels wide. Windows get theirs from dwl (compiled in, so it suits both themes): vermilion around the focused window, warm grey around the others, asagi around one that asks for attention. Menus, notices and the level display are solid cards (nothing of the wallpaper shows through) with a warm-grey border of their own theme; a critical notice's border is vermilion.

4.5 Wallpapers



The two default wallpapers, redrawn as pixel art: dawn-mart on the dark theme, station-street on the light.

Super+W (or *wallpaper* in quick settings) opens the picker: a small window with the current theme's wallpapers listed on top and, underneath, the one under the cursor — the picture itself, as your screen will show it, scaled down by a whole number so the pixel art stays crisp. Move with ↑/↓ or the mouse, type to search; **Enter** sets it, **Esc** leaves everything as it was. A ◖ after a name means the place also comes in the other theme's light (see the pairs below), and choosing it sets both.



The wallpaper picker (Super+W) on both themes: the list on top, the picture under the cursor as the screen will show it.

Each theme remembers its own wallpaper. The choices:

- **default** — a retro railway town from the pack: **dawn-mart** on the dark theme (a konbini glowing by a level crossing under the stars) and **station-street** on the light one (the same kind of crossing on a shop street under cherry blossom). Without the pack, the default is *fuji*.
- **fuji** — drawn here: the seigaiha (blue-sea wave) pattern tiled 1:1, with Mt Fuji, a torii in the water, birds and a woodblock-print cartouche along the bottom; by night on the dark theme, by day on the light one.
- **the pack** (`WALLPAPER_PACK="yes"`) — 70 pictures (38 for the dark theme, 32 for the light) fetched once from GitHub at pinned commits and checked by SHA-256, then redrawn as 640×360 pixel art with 32 colours. Every one is Japanese and retro, and sits in the themes' own colours — ink and paper, vermilion, teal and indigo (each was measured against the two palettes; nothing neon, nothing off-key) — and no two look alike. The picker lists them grouped by kind:

- *dark*: towns and streets by night (the konbini at the crossing, a lantern-lit pixel alley, a station's vending machines, a two-car train by the sea, Yokohama's Meiji harbour); shrines, gardens, rain and snow (a torii in the rain, a castle in the pines, a stone lantern by a moonlit pond, an onsen in snow); rooms and baths (a tatami room, a sentō, a CRT by a rainy window); ink and woodblock (Hokusai's waves and Lake Suwa, an ensō, a plum branch at night, a ronin); and anime and film (Jujutsu Kaisen, My Hero Academia, Hunter × Hunter, Final Fantasy VII, Totoro, Your Name, Girls' Last Tour, Chainsaw Man).
- *light*: streets and villages by day; Hiroshige's Tōkaidō road (driving rain at Shōno, night snow at Kanbara, the Hakone pass, the mist at Mitsuke, the crests at Seki, the crowd fording the Ōi); Hokusai (the Great Wave, the fisherman at Kajikazawa, the gust at Ejiri, Mannen bridge, the roofers at Mitsui); ukiyo style, ink and brush (kimono walkers under cherry trees, an ink dragon, a plum branch, an ensō, koi, a tiger, Musashi); and anime (Demon Slayer, Frieren and three illustrations).

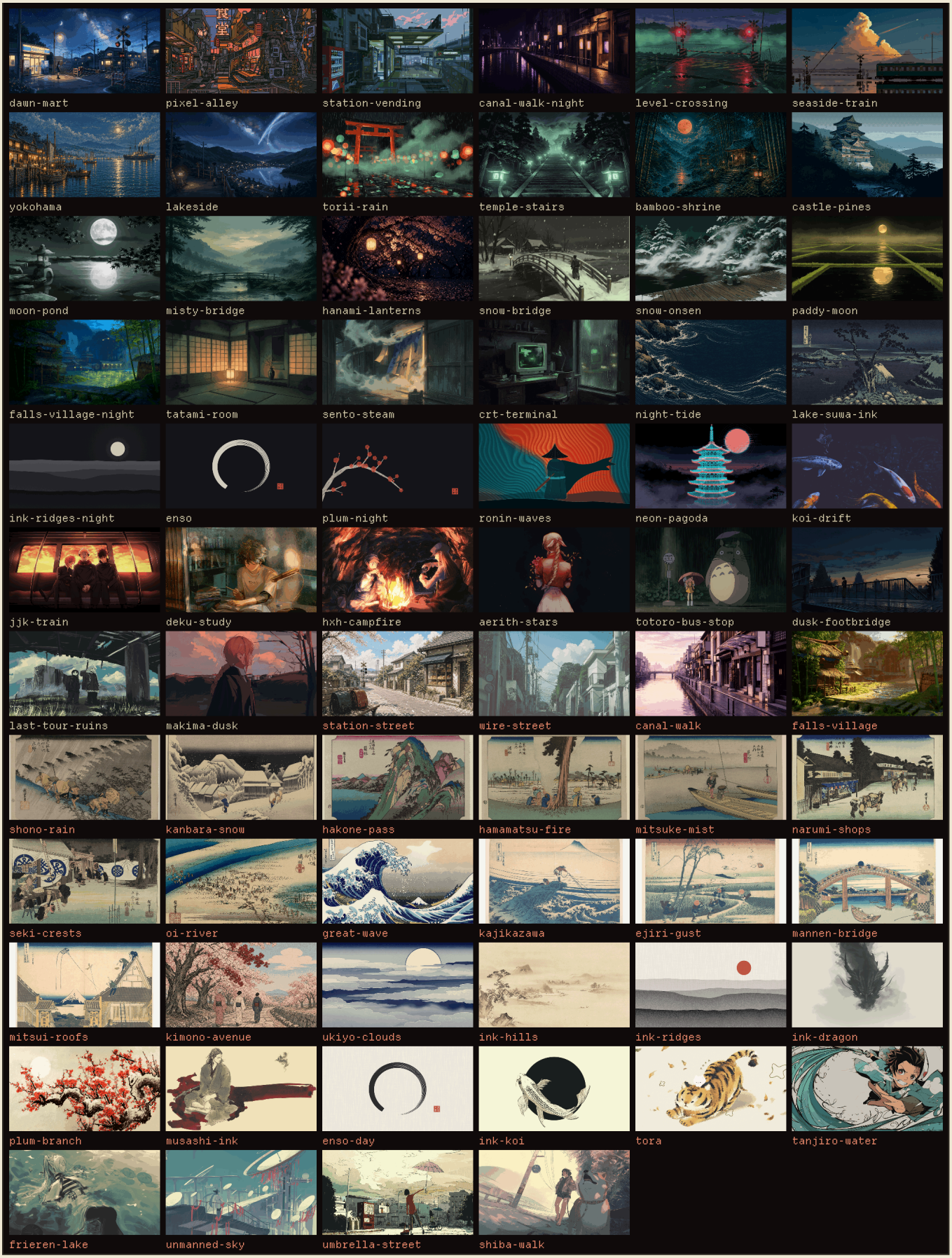
Five come in pairs — the same picture by day and by night (a canal walk, a waterfall village, ink ridges, the ensō, the plum branch) — and choosing one sets the other theme's too. Sources and licences, one line each: `~/.local/share/fedokawaii/wallpapers.list`. Nothing is stored in the repository; the files are fetched for your own use. A picture dropped from a later list is deleted from your pack the next time it is fetched. * **your own** — anything in `~/Pictures/Wallpapers`, shown as it is on either theme.

Pixel art is enlarged by a whole number to cover each monitor, so it never blurs: every pixel of the 640×360 picture becomes a square block of screen pixels.

Screen	Each art pixel becomes	640×360 becomes
1080p	3 × 3	1920×1080
1440p	4 × 4	2560×1440
4K	6 × 6	3840×2160

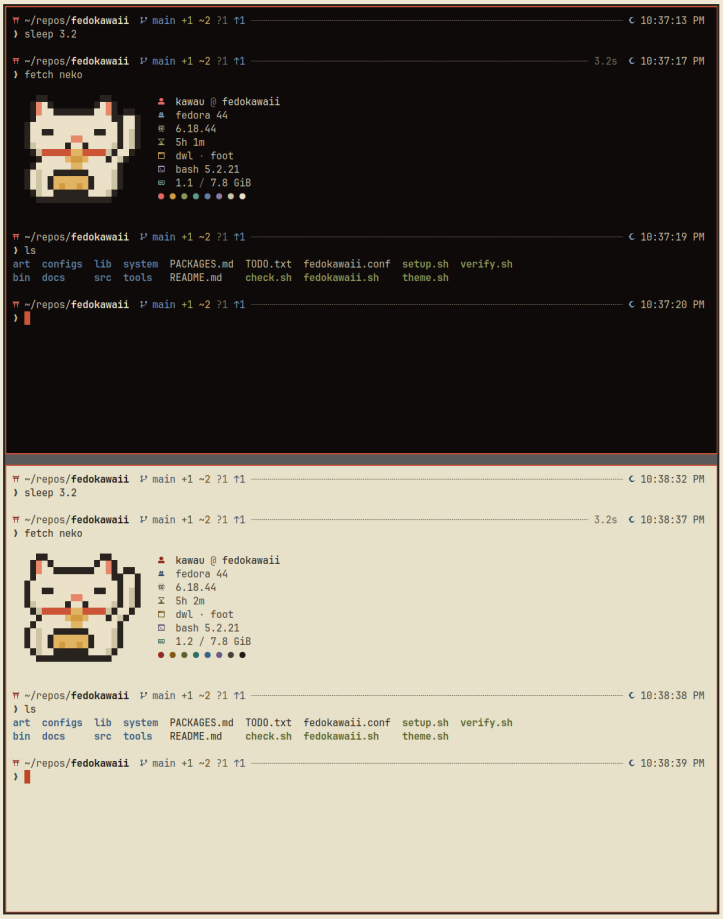
The results — and the picker's previews — are cached in `~/.cache/kawa-wallpaper`. A new choice dissolves in over the old one in an ordered dither whose grain is exactly one pixel of that art.

Tip: `~/.cache/kawa-wallpaper` can be deleted at any time; `kawa-wallpaper --restore` draws the current wallpaper again.



The whole pack: the dark theme's pictures first, then the light theme's.

4.6 The terminal and the prompt



A real terminal: the prompt's ledger line per command, a command that took over three seconds, fetch, and ls in a git repository with changes.

```
⛩ ~/repos/site   main +2 ~1 ····· 4.2s   × 2   ☾ 08:52:07 PM
❱
```

Part	Shows
torii	where every entry starts
path	the folders above in the text colour, the one you are in bright and bold (the last three; `…/` for the rest); `user@host` first over SSH
git	branch, then only what is non-zero: `+` staged, `~` modified, `?` untracked, `!` conflicted, `↑` ahead, `↓` behind — one `git status` call, parsed in bash
rule	a dotted line to the right edge, so each command reads as a new entry
right end	time taken (3 s or more) · the exit code in a vermilion seal when the command failed · sun by day, moon by night · the time to the second
`❱`	the text colour; vermilion after a failure

(The terminal's "vermilion" and "paper" are its ANSI red and white, so they follow the theme like everything else in it.) Nothing on the header line has a background, so the cursor below never touches it. Outside a git repository the prompt runs entirely inside bash — not one process is started to draw it.

Feature	How
new terminal in the current folder (Ctrl+Shift+N)	the prompt reports the folder (OSC 7)

Feature	How
jump between prompts (Ctrl+Shift+Z / X)	the prompt marks itself (OSC 133)
last command's output in neovim (Ctrl+Shift+G)	OSC 133 marks around the output + foot's `pipe-command-output`
history	50 000 entries, duplicates erased, each command saved as it runs, so every new window has it; Up/Down search by what you typed
instant terminals	`foot --server` at login; Super+Return opens a client

OSC (Operating System Command) sequences are short invisible messages a program prints to the terminal. The prompt sends four, and foot acts on them:

The prompt prints	When	foot uses it for
`OSC 7 ; file://host/folder`	before each prompt	**Ctrl+Shift+N** opens the new terminal in that folder
`OSC 133 ; A`	where each prompt starts	**Ctrl+Shift+Z** / **X** jump between prompts
`OSC 133 ; C`	when a command starts	the start of its output
`OSC 133 ; D`	when it has finished	the end of its output: **Ctrl+Shift+G** sends what lies between to neovim

Read more: foot's README, *Shell integration* (codeberg.org/dnkl/foot).

`fetch` prints one of ten pictures drawn for this desktop — a beckoning cat, a daruma, a torii at sunset over the sea, a beige computer showing Mt Fuji, Fuji at dawn, an onigiri, a paper lantern, a cassette, a fox mask and a bowl of ramen — beside the system facts. `fetch NAME` picks one (`neko`, `daruma`, `torii`, `pc`, `fuji`, `onigiri`, `chochin`, `cassette`, `kitsune`, `ramen`). It is pure bash and prints in a few milliseconds.

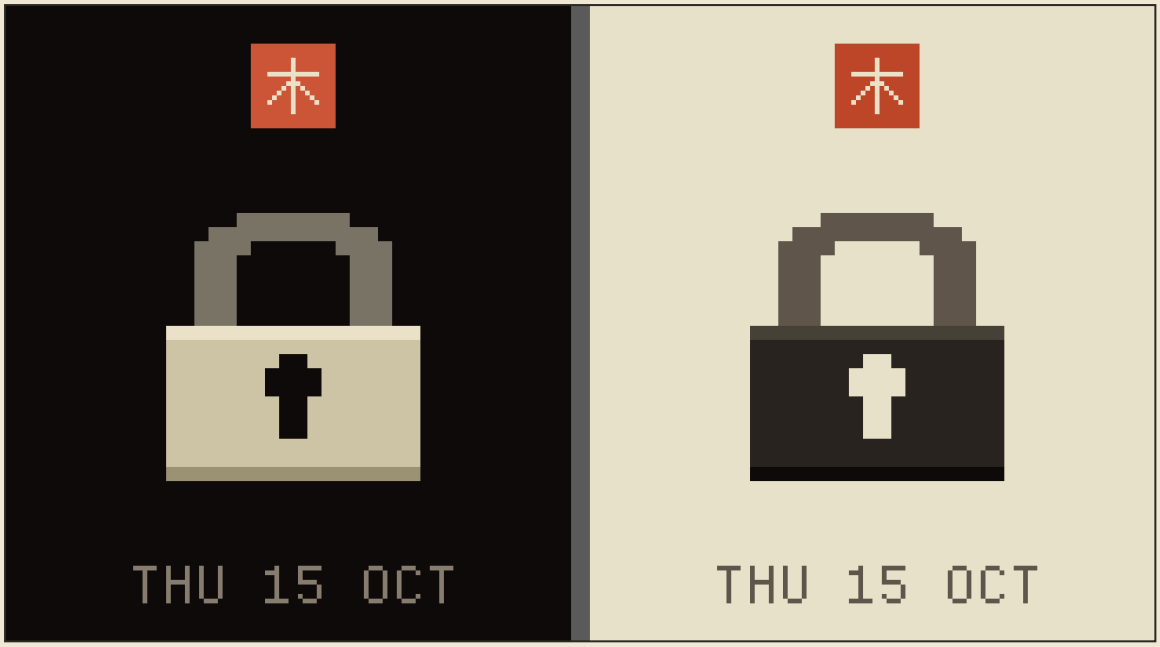


The ten fetch pictures, on ink and on paper: two pixels per character cell, drawn with half blocks.



The terminal's sixteen colours on both themes, with bold, dim, italic, underline, reverse and strike.

4.7 Lock, login, boot



The middle of the lock screen on both themes, at twice its size: the day's kanji seal, the pixel padlock (the typing indicator appears in its keyhole) and the date.

The **lock screen** is the theme's plain ground with a pixel padlock in the exact centre — paper on ink, or ink on paper. Above it the day of the week sits in a small vermilion seal, in kanji (月 Monday, 火 Tuesday, 水 Wednesday, 木 Thursday, 金 Friday, 土 Saturday, 日 Sunday), and below it the date in the bar's pixel letters; both are drawn into the picture as it locks, in a few tenths of a second. Before sleep it locks with the bare padlock at once, so the lid can close. swaylock's indicator is shrunk to a 15-pixel ring that sits inside the padlock's keyhole, clear of its face: it is invisible until you type, then the keyhole glows — vermilion while typing, teal while checking, red when wrong. It locks before sleep, with **Super+Escape**, and by itself when you are away:

Idle for	What happens
9 min 50 s	a quiet notice, *locking in 10 s* — touch a key and nothing more happens
10 min	the screen locks
15 min	the screens power off (windows stay where they were); the first key wakes them

The password goes in in the keyboard layout that was active (the bar showed it before the lock); if the keyhole turns red although the password is right, press Alt+Shift and type it again.

The **login** is greetd's plain text prompt (agreety) on the console: user name, password, nothing else. That console (tty1) always uses the dark theme's 16 colours (set as the kernel parameters `vt.default_red`, `vt.default_grn` and `vt.default_blu`, sixteen numbers each) and Terminus bold — 12×24 below 1440p, 16×32 from 1440p up. On Fedora 44 the spare consoles (**Ctrl+Alt+F2…F6**) look the same; Fedora 45 runs kmscon there — a console that runs as an ordinary program instead of inside the kernel — which keeps its own colours and font (`/etc/kmscon/kmscon.conf`). To see the palette the kernel was given (the first number is the red of the ink ground, `0f` in hex):

```
grep -o 'vt.default_red=[0-9,]*' /proc/cmdline    # → vt.default_red=15,… (15 = 0x0f)
```

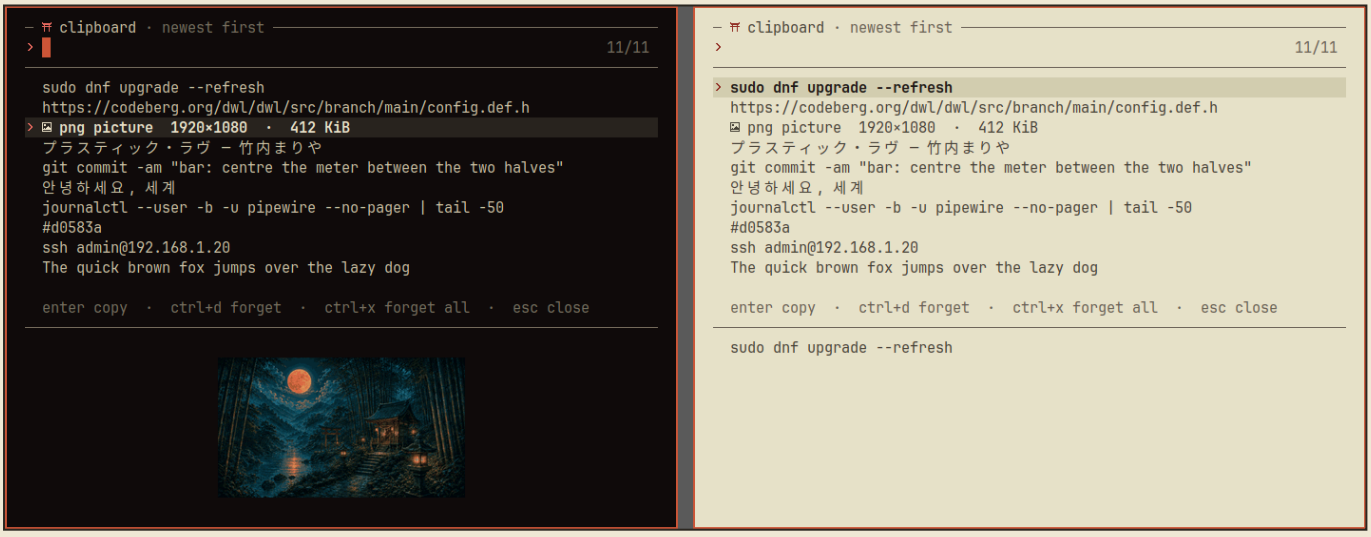
Read more: Fedora's change to kmscon.



The boot menu at 1080p: sumi ink, a hem of seigaiha waves fading upwards in a dither, Departure Mono at twice its grid.

The **boot menu** is a GRUB theme in the same language, at your monitor's native resolution: plain sumi ink with a hem of seigaiha waves along the bottom edge, fading upwards in an ordered dither; the torii above a quiet box of entries, the chosen one marked by a vermilion bar, a thin vermilion line counting down. Its text is Departure Mono at two to four times its grid (22, 33 or 44 px, by the screen's height). It shows for `BOOT_MENU_SECONDS`; with `0` Fedora's hidden menu returns (it then appears only after a failed boot, or when you hold **Shift** or press **F8** as the machine starts). Re-run **Theme** after changing monitors so it matches the new resolution.

4.8 Copy and paste



The clipboard history (Super+V) on both themes: newest first, the full text or the picture itself underneath.

Copying works the same way in every program: Ctrl+C / Ctrl+V (Ctrl+Shift+C / V in the terminal), and selecting text in the terminal copies it as well. Two things make it dependable:

- **A copy outlives its program.** On Wayland a copy is only a promise — the program that copied hands the data over when something pastes — so normally, closing it empties the clipboard. Here `kawa-clipkeep` (C, in `src/kawa-clipkeep/`, built by **Theme**) reads every format of each copy the moment it is made, and when the clipboard is then emptied because its owner closed, it offers the same copy again, in all its formats. While the program that copied is open nothing changes: it keeps serving

its own copy. Copies a password manager marks as secret are never kept, nor copies over 64 MiB, nor the primary selection (select + middle-click). One limit, from Wayland itself: clearing the clipboard on purpose while the program that copied is still open looks exactly like that program closing, so the copy comes back once — clear it again, or close that program first.

- **Between Wayland and X11 programs** (Steam, older games) the compositor passes copies both ways by itself. (*X11* is the older display system Wayland replaces; programs written only for it run through *XWayland*, a compatibility layer dwl starts when one needs it.)

Two commands show what the clipboard holds right now:

```
wl-paste --list-types    # the formats the current copy is offered in, one per line
wl-paste                 # the copy itself, as text
```

Super+V opens the clipboard history in a small floating window: the last copies, newest first, pictures included. Type to search; ↑/↓ or the mouse move; the full text — or the picture itself — shows underneath.

Keys	Does
Enter	put that entry back on the clipboard and close; then paste as usual
Ctrl+D	forget that entry
Ctrl+X	forget them all (it asks first; the clipboard itself stays)
Esc	close, the clipboard exactly as it was

The history is cliphist's (`~/.cache/cliphist`), fed by `wl-paste --watch` from the moment you log in; it keeps the last 750 copies. Its window uses the terminal's own colours, so it follows dark and light like foot.

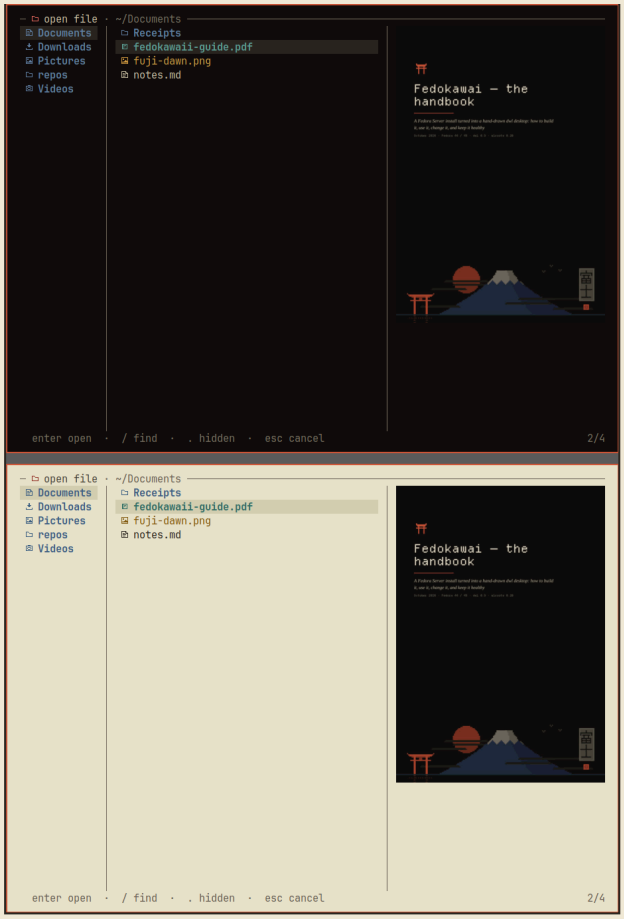
Tip: copied a password by hand from somewhere that does not mark it secret? Open Super+V and press Ctrl+D on it, so it does not stay in the history.

4.9 Files, file dialogs and default programs

Files — Super+E opens yazi in a window of its own; folders opened from anywhere (a download's *Show in folder*, `xdg-open ~/Pictures`) open the same way (§11.4).

File dialogs

Every program's *Open*, *Upload* and *Save as* — Brave's uploads and downloads, every GTK program's dialogs — opens **yazi** (§11.4) in a small floating terminal: the same file manager, keys and previews you already use, instead of a separate dialog with its own ways. It is dressed like the other popups — what it is for in a rule along the top (*open file*, *open files*, *open folder*, *save file*, with the folder you are in), the keys that matter along the bottom. It starts in the folder the program suggests (Brave: where you last saved or uploaded), else your home.



The file dialog an upload opens: yazi in a floating window, the PDF under the cursor previewed.

In the dialog	Open a file	Open a folder	Save
move · into a folder · back	↑ ↓ (or j k) · → or Enter (l) · ← (h)	the same	the same
choose	**Enter** on a file	**Enter** on a folder, or **q** in the one you are in	**Enter** on the file the program suggested (it is already there, under its name; **r** renames it first)
jump	**z** a few letters of a name below here · **Z** a folder you use often · **g d** Downloads	the same	the same
a USB stick, disk or SD card	**M**: the drives; **Enter** on one goes there (§4.13)	the same	the same
several at once	**Space** marks each, then **Enter** (where the program accepts several)	Space, then Enter	—
save into another folder	—	—	go there and press **Enter** on the folder
replace a file	—	—	**Enter** on it; it asks first
search · hidden files	`/` · `.`	the same	the same
cancel	**Esc**	**Esc**	**Esc** (the suggested file goes again)

Saving shows the file before it is written: the program's suggested name is waiting in the folder, standing in for the file until you choose — its preview says *the file being saved* — and Enter keeps it there. Cancel, and the stand-in is removed again: nothing is left behind.

How it is wired, should you look. A *portal* is a D-Bus service through which a program asks the desktop for something it should not do alone — a file dialog, a screen share, the colour scheme; *xdg-desktop-portal* takes each request and hands it to a *backend* named in a small config file. Here the file-dialog backend `xdg-desktop-portal-termfilechooser` (built by setup from a pinned release, `/usr/local/libexec/`) hands every dialog to `~/.local/bin/kawa-pick`, which opens yazi; `~/.config/xdg-desktop-portal/wlroots-portals.conf` routes dialogs there, with GTK's own dialog as the fallback — should the portal ever be missing, programs show that one, in the desktop's colours, instead. The portal's settings are in `~/.config/xdg-desktop-portal-termfilechooser/config`.

The routing file's settings, one comment each:

```
[preferred]
# anything not named below: the GTK backend
default=gtk
# screen sharing and screenshots: the wlroots backend
org.freedesktop.impl.portal.ScreenCast=wlr
org.freedesktop.impl.portal.Screenshot=wlr
# the dark / light setting that GTK 4 and Brave follow
org.freedesktop.impl.portal.Settings=gtk
# file dialogs: yazi first, GTK's own if it is missing
org.freedesktop.impl.portal.FileChooser=termfilechooser;gtk
```

Its name comes from `XDG_CURRENT_DESKTOP=wlroots`, which the session sets: the portal looks for `wlroots-portals.conf`. *Read more:* portals.conf(5).

Default programs come from `~/.config/mimeapps.list` (set by Theme), so `xdg-open`, yazi, Brave's downloads and every other program agree. Each kind of file has a *MIME type* — a standard name such as `text/plain` or `application/pdf` — and the list names one program for each. `xdg-mime` asks it:

```
xdg-mime query default text/plain        # → kawa-edit.desktop (neovim in a terminal)
xdg-mime query default inode/directory   # → kawa-files.desktop (yazi in a terminal)
```

The whole list:

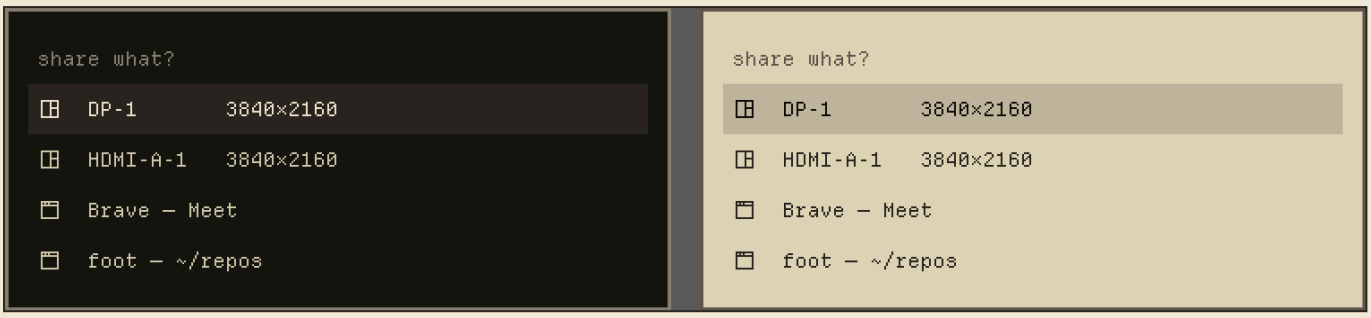
Type	Opens in
web pages, links	Brave Origin
folders	yazi (in a terminal window)
text, code, config files, Markdown	neovim (in a terminal window)
pictures	imv
video, music, playlists	mpv
PDF, EPUB, XPS, FictionBook	zathura
comic books (CBZ/CBR)	zathura
Word / OpenDocument text, RTF	AbiWord
spreadsheets, CSV	Gnumeric
Xournal++ notes	Xournal++
magnet links, `.torrent` files	torrra (§12)

Programs that ask for a terminal (yazi's own launcher entry, say) get foot.

4.10 Screen sharing — picture and sound

A call in Brave (or any program) that shares a screen asks the desktop portal, which asks what to share with the **share what?** menu: every screen with its size, and every open window by its title. With only one screen and no choice to make, it is taken without

asking. Sharing runs at up to 60 frames a second. While anything is shared, notifications are held back so they never appear in the call — critical ones and the level display still show.



"share what?" — two screens with their sizes, and the open windows by title.

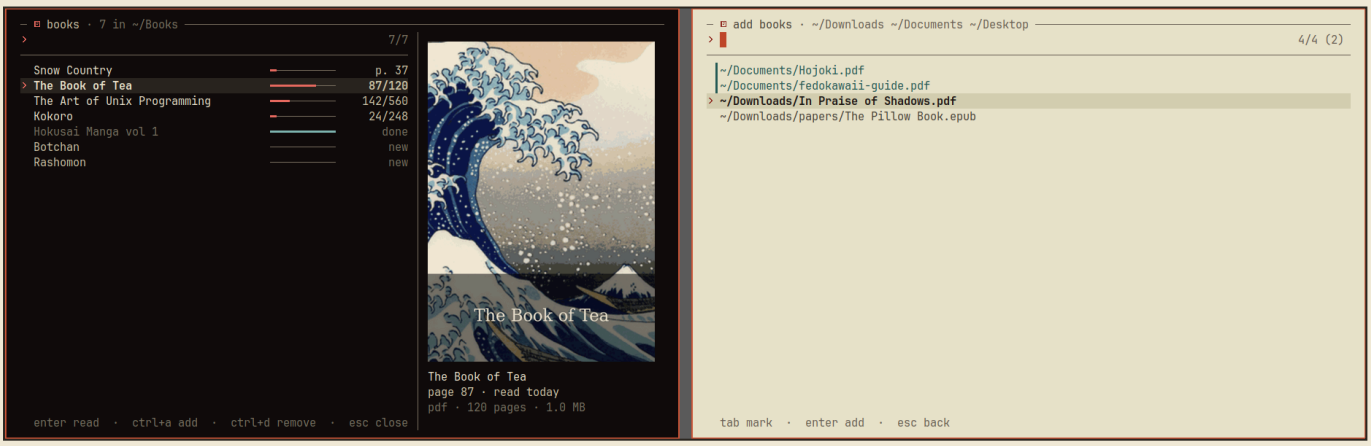
Sound has three ways in, depending on what the call needs:

You want the others to hear	Do
the sound of a **Brave tab** (a video you are showing)	share the tab and keep *Also share tab audio* ticked — Brave captures it itself
the **computer's sound** with a whole screen or a window	tick *Also share system audio* in Brave's picker (Theme turned this Brave feature on). It carries everything you hear — including the call itself, so others may hear an echo of their own voices. Headphones do not help; use it for a moment, or use the next row instead
only certain programs (a game, a player), in any call program	in pavucontrol (audio menu → mixer → *Playback*) move those programs to the **Share** output — you still hear them — then pick **Share (as microphone)** as the microphone, or as a second audio input, in the call program. Only what you moved there is sent, never the call

PipeWire is the sound server: it carries every program's sound to the speakers — and the pictures of a screen share to the call. **Share** is a pair of devices of its own, set up in `~/.config/pipewire/pipewire.conf.d/60-share.conf`: an output you can send programs to, and *Share (as microphone)*, which a call can record from. `wpctl status` lists both with every other output and input.

Your microphone itself travels as usual; check it in the audio menu (Super+A → input).

4.11 Books — Super+R



The books window (Super+R). Dark: the ones being read first, each with its page and a bar, the cover of the one under the cursor beside. Light: adding books (Ctrl+A) from Downloads, Documents and Desktop.

Super+R opens your library: every book in `~/Books` (and its folders) — PDF, EPUB, MOBI, FB2, XPS and comic books (CBZ, CBR, CB7) — in a small floating window, the ones you are reading at the top, the most recent first. Each line shows how far in you are — a bar (vermilion while reading, teal once finished) and, at the right, one of four marks:

Mark	Means
`87/120`	reading: on page 87 of 120 (the page zathura remembered when you last closed it)
`0`	

Mark	Means
`p. 37`	reading a book without fixed pages — EPUB, MOBI, FB2 flow their text to the window's size, so there is a page but no total (its bar shows one step: begun)
`done`	finished: you closed it on its last page
`new`	not opened yet (its kind, page count and size show under its cover)

The cover of the book under the cursor shows beside the list, centred in its pane, with the title, where you are and when you last read it under it.

Keys	Does
type	search by title
Enter	read it in zathura — it opens on the page where you stopped (§11.12)
Ctrl+A	add books: every book in `~/Downloads`, `~/Documents` and `~/Desktop` is listed; **Tab** marks several, **Enter** moves them into `~/Books`, **Esc** goes back
Ctrl+D	remove it: the file goes to the trash (yazi: **g t** brings it back)
Esc	close

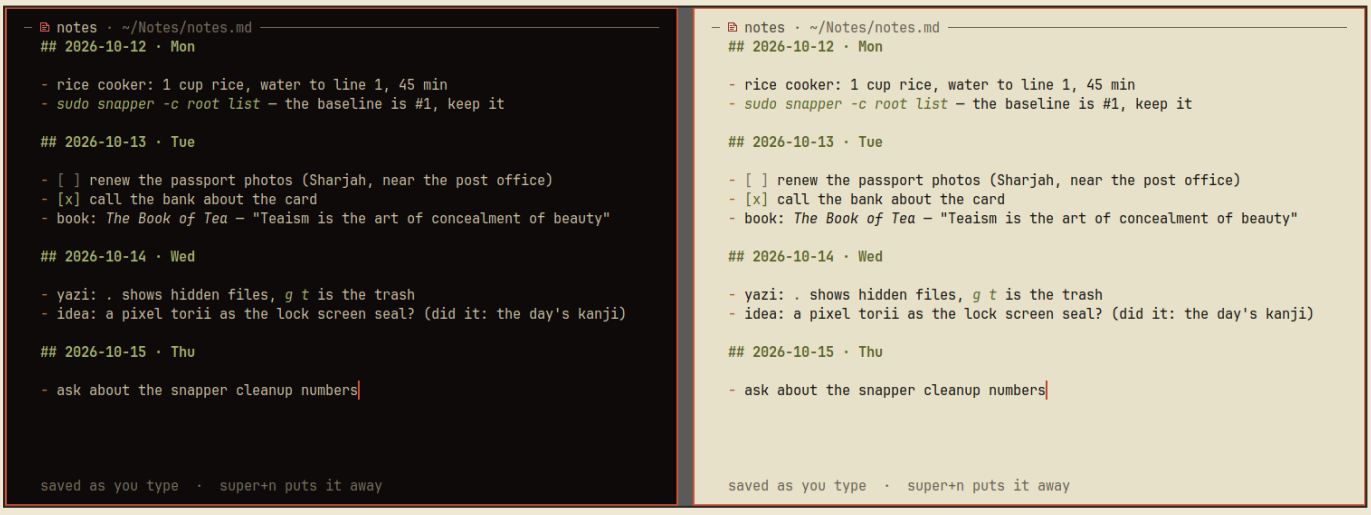
How it keeps track

Nothing new runs in the background and nothing is stored twice: the page comes from zathura's own memory, which zathura writes when you close a book; the window only reads it. Page counts and covers are worked out once per book and kept.

What	Where
your books	**`~/Books/`** — folders inside it are fine
the page you are on, per book	`~/.local/share/zathura/bookmarks.sqlite` (zathura's; read, never written)
covers and page counts	`~/.cache/kawa-books/` — safe to delete, drawn again when needed
the window	`~/.local/bin/kawa-books` (Super+R)

Tips. Folders inside `~/Books` (`~/Books/Japanese`, `~/Books/Work`) keep a large library tidy; the search matches titles anywhere in it. A book moved to another folder keeps its page: the window finds it by its name and opens it there. In zathura, **Ctrl+r** turns the pages into the theme's colours for reading at night (§11.12).

4.12 Notes — Super+N



The notes window (Super+N): one file, a heading per day, open at the end and ready to type.

Super+N opens your notes in a small floating window; **Super+N again** puts them away. It is one plain file, **`~/Notes/notes.md`**, that reads top to bottom like a diary: the first time you open it on a day, that day's date becomes a heading, and the window opens at

the end, ready to type. Write anything — a phone number, a command, a thought — and press Super+N; it is saved as you type, so closing the window any other way (Super+Q included) loses nothing. A day you opened but wrote nothing under leaves no empty heading behind.

What	Where
your notes	**`~/Notes/notes.md`** — plain Markdown, one file
the window	`~/.local/bin/kawa-notes` (Super+N): neovim, the desktop's editor, dressed as a page — the name in a rule across the top and a quiet line of keys along the bottom, like every popup; no line numbers, the text three columns in (`~/.config/fedokawaii/notes.lua`)

It is neovim (§11.5), so its keys work: **Esc** then `/word` finds something, `gg` goes to the top, `u` undoes. *Markdown* is plain text with a few marks for structure. It is optional, but it makes notes easy to scan later:

You type	It means
`## Japan trip`	a heading
`- milk`	an item in a list
`- [ ] call the bank`	a to-do; `- [x]` once it is done
`**bold**`	what matters

Tips. To find something from any terminal: `rg -i rice ~/Notes`. Notes are plain text in your home, so the snapshots of `/home` keep their history too (§7.2). Want a separate file for a project? Start one in `~/Notes` from neovim (`:e ~/Notes/japan-trip.md`); Super+N always opens `notes.md`.

4.13 Drives — USB sticks, disks, SD cards



A stick that opened (click the notice for its files), then eject's "safe to unplug" — dark and light.

Plug a drive in and it opens by itself: a notice names it, the bar shows the drive mark, and its files are one click away. **Eject** before you pull it out; the notice **safe to unplug** says when you can.

Terms.

Term	Plain meaning
file system	how a drive keeps files: *exFAT* (most sticks and SD cards), *FAT32* (small or old ones), *NTFS* (Windows disks), *ext4* or *Btrfs* (Linux disks)
mount · mount point	to open a file system so its files show in a folder · that folder: `/run/media/YOU/LABEL` (a file system with no label: its UUID instead)
label	the name a file system carries (`KINGSTON`, `CANON EOS`); a drive with none is shown by its maker and model (a card in the machine's own reader: *SD card*)

Term	Plain meaning
eject	write out what is still on its way to the drive, close every file system on it, then switch the drive off — or, when it says its media come out (a card in a reader, and many sticks say so of themselves), eject the media. Either way it is safe to unplug
udisks · udiskie	the system service that mounts drives for the person at the screen, without a password · the small program the session runs that asks it to mount whatever you plug in
LUKS	Linux's standard disk encryption: an encrypted drive opens only with its passphrase

What happens, step by step:

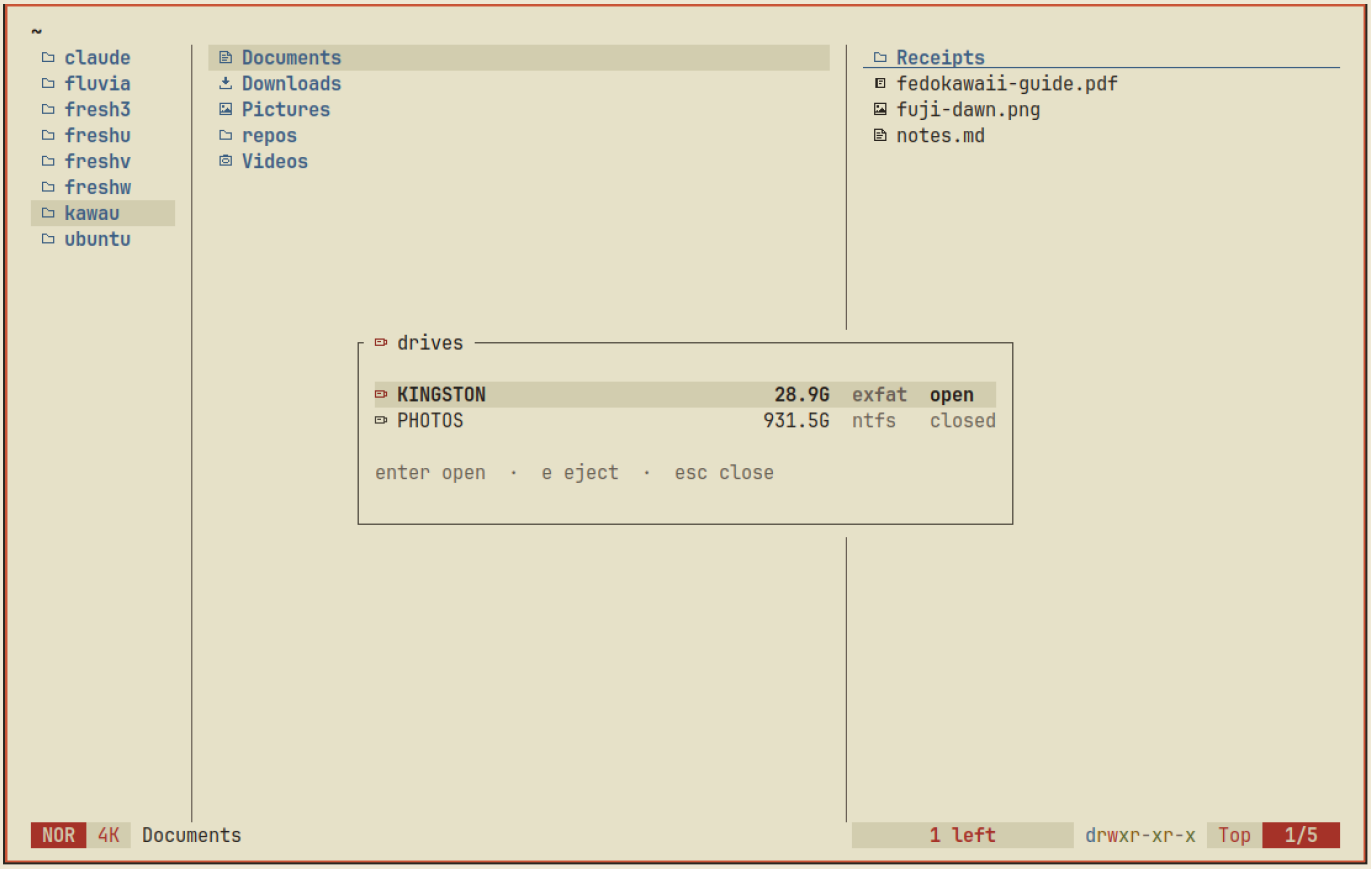
You	You see
plug a stick in	a notice: **KINGSTON** — *28.9G vfat · click to open it in the files*; the drive mark in the bar
click the notice	yazi opens on the stick, in a window of its own
plug in an encrypted drive	a small box asks for its passphrase (typing hidden); **Esc** leaves it locked
click the drive mark (or quick settings → **drives**)	the drives menu: each file system with its size, *open* or *closed*, then **eject** for each drive
choose **eject KINGSTON**	**safe to unplug** — or, if a program still has a file open there, which one: close it, eject again. Right after a big copy it first says it is *writing what is still on its way*, and that notice turns into **safe to unplug** when done
a drive that will not open (a file system Linux cannot read, a wrong passphrase)	a notice titled **not opened**, with the reason
pull a drive out without ejecting	a critical notice (the vermilion border) says so, and stays until you click it: what was still being written may be lost



The drives menu, from the bar's drive mark (the 2: two drives plugged in): one open, one closed, each to eject.

In yazi and in every file dialog (uploads in Brave included), **M** shows the drives:

Key	Does
M	the drives: each with its size, file system, open or closed
j k · ↑ ↓	move through the list
Enter · l · →	open it (mounted first if need be; an encrypted one asks for its passphrase) and go there
e	eject its drive: yazi steps out of it first, the list closes, and a notice says when it is safe to unplug. Standing in a drive, the list opens on it — **M** then **e** ejects the drive you are looking at
Esc · q · M	close the list
g m	go to `/run/media/YOU` , where every open drive is (no drive open: a note says so)



M in yazi: the drives, the open one lit; Enter goes there, e ejects it.

So to attach a photo from an SD card in Brave: **Upload** → the dialog opens → **M** → Enter on the card → Enter on the photo.

From a terminal — the same things, by hand:

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINTS     # every disk and what is on it; sdb, sdc… are drives
kawa-drives list                                 # just the drives: a line per file system
udisksctl mount -b /dev/sdb1                     # open one → "Mounted /dev/sdb1 at /run/media/…"
kawa-drives eject /dev/sdb1                      # write out, close and switch off that drive
```

`lsblk` is the map: a whole disk (`sdb`), its partitions (`sdb1`), and for each its size, file system, label and where it is open. The system's own disk is `nvme0n1` (or `sda` on SATA); the drives menu never lists it, and nothing on the machine's own disks opens by itself — a Windows partition or a second internal drive stays closed until you open it (`udisksctl mount -b …`).

A **disk image** (an `.iso`, an `.img`) opens the same way: `udisksctl loop-setup -f FILE.iso` makes it a drive of its own, it opens with a notice, and the drives menu and **M** list it under its label (an image with none: under its file's name); eject closes it and frees the file.

Looking after a drive — the tools for the usual file systems are installed. Each works on a *closed* file system: close it with `udisksctl unmount -b /dev/sdb1`, and the drive stays plugged in and on. (Ejecting does not help here: the drive is switched off, and plugged back in it opens again by itself.) Read the name with `lsblk` twice: these act on whatever you name.

To	exFAT	FAT32
check and repair	`sudo fsck.exfat /dev/sdb1`	`sudo fsck.vfat -a /dev/sdb1`
rename (the label)	`sudo exfatlabel /dev/sdb1 TRIP`	`sudo fatlabel /dev/sdb1 TRIP`
erase and start over	`sudo mkfs.exfat -L TRIP /dev/sdb1` — **everything on it is gone**	`sudo mkfs.vfat -F 32 -n TRIP /dev/sdb1` — the same

An NTFS disk is Windows' own: check and repair it there (`chkdsk`); here it opens as it is.

exFAT is the one to choose for a stick shared with Windows, macOS, cameras and TVs: every one of them reads and writes it, and a file may be larger than 4 GB, which FAT32 refuses.

Pitfalls.

- An NTFS disk from a Windows PC that was not fully shut down — *Fast Startup* or hibernation leaves it marked as in use — opens read-only or not at all. Shut Windows down with **Shift** held on *Shut down*, or turn Fast Startup off in Windows' power settings.
- "still open in: bash" (eject's notice) means a terminal stands in a folder on the drive: `cd` out of it (or close that terminal), then eject.
- A drive with nothing Linux can read on it — brand new, or a file system it does not know — shows no mark and opens nothing; `lsblk` still lists it.

What	Where
what is mounted, and how	udiskie: `~/.config/udiskie/config.yml` (started by the session)
the notices, the mark, the menu, eject, the passphrase box	`~/.local/bin/kawa-drives` (`kawa-drives --help`)
the drives in yazi	`~/.config/yazi/plugins/drives.yazi/`, the keys in `~/.config/yazi/keymap.toml` (the file dialog's: `~/.config/fedokawaii/yazi-pick/keymap.toml`)
where drives open	`/run/media/YOU/LABEL`

Read more: udisks · udiskie · `man lsblk` · `man udisksctl`.

5. How the look is built

5.1 Fonts

Font	Where	Why
Departure Mono (Nerd Font build)	bar, menus, notifications, boot menu, the game overlay	a pixel font on an 11 px grid — set at 11 px (or 22, 33), never in points, with smoothing off
Sumi Icons	every icon in the bar, menus and notifications — and, as *Sumi Icons Term*, in the prompt, fetch and the clipboard and wallpaper windows	the desktop's own icon font, built from `art/icons.txt`
PixelMplus 12	Japanese in the pixel UI (song titles in the bar, menus, notices) and in the terminal	a 12 px Japanese pixel font, so a Japanese title in the bar stays sharp
JetBrains Mono (Nerd Font)	the terminal and neovim	readable code at 11.5 pt
Noto Sans CJK	Japanese, Chinese and Korean in programs, web pages and subtitles	a full, smooth face where text is read at length
Terminus	the text console	sharp bitmap font for the login
Noto Sans, Noto Naskh Arabic, Amiri, Liberation, Noto Color Emoji	GTK and Qt programs, documents, Arabic, emoji	

A *point* is a length (1/72 inch) that turns into a different number of pixels with each screen's scaling; a pixel size is exact, which is what a pixel font needs. *Fontconfig* is the library every program asks which font to use and how to draw it. Its rules in `~/.config/fontconfig/fonts.conf` switch three things **off** for the three pixel fonts (that rule comes after the general one, because in fontconfig the last matching edit wins):

Setting	What it does	Other fonts	Pixel fonts
`antialias`	smooths edges with in-between shades	on	off
`hinting`	nudges outlines onto the pixel grid	slight	off — they sit on it already
`rgba`	colour fringes (subpixel smoothing)	`rgb`	`none`

PixelMplus is drawn on a 12-pixel grid, so it stays unsmoothed only at 12 and 24 px; at any other size (Japanese inside the 11 px bar, say) it keeps the normal smoothing, which reads better than broken pixels. To see which font a program gets:

```
fc-match monospace              # → …: "JetBrainsMono Nerd Font" "Regular"
fc-match 'sans-serif:lang=ar'   # → …: "Noto Naskh Arabic" "Regular"
fc-match 'sans-serif:lang=ja'   # → …: "Noto Sans CJK JP" "Regular"
```

Read more: the Arch Wiki's Font configuration.

5.2 Icons



The icons of Sumi Icons, eight times their size: each square is one pixel on screen. Here each is centred in its tile; in the font it sits on the text's middle line.

Every icon is drawn as text in `art/icons.txt` — `#` for a pixel, `.` for nothing — on the same 11 px grid as Departure Mono, ten pixels tall and centred on the middle of the text line. `tools/build-icons.py` merges each drawing into a few rectangles and writes `SumiIcons.ttf` (code points U+FE000 onward, clear of every Nerd Font), its terminal cut `SumiIconsTerm.ttf` ("Sumi Icons Term": the same drawings two pixels higher, centred on the terminal font's taller capitals) and `~/.config/fedokawaii/icons.sh`, which gives the scripts names like `$ICON_SPEAKER`. Each font carries one harmless hinting instruction, so FreeType never auto-hints it: every pixel lands exactly where it is drawn.

A drawing is a block that starts with `@name`; this is the first in the file, the torii:

```
@torii
#........#
##########
.########.
..#....#..
##########
..#....#..
..#....#..
..#....#..
..#....#..
..#....#..
```

To change an icon:

1. Edit its drawing in `art/icons.txt`, in your working copy (chapter 15). Add new icons at the end of the file — inserting one in the middle renumbers the rest.

2. Build the fonts in a throwaway Python environment with `fonttools` (and `Pillow` for `--preview out.png`, a picture of every icon) — the commands below.
3. Run **Theme** from the working copy, or `fedokawaii apply` (chapter 15).

```
python3 -m venv /tmp/v                    # a private Python, thrown away later
/tmp/v/bin/pip install fonttools pillow   # the two libraries the script uses
/tmp/v/bin/python tools/build-icons.py    # writes both fonts and icons.sh
```

The meters are made of icons too: `eq-0` … `eq-5` are 3-pixel columns with up to five lit segments and a 4-pixel advance, and `eq-cap` has no width at all — printed after a full column in vermilion, it repaints that column's top segment. The music meter and the CPU history are built from them, and so is the level display's wedge (at twice the size); the brightness menu and the volume menu's title use the framed-bar icons `lv-*` (cells 8 px tall).

5.3 Transitions

`kawa-veil` (C, about 750 lines in `src/kawa-veil/`, built by **Theme** with the tools that build dwl) makes the theme and wallpaper changes smooth. It takes a still of every screen (the compositor's screencopy), shows it on the topmost layer with every click passing through, runs the change underneath, waits a moment for the apps to redraw, then lets the still go block by block over about two thirds of a second, with an ease in and out. Each block is one pixel of the wallpaper pack's art on that screen (3 px at 1080p, 4 at 1440p, 6 at 4K), and a block is only ever cleared, never blended, so every frame stays crisp. The order the blocks go in is the design: an 8×8 ordered dither for a wallpaper, and for a theme a front of seigaiha crests — arcs 96 px wide at 1080p (128 at 1440p, 192 at 4K) with a dithered fringe — rising or falling across the screen. It works on rotated screens too. If anything is missing (no compositor, no screencopy) the change simply happens without it, and a watchdog removes the veil after 20 seconds whatever happens.

Change	Style	Takes
to the light theme	crests rising from the bottom edge (`--rise`), like dawn	about 0.65 s
to the dark theme	crests falling from the top edge (`--fall`), like nightfall	about 0.65 s
a new wallpaper	an 8×8 ordered dither (`--dissolve`)	about 0.5 s

An *ordered dither* uses a fixed 8×8 pattern of thresholds to decide which blocks go first, so the change spreads as an even, fine grain rather than as random noise.

5.4 Where every app gets its colours

All theme files live in `~/.config/fedokawaii/themes/dark/` and `…/light/`, written from one palette by `tools/build-themes.py`. **`~/.config/fedokawaii/current`** is a link to one of them; every app reads its colours through that link, and `kawa-theme` switches it, then tells whatever is running to look again. A *symbolic link* is a small file that points to another file or folder; replacing it in one step switches every reader at once.

```
readlink ~/.config/fedokawaii/current                    # → themes/dark
ls ~/.config/fedokawaii/current/                         # the files the apps below read
gsettings get org.gnome.desktop.interface color-scheme   # → 'prefer-dark'
```

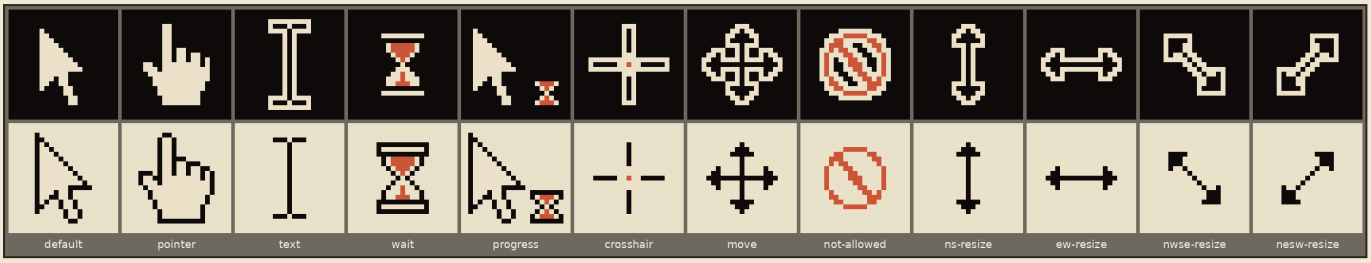
App	Reads the theme by	On a switch
foot (and everything in it: neovim, btop, yazi, lazygit, torrra, spotify_player, bat, nmtui, the clipboard window)	both palettes in `foot.ini`; `include=` picks the one to start in	SIGUSR1 / SIGUSR2 to the foot server switches every window; neovim notices by itself
neovim	gruvbox.nvim with the palette remapped; `background` is never forced	follows the terminal (DEC mode 2031)

App	Reads the theme by	On a switch
yazi, lazygit, bat, torrra, the clipboard window, nmtui	the terminal's 16 colours by name (yazi's `theme.toml`, lazygit's config, bat's `ansi` theme, torrra's `ansi-dark` (right on both: the palettes keep "black" a shade of the ground), `NEWT_COLORS` for nmtui)	with foot, at once
waybar	`@import` of `current/waybar.css`	SIGUSR2 (reload)
fuzzel	`include=current/fuzzel.ini`	read fresh at every menu
mako	`include=current/mako` (outside any section, as mako requires); the file carries its own sections for low-urgency, critical and level notices	`makoctl reload`
GTK 4 / libadwaita	one `gtk.css` with both palettes, chosen by GTK's color-scheme media query	gsettings `color-scheme`
GTK 3	two GTK themes of its own, `Fedokawaii` and `Fedokawaii-dark` (adw-gtk3 + the palette, in `~/.local/share/themes`)	gsettings `gtk-theme` — every open window, the file dialog included
Brave	GTK mode (set by Theme): its tab strip, toolbar and menus come from the GTK theme, a few rules in it reach Brave alone (the address bar keeps Brave's own colours); the new-tab page is the ground colour (`light-dark()`); web pages follow the portal's dark / light	live
Qt	`qt6ct.conf` is a link into the theme	the link is replaced; qt6ct watches its folder
mpv	`include` of `current/mpv.conf`: on-screen text, the controller, menus	every open player loads the new file over its socket
btop	`color_theme = "fedokawaii"`: `~/.config/btop/themes/fedokawaii.theme` is a link through `current` (btop takes themes only by name)	SIGUSR2
zathura	`include ../fedokawaii/current/zathurarc`	`SourceConfig` over D-Bus
imv, presenterm	their config file is a link into the theme	next start
swaylock	`kawa-lock` passes `-C current/swaylock`	next lock
the screenshot region	`kawa-screenshot` reads `current/colors.sh`	next screenshot
wallpaper	each theme's own choice	`kawa-wallpaper --restore`
MangoHud (games)	`MangoHud.conf` in the dark palette and Departure Mono	—
the console, boot menu	always the dark theme's colours	—

A *signal* is a short message the kernel delivers to a running program; `SIGUSR1` and `SIGUSR2` are left for each program to use as it likes — foot switches its palette on them, waybar reloads its style on `SIGUSR2`. `kawa-theme` sends them only to programs that already catch them, since an uncaught `SIGUSR1` would end the program.

To change a colour: edit the palette at the top of `tools/build-themes.py`, run `python3 tools/build-themes.py`, then **Theme**. A few files cannot read a generated one — dwl's `config.h` (compiled in), mpv's subtitle colours, satty, MangoHud, `fetch` and the scripts' fallbacks — so they hold the colours themselves, and every run of the generator checks that each one is still a palette colour (it stops if not).

5.5 Pointers



The mouse pointers at 8× their size, on ink and on paper.

The mouse pointer is drawn for the desktop too: the theme **Fedokawaii**, in `~/.local/share/icons/Fedokawaii/`. The arrow and the hand are the classic shapes in ink with a paper body, so they read on both themes; the thin ones — the text cursor, the cross for screenshots, the arrows for moving and resizing a window — are ink lines with a one-pixel paper rim; the hourglass's sand, the cross's centre and the "not here" sign are vermilion. Each is a drawing in `art/cursors.txt` at the desktop's own pixel scale: 24 px, every pixel one screen pixel, like the fonts and icons, and 48 px with every pixel doubled for a 2× screen — so a pointer is never smoothed or blurred.

Where	What
`~/.local/share/icons/Fedokawaii/cursors/`	the twelve pointers, and links for the older names programs ask for (`left_ptr`, `hand2`, `xterm`…)
`~/.local/share/icons/default/index.theme`	makes Fedokawaii the default for dwl itself and older (XWayland) programs
`XCURSOR_THEME`, `XCURSOR_SIZE` · `cursor-theme`	the same for everything else: the session's environment (`start-dwl`) and GTK's setting (`/etc/dconf`)

Shapes not drawn here — the rarer ones, like a "copy" or "zoom" pointer — come from Adwaita underneath, through one line of the theme's `index.theme` (the comment is added here):

```
[Icon Theme]
Name=Fedokawaii
Comment=Pixel pointers in ink and paper (tools/build-cursors.py)
# any shape this theme does not draw is taken from Adwaita
Inherits=Adwaita
```

To change one: edit its drawing, run `python3 tools/build-cursors.py` (standard Python only; `--preview out.png` draws them all), then **Theme**; the change shows at the next login.

6. Keeping it healthy

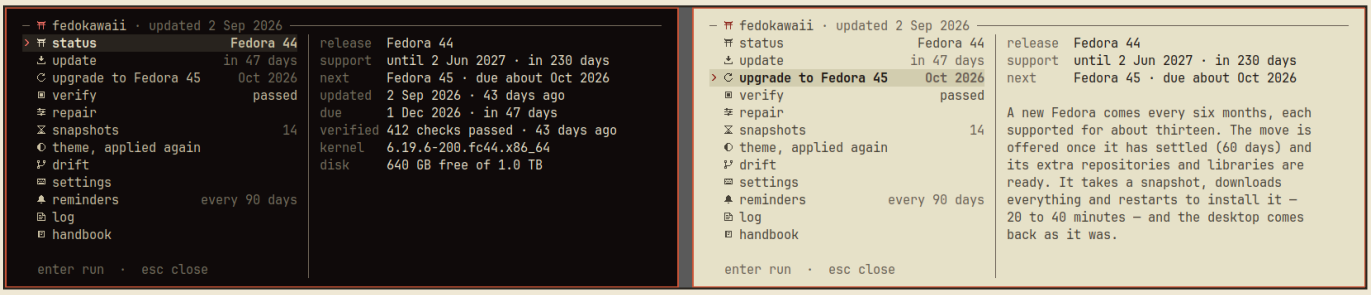
The machine is built to be left alone for months and to run for years on a toolkit that is never edited again: once a season you **update** it, about once a year you **move** it to the next Fedora, and both are one entry in one menu.

Terms.

Term	Plain meaning
update	newer versions of the packages of the release you have: Fedora 44 stays 44
upgrade, the move	going to the next release, 44 → 45: nearly every package is replaced in one go
kernel	the core of Linux, which drives the hardware; each new one is installed beside the one before
transaction	one `dnf` run that installs, updates or removes a set of packages together
SELinux	Fedora's mandatory access control: rules in the kernel that limit what each program may touch, even when it runs as root. *Enforcing* means the rules are applied, not only logged; `getenforce` says which
linger	systemd keeps your own services (Prowlarr, the portals) available with nobody logged in, so a run as root can reach them

Read more: SELinux on Fedora.

6.1 The control — Super+U



The system's control (Super+U): what is due at the right of each entry, how things stand in the pane — the status on the dark theme, the move to the next Fedora on the light.

Super+U opens it (also quick settings → *system*, or `fedokawaii` in a terminal): a small window with nothing to search, only twelve entries — ↑ ↓ (or **j k**) to choose, the mark at the right of each saying what is due, and beside them a pane with how things stand for the entry under the cursor. **Enter** runs it in the same window — what changes the system asks for your password — and a key afterwards brings the menu back with its marks brought up to date; **Esc** (or **q**) closes it.

Entry	Does	Its mark
status	how things stand, on one page	the release
update	the season's update (§6.2)	when the next is due; **due** in red once it is
upgrade to Fedora N	the move to the next release (§6.4)	when it is expected or offered; **ready** in red once it has settled
verify	every check (§6.7); changes nothing	**passed**, or how many to fix

Entry	Does	Its mark
repair	verify, put right what it found — packages, this machine's drivers and firmware (§6.9), anything doubled or fighting (a package twice, a second login manager, an unneeded dependency…), services, the login, dwl (rebuilt if it will not start; work of yours in its folder is set aside as a branch or a stash, never lost), the session, the tools, neovim's plugins, the configs where the theme was chosen — and verify again; offline too, as far as it can	
snapshots	the snapshots of `/` (§7.2)	how many
theme, applied again	every config back as the toolkit has it (yours kept in the backup)	
drift	the configs that differ from the toolkit's; changes nothing	
settings	a copy of `fedokawaii.conf` in neovim; when you close it every value is checked, and only a valid file replaces the toolkit's (a wrong one is shown and nothing changes); the next update applies it	
reminders	Enter changes the season: every 60 · 90 · 120 days · off	the season
log	the last run's log, at its end	
handbook	this book	

Every entry is a command too — `fedokawaii status`, `update`, `upgrade` (`--now`: without waiting), `verify`, `repair`, `theme`, `drift`, `snapshots`, `log`, `settings`, `guide`, `remind [check | every DAYS | wait DAYS | on | off]` — and `update`, `verify` and `repair` work on their own in a terminal. On a text console (Ctrl+Alt+F3), or should fzf ever fail to start, `fedokawaii` shows the same menu as a numbered list: the system can be looked after even when the desktop cannot start.

`fedokawaii status` prints the page of the *status* entry. On Fedora 44 in mid-October 2026, ninety days after an update, it reads something like:

```
release  Fedora 44
support  until 2 Jun 2027 · in 230 days
next     Fedora 45 · due about Oct 2026
updated  17 Jul 2026 · 90 days ago
due      now · every 90 days
verified 212 checks passed · 90 days ago
…        (then the running kernel and the free disk space)
```

Reminders. Two minutes after you log in, and every six hours while you stay, a quiet check reads what is known — your last update, your release's end of support, whether the next release has settled — and shows at most one notice a day, only when something is due. It never goes on the network. The season is in `~/.config/fedokawaii/upkeep.conf`: `EVERY` (days between updates, 90), `WAIT` (days a new release settles before the move is offered, 60), `REMIND` (`yes` or `no`). The file is written the first time you change the season; until then the defaults hold:

```
# ~/.config/fedokawaii/upkeep.conf — the update season (fedokawaii remind; Super+U → reminders)
EVERY=90     # days between updates: a reminder once they have passed (7–365)
WAIT=60       # days a new Fedora release settles before the move to it is offered (0–180)
REMIND=yes   # yes | no
```

```
fedokawaii remind every 120   # an update season of 120 days
fedokawaii remind wait 30     # offer a new release 30 days after it is out
fedokawaii remind check       # what is due, in words, for example:
#   the update season is here: 90 days since the last update (Super+U)
systemctl --user list-timers fedokawaii-remind.timer    # when the quiet check runs next
```

The toolkit. What root runs — every update, repair and move — is the copy setup installed in `/usr/local/lib/fedokawaii`, owned by root, so nothing running as you can change it; the `fedokawaii` command is installed from it. The folder you installed

from can go once setup is done. *settings* edits the toolkit's own `fedokawaii.conf`. Every other change is made in your working copy and applied from there (chapter 15): the copy becomes the toolkit.

Its `fedokawaii.conf` is merged key by key — a value you saved through *settings* stays, unless the copy changed that key since the toolkit was last made from a copy; then the copy's is meant. For example:

Key	Saved through *settings*	In the copy	The toolkit gets
`SSH_SERVER`	`yes`	`no`, as it always was	`yes` — yours stays
`MONITOR_HZ`	`144`	`165`, changed in the copy	`165` — the copy's change
`DWL_COMMIT`	(a pin)	a newer pin	the copy's

The log names every value taken from the copy, and the toolkit's file before is kept as `/var/lib/fedokawaii/fedokawaii.conf.before`. The toolkit holds plain files and folders only: a copy with a symbolic link in it is refused — the toolkit stays as it was, and the run stops there, before anything is applied.

6.2 The season's update

update brings everything up to date in one run, in this order:

Step	What happens	If it fails
your working copy	only when the update is run from it (chapter 15): the copy becomes the toolkit, your saved settings merged in	a copy with a link in it is refused: it stops before anything has changed
checks	10 GB free on `/` and 300 MB on `/boot`, no earlier update waiting for a restart, dnf's own state (packages left twice by an interrupted run are tidied); a clock that is not synchronised is only reported	it stops before anything has changed, and says what to do
snapshot	a numbered snapshot of `/`, *before update DATE*, checked to exist	it stops: nothing is updated without one
packages	one `dnf upgrade --refresh` — Fedora, RPM Fusion, Brave, Claude Code, the COPRs — run as a system service of its own, its output followed in the window and kept in the log: closing the window, or the session ending, cannot cut it off halfway	tried again; if a third-party repository is what fails, Fedora's own updates go ahead without it and the report says so; otherwise it stops with nothing changed
`fedokawaii.conf`	its system values applied again: keyboard, time zone, DNF, SSH, the performance profile	
the manifest	anything missing installed; a package a newer Fedora no longer has is listed, never fatal	
this machine's drivers	the firmware the kernel asked for and did not find, the CPU's microcode, Wi-Fi's and the sound card's parts — whatever this hardware needs (§6.9); device firmware updates are listed, never applied by themselves	each on its own
nothing doubled or fighting	packages left twice removed; what `dnf check` finds brought in line; services that fight the desktop's own switched off; the login made greetd's again; dependencies nothing needs removed; what a stopped run left cleared. Configuration files an update could not merge are listed in `/var/lib/fedokawaii/rpmnew`, never touched	each on its own
the desktop's own parts	dwl's wlroots, the pinned tools and fonts (§6.3), GE-Proton, the file dialog	each on its own: one that fails — GitHub down, say — is reported, the rest goes on, and the next update tries it again
torrra · Prowlarr	libtorrent to its newest under torrra; Prowlarr's newest image, then started and checked	
neovim's plugins	your current versions kept first (`~/.local/state/fedokawaii/lazy-lock/`, the last six), then `Lazy! sync`	
verify	every check, the outcome recorded for the menu	

The dnf transaction itself also gets its pre/post pair of snapshots. At the end the update says whether a new kernel waits for a restart, and notes whether the next Fedora is out. The outcome — complete, *partial* (a part to retry next time), *checks* (verify found something: run *repair*) or *stopped* — shows in the menu, and the run's last line says it too (`── update done (ok) — the next one in 90 days`); the log keeps every line (`fedokawaii log`). Not included: language servers you installed through Mason in neovim (`:Mason`, then `U`).

6.3 Pinned, and why

The few tools Fedora does not package come from their official releases, each **pinned**: an exact version and the SHA-256 of its file. The desktop's configs are written for exactly these — yazi's layout code, the fonts' pixel metrics every alignment is measured in — so an update never moves them, and a download that differs by one byte is refused.

Tool	Version	Held because
yazi	26.9.1	its theme, and the file dialog's layout code, use its internals
satty · presenterm · spotify_player	0.22.0 · 0.16.1 · 0.25.1	their configs are written for these
JetBrains Mono and Departure Mono (Nerd Fonts)	3.5.1	every alignment is measured in their pixels
PixelMplus 12	2013	its only release
torrra	2.5.1	its key wrapper replaces three of its methods; libtorrent under it moves to the newest
GE-Proton	the newest; two kept	games are what it is for, and it changes nothing else

dwl is built from a pinned commit (`DWL_COMMIT`), and so is the file dialog's portal (`TFC_COMMIT`).

A *SHA-256* is a 64-character fingerprint of a file's exact bytes: change one byte and it changes completely. The pins live in `lib/upstream.sh`; the version in place is recorded in your home:

```
cat ~/.local/share/fedokawaii/upstream/yazi     # → v26.9.1
sha256sum yazi-x86_64-unknown-linux-musl.zip    # a download of it must print exactly:
# 9b9c39decccf8cb0ff53a7d637d38f8a79d93bbd0099f4ea9c619ef6bb392f5d  yazi-x86_64-unknown-linux-musl.zip
```

6.4 A new Fedora — upgrade

A new Fedora comes about every six months, in April and October (Fedora 43 on 28 October 2025, Fedora 44 on 28 April 2026); each is supported for about thirteen, until four weeks after the release two later — Fedora 44 until 2 June 2027. The menu shows your release's date from Fedora's own record of it, the line `SUPPORT_END=YYYY-MM-DD` in `/etc/os-release` (`grep SUPPORT_END /etc/os-release` shows it), and offers the move when it is sensible:

1. **Out.** A release is out once Fedora publishes its release tree. An update finds that out, and so does opening the menu — one request to Fedora's own servers, at most once a day.
2. **Settled.** The move is offered `WAIT` days (60) after the release day, once its first fixes are in — sooner when your release ends within 60 days. `fedokawaii upgrade --now` does not wait.
3. **Ready.** When you choose it, the move checks first, and stops with nothing changed unless RPM Fusion already has the new release (ffmpeg and the drivers come from there), there are 20 GB free, and every library dwl needs is in it (one that a tool of the desktop's own needs is reported). A COPR that has not built for the new release yet is left out, its packages kept as they are, until it has.
4. **The move.** Your release fully updated, a snapshot of `/`, everything downloaded — should dnf report conflicts, it lists what it would remove and asks — then a restart into Fedora's own installation: 20–40 minutes with nothing else running. Leave the computer on.
5. **After.** Once, by itself, after the restart: the manifest again, packages left twice tidied, torrra for the new Python, the theme (dwl rebuilt if needed), a new baseline snapshot for the new release, the checks — and the outcome, told once in a notice when you

next log in (nobody is logged in while it runs). Configuration files the new packages could not merge (an `.rpmnew` beside yours) and packages no longer in any repository are listed in the status, for you to look at — never changed by themselves.

If the installation fails, the computer restarts into your old release as it was, and the notice at your next login says so. If the new release starts but the desktop does not, the boot menu still has your previous kernel, and the snapshot taken before the move holds the old system (§7.2).

Underneath, the move is Fedora's own *offline upgrade*: everything is downloaded first, then installed in a special start before anything else runs. `fedokawaii upgrade` puts the checks above around these two commands of dnf's, so you never type them yourself:

```
sudo dnf system-upgrade download --releasever=45   # all of Fedora 45; nothing installed yet
sudo dnf system-upgrade reboot                      # restart into the installation
```

Read more: Upgrading Fedora Linux using DNF.

6.5 Built to be left alone

The toolkit is never edited after it is installed, so whatever changes around it over the years is handled one way — checked for, worked around, or reported, never a silent break:

In time	What happens
Fedora moves to a newer wlroots and drops the one dwl is written for	dwl uses its own copy, kept from Fedora's build at the first install (`/usr/local/lib/fedokawaii-wlroots-0.20`); a move checks the new release has every library under it
dwl cannot start — a library missing	the login starts the build before it (`dwl.prev`); with neither, a rescue screen on the console says what to do, instead of a login that loops back
a new kernel misbehaves	the one before it stays installed, in the boot menu
a package is renamed or retired	listed, never fatal: the rest goes on without it
a third-party repository is down, changes its key, or has nothing yet for a new release	skipped with a warning; Fedora's own updates never wait on it
a release tool's download changes	refused by its checksum; the version in place stays
a run is interrupted — the window closed, the power cut	dnf runs apart from the window; the next run tidies a half-finished transaction; every record is written whole or not at all
the disk fills	every run checks for room first; snapshots stay within bounds (a fifth of the disk stays free)
dnf's clean-up (`autoremove`) would take a package the desktop needs	never: every package in the manifest is marked as yours. Nothing is *protected* by name, so a later Fedora can still replace one (say, sudo by sudo-rs) without holding back every update
a tool's output changes its language or format	setup, verify and the `fedokawaii` command read it in one fixed language (`LC_ALL=C.UTF-8`), whatever yours is
two runs at once	one at a time; the menu shows when one is going on

6.6 Kernels, DNF, snapshots

DNF keeps two kernels: the newest and the one before it (`installonly_limit=2`, the DNF 5 minimum; DNF's default keeps three). Should a new one fail to boot or to start the desktop, choose the older one in the boot menu (if it is hidden, hold **Shift** or press **F8** as the machine starts); the next update replaces it.

```
rpm -q kernel-core    # the kernels installed: at most two lines
uname -r              # the one running now
```

`/etc/dnf/libdnf5.conf.d/90-fedokawaii.conf` holds only what differs from Fedora's defaults — its six settings, each with what it does and DNF's own default:

```
[main]
# ten downloads at once, from DNF_PARALLEL (default 3; 20 at most)
max_parallel_downloads=10
# use the mirror that connects fastest (default off)
fastestmirror=True
# a mirror sending 50 kB/s or less …
minrate=50k
# … for 20 seconds is dropped (defaults: 1000 bytes/s, 30 s)
timeout=20
# keep two kernels (default 3)
installonly_limit=2
# Enter means yes at dnf's question (default no)
defaultyes=True
```

Setup and `verify` check all six with `dnf --dump-main-config`, which prints every setting as DNF sees it once all its files are read:

```
dnf --dump-main-config |
  grep -E '^(defaultyes|fastestmirror|installonly_limit|max_parallel_downloads|minrate|timeout) '
```

```
defaultyes = 1
fastestmirror = 1
installonly_limit = 2
max_parallel_downloads = 10
minrate = 51200
timeout = 20
```

(`minrate` is shown in bytes: 50k = 51 200.) *Read more:* dnf5.conf(5).

snapper keeps timeline snapshots of `/` (and `/home` when it is its own subvolume), a *baseline* taken right after the verified build, a pre/post pair around every DNF transaction, and one before every update and every move. For a machine left alone for months they stay within bounds — far fewer than snapper's own defaults:

Kept	snapper's default	here
numbered: the pre/post pairs and the one before each update	50	4–20
numbered and marked *important*: the one before each move	10	4–10, counted apart from the others
hourly	10	2–8
daily	10	2–7
monthly	10	0–2
yearly	10	none

A range such as `4-20` means: keep twenty, and when free space runs low go down as far as four, so that a fifth of the disk stays free (`FREE_LIMIT=0.2`). No snapshot younger than an hour is cleaned up. The baseline stays until you delete it — or until the next Fedora: a move takes a new baseline for the new release, and the old one then ages out with the numbered ones. `~/Downloads` is a subvolume of its own (when it was still empty at setup; `verify` warns otherwise), so downloads stay out of `/home`'s snapshots: the space of a deleted download comes back at once.

```
sudo snapper -c root get-config    # every limit above, as snapper has it
sudo snapper -c root list          # the snapshots themselves (or: fedokawaii snapshots)
```

Read more: snapper-configs(5).

The machine runs the **tuned** profile `fedokawaii` (`system/tuned.conf`). *tuned* is a service that applies a named set of kernel and hardware settings, a *profile*. This one takes Fedora's *throughput-performance* (the CPU at full speed — this is a desktop that is always plugged in) and changes the memory settings for a desktop whose swap is *zram*, compressed memory rather than a disk, so a large copy never stalls the desktop:

Setting	Kernel default	throughput-performance	here	Why
`vm.swappiness`	60	10	100	swapping to zram is cheap: use it rather than drop file cache
`vm.dirty_ratio`	20 %	40 %	20 %	a big copy is written out long before it fills 40 % of memory
`vm.dirty_background_ratio`	10 %	10 %	10 %	writing starts early, in the background
`vm.page-cluster`	3 (8 pages)	—	0 (1 page)	reading back from zram is a memory copy: no read-ahead
`vm.watermark_boost_factor`	15000	—	0	no extra bursts of memory reclaim, so frames stay even under load

```
tuned-adm active                      # → Current active profile: fedokawaii
sysctl vm.swappiness vm.dirty_ratio   # → vm.swappiness = 100, vm.dirty_ratio = 20
```

Wi-Fi power saving is off as well (`wifi.powersave = 2`, meaning *disable*, in `/etc/NetworkManager/conf.d/90-fedokawaii.conf`), so the connection never naps.

Read more: tuned's throughput-performance profile · the kernel's vm settings.

6.7 What verify proves

Every check fails if its probe fails — a broken tool can never produce a pass. (A *probe* is the small command a check runs: `getenforce` for SELinux, `findmnt` for the file system.) The output is one line per check under each area, and a verdict at the end:

```
system
  ✔ Fedora 44 (44 or newer)
  ✔ SELinux enforcing
  ✔ / is Btrfs
  …
  ! Prowlarr has no indexers yet — open http://localhost:9696 → Indexers → Add Indexer (torrra finds nothing until then)
  …
✔ verified — 212 checks passed, 1 warning(s)
```

✔ passed · ✘ failed (`fedokawaii repair` fixes most) · ! worth knowing, never a failure; a line after a dim dot only reports (which driver opens NTFS, say). The outcome is kept for the menu as one line — when, passed, failed, warnings:

```
cat /var/lib/fedokawaii/last-verify    # → 1792039927 212 0 1
```

Area	Checks
system	Fedora ≥ 44 · SELinux enforcing · Btrfs root · no root-owned files in your home · groups · neovim as the editor · the tuned profile · Wi-Fi power saving off · `secrets.sh` private · linger · ntsync and i2c-dev · the SSH policy and firewall ports (6881 for torrents)
packages and redundancy	every manifest package installed · none of the forbidden ones · nothing user-installed beyond the pre-setup record and the manifest · no Flatpak or Snap · no package installed twice · `dnf check` clean · no service that fights the desktop's own (a second login manager, power-profiles-daemon, a second notification daemon…) · the login greetd's alone · dependencies nothing needs (a warning) · configuration files an update could not merge (a warning) · nothing left behind by a stopped run

Area	Checks
drivers and firmware	every package this hardware needs (microcode, Wi-Fi, sound DSP, missing firmware, Intel's video decoding beside an AMD card; one missing is a warning: update and repair install it) · microcode loaded at boot · every network, display, sound and USB device held by a driver · Wi-Fi managed by NetworkManager · device firmware updates, when fwupd knows of any (shown, never a fault) · udisks answers, so drives open · which driver opens NTFS
repositories and DNF	RPM Fusion, Brave, Claude Code, COPRs · all six DNF values
upkeep	the toolkit root's alone · the `fedokawaii` command the toolkit's · the manifest marked as yours · the snapshots' bounds
boot	the boot-menu setting · at most two kernels, the newest running · no rescue image · the GPU kernel argument
GPU and media	hardware video decoding · Vulkan · ffmpeg encoders
compositor and login	dwl version and pinned commit, XWayland, its own wlroots and every library, the patch (keys, gaps, frames, centring, layout), built from `config.h` · the session launcher and its rescue screen · the greetd SELinux module, agreety · the boot-success timer
services	greetd, Bluetooth, snapper timers, fstrim, PipeWire, Prowlarr · the monthly Btrfs scrub, and what the last one found
torrents	the key in `secrets.sh` · Prowlarr's container file current, running, answering, taking the key · its files yours · indexers (a warning when there are none) · podman's database · `~/Downloads` outside the snapshots
release binaries	present, every library resolves, `--version` runs · fonts
applications	Brave Origin actually starts · Claude Code · LazyVim · torrra loads and runs through its wrapper, with no key in its config · mpv in the bar · yt-dlp's JavaScript runtime · lazygit · qalc
theme	every file in sync · foot and fuzzel accept their configs · waybar config parses · GTK and the portal follow the current theme · no window buttons · default programs · Brave in the desktop's colours · Qt palette · fonts resolve (Arabic, Japanese, Chinese, Korean) · mpv's colours · pixel art drawn for both themes · the two C helpers built · the clipboard window · the wallpaper pack · the pixel pointers · the yazi file dialog · cava · the boot menu theme
live session	bar, notifications, terminal server, wallpaper, idle, automount, status readers, polkit agent, clipboard history and keeper, the update reminder · the graphical session known to systemd · the file-dialog and screen-sharing portals · the "Share" sound devices · the refresh rate is the best available · a monitor answers DDC/CI · night light (shown, never a fault: it is yours to switch off)

6.8 Between seasons

In between, nothing changes underneath you — nothing updates, or even checks for updates, in the background:

Would otherwise	Here
DNF fetches package lists every three hours or so (its *makecache* timer: ten minutes after boot, then three hours after each run, plus up to an hour at random)	off: lists are fetched when you update (`dnf upgrade --refresh`); `systemctl is-enabled dnf5-makecache.timer` says `disabled`
neovim's plugin manager looks for new plugin versions	off (`checker = { enabled = false }` in `lua/config/lazy.lua`); plugins move only with an update, which keeps the old versions first
Prowlarr's image is replaced as soon as a new one appears	fetched by an update only; its next start uses it
tldr refreshes its pages	at most once a quarter, and only when you run it
the release tools and fonts	pinned (§6.3)

Two programs live on the internet's schedule rather than yours, and are worth updating on their own when they ask — each command touches that one program only:

When	Command
Brave reports a security fix you want now	`sudo dnf upgrade --refresh brave-origin`

When	Command
YouTube stops playing in mpv, or yt-dlp says to update	`sudo dnf upgrade --refresh yt-dlp`
security fixes only, between seasons (optional)	`sudo dnf upgrade --refresh --security`

If an update breaks neovim

An update keeps the plugin versions you had in `~/.local/state/fedokawaii/lazy-lock/` (the last six), as *lockfiles*: lists of every plugin and the exact version it was at. To go back:

1. Find the newest — the versions from just before the last update: `ls -t ~/.local/state/fedokawaii/lazy-lock/ | head -n 1`
2. Copy it over neovim's own lockfile, for example `cp ~/.local/state/fedokawaii/lazy-lock/lazy-lock.20261015-0852.json ~/.config/nvim/lazy-lock.json`
3. Open neovim and run `:Lazy restore`: every plugin goes back to the version in the file.

Anything else an update changed, snapper undoes (§7.2): every `dnf` run is wrapped in a pre/post pair, and the update itself starts from a snapshot.

Read more: lazy.nvim, Lockfile.

6.9 Drivers and firmware

The kernel carries the drivers — the code that runs a device. Many devices also need **firmware**: a small program loaded *into* the device (a Wi-Fi card, a GPU, a sound DSP) each time it starts, and **microcode**, the CPU's own corrections, loaded at every boot. Fedora ships firmware in separate packages, and a minimal install does not bring them all: Intel's Wi-Fi and Bluetooth firmware, for one, comes only when asked for. So setup, every update and every repair look at the hardware itself and install what it needs:

Looks at	Finds	Installs
the kernel's own log, this boot	firmware a driver asked for and did not find: `Direct firmware load for iwlwifi-so-a0-gf-a0-89.ucode failed with error -2`	the package that provides that file (one of Fedora's `iwlwifi-…-firmware` packages, here)
`/proc/cpuinfo`	the CPU's maker: `GenuineIntel` or `AuthenticAMD`	its microcode (`microcode_ctl` or `amd-ucode-firmware`), then the boot images again, so the next start loads it early
`/sys/class/ieee80211`	a Wi-Fi radio	NetworkManager's Wi-Fi part (without it the card is not managed at all) and the regulatory database
the sound card's driver	Intel's audio DSP (`sof-audio-pci-intel-…`)	its firmware, `alsa-sof-firmware`
the GPUs	an Intel GPU beside the AMD card the desktop drives	Intel's video decoding too (`intel-media-driver`)

What it installs is recorded in `/var/lib/fedokawaii/hardware-pkgs` and belongs to the manifest from then on: `verify` counts it as the desktop's, and dnf's clean-up never takes it. A file the kernel asked for that no package has is not a fault — a driver tries the newest firmware first, then loads one it has. A device no driver holds at all is reported (`verify`, *drivers and firmware*): that is a kernel matter, and that device will not work here.

Firmware inside the devices — the SSD's, the motherboard's, a dock's — comes from the device makers through the LVFS, with **fwupd**. The update fetches the list and says what is new; installing it is yours to decide, because some updates restart the machine:

```
fwupdmgr get-devices      # every device fwupd can update, with its firmware version
fwupdmgr get-updates      # what is new for them (the update has fetched the list)
fwupdmgr update           # install them; it says first which ones need a restart
```

To see what holds a device:

```
lspci -nnk | rg -A3 -i 'vga|audio|network'   # PCI devices: name, IDs, "Kernel driver in use"
lsusb                                       # USB devices: the dongle, the keyboard…
journalctl -k -b -g 'firmware'              # what the kernel said about firmware this boot
```

Read more: Fedora's firmware packages · fwupd · LVFS.

7. Keeping it clean

Chapter 6 keeps the machine working. This one keeps it tidy: where files belong, how the snapshots work and how to use them, how to keep the package list short, and how to find and free disk space. Nothing here runs by itself unless it says so.

Terms used in this chapter

Term	In plain words
Btrfs	the file system this desktop runs on (setup requires it): it checksums what it stores and can take snapshots
subvolume	a folder that Btrfs treats as a tree of its own: it is snapshotted, or left out of a snapshot, as a whole
snapshot	a copy of a subvolume as it was at one moment; snapper's are read-only
copy-on-write	Btrfs writes a change to a new place instead of over the old data, so a snapshot can keep the old blocks — and a fresh snapshot takes almost no space
dependency	a package installed because another package needs it, not because anyone asked for it
user-installed	a package asked for by name: by you, or by setup from the manifest
autoremove	`dnf autoremove`: removes the dependencies that no user-installed package needs any more
journal	systemd's log of the system and of your own services, read with `journalctl`
cache	files a program can make again: deleting them costs time, not work (a few programs keep more there: §7.4)
XDG base directories	the freedesktop.org rule for where programs keep settings, data, state and cache (§7.1)
drop-in folder	a `NAME.d/` folder beside a config file: each file in it adds or overrides settings, and the main file stays untouched (§10.3)

7.1 Where things go

Each kind of file has one place. Knowing which places belong to the scripts, which to you, and which can be thrown away is most of keeping a desktop clean.

Most of your home follows one rule, the XDG Base Directory specification: a program splits its files into settings, data, state and cache, and each kind has its own folder. No `XDG_*` variable is set on this desktop, so every program uses the default folders:

Kind	Folder (variable)	For example
settings	`~/.config` (`XDG_CONFIG_HOME`)	`~/.config/foot/foot.ini`
data: what you would miss	`~/.local/share` (`XDG_DATA_HOME`)	`~/.local/share/zathura/bookmarks.sqlite`: the page you reached in each book
state: worth keeping, not precious	`~/.local/state` (`XDG_STATE_HOME`)	`~/.local/state/mpv/`: where each video stopped
cache: can be made again	`~/.cache` (`XDG_CACHE_HOME`)	`~/.cache/kawa-wallpaper/`: the scaled wallpapers
your own programs	`~/.local/bin`	the `kawa-*` helpers

Read more: XDG Base Directory Specification.

Place	Holds	Who writes it	Safe to delete?
`/etc`	system-wide settings: DNF (`/etc/dnf/libdnf5.conf.d/90-fedokawaii.conf`, the snapshot hook in `/etc/dnf/libdnf5-plugins/actions.d/`), the tuned profile, NetworkManager, `/etc/dconf`, `/etc/default/grub`, snapper's configs	Setup and Theme (as root), and Fedora's packages	no. Change Fedokawaii's own files through `fedokawaii.conf` and the toolkit (§2); anything else with `sudoedit FILE`, as a drop-in where the program reads one (§10.3)

Place	Holds	Who writes it	Safe to delete?
`~/.config`	every program's settings	Theme owns the files listed at the start of chapter 11; the rest are yours	no — but a managed file edited here is replaced at the next Theme run
`~/.config/fedokawaii/`	`env`, `themes/`, `current`, `icons.sh`, `secrets.sh`	Theme (`env` is written from `fedokawaii.conf`); `secrets.sh` is yours	no
`~/.local/bin`	`kawa-*` helpers, `fetch`, `brave-origin`, `kawa-veil`, `kawa-clipkeep`, the release binaries (`yazi`, `ya`, `satty`, `presenterm`, `spotify_player`), `torrra`	Theme, `update` and pipx	no. Your own scripts may live here too, under names the desktop does not use there
`~/.local/share`	program data: Steam and its games, pipx's torrra, podman's images, the trash, neovim's plugins and Mason tools, the pixel art and the wallpaper pack	the programs themselves	no, not as a whole; see §7.3 and §7.4 for the parts you can trim
`~/.local/state/fedokawaii/`	theme stamps, each theme's wallpaper choice, `backup/<time>/`, `lazy-lock/` (neovim's plugin versions before each update)	Theme, `update` and the session	the old folders in `backup/` — the theme itself keeps only its newest 20
`~/.cache`	logs, previews, the scaled wallpapers, the clipboard history, launcher counts	the programs	yes, piece by piece; every program rebuilds its part (§7.4)
`~/Downloads`	what Brave, torrra and yt-dlp fetch	the programs	yes — and its space comes back at once: it is its own Btrfs subvolume, outside every snapshot
`~/Pictures/Screenshots`	Print, Shift+Print, satty's Ctrl+S, mpv's `s`	the screenshot tools	yes, whenever you like
`~/Pictures/Wallpapers`	your own wallpapers, shown on both themes	you	yes
`~/Books`	your library: what the books window (Super+R) lists	you (and its Ctrl+A)	yes, book by book — Ctrl+D in the window sends one to the trash
`~/Notes/notes.md`	your notes (Super+N)	you	only if you mean it: it is your writing

The one rule for configuration

A file the theme manages is changed in your working copy and applied (chapter 15) — never only where the program reads it. Trying a change in place first is fine; to make it last:

1. Edit the file where the program reads it, for example `~/.config/foot/foot.ini`, and try it.
2. `fedokawaii keep ~/.config/foot/foot.ini` copies it into your working copy (`fedokawaii copy` makes the copy, once). For a file that is drawn, or filled in from `fedokawaii.conf`, it names the source to change instead.
3. `fedokawaii apply` makes the copy the toolkit's, so every later theme run keeps your version.

A file changed only in place is listed by `fedokawaii drift`, and the next theme run puts the toolkit's back (yours is kept in `~/.local/state/fedokawaii/backup/<time>/`). Files the theme never writes are yours to edit where they are (§15.1).

Your own files

A short, fixed layout makes them easy to find, to back up and to clean:

Folder	For	Why
`~/repos/`	every git repository, one folder each	the prompt shows the branch and changes there; lazygit and neovim find the project from the folder. The Fedokawaii clone can stay at `~/fedokawaii` (§3.2)
`~/Documents/`	papers, letters, spreadsheets, PDFs you keep	one place for everything you would hate to lose
`~/Pictures/` · `~/Videos/` · `~/Music/`	what you keep of each kind	the standard folders; file dialogs and yazi know them
`~/Downloads/`	arrivals only	not in snapshots — move what you keep into one of the folders above, delete the rest
`~/Books/`	books and comics, in folders of your own if you like	the books window reads it (§4.11); its Ctrl+A moves books here from `~/Downloads`, `~/Documents` and `~/Desktop`
`~/Notes/`	`notes.md` (Super+N), and any other notes you start	plain Markdown: searchable with `rg`, readable anywhere (§4.12)

Snapshots are not backups: they live on the same disk. Copy the folders in the second table to another disk or machine now and then.

7.2 Snapshots

snapper does here what Time Machine or Timeshift do elsewhere: it keeps earlier states of the system so a change can be undone. Setup made two configurations, both Btrfs snapshots:

Config	Covers	Snapshots kept in
`root`	`/` — the system, `/etc`, `/usr`, `/var`	`/.snapshots/`
`home`	`/home` (only when it is its own Btrfs subvolume, as the installer makes it) — **except** `~/Downloads`	`/home/.snapshots/`

What makes a snapshot

Trigger	Config	Cleaned up by
every DNF transaction: a *pre* before and a *post* after, described by the command that ran (`dnf install htop`; the date when the command cannot be read). dnf5's actions plugin runs `/usr/local/sbin/kawa-snap` from `/etc/dnf/libdnf5-plugins/actions.d/fedokawaii-snapper.actions`	`root`	the *number* rule
every hour (`snapper-timeline.timer`)	`root` and `home`	the *timeline* rule
before every update: a single snapshot, *before update YYYY-MM-DD*, marked important; `fedokawaii update` checks that it exists before it changes anything	`root`	the *number* rule, counted among the important ones — so a busy month of dnf runs never pushes it out
before every move to a new Fedora: *before Fedora N*, marked important	`root`	the *number* rule, counted among the important ones
the *fedokawaii baseline*, taken once after the verified build — and again for each new Fedora	`root`	none — kept until you delete it, or until the next release's baseline replaces it (then the *number* rule)
`sudo snapper -c CONFIG create -d "…"`	either	none — kept until you delete it

`snapper-cleanup.timer` applies the rules once a day, and never removes a snapshot younger than an hour (snapper's `NUMBER_MIN_AGE` and `TIMELINE_MIN_AGE`, 3600 seconds). Setup gives both configs the same limits (the bounds in §6.6). Each is

a range, *min-max*: the cleanup keeps up to the *max*, and goes down towards the *min* only while less than a fifth of the disk is free (`FREE_LIMIT=0.2`).

Setting	Counts	Keeps
`NUMBER_LIMIT=4-20`	DNF pairs, *before update*, old baselines	the newest 20; down to 4 when space is short
`NUMBER_LIMIT_IMPORTANT=4-10`	*before Fedora N* (counted apart from the others)	the newest 10; down to 4
`TIMELINE_LIMIT_HOURLY=2-8`	the first snapshot of each hour	the last 8; down to 2
`TIMELINE_LIMIT_DAILY=2-7`	the first of each day	a week; down to 2
`TIMELINE_LIMIT_MONTHLY=0-2`	the first of each month	two; down to none
`TIMELINE_LIMIT_WEEKLY=0` · `TIMELINE_LIMIT_YEARLY=0`	—	none

Read them with `sudo snapper -c root get-config`. Change one with, for example:

```
sudo snapper -c root set-config NUMBER_LIMIT=4-30     # a range, or low space no longer trims it
sudo snapper -c home set-config TIMELINE_LIMIT_DAILY=2-5 TIMELINE_LIMIT_MONTHLY=0-1
sudo snapper -c home set-config TIMELINE_CREATE=no    # no hourly snapshots of /home at all
```

`fedokawaii repair` sets the limits back to setup's (`snapper_limits` in `setup.sh`); to keep yours, change them there, in your working copy, and apply. It leaves `TIMELINE_CREATE` as you set it.

The baseline has no cleanup rule, so the automatic cleanup never removes it: it stays as the known-good state of the freshly built system until you delete it yourself, or until a move to the next Fedora takes a new one (*fedokawaii baseline (Fedora N)*) and hands the old one to the numbered cleanup. To keep any other snapshot for good the same way: `sudo snapper -c root modify --cleanup-algorithm "" NUMBER` (the number from `snaps`).

Looking and comparing

Command	Does
`snaps`	the list for `/` (an alias for `sudo snapper -c root list`)
`sudo snapper -c root list --type pre-post`	only the DNF pairs, one line each
`sudo snapper -c home list`	the list for `/home`
`sudo snapper -c root status 41..42`	every file changed between two snapshots
`sudo snapper -c root diff 41..42 /etc/some.conf`	the change inside one file
`sudo snapper -c root status 41..0`	everything changed since 41 — `0` is the system as it is now

What `snaps` shows, shortened to the columns that matter here:

```
 # | Type   | Pre # | Cleanup  | Description
---+--------+-------+----------+-------------------------
 0 | single |       |          | current
 1 | single |       |          | fedokawaii baseline
40 | single |       | number   | before update 2026-10-15
41 | pre    |       | number   | dnf install hyperfine
42 | post   |    41 | number   | dnf install hyperfine
57 | single |       | timeline | timeline
```

In `status`, each line starts with six characters: `+` created, `-` deleted, `c` content or `t` type changed, then `p`, `u`, `g`, `x` and `a` for permissions, owner, group, extended attributes and ACL; `.` means unchanged. For a pair around a transaction that installed `htop` (say 45 and 46), shortened:

```
$ sudo snapper -c root status 45..46
+..... /usr/bin/htop
c..... /usr/lib/sysimage/rpm/rpmdb.sqlite
```

Undoing

You want	Do
undo one DNF transaction	`sudo snapper -c root undochange 41..42`, reboot, `verify` (§13.9)
one system file as it was	`sudo snapper -c root undochange 41..42 /etc/some.conf`
one of your files as it was	copy it out of the snapshot: `sudo ls /home/.snapshots/57/snapshot/$USER/` to find it, then `sudo cp -a /home/.snapshots/57/snapshot/$USER/Documents/letter.odt ~/Documents/` (`-a` keeps you as its owner)
a file deleted by mistake	the same, from the newest hourly snapshot that still has it
the whole system back to an earlier state	`sudo snapper -c root undochange 41..0` — every file changed since 41 goes back. There is no one-step rollback on this layout (§11.21): the boot entries name the `root` subvolume, so `snapper rollback` would boot the same system again

A whole-system undo is the same tool as undoing one transaction, only wider: it puts files back and runs no package scripts, and `/boot` (the kernels) is not in the snapshots. Reboot and run `verify` afterwards.

Before a risky change — a new repository, a driver, a big edit in `/etc` — take your own:

```
sudo snapper -c root create -p -d "before new repo"   # -p prints its number, say 58
sudo snapper -c root status 58..0                    # later: everything changed since
sudo snapper -c root undochange 58..0                # if it went wrong: all of it back, then reboot
```

A snapshot made this way has no cleanup rule: delete it yourself once the change has proved itself (§7.5).

Deleting

`sudo snapper -c root delete 30-40` (a range) or `delete 31 35`; the same with `-c home`. Delete a *pre* together with its *post*. Btrfs frees the space in the background; `delete --sync` waits until it is free. Snapshots hold every file that was there when they were taken, so deleting a large file frees nothing until the last snapshot holding it is gone:

Step	Space used
you save a 4 GB video in `~/Videos`	4 GB
the hourly snapshot of `/home` is taken	still 4 GB: the snapshot shares the same blocks
you delete the video	still 4 GB: the snapshot holds it
the last snapshot holding it is cleaned up — up to two months later if a monthly one caught it, sooner when space is short	4 GB free again

Not in any snapshot:

- `~/Downloads` — its own subvolume (setup made it while the folder was empty; `verify` warns if it is not). A Btrfs snapshot never includes a subvolume nested inside it.
- `/boot` and `/boot/efi` — separate partitions; the previous kernel is the safety net (§6.6).
- everything else in your home **is** in the `home` snapshots: `~/.cache`, Steam's games, podman's images. Deleting them frees their space only as the snapshots holding them expire.

To keep another big folder out the way `~/Downloads` is, make it a subvolume while it is still empty — as yourself, with no `sudo`, the way setup made `~/Downloads`:

```
btrfs subvolume create ~/Games   # a new, empty subvolume, owned by you
restorecon ~/Games               # its SELinux label, as for any folder in your home
stat -c %i ~/Games               # prints 256: the top folder of every subvolume has inode 256
```

To remove such a subvolume later, empty it and `rmdir` it, as with a plain folder.

Read more: snapper(8) · snapper-configs(5) · Btrfs subvolumes · Working with Btrfs – Snapshots (Fedora Magazine).

7.3 Keeping packages lean

Every package you want lives in `lib/manifest.sh`, and `verify` fails on anything user-installed beyond the manifest and what was there before setup. That rule keeps the list honest; these keep the disk lean.

Command	Does
`dnf repoquery --userinstalled`	packages asked for by name, or brought by a group such as the installer's; the rest came in as dependencies. `verify` compares this list with the manifest and its record from before setup
`dnf repoquery --unneeded`	what `sudo dnf autoremove` would remove: a preview that changes nothing
`sudo dnf autoremove`	removes dependencies nothing needs any more. `update` and `repair` run it for you (in a snapshotted dnf run, the list in the log); by hand, read the list before saying yes. Manifest packages are user-installed, so it never touches them
`dnf repoquery --installed --whatrequires NAME`	which installed packages require NAME
`rpm -qf FILE`	which package a file belongs to, for example `rpm -qf /usr/bin/foot`
`sudo dnf clean all`	empties DNF's download and metadata caches in `/var/cache/libdnf5` (they come back at the next upgrade). `dnf` run without `sudo` keeps its own copy in `~/.cache/libdnf5`; `dnf clean all` without `sudo` empties that
`rpm -q kernel-core`	two kernels at most: the newest, and the one before it; `uname -r` names the one running

Adding a package properly

A package you keep goes into the manifest, in your working copy: its name in `PKG_APPS`, a row in `PACKAGES.md`, then *apply* — apply runs repair, which installs it (§13.6 has the steps). An `update` alone does not read your working copy. A one-off — something to try for an afternoon — can be `sudo dnf install NAME`; `verify` will list it as *installed outside the manifest* until you `sudo dnf remove NAME` again, which also removes the dependencies it brought (DNF's `clean_requirements_on_remove`, on by default). Never leave it there to be forgotten.

Taking one out

In your working copy, move it from its list to `RETIRED_PKGS` in `lib/manifest.sh`, then *apply*. Repair (and every update after it) marks it as a dependency (`dnf mark dependency`) rather than removing it — unless it was already user-installed before setup:

	Before	After apply
listed by `dnf repoquery --userinstalled`	yes	no
counted by `verify`	as a manifest package	not at all
`sudo dnf autoremove` (the next update or repair runs it)	leaves it	removes it once nothing needs it

Read more: dnf5 autoremove · repoquery · mark · clean · dnf5.conf.

Kernels

Two at most: the newest and the one before it, kept as the way back should a new one misbehave (§6.6). DNF removes the oldest by itself when a third arrives — never the one running — and `dnf autoremove` never removes a kernel; there is nothing to clean by hand:

```
rpm -q kernel-core   # two lines, one per kernel
uname -r             # the one running: normally the newest
```

The rest of what this desktop installs

What	Where	Clean with
torrra (pipx)	`~/.local/share/pipx/venvs/torrra`	nothing to do: torrra stays at its version, and `update` moves libtorrent under it to the newest. `pipx list` shows it; do not `pipx uninstall` it — `verify` checks it
Prowlarr's image	podman's storage in `~/.local/share/containers`	`podman system df` shows the size; `podman image prune` removes *dangling* images — older versions left without a name when a newer one was pulled (`update` already runs it). Not `podman image prune -a`: Prowlarr's container exists only while it runs, so that would delete its current image too
GE-Proton	`~/.local/share/Steam/compatibilitytools.d`	nothing: `update` keeps the newest two
games	`~/.local/share/Steam/steamapps`	Steam's *Settings → Storage*: uninstall games there. Each game's Proton prefix (`steamapps/compatdata`) and shader cache (`steamapps/shadercache`) can grow to gigabytes; a folder there for a game you no longer have can go
neovim's tools	Mason, in `~/.local/share/nvim/mason`	`:Mason`, `X` on a tool you no longer use

7.4 Disk space and logs

How full is it? On Btrfs, `df` and `du` cannot tell what the snapshots hold: `du` counts a file in full even when a snapshot shares its blocks, so neither says what deleting something would free. Ask Btrfs:

Command	Shows
`sudo btrfs filesystem usage /`	the real picture: *Free (estimated)*, and *Device unallocated*
`sudo btrfs filesystem du -s ~/Downloads ~/.local/share/Steam`	how much of a folder is its own and how much is shared with snapshots
`du -xh --max-depth=1 ~ | sort -h | tail`	the biggest folders in your home (`-x` stays on one subvolume, so it skips `~/Downloads`; run it there separately)
`fd -HI -S +1g . ~`	single files of 1 GB or more (`-H`: hidden ones too; `-I`: also those a `.gitignore` hides, such as build folders in `~/repos`)

What the first two print, shortened:

```
$ sudo btrfs filesystem usage /
Overall:
    Device size:                 931.51GiB
    Device allocated:            212.02GiB
    Device unallocated:          719.49GiB
    Free (estimated):            741.22GiB      (min: 381.47GiB)
$ sudo btrfs filesystem du -s ~/Downloads ~/.local/share/Steam
     Total   Exclusive  Set shared  Filename
  38.40GiB    38.40GiB       0.00B  /home/you/Downloads
  61.20GiB     2.10GiB    59.10GiB  /home/you/.local/share/Steam
```

Free (estimated) is roughly what you can still write. *Device unallocated* is space Btrfs has not yet handed to data or metadata (see *balance* below). *Exclusive* is what deleting the folder would free at once: all of `~/Downloads`, which no snapshot holds, but little of Steam's — the rest is shared with the `home` snapshots and comes back only as they expire.

`compsize`, `gdu` and `ncdu` are not in the manifest; `du`, `btrfs filesystem du` and `fd` do the job.

The journal

The journal is kept on disk in `/var/log/journal`. It stays within systemd's default limits — at most 10 % of the file system, while leaving 15 % of it free, each capped at 4 GB; nothing here changes them.

Command	Does
`journalctl --disk-usage`	how much the journal uses: `Archived and active journals take up 1.1G in the file system.`
`sudo journalctl --rotate --vacuum-time=4weeks`	close the files in use, then delete entries older than four weeks
`sudo journalctl --rotate --vacuum-size=500M`	the same, down to 500 MB

Vacuuming deletes only archived (closed) journal files; `--rotate` closes the active ones first, so their old entries can go too.

For a lasting limit, write a drop-in of your own:

```
sudo mkdir -p /etc/systemd/journald.conf.d        # the folder may not exist yet
sudoedit /etc/systemd/journald.conf.d/50-size.conf
```

```
[Journal]
# the most the journal may use on disk
SystemMaxUse=500M
```

`sudo systemctl restart systemd-journald` applies it; `journalctl --disk-usage` shows the result.

Read more: journald.conf(5) · journalctl(1).

Caches you can clear

What	Clear with	Comes back
scaled wallpapers and picker previews	`rm -r ~/.cache/kawa-wallpaper`	at the next wallpaper change, or `kawa-wallpaper --restore`
thumbnails other programs share (the freedesktop.org folder; yazi keeps its own previews under `/tmp`, emptied at every restart)	`rm -r ~/.cache/thumbnails`	when a program needs them again
the clipboard history	Super+V, then Ctrl+X (or `cliphist wipe`)	from your next copy
the launcher's counts (a single file)	`rm ~/.cache/fuzzel`	the order of programs starts afresh
book covers and page counts	`rm -r ~/.cache/kawa-books`	as the books window (Super+R) draws them again
old config backups	`rm -r ~/.local/state/fedokawaii/backup/<time>`	— keep the newest until you are sure
the trash	**g t** in yazi → *Empty trash bin*, or `gio trash --empty` (`gio trash --list` shows what is there first)	—
DNF's caches	`sudo dnf clean all`	at the next upgrade

Clear single folders while you are logged in; the logs (`dwl.log`, `waybar.log`) are open by running programs and are best left alone.

Btrfs care

TRIM tells the SSD which blocks are free, so it can keep writing at full speed. Btrfs does it as it goes (`discard=async`, its default on SSDs since Linux 6.2), and `fstrim.timer` catches up once a week. Two more tools exist; neither is needed often:

- **scrub** reads every block and checks it against its checksum, to find a failing disk early. The Btrfs documentation suggests once a month, and a timer does exactly that (`fedokawaii-scrub.timer`, at idle disk priority); `verify` reports what the last one found. By hand: `sudo btrfs scrub start /`, then `sudo btrfs scrub status /`. On one disk it can repair only what Btrfs keeps two copies of — normally its metadata (`sudo btrfs filesystem df /` shows *DUP* there). Damaged file data it can only report; the copy to restore is your backup.
- **balance** rewrites block groups — the chunks Btrfs hands out to data and metadata — so the free space inside half-empty ones becomes *unallocated* again. It is almost never needed: only when `btrfs filesystem usage /` shows *Device unallocated* close to zero while data is far from full. Then a gentle one: `sudo btrfs balance start -dusage=50 /` (only data block groups less than half full). Never run a full balance as routine: it rewrites everything.

Read more: btrfs filesystem · btrfs scrub · btrfs balance · mount options.

7.5 A seasonal routine

Once a season, with the update (§6.2):

Step	Command
update everything	**Super+U** → update (or `update` in a terminal) — it ends with `verify` (§6.2)
check the result	the status in the same menu: every check passed, nothing listed to look at
once a year	**Super+U** → upgrade, once the menu offers the next Fedora (§6.4)
drop unneeded dependencies	nothing to do: the update does it (`dnf repoquery --unneeded` shows what is left)
look at the space	`sudo btrfs filesystem usage /`
sort the downloads	move keepers out of `~/Downloads`, delete the rest; same for `~/Pictures/Screenshots`
empty the trash	**g t** in yazi → *Empty trash bin*, or `gio trash --empty`
prune snapshots you made by hand	`snaps`, then `sudo snapper -c root delete N`
trim the journal	`sudo journalctl --rotate --vacuum-time=4weeks`
check the disk	nothing to do: a scrub runs every month; `sudo btrfs scrub status /` shows the last one
back up	copy `~/repos`, `~/Documents` and your other folders to another disk

8. Software, and why each piece

The full table is `PACKAGES.md`; these are the choices worth knowing.

Job	Tool	Why this one
compositor	dwl 0.9	small and fast; patched for gaps, retro frames, centred dialogs, Arabic-proof shortcuts, the layout in the bar
bar	waybar	its own clock; custom modules fed by one status reader
launcher and every menu	fuzzel	one look for all menus; no second menu program
terminal	foot	fast, live dark/light palettes, prompt integration, pictures (Sixel)
login	greetd + agreety	a plain text login, no extra greeter
lock / idle	swaylock · swayidle · wlopm	the padlock is a picture, the ring sits in its keyhole; wlopm powers the screens off without moving windows
transitions	`kawa-veil` (this repository, C)	nothing packaged does animated changes on dwl; built locally
clipboard	wl-clipboard · cliphist · fzf · `kawa-clipkeep` (this repository, C)	history with a searchable window and picture previews (ImageMagick, as sixels); copies that outlive their program — the one clipboard keeper for wlroots is not packaged for Fedora, and a small one is easy to prove
notifications	mako	
drives	udiskie (udisks) · `kawa-drives` (this repository)	udiskie only mounts; the notices, the bar's mark, the menu, yazi's **M** and an eject that says *safe to unplug* are one small script, so a GUI file manager and its daemons are not needed
music meter	cava	the only extra process for the bar, asleep in silence
brightness	ddcutil	the monitor's real backlight
fetch	`fetch` (this repository)	replaces fastfetch: a few milliseconds, pictures drawn here
torrents	torrra + Prowlarr	a terminal client that searches through Prowlarr's hundreds of indexers
video	mpv (+ mpv-mpris, yt-dlp, Node.js)	plays nearly any file, and web links through yt-dlp; Node.js answers YouTube's JavaScript challenges for yt-dlp

Removed as redundant across versions: tuigreet (agreety is part of greetd), wlogout (a fuzzel menu), fastfetch (own `fetch`), a separate clock script (waybar's clock), starship (native prompt), GStreamer plugins (nothing uses them), and the Rust toolchain (release binaries instead).

9. Troubleshooting

Symptom	Look at
setup stopped	the red line on screen and `/var/log/fedokawaii.log`; fix; run it again
something feels off	`verify` — it names the exact piece
the login loops back	`~/.cache/dwl.log`; `sudo journalctl -b -u greetd`
a rescue screen instead of the desktop	dwl could not start (it says why): `fedokawaii repair` there, then log out (Ctrl+D) and in again
an update stopped, or did only part	the menu's status line (Super+U) and `fedokawaii log`; most causes go with `fedokawaii repair`, then update again
the move to a new Fedora was refused	it says why (not settled, RPM Fusion not ready, a library missing, too little room) — nothing changed; the release you have stays supported until the date in the status
the bar shows no workspaces	`~/.cache/waybar.log`; `pgrep -f kawa-dwlstatus`
the bar, the terminal server or the clipboard keeper vanished	they start again by themselves within a second; if one keeps failing, a notice says so once — `~/.cache/dwl.log`. mako is started again by the next notification
a theme or wallpaper change has no transition	`ls ~/.local/bin/kawa-veil`; re-run **Theme** (it builds it; the build log is in the theme log)
a copy is gone after closing its program	`pgrep kawa-clipkeep`; re-run **Theme**; Super+V still has it
an upload button does nothing	`verify` (the portal checks); then restart Brave — it looks for the file-dialog portal when it starts
a USB drive does not open	`lsblk -f` (is a file system there?); `pgrep -a udiskie`; open it by hand: `udisksctl mount -b /dev/sdb1` says why not; an NTFS disk from Windows: §4.13, *Pitfalls*
"is in use" when ejecting	the notice names the program: close it (a terminal standing in the drive: `cd` out), then eject again
no music meter	`cava -v`; the meter appears only while an MPRIS player plays
brightness does nothing	switch **DDC/CI on** in the monitor's own menu, then try Super+]
Brave not in the desktop's colours	it was open while Theme ran: close Brave, run **Theme** (or Settings → Appearance → Theme → GTK)
torrra says Prowlarr has no indexers	type `prowlarr` in a terminal (it starts Prowlarr and opens its page) → Indexers → Add Indexer (§12)
the boot menu has the wrong size	re-run **Theme** after changing monitors
a wallpaper-pack picture is missing	`kawa-wallpaper --pack` when online
SELinux denial	`sudo ausearch -m avc -ts recent`; never disable SELinux
a config change didn't apply	`fedokawaii drift`; foot fonts and dwl changes apply at the next login
want an old config back	`~/.local/state/fedokawaii/backup/<time>/`

Known issue	Status
foot does not reorder right-to-left text	Arabic is complete in GTK/Qt programs and Brave; in the terminal the letters show but are not reordered — a foot limitation
Steam's interface	not themeable
Brave's own pages	settings pages, the Shields panel, about:blank and error pages keep Brave's own greys; private windows keep its private colours
sharing a screen with system sound	carries the call's own sound too (an echo for the others); use a tab's sound or the Share output instead (§4.10)

Reading the logs. Most rows in the first table end in a log; these commands read them:

Log	Read it with
setup, update, repair, moves	`fedokawaii log` — `/var/log/fedokawaii.log`, opened at its end
the session: dwl and what it starts	`tail -n 40 ~/.cache/dwl.log`
one system service, since the last start	`sudo journalctl -b -u greetd`
one of your own services	`journalctl --user -u prowlarr -n 50` (the last 50 lines)
every error since the last start	`journalctl -b -p err`

`ausearch -ts recent` in the SELinux row means the last ten minutes.

Read more: Troubleshooting SELinux (Fedora Quick Docs) · journalctl(1).

10. Where everything lives

Every path this desktop uses, in three tables: your own files, each program's files, and what the scripts put on the system. Settings that Theme manages are replaced at its next run if edited in place (§7.1); data and caches belong to the programs. The columns of §10.2 follow the XDG folders (§7.1): settings in `~/.config`, data in `~/.local/share`, state in `~/.local/state`, cache in `~/.cache`.

10.1 Your files

Path	What	Notes
`~/Documents/` · `~/Pictures/` · `~/Videos/` · `~/Music/`	what you keep	the standard folders: file dialogs and yazi know them
`~/Downloads/`	arrivals: Brave, torrra and yt-dlp save here	its own Btrfs subvolume, outside every snapshot (§7.2)
`~/Books/`	your library — the books window lists it (Super+R, §4.11)	folders inside are fine
`~/Notes/notes.md`	your notes (Super+N, §4.12)	plain Markdown
`~/Pictures/Screenshots/`	Print, Shift+Print, satty's Ctrl+S, mpv's `s`	
`~/Pictures/Wallpapers/`	your own wallpapers, on both themes	
`~/repos/`	your git repositories	
`~/.config/fedokawaii/secrets.sh`	API keys, as `export NAME=value` (and Prowlarr's, deliberately without `export`)	yours, mode 600; nothing else holds a key

The standard folders were made at setup by `xdg-user-dirs-update`, which records them in `~/.config/user-dirs.dirs`; programs ask for them by name:

```
$ xdg-user-dir DOWNLOAD
/home/you/Downloads
```

10.2 Each program's files

Program	Settings	Data	Cache, state, logs
the desktop (dwl, bar, menus)	`~/.config/fedokawaii/` (`env`, `themes/`, `current`, `icons.sh`, `keybinds.txt`, `cava.conf`, `fzf.opts` — the look of every list) · `~/.config/{waybar,mako,fuzzel,dwl}/` · dwl's `~/.local/src/dwl/config.h`	`~/.local/share/fedokawaii/` (pixel art, `wallpapers.list`, the pack in `wallpapers/`, `upstream/`)	`~/.cache/dwl.log`, `waybar.log`, `fedokawaii-theme.log`, `kawa-wallpaper/` · `~/.local/state/fedokawaii/` (each theme's wallpaper, stamps, `backup/`, `lazy-lock/`)
upkeep (Super+U)	`~/.config/fedokawaii/upkeep.conf` (the season) · `~/.config/systemd/user/fedokawaii-remind.{timer,service}` (the reminder)	—	`~/.cache/fedokawaii/` (when the next Fedora came out, the day of the last reminder)
foot and bash	`~/.config/foot/foot.ini` · `~/.bashrc.d/fedokawaii.sh` · `~/.inputrc`	`~/.bash_history` · zoxide's folders: `~/.local/share/zoxide/`	`~/.cache/fedokawaii/` (fzf's and zoxide's shell code)
the clipboard	—	`~/.cache/cliphist/db` (the history)	
books and notes	—	`~/Books/` · `~/Notes/`	`~/.cache/kawa-books/` (covers, page counts)

Program	Settings	Data	Cache, state, logs
neovim	`~/.config/nvim/` (`lua/config/lazy.lua`, `lua/plugins/colorscheme.lua` from Theme; the rest is LazyVim's starter, yours)	`~/.local/share/nvim/` (plugins in `lazy/`, Mason's tools in `mason/`) · `~/.config/nvim/lazy-lock.json` (plugin versions)	`~/.local/state/nvim/` (undo history, swap files, logs)
lazygit	`~/.config/lazygit/config.yml`	—	`~/.local/state/lazygit/`
yazi (files and dialogs)	`~/.config/yazi/theme.toml`, `init.lua`, `keymap.toml`, `plugins/drives.yazi/`, `plugins/smart-enter.yazi/` · the dialog's keys: `~/.config/fedokawaii/yazi-pick/` · `~/.config/xdg-desktop-portal-termfilechooser/config`	the trash: `~/.local/share/Trash/`	`~/.local/state/yazi/`
drives (udiskie, kawa-drives)	`~/.config/udiskie/config.yml`	—	which drives are open: `$XDG_RUNTIME_DIR/kawa-drives/` (cleared at each login)
zathura	`~/.config/zathura/zathurarc`	`~/.local/share/zathura/bookmarks.sqlite` (each book's page, bookmarks)	
mpv and yt-dlp	`~/.config/mpv/` (`mpv.conf`, `scripts/`) · `~/.config/yt-dlp/config`	—	`~/.local/state/mpv/` (where each video stopped)
torrra (torrents)	`~/.config/torrra/config.toml` — Prowlarr's address, the download folder, colours; never the key	the program: `~/.local/share/pipx/venvs/torrra/` · downloads: `~/Downloads/`	
Prowlarr	`~/.config/containers/systemd/prowlarr.container` (how it runs) · its key: `secrets.sh`	`~/.config/prowlarr/` (its database: indexers, login, history) · the image: `~/.local/share/containers/`	`journalctl --user -u prowlarr`
Brave Origin	`~/.config/BraveSoftware/Brave-Origin/` (profile, settings, extensions)	the same folder	`~/.cache/BraveSoftware/`
spotify_player	`~/.config/spotify-player/` (`app.toml`, `theme.toml`)		`~/.cache/spotify-player/` (login, cover art)
btop	`~/.config/btop/btop.conf`		
tldr	its colours and refresh in `~/.bashrc.d/fedokawaii.sh`		`~/.cache/tldr/` (the pages)
Steam and games		`~/.local/share/Steam/` · GE-Proton in `compatibilitytools.d/` there	
GTK, Qt, pointers	`~/.config/gtk-4.0/` · `~/.config/qt6ct/` · `/etc/dconf`	`~/.local/share/themes/Fedokawaii*` · `~/.local/share/icons/Fedokawaii/` (pointers)	

10.3 On the system

Path	What
`/usr/local/lib/fedokawaii/`	the toolkit: this project, root's own copy, run by every update, repair and move (§2)

Path	What
`/usr/local/bin/`	`fedokawaii` (the control, Super+U), `dwl` and `dwl.prev` (the build before it), `start-dwl`, `kawa-greeter`, `kawa-torrra`
`/usr/local/lib/fedokawaii-wlroots-0.20/`	dwl's own copy of wlroots, kept from Fedora's build (§6.5)
`/usr/local/sbin/`	`kawa-snap` (the DNF snapshots)
`/usr/local/libexec/xdg-desktop-portal-termfilechooser` (+ its service files under `/usr/local/lib` and `/usr/local/share`)	the yazi file dialog's portal, built by setup
`/etc/dnf/libdnf5.conf.d/90-fedokawaii.conf` · `/etc/dnf/libdnf5-plugins/actions.d/`	DNF's settings · the snapshot around every DNF run
`/etc/tuned/profiles/fedokawaii/` · `/etc/NetworkManager/conf.d/90-fedokawaii.conf`	the performance profile · Wi-Fi power saving off
`/etc/dconf/db/local.d/00-fedokawaii`	GTK's defaults: theme, icons, pointers, fonts
`/etc/greetd/fedokawaii.toml` · `/etc/systemd/system/greetd.service.d/fedokawaii.conf`	the login: agreety on tty1 · greetd pointed at that file
`/etc/snapper/configs/` (`root`, `home`) · `/.snapshots` and `/home/.snapshots`	the snapshot settings · the snapshots
`/boot/grub2/themes/sumi/`	the boot menu theme
`/var/lib/fedokawaii/` · `/var/log/fedokawaii.log` · `/var/log/fedokawaii-{check,verify}.log`	setup's progress and the records the menu reads (`last-update`, `last-verify`, `last-upgrade`, `release-N`, `snapshots`, and what to look at after a move: `unavailable`, `rpmnew`) · the log · the check and verify logs (root's alone: `fedokawaii log`)

Everything system-wide goes into a drop-in folder (`*.d/`) or under `/usr/local`, never into a file Fedora's packages own — so a Fedora update never fights these settings, and removing one of them removes it cleanly. A drop-in folder sits beside the file it changes; each `.conf` file in it is read after the main file and overrides only what it sets. greetd's is a small example (shortened: the real file also keeps tty1 for greetd alone):

```
# /etc/systemd/system/greetd.service.d/fedokawaii.conf
[Service]
# an empty ExecStart= first clears the command in Fedora's greetd.service
ExecStart=
# then the one used instead: greetd with Fedokawaii's own config file
ExecStart=/usr/bin/greetd --config /etc/greetd/fedokawaii.toml
```

Command	Shows
`systemctl cat greetd.service`	Fedora's unit, then each drop-in, each under a `# /path` line
`rpm -qf /etc/greetd/fedokawaii.toml`	`file /etc/greetd/fedokawaii.toml is not owned by any package`: Fedokawaii's own
`rpm -qf /etc/greetd/config.toml`	`greetd-…`: Fedora's file, left as the package made it

Had a script edited a packaged config file instead, an update that brings a new version of that file would leave it beside yours as `.rpmnew`, to merge by hand; after a move to a new Fedora the menu lists any there are (`rpmnew` in the last row above).

Read more: systemd.unit(5), drop-in directories.

11. Using the software

Chapter 4 covers the desktop itself: the bar, the menus, the keys, the themes. This chapter covers the programs you use on it, one at a time: what each is for here, how you open it, the handful of things worth knowing, and where it trips people up. Keys are given as this desktop configures them; where a program keeps its own defaults, they are the defaults of the version Fedora 44 ships, or of the release `update` fetches. Each program also documents itself for the version installed: `man foot`, `man fuzzel`, `~` in yazi, `:help` in neovim; for any other command, `tldr` and `man` (§11.2).

Who owns which config

Every file under `configs/home/` in the toolkit belongs to **Theme**: change it in your working copy and apply (chapter 15; **Super+Shift+U** opens each by name). An edit made only in `~/.config` is listed by `fedokawaii drift` and replaced at the next theme run (the old copy goes to `~/.local/state/fedokawaii/backup/`). Files the toolkit does not ship are yours and are never touched.

Managed by Theme (edit in the repository)	Yours (edit in place)
`foot/foot.ini`, `fuzzel/fuzzel.ini`, `mako/config`, `waybar/`, `btop/btop.conf`, `lazygit/config.yml`, `bat/config`, `mpv/mpv.conf` and `mpv/scripts/kawa-*.lua`, `yt-dlp/config`, `zathura/zathurarc`, `satty/config.toml`, `spotify-player/`, `yazi/theme.toml`, `yazi/init.lua`, `yazi/keymap.toml` and `yazi/plugins/` (drives, smart-enter), `udiskie/config.yml`, `nvim/lua/config/lazy.lua`, `nvim/lua/plugins/colorscheme.lua`, `nvim/after/ftplugin/sh.lua`, `MangoHud/MangoHud.conf`, `mimeapps.list`, `xdg-terminals.list`, the portal files (`xdg-desktop-portal/`, `xdg-desktop-portal-wlr/`, `xdg-desktop-portal-termfilechooser/`), the pointers in `~/.local/share/icons/`, `pipewire/pipewire.conf.d/60-share.conf`, `~/.bashrc.d/fedokawaii.sh`, `~/.inputrc`, `dwl/autostart.sh`, the launcher entries in `~/.local/share/applications`	every other file in `~/.config/nvim`, `yazi/yazi.toml`, `mpv/input.conf`, `cava/config`, `~/.config/fedokawaii/secrets.sh`, `~/.config/torrra/config.toml`, the browser and Steam profiles (except the two Brave settings in §2)

11.1 foot — the terminal

Super+Return opens a terminal: the window that shows text and passes your keys to the program running in it, usually the shell (§11.2). Each one is a window of the foot *server* — one foot process, started at login, that draws every terminal window — which is why it appears at once. **Ctrl+Shift+N** inside a terminal opens another in the same folder. The prompt and its marks are described in §4.6.

Keys	Does
Ctrl+Shift+C · Ctrl+Shift+V	copy · paste. Selecting text with the mouse already copies it
Shift+PageUp / PageDown	scroll back through the last 20 000 lines (also the mouse wheel)
Ctrl+Shift+R	search the scrollback: type, then Ctrl+R for the older match, Ctrl+S for the newer; Ctrl+W widens the match by a word; Enter ends the search with the match selected; Esc cancels
Ctrl+Shift+O	URL mode: every link on screen gets a short label; type the label to open it in Brave; Esc leaves
Ctrl+Shift+Y	copy mode: every commit hash, IP address and file path on screen gets a label; type it and that text is on the clipboard — no careful selecting. Type the label in capitals and it is pasted where you are typing as well
Ctrl+Shift+Z / X	jump to the previous / next prompt
Ctrl+Shift+G	the last command's output in neovim (§4.6)
Ctrl+Shift+U	type a character by its Unicode number, in hexadecimal: Ctrl+Shift+U, `e9`, Enter types *é*. Nothing shows while you type the number; Esc gives up
Ctrl++ · Ctrl+- · Ctrl+0	larger · smaller · normal text (Ctrl+wheel too)
double · triple · four clicks	select a word · the text inside the quotes around it (the whole line where there are none) · the whole line

Keys	Does
Ctrl+drag	select a rectangle, such as one column of `ll`'s output

Recommended. A terminal started as `footclient -a float` opens floating and centred, like the network window from quick settings, btop from the bar and the clipboard history; good for a quick look at something without disturbing the tiling. In programs that use the mouse themselves (btop, yazi, lazygit), hold **Shift** while dragging to select text anyway.

Tip: to keep a long output, press Ctrl+Shift+G and save it from neovim with `:w ~/Documents/output.txt`.

A notice when a long command ends

A command that runs 10 seconds or more sends a notice when it finishes — *done in 2m14s*, or *failed (1) after 40.3s*, with the folder it ran in — but only while its terminal is not the window you are looking at. Start an upgrade, go elsewhere, and hear when it is done.

Safe clipboard

Programs in the terminal may *put* text on the clipboard (neovim over SSH does that with OSC 52 — an *escape sequence*, an instruction a program prints to the terminal in place of text), but never *read* it: a file you `cat` or a server you `ssh` into cannot see what you copied (`[security] osc52=copy-enabled`).

Pitfalls.

- Every selection replaces the clipboard (`selection-target=clipboard`) and lands in the clipboard history (Super+V). Middle-click pastes the *primary* selection — a second, older clipboard that holds whatever was last selected — which foot itself no longer fills; use Ctrl+Shift+V.
- All Super+Return windows live in one server process. If it ever stops, they all close together; a new server starts within a second, but the windows do not come back. For something that must not be cut off — a long copy, an upgrade — start the plain **Foot** entry from the launcher, which runs as a process of its own.
- The server reads `foot.ini` once, when it starts: any change — the font, a key, the scrollback — reaches new windows only after the server restarts, which in practice means the next login (§9). The theme's colours are the exception: they switch in every open window.
- Right-to-left text is shown but not reordered (§9, known issues).

Read more: `man foot` and `man foot.ini` describe every key and option of the installed version; the project lives at codeberg.org/dnkl/foot.

11.2 The shell

Terms.

Term	In one line	Here
terminal	the window that shows text and passes your keys on	foot (§11.1)
shell	the program in it that reads a command line, runs it and prints the next prompt	bash
readline	bash's line editor: what Tab, Up and the word keys do while you type	`~/.inputrc`
variable · environment variable	a named value, read as `$NAME` · one marked with `export`, so every program started from this shell gets a copy	`$HOME` · `EDITOR=nvim`
`PATH`	the folders searched, in order, for a command typed by its name	`~/.local/bin` first
alias · function	a word typed in place of a longer command · a named piece of shell code that takes arguments and can decide	`ll` · `prowlarr`, `prowlarr stop`
exit status	the number a command ends with: 0 is success, anything else a failure	`$?`; the prompt's seal
pipe · redirection	one command's output into the next command · output into a file, or input from one	*By example*, below

Term	In one line	Here
glob	a pattern that bash turns into the matching file names before the command runs	`*.pdf`
quoting	`'…'` keeps text exactly as typed; `"…"` keeps it one word but still expands `$NAME`	`"$file"`

Bash, with Fedora's own `~/.bashrc` reading `~/.bashrc.d/fedokawaii.sh` (should a `~/.bashrc` not read that folder, Theme adds Fedora's few lines for it). The prompt, the OSC 7 / 133 marks and the history size are in §4.6; this is the rest of what the file and `~/.inputrc` set. A handful of small tools make it quicker, each doing one job:

Tool	Job	Replaces
fzf	find by typing a few letters of anything: history, files, folders	scrolling, `Ctrl+R`'s one-match-at-a-time search, long paths
zoxide	`cd` remembers the folders you use and jumps to them by a word	typing `cd ../../repos/project`
eza	`ls`, with folders first, sizes you can read, each file's git state, trees	`ls -la`, `tree`
tldr	the five commands you actually wanted, with examples	reading a whole man page
bat, **ripgrep**, **fd**	reading a file, searching inside files, finding files (§11.7)	`cat`, `grep -r`, `find`

Keys or input	Does
Up / Down	earlier / later commands that start with what you have already typed: `git c`, then Up, offers `git commit …`, `git checkout …` in turn
Ctrl+R	search the whole history in fzf: type any part of a command, Enter puts it on the line. Ctrl+R again lists in time order instead of best match first; Shift+Delete removes the highlighted command from the history (a password typed by mistake)
Ctrl+T	pick a file or folder below here (previewed: a file's start, a folder's tree) and insert its path where you are typing
Alt+C	pick a folder below here and go into it
`nvim **`, then Tab	fuzzy completion for any command: `**` + Tab opens fzf for that argument
`cd proj`	jumps to the folder you use most whose name matches *proj*, from anywhere (`~/repos/project`); `cd re pro` matches both words in order; a real path still works exactly as always
`cdi`	the folders zoxide knows, in fzf
Tab	complete, ignoring case; one press lists every choice when there are several
Ctrl+Left / Right	move by word
Ctrl+Backspace · Ctrl+Delete	delete a word backwards · forwards
Alt+.	insert the last argument of the previous command (press again for the one before): after `mkdir ~/repos/notes`, typing `cd` and Alt+. gives `cd ~/repos/notes`
a folder name on its own	changes into it (`autocd`)
`**` in a path	any depth of folders: `ls **/*.pdf` (`globstar`)

In every fzf list — Ctrl+R, Ctrl+T, Alt+C, the clipboard history — each word you type narrows the list, matched loosely:

Type	Keeps
`dnf inst`	lines that contain both words, in any order
`'ssh`	lines with exactly *ssh* (a leading `'` turns the loose match off)

Type	Keeps
`^bin` · `.pdf$`	lines that start with *bin* · end in *.pdf*
`!sudo`	lines without *sudo*

Alias or setting	Does
`ls` · `ll` · `lt`	eza: names, folders first · everything, with sizes, dates (`2026-10-15 13:04`) and each file's git state (`N` new, `M` modified) · a tree, two levels deep
`tldr tar`	examples for `tar` (any command); the pages are fetched on first use and refreshed at most once a quarter, when you ask
`update` · `verify` · `repair`	§6.1
`snaps`	the snapshot list (§11.21)
`dwl-log`	the compositor's log, opened at the end
`torrra` · `prowlarr`	torrra through its key wrapper · Prowlarr's web page, started first (`prowlarr stop` when done) (§12)
`ssh`	always reaches the server as `xterm-256color` : few servers know foot's own terminal type
`history`	shows the date and time of each command
`EDITOR` , `VISUAL` , `vi`	all neovim; `man` pages open in neovim too (`q` closes; `gO` lists the sections)
`less`	colours kept, mouse wheel works, searches ignore case unless you type a capital

By example

Small, safe commands for the ideas in the *Terms* table; try them in any folder. Anything after `#` is a comment, which bash ignores.

Alias, function or program. `type -t` names what a word is:

```
type -t ll prowlarr nvim cd
```

```
alias
function
file
function
```

`ll` is an alias, `prowlarr` a function (it looks at its argument: `prowlarr stop`), `nvim` a program found through `PATH` (`command -v nvim` prints `/usr/bin/nvim`), and `cd` is zoxide's function. `type ll` shows what the alias stands for. Your own go in `~/.bashrc.d/local.sh` , which every new shell reads after this desktop's file, and which Theme never touches:

```
# ~/.bashrc.d/local.sh — yours
alias gs='git status'                          # an alias: typing gs runs git status
mkcd() { mkdir -p -- "$1" && cd -- "$1"; }   # a function: mkcd photos makes the folder, then goes in
```

An alias only swaps one word for others. Anything that needs an argument in the middle, or a decision, is a function.

Environment variables. Only an exported variable reaches the programs a shell starts:

```
NAME=hello; bash -c 'echo "[$NAME]"'    # prints []       — a plain variable stays in this shell
export NAME; bash -c 'echo "[$NAME]"'   # prints [hello]  — exported: the new bash has a copy
printenv EDITOR                         # prints nvim     — how git and sudoedit know your editor
```

Exit status, `&&` and `||` .

```
grep -q autocd ~/.bashrc.d/fedokawaii.sh; echo $?   # 0 — found
grep -q zsh ~/.bashrc.d/fedokawaii.sh; echo $?      # 1 — not found
mkdir -p ~/tmp && cd ~/tmp                          # && runs the cd only if mkdir succeeded
grep -q zsh ~/.bashrc.d/fedokawaii.sh || echo none  # || runs the echo only after a failure
```

Pipes. The vertical bar `|` hands one command's output to the next command as its input. Your five most-used commands:

```
history | awk '{print $4}' | sort | uniq -c | sort -rn | head -5
```

Step	Passes on
`history`	every saved line: `812 2026-10-14 21:03:11 git status`
`awk '{print $4}'`	the fourth field of each line, the command's name: `git`
`sort`	the names in order, so the same ones stand together
`uniq -c`	each name once, with how many times it came: `251 git`
`sort -rn`	by that number (`-n`), largest first (`-r`)
`head -5`	the first five lines only

Redirection.

```
ls ~/Downloads > ~/list.txt    # > writes the output into a file, replacing what it held
date >> ~/list.txt             # >> adds to its end instead
ls /nope 2> /dev/null          # 2> sends the error messages away; /dev/null discards them
```

`>` empties its file before the command starts, so `sort list.txt > list.txt` leaves an empty file: write to a new name, then rename it.

Globs. Bash turns the pattern into file names before the command sees it:

Pattern	Matches
`*.pdf`	every name ending in `.pdf`, in this folder
`IMG_?.jpg`	`?` is exactly one character: `IMG_1.jpg`, not `IMG_12.jpg`
`[ab]*`	names that start with `a` or `b`
`**/*.pdf`	PDFs here and in every folder below (`globstar`)
`!(*.jpg)`	everything except JPEGs (`extglob`, on here)

A pattern matches a hidden name (`.bashrc`) only when its own first character is a dot. When nothing matches, bash passes the pattern on unchanged, and `ls` reports that `*.pdf` does not exist.

Quoting.

```
f='old notes.txt'
ls $f           # wrong: bash splits at the space, so eza looks for "old" and "notes.txt"
ls "$f"         # right: one name, space and all
echo '$HOME'    # prints $HOME    — single quotes keep every character as typed
echo "$HOME"    # prints /home/…  — double quotes keep the words together but expand $NAME
```

Put double quotes round every `"$variable"`: ShellCheck's most frequent warning, SC2086, is exactly this (§11.5).

Fast to start

Every helper's shell code is written once to `~/.cache/fedokawaii/` and read from there: a new terminal starts no extra program, and the prompt itself runs none outside a git repository. zoxide learns a folder only when you arrive in it — one quick call then,

nothing on any other prompt. (`time bash -ic exit` shows the cost: a few hundredths of a second.) After an `update` changes one of the tools, its cached code is written again by itself.

History rules

A command typed with a **leading space** is not saved: use it for anything with a password or token in it. Bare `ls`, `ll`, `lt`, `cd`, `cd -`, `pwd`, `exit`, `clear` and `history` are not saved either (`ls -la` is: a pattern must match the whole line), and a repeated command keeps only its newest copy.

Secrets

API keys go in `~/.config/fedokawaii/secrets.sh` (mode 600), which every new shell reads — and only while the file is yours alone: a copy others could read is not loaded, and the shell says so. Write them as `export NAME=value` — without `export` the programs you start never see them. Setup keeps Prowlarr's key there too (§12), deliberately without `export`: only the torrent search reads it, so no other program carries it. Nothing in the repository holds a key.

Pitfalls. `cd` here is zoxide's: a name that is not a folder where you are is looked up among the folders you have used, so `cd notes` typed in the wrong place lands in `~/repos/notes` if that is the *notes* you use most. Check the prompt before running something destructive there. A misspelt name is not corrected either: `cd Dcouments` answers *zoxide: no match found* (`builtin cd Dcouments`, bash's own `cd`, does correct it, through `cdspell`). Outside foot — the text consoles (Ctrl+Alt+F2…F6), a login from another machine — the prompt uses plain characters, because only foot has the icon font.

Read more: the GNU Bash manual — *Quoting*, *Pattern Matching* (globs), *Pipelines*, *Redirections*, *The Shopt Builtin*; its *Command Line Editing* chapter is readline (`man bash` holds the same text). fzf's keys and search syntax: github.com/junegunn/fzf; zoxide: github.com/ajeetdsouza/zoxide.

11.3 fuzzel — the launcher



The launcher (Super+Space): Papirus icons and names, on both themes.

Super+Space, or the torii key at the left of the bar. Type part of a name, Enter starts it, Esc closes.

- It matches application **names only**: *brave* finds Brave Origin, *browser* finds nothing.
- The programs you start most often move to the top (the counts live in `~/.cache/fuzzel`).
- Terminal programs with a launcher entry, such as Neovim and btop, open in a terminal window. **Files** (yazi) and **Torrents** (torrra) are entries of this desktop's own.

- The other tools installed as release binaries — presenterm, spotify_player, satty — have no launcher entry; start them in a terminal.

Keys	Does
Up · Down (or Ctrl+P · Ctrl+N)	move in the list
Enter	start the highlighted program
Tab	start it when only one match is left; otherwise move down the list
Shift+Delete	forget how often you started the highlighted program, so it loses its place near the top
Esc	close

Read more: `man fuzzel` and `man fuzzel.ini`; codeberg.org/dnkl/fuzzel.

11.4 yazi — files



yazi: the parent folder, the current folder, and a preview of the file under the cursor.

Super+E opens yazi in its own terminal window, starting in your home folder; `q` quits and closes the window. Typed in a terminal, `yazi` starts in the folder you are in. Three columns: the parent folder, the current one, a preview of the file under the cursor. Its colours are the terminal's, so it switches with the theme. The file under the cursor is the *hovered* one; files marked with Space are *selected*. Most keys act on the selected files, or on the hovered one when none are. Press **~** or **F1** for the complete key list of the installed version.

Keys	Does
h j k l (or arrows)	parent folder · down · up · open folder
gg · G · H · L	top · bottom · back · forward through visited folders

Keys	Does
g h · g d · g c · g t	home · Downloads · `~/.config` · the trash
M · g m	the drives: open or eject a USB stick, disk or SD card · go where they open (§4.13)
g Space	type a path and jump there
Enter · O	open the file (a folder: go into it) · choose what to open it with
Space · v · Ctrl+A	select a file · select a range · select all
y · x · p · P	copy · cut · paste · paste, overwriting
d · D	move to the trash · **delete for good**
a	create a file; end the name with `/` for a folder
r	rename; with several files selected, rename them all in neovim
.	show / hide hidden files
s · S	search file names (fd) · file contents (ripgrep) below this folder
f · / · n · N	filter the list as you type · find · next · previous match
z · Z	jump to any folder or file below this one by typing part of its name (fzf) · jump to a folder you often use, from anywhere, by a word (zoxide — the same list as the shell's `cd`)
c c · c f · c d	copy the file's full path · its name · the folder's path
t t · 1…9 · [ ]	new tab · go to a tab · previous / next tab
Tab	details of the file (type, size, picture dimensions)
, then a letter	sort: `m` modified, `s` size, `n` natural, `a` alphabetical (capital = reversed)
K · J	scroll the preview
; · :	run a shell command in the background · run one in this window (yazi returns when it ends)
w	running tasks (copies, moves)

Opening

On a folder, Enter goes into it (yazi's own Enter would open the folder in neovim: `~/.config/yazi/plugins/smart-enter.yazi`). On a file, it uses the first *opener* — a command yazi runs to open a file — for the file's type (its MIME type, such as `image/png`): text and JSON open in neovim inside the same window (`:q` brings you back to yazi); an archive is unpacked next to itself (yazi's *Extract here*, with 7-Zip); pictures, PDFs, videos and the rest go to `xdg-open`, which hands them to the default program for that type (§4.9). **O** lists every choice for the hovered file.

Previews that work here: text and code (in the terminal's colours), JSON (jq), pictures (drawn in foot as Sixel), the first page of PDFs (pdftoppm), video stills (ffmpeg), fonts and HEIC/AVIF/JXL pictures (ImageMagick), and the contents of archives (7-Zip). SVG pictures show no preview, because resvg is not packaged for Fedora.



yazi's preview column showing a picture, drawn in the terminal as Sixel by foot.

Bulk rename

Select the files (Space or v), press **r**: neovim opens with one name per line. Change the lines, save and quit (`:wq`), and yazi asks before renaming. Keep the lines in the same order and do not add or remove any. Every neovim edit works on the list: `:%s/^IMG_/trip_/` turns `IMG_0412.jpg` into `trip_0412.jpg`, and every other line alike (§11.5, where a macro also numbers them).

Trash

d moves files to the desktop trash (`~/.local/share/Trash`). **g t** opens it; there, **O** on a file offers *Restore selected files* and *Empty trash bin*. **D** skips the trash entirely.

Tip: files moved with **d** still take their space until the trash is emptied; make it part of the seasonal routine (§7.5).

Commands on files

In `;` and `:` commands, `%h` stands for the hovered file and `%s` for the selected ones (the hovered one when none are), each already quoted, so names with spaces are safe. The command runs in the current folder.

Type	Does
`;` then `chmod +x %h`	makes the hovered script executable
`;` then `7z a backup.7z %s`	packs the selected files into `backup.7z`, here
`:` then `du -sh %s; read`	shows their sizes until you press Enter (without the `read`, yazi comes back before you can read them)

Recommended: yazi, neovim and lazygit together. Work in one terminal: browse in yazi, press Enter on a file to edit it in neovim, `:q` to return; when the work is done, `:` then `lazygit` to stage and commit, and `q` in lazygit brings you back to yazi. Uploading a file in Brave opens yazi too, as the file dialog (§4.9, §13.1).

Pitfalls.

- Input boxes (rename, create, filter) work like a tiny vi: Esc first leaves typing, a second Esc — or Ctrl+C — cancels.
- Three files are the desktop's: `theme.toml` (colours, the file-kind icons — the desktop's own pixel icons), `init.lua` (the frame: one free column at each edge, as every popup has, and the file dialog's title rule and key line) and `keymap.toml` (**M** and **g m**, the drives), with the drives plugin in `plugins/`. Your own openers go in `yazi.toml`, which you create; your own keys go in `keymap.toml` in your working copy (chapter 15), so the next Theme keeps them (an example follows this list). yazi itself is held at the version these files are written for (§6.3).
- Search results (s, S) are a temporary list; Esc leaves it, Ctrl+S stops a search still running.

A key of your own, as an example — `g` then `r` to go to `~/repos`. Add its line to the list in `configs/home/.config/yazi/keymap.toml` in your working copy (Super+Shift+U → *files*), then apply:

```
# keymap.toml — these keys are tried before yazi's own
[mgr]
prepend_keymap = [
  { on = "M",          run = "plugin drives",          desc = "Drives: open or eject one" },
  { on = [ "g", "m" ], run = "plugin drives -- media", desc = "Go to the open drives" },
  { on = [ "g", "r" ], run = "cd ~/repos",             desc = "Go to ~/repos" },   # yours
]
```

yazi reads it when it starts; `~` then lists the new key with its *desc*. The file dialog reads its own list instead, `configs/home/.config/fedokawaii/yazi-pick/keymap.toml` (the menu's *files* opens it beside the others): a key you want there too goes in both.

Read more: yazi-rs.github.io/docs/quick-start for the keys, and the *Configuration* pages beside it for `keymap.toml` and `yazi.toml`. The site describes the newest yazi; `~` lists the keys of the version installed here.

11.5 neovim and LazyVim — the editor



Neovim with LazyVim, the palette's gruvbox, on a shell script.

Neovim is the only editor here (`nvim` , `vi` , `$EDITOR` , the man pages, yazi's Enter). It runs the **LazyVim** starter that setup copied into `~/.config/nvim` ; the repository adds only `lua/config/lazy.lua` (the starter's own, with the background check for new plugin versions switched off, so plugins move only with `update`), `lua/plugins/colorscheme.lua` , the remapped gruvbox that follows the dark and light theme by itself (§5.4), and `after/ftplugin/sh.lua` , which keeps shell scripts from being reformatted on save (*Formatting*, below). Started without a file, it shows a start screen with one-key actions: `f` find file, `n` new file, `g` find text, `r` recent, `c` config, `s` restore the last session in this folder, `x` extras, `l` plugins, `q` quit.

Neovim differs from other editors in one way that matters more than all the others: most of the time, the keys you press are **commands**, not text. That is what makes it fast once learned, and baffling for the first ten minutes. This section starts from nothing — how to stay in control, a first hour of practice, the small grammar everything else is built from — and then covers what LazyVim adds. Read it with neovim open beside it.

Terms.

Term	In one line
mode	what keys do at the moment: give commands (Normal), type text (Insert), select (Visual)
buffer	a file's text held in memory while you edit it; the file on disk changes only when you save
window	a view onto a buffer: two windows can show two files, or two places in one
tab	a whole layout of windows — not one file each, as in a browser
operator · motion · text object	a verb (`d` delete) · a move (`w` to the next word) · a named piece of text (`iw` this word)
register	one of neovim's clipboards, named by a character: `a` … `z` , and `+` for the desktop's
leader	the key that starts LazyVim's own commands: Space
plugin	an add-on written in Lua; LazyVim is a collection of them, installed and updated by lazy.nvim
picker	a search window: type, choose from the list, see a preview
language server · LSP	a separate program that understands one language and answers neovim's questions — where is this defined, what is wrong here · the protocol they speak (*Language Server Protocol*)
treesitter	a parser in neovim that reads the structure of code — functions, strings, comments — for colours, folds and text objects; LazyVim installs the parsers for common languages
formatter	a program that rewrites a file's layout (indentation, spacing) without changing what it does
quickfix list	a list of places — file and line — to step through, such as search results

Modes

Neovim is always in one *mode*. Its name stands in the box at the left end of the status line, each in a colour of its own. Esc leaves every other mode for Normal.

Mode	The box	You are	Get there with
Normal	NORMAL, vermilion	giving commands: moving, deleting, copying. Neovim starts here	Esc
Insert	INSERT, asagi	typing text, as in any editor	`i` , `a` , `o` and a few others (below)
Visual	VISUAL, yamabuki	selecting text, to act on it	`v` , `V` , Ctrl+V
Command-line	COMMAND, blue	typing a command after `:` , or a search after `/` ; Enter runs it, Esc drops it	`:` or `/`
Replace	REPLACE, beni	typing over the existing text	`R`
Terminal	TERMINAL, asagi	in a shell inside neovim; Esc Esc, quickly, stops typing into it	Ctrl+/ (again to hide it)

`:` opens a small box near the top of the window with suggestions under it (Tab completes); a search with `/` stays at the bottom.

Never stuck

Whatever happened, one of these gets you out of it:

Situation	Do
you do not know which mode you are in, or keys do odd things	**Esc** — twice if unsure. You are in Normal mode (Ctrl+[is the same key)
save and quit	`:wq` Enter, or `ZZ`. Ctrl+S saves, also while typing
quit without saving	`:q!` Enter; `:qa!` closes every file. Plain `:q` on a changed file asks *Save changes?* — Yes, No or Cancel
undo · redo	`u` · Ctrl+R, as many times as needed
throw away everything since the last save	`:e!` Enter
the status line says *recording @q*	you pressed `q` and a letter: press `q` once more
a window of old commands opened at the bottom	you typed `q:` : `:q` Enter closes it. (Ctrl+C there copies the line under the cursor to the command line; Esc then drops it)
letters come out in Arabic, or Normal-mode keys do nothing	the keyboard layout changed (the bar shows AR): Alt+Shift. The desktop's shortcuts always read the US layout; neovim reads the letters you type
neovim vanished and the shell says *Stopped*	Ctrl+Z put it to sleep: type `fg`
Found a swap file on opening a file	a neovim crashed while editing it: `R` recovers the unsaved text — save it, and answer `D` (delete the old swap file) the next time the question comes. (A file open in another running neovim opens anyway, with the warning *W325*: save it in one place only)
readonly or *Permission denied* on a system file	`:q!`, then `sudoedit /etc/the-file` in the shell: your own neovim edits a copy, and sudo puts it in place (§7.1)

Tip: undo survives closing the file — LazyVim keeps the undo history on disk, so `u` the next morning still takes back yesterday's edits. Space s u lists the whole history to pick a point from.

The first hour

Practise on a long file you cannot damage; a copy of this handbook's source does well:

```
cp ~/fedokawaii/docs/fedokawaii-guide.md /tmp/practice.txt
nvim /tmp/practice.txt
```

Spend ten minutes on each block below, in order, and do every key at least five times. Plain text files wrap long lines and underline spelling mistakes in red (Space u s hides them).

Moving. Arrows work, but h j k l sit under your right hand, and the hand never leaves the keys.

Keys	Move
`h` `j` `k` `l`	left · down · up · right
`w` · `b` · `e`	to the next word · back a word · to the end of the word
`W` · `B` · `E`	the same, counting only spaces as gaps (`~/.config/nvim` is one word)
`0` · `^` · `$`	start of the line · its first character after the indentation · its end
`gg` · `G` · `42G` (or `:42`)	first line · last line · line 42
Ctrl+D · Ctrl+U	half a screen down · up
`}` · `{`	to the next · previous blank line: a paragraph at a time
`%`	to the matching bracket

Keys	Move
Ctrl+O · Ctrl+I	back · forward to where you were before a jump
`s`, a letter or two, a label	**flash**: every visible place with those letters gets a one-letter label; type it to land there

The numbers in the left margin count lines **from the cursor**: the line you are on shows its real number, every other line how far away it is. Any move takes a count in front — `5j` goes five lines down, and the 5 can be read off the margin.

Typing.

Keys	Start typing
`i` · `a`	before · after the cursor
`I` · `A`	at the start · at the end of the line
`o` · `O`	on a new line below · above
Esc	stop typing: back to Normal mode

Changing.

Keys	Do
`x`	delete the character under the cursor
`r`, then a character	replace one character
`dd` · `yy`	cut · copy (*yank*) the line
`p` · `P`	paste after · before the cursor; a whole line goes below · above
`J`	join the next line onto this one
`~`	switch the case of a letter
`.`	**repeat the last change** — the most useful key in neovim
`u` · Ctrl+R	undo · redo

Searching. Searches ignore case unless you type a capital letter.

Keys	Do
`/word` Enter · `?word` Enter	search forward · backward
`n` · `N`	next · previous match
`*` · `#`	the word under the cursor, forward · backward
Esc	clear the highlighting

Replacing. While you type a replacement, the file shows the result live.

Command	Replaces
`:s/old/new/`	the first *old* on this line
`:s/old/new/g`	every *old* on this line
`:%s/old/new/g`	every *old* in the file (`%` means all lines)
`:%s/old/new/gc`	the same, asking at each: `y` yes, `n` no, `a` all the rest, `q` stop
`:%s/\<old\>/new/g`	whole words only — not the *old* in *golden*
`:'<,'>s/old/new/g`	in the selected lines: select, press `:`, and the `'<,'>` is filled in

The *old* is a **regular expression**: a pattern in which some characters stand for others. The few worth knowing first:

Pattern	Matches	For example
`.`	any one character	`c.t`: *cat*, *cut*
`^` · `$`	the start · the end of the line	`^#`: a `#` at the start
`\<` · `\>`	the start · the end of a word	`\<old\>` (above)
`\d` · `\s`	a digit · a space or a tab	`\d\d`: two digits
`*` · `\+`	the thing before it, any number of times · at least once	`\s\+$`: spaces at a line's end
`[…]`	one of the characters in the brackets	`[0-9]`, `[aeiou]`
`\.`	a real dot: the backslash takes the special meaning away	`\.sh$`

So `:%s/\s\+$//e` removes the spaces at the ends of all lines (the `e` keeps it quiet when there are none). ripgrep, behind Space / and yazi's **S**, writes the same ideas a little differently: `+` without the backslash, `\b` for the edge of a word.

Finally `:w` to save and `:q` to quit. Delete `/tmp/practice.txt` whenever you like.

The grammar: verb, count, motion

Most of neovim is three parts said in order: *what to do* (a verb — neovim calls it an operator), *how often* (a count, optional), and *over what* (a motion, or a text object). `d` deletes and `w` moves a word, so `dw` deletes a word, `d3w` three words, `d$` to the end of the line. Every verb works with every motion, so each new motion gives you a new command with every verb. A verb typed twice acts on the whole line.

Verb	Does	Twice: the whole line
`d`	delete (a cut: the text goes to the clipboard)	`dd`
`c`	change: delete, then type in its place	`cc`
`y`	yank: copy	`yy`
`>` · `<`	indent · outdent	`>>` · `<<`
`gu` · `gU` · `g~`	lower case · UPPER CASE · switch case	`guu` · `gUU` · `g~~`
`=`	re-indent	`==`
`gc`	comment out · back in	`gcc`

Any move from the first hour works as the motion — `w`, `b`, `e`, `0`, `$`, `G`, `gg`, `}`, a `/search` — and `j` or `k` make the verb take whole lines. Two more are made for verbs: `fx` reaches onto the next *x* on the way, `tx` up to just before it (any character in place of *x*; `F` and `T` look backwards). Text objects (next block) work as motions too.

You type	It does
`dw` · `d3w`	delete to the next word · three words
`D` (the same as `d$`)	delete to the end of the line
`dt)` · `df,`	delete up to the next `)` · up to and including the next comma
`5dd` · `dG`	delete five lines · everything to the end of the file
`cw` · `ciw`	change to the end of the word · the whole word, wherever the cursor is in it
`ci"` · `ci(`	change what is inside the quotes · the brackets
`yip` · `>ip` · `gcip`	copy · indent · comment out the paragraph
`gUiw`	the word in capitals
`d/end` Enter	delete up to the next *end*

One line, the cursor on the *o* of `old_name`, and what each command leaves:

You type	The line becomes
(nothing yet)	`mv old_name.txt "new name.txt"`
`dw`	`mv .txt "new name.txt"` — `w` stops at the dot
`dW`	`mv "new name.txt"` — `W` runs on to the next space
`dt.` · `df.`	`mv .txt "new name.txt"` · `mv txt "new name.txt"` — up to the dot · the dot too
`ci"`	`mv old_name.txt ""`, typing between the quotes: the object finds the next quotes on the line
`d$`	`mv` and the space after it

The dot

`.` repeats the last change, verb and typed text together. To rename a word in a few chosen places:

1. `*` on the word: neovim searches for it, highlights every copy and moves to the next one.
2. `ciw`, the new word, Esc: this copy is changed.
3. `n` to the next copy and `.` to change it the same way — or `n` again to leave one as it is.

Tip: `f` and `t` are LazyVim's (flash): they look past the end of the line too, highlight every possible target, and pressing `f` again moves on to the next one.

Text objects

A text object names a piece of text by what it is rather than where it ends, so it works wherever the cursor sits inside it. `i` takes the inside, `a` takes it *around*, with its surroundings: `iw` is a word, `aw` the word and the space after it. They follow a verb, or `v` to select:

Object	`i…` (inside)	`a…` (around)
`w` · `W`	the word	the word and a space
`s` · `p`	the sentence · the paragraph	with the space · the blank line after it
`"` · `'`	inside the quotes	with the quotes
`(` `[` `{` `<`, or `b` for the nearest	inside the brackets	with the brackets
`q`	inside the nearest quotes of any kind	with them
`t`	inside an HTML or XML tag	with the tags
`a`	an argument between commas	with its comma
`f` · `c`	the body of a function · a class (in code that treesitter can read)	the whole function · class
`g`	the whole file	the whole file

`daw` deletes a word and its space; `yi"` copies a quoted path without its quotes; `dap` deletes a paragraph; `yag` copies the whole file. LazyVim's mini.ai adds `q`, `a`, `f`, `c` and `g`, and makes all of them look ahead: `ci"` works from the start of the line, before the quotes. It adds a few more as well — `u` a function call, `d` a number, `o` a block, an *if* or a loop; pause after `vi` and the key panel lists them all.

Surround comes with an extra, `coding.mini-surround` (switch it on in `:LazyExtras`, below). Then `gsaiw"` puts quotes round a word — `gsa`, an object, the character — and `gsa` works on a selection too; `gsd"` removes the quotes round the cursor; `gsr"'` turns double quotes into single ones.

Visual mode and block mode

Select first, then act — the way to go when the piece has no name.

Keys	Does
`v` · `V` · Ctrl+V	select characters · whole lines · a rectangle (a block)
any move or object	extends the selection: `vip` a paragraph, `v$` to the end of the line
`o` · `gv`	jump to the other end · select the last selection again
`d` · `y` · `c`	delete · copy · change the selection
`>` · `<`	indent · outdent; the selection stays, so press again for more
`u` · `U` · `~`	lower case · upper case · switch case
`J` · `gc`	join the lines · comment them out or back in
Alt+J · Alt+K	move the selected lines down · up (a single line too, in Normal mode)
`p` · `P`	replace the selection with the clipboard; `P` leaves the clipboard as it was
Ctrl+A · `g` Ctrl+A	add one to every number · count up down the selection: a column of `0`s becomes 1, 2, 3 …
`:`	a command for the selected lines (`:'<,'>` appears)

Block mode selects a rectangle, which is how columns are edited. Press Ctrl+V, then move down (`j` , or `3j`) and across. With a block selected:

- `I` , the text, Esc — the text goes in at the left edge of the block on **every** line. It shows on the first line while you type and on the others when you press Esc: Ctrl+V, `3j` , `I` , then `- [ ]` and a space, then Esc turns four lines into a checklist.
- `$` , then `A` , the text, Esc — appended at the end of every line, whatever its length.
- `c` replaces the block's contents on every line; `d` deletes the column; `r` and a character fills it.

LazyVim lets the cursor go past the end of short lines in block mode, so a block can be as wide as the longest line. (foot's own paste is Ctrl+Shift+V, so Ctrl+V reaches neovim.)

Registers and the clipboard

Neovim keeps several clipboards, called *registers*, each named by one character. To use one, type `"` and its name before the command: `"ayy` copies the line into register `a` , `"ap` pastes it back. Typing `"` alone — or Ctrl+R in Insert mode — shows a popup of what each register holds; Space s " lists them in the picker, and `:reg a0` prints just `a` and `0` .

Here, the plain commands use the desktop's clipboard. LazyVim sets `clipboard=unnamedplus` (except in an SSH session), so `y` , `d` , `c` , `x` and `p` all go through the system clipboard: a line yanked in neovim pastes into Brave with Ctrl+V, a link copied in Brave pastes in neovim with `p` , and every yank lands in the clipboard history (Super+V, §4.8) — and, thanks to `kawa-clipkeep` , stays on the clipboard after neovim closes. `"+y` and `"+p` , the way plain Vim reaches the clipboard, work too and do exactly the same.

Register	Holds
`"` (unnamed) and `+`	the last yank or delete — here, the system clipboard
`0`	the last **yank** only; deletes leave it alone, so `"0p` pastes what you copied even after deleting since
`1` … `9`	the last nine deletions of a line or more, newest first
`-`	the last deletion within a line (`dw` , `x`)
`a` … `z`	your own: `"ay` stores, `"Ay` (the capital) adds to the end
`_`	the black hole: `"_dd` deletes a line without touching the clipboard
`*`	the primary selection (select, then middle-click)
`/` · `:` · `.` · `%`	the last search · the last command · the last text typed · this file's name

Tip: since deletes go to the clipboard, a `dd` replaces the link you copied in Brave a minute ago. It is still in Super+V; to keep it on the clipboard, delete with `"_d` , or paste your last yank with `"0p` .

Macros

A macro is a recording of keys that neovim plays back: any edit you can do once by hand, it does a hundred times. The recording goes into a register, like a copy.

Keys	Does
`qa`	start recording into register `a` (any letter); the status line shows *recording @a*
`q`	stop recording
`@a` · `@@`	play it once · play the last played macro again
`10@a`	play it ten times in a row
`qA` … `q`	record more onto the end of `a`
`:5,20normal @a`	play it once on each of lines 5 to 20
`:'<,'>normal @a`	play it once on each selected line
`:g/pattern/normal @a`	play it once on each line containing *pattern*

Three habits make a macro reliable:

- **Start from a known place.** Begin with a move that does not depend on where the cursor happened to be — `0` or `^` — and end by moving to where the next run should start, usually `j`, the next line.
- **Move by meaning, not by counting.** `t.`, `f,`, `w`, `/word` find their place on lines of any length; `llll` works only on lines exactly like the first.
- **A macro stops at the first command that fails.** On the last line `j` fails, and playback ends there — so `100@a` simply means "to the end".

A worked example: numbering photos. Select some pictures in yazi and press **r** (bulk rename, §11.4): neovim opens with one name per line.

```
IMG_20261003_101512.jpg
IMG_20261003_101530.jpg
IMG_20261003_102244.jpg
IMG_20261003_103901.jpg
```

They should become `beach_01.jpg`, `beach_02.jpg` and so on.

1. Do the first line by hand: `gg`, then `0ct.` (change everything up to the dot), type `beach_01`, Esc. It reads `beach_01.jpg`.
2. Record how to get from one line to the next. On line 1, `qa` starts the recording, then: `0yt.` copies `beach_01` (everything before the dot); `j0ct.` goes down and changes that line up to its dot; **Ctrl+R** then `0` types the text just copied; Esc; **Ctrl+A** adds one to the number under the cursor; `q` stops. Line 2 reads `beach_02.jpg`.
3. `100@a` does the rest: each run copies the line it is on, moves down, writes the name and counts up, until the last line stops it.
4. `:wq` — and yazi asks once more before renaming.

Ctrl+A keeps the leading zero (`beach_09` becomes `beach_10`). It reads a `-` just before a number as a minus sign, which is why the example uses `_`: from `beach-01`, Ctrl+A makes `beach00`.

Editing a macro. A macro is only text in a register, so it can be corrected instead of recorded again. On an empty line, `"ap` pastes it, with Esc shown as `^[` and Ctrl+A as `^A`. Edit it like any text — to type a special key into it, press Ctrl+V first in Insert mode, then the key — then `0"ay$` copies the line back into `a` (`$` rather than `yy`, so no line break comes along) and `dd` removes the line.

Without a recording: `:normal` and `:g`. `:normal` runs Normal-mode keys on every line of a range, as if you typed them there with the cursor at the start of each — often all a macro would have done. Insert mode ends by itself at the end of each line's keys.

Command	Does
`:%normal A;`	adds `;` to the end of every line
`:'<,'>normal I#`	puts `#` at the start of each selected line

Command	Does
`:g/^TODO /normal 0cw- [ ]`	turns every line starting `TODO` into a checklist item, `- [ ] …`
`:g/^$/d`	deletes every empty line
`:v/keep/d`	deletes every line that does **not** contain *keep*
`:sort` · `:sort u`	sorts the lines of the file (or of a selection) · and drops repeated ones

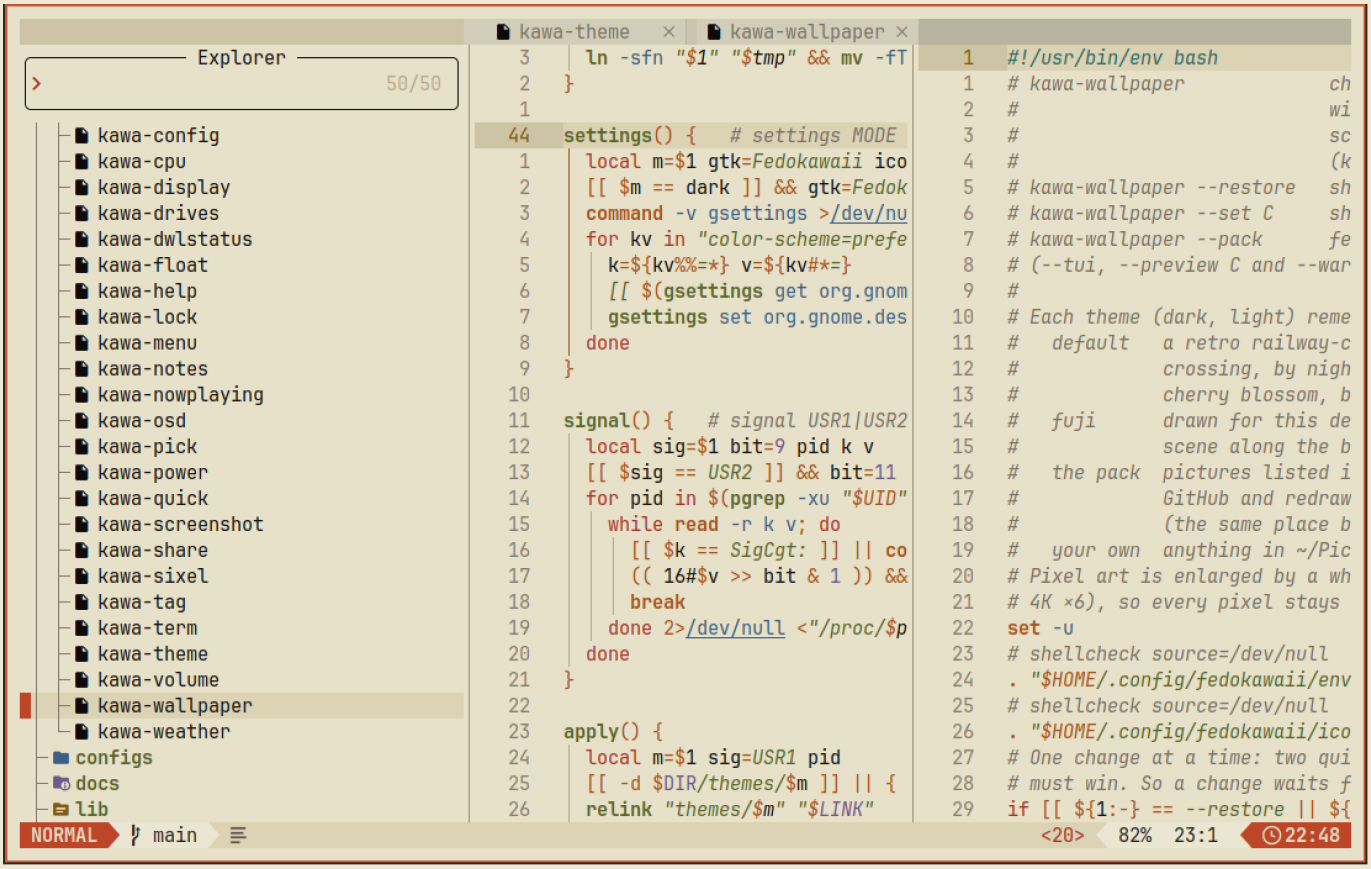
`:g/pattern/` picks lines by what they contain and runs the command after it on each — `d` to delete, `normal` and any keys, `normal @a` a macro; `:v` picks the lines that do not match.

Files, buffers and windows

Every file you open becomes a *buffer* and stays open until you close it; once there are two or more, LazyVim lists them along the top. A *window* is a view onto a buffer: splitting the screen shows two files side by side, or two places in one.

To see the difference: `:e notes.md`, then `:e todo.md` — two buffers in one window; Shift+H brings *notes* back. Space | splits the window, and both halves show *notes* (two places in one file); Shift+L in one half shows *todo* there. Space w d closes a window, but its buffer stays open until Space b d.

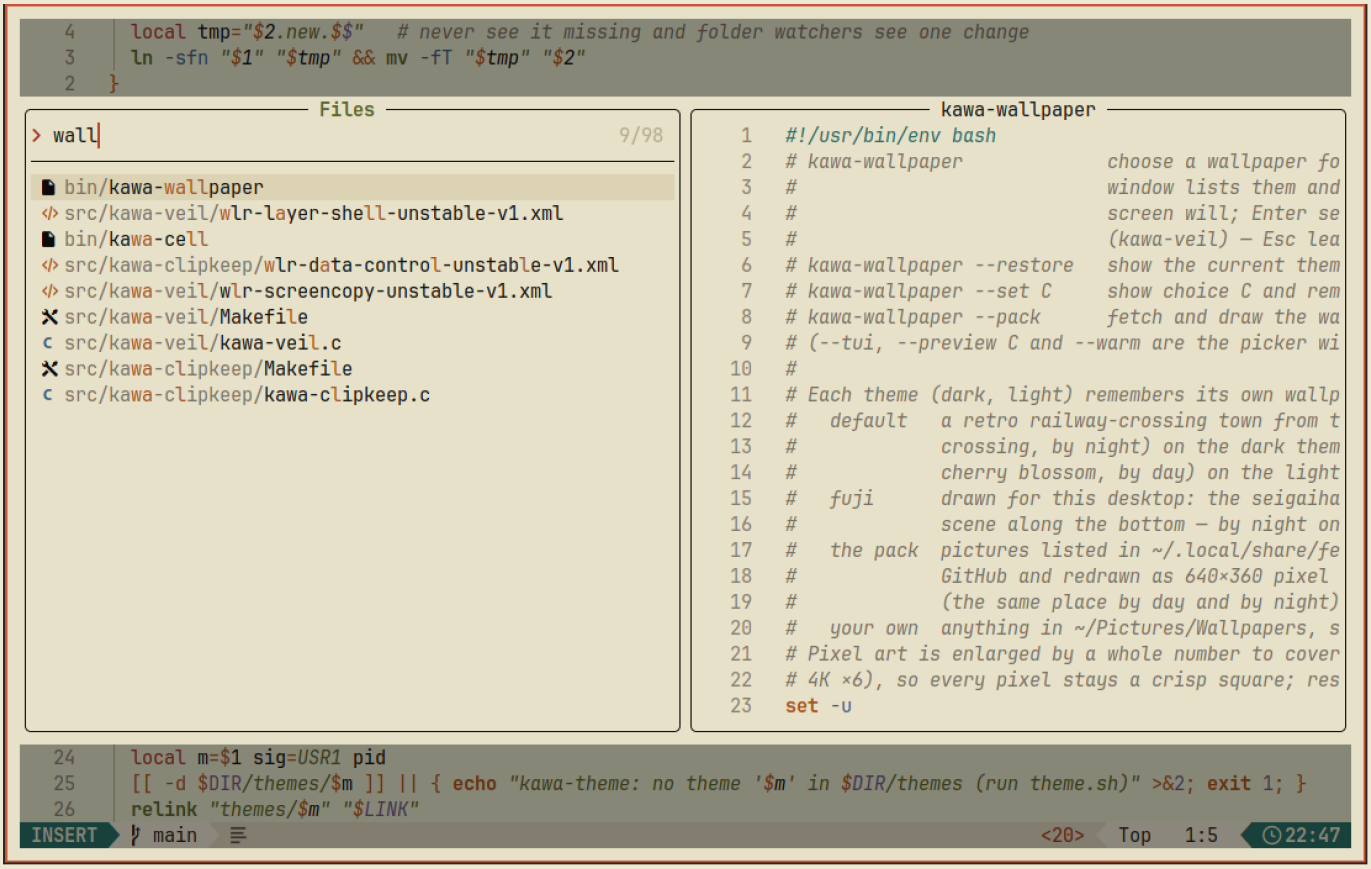
Keys	Does
Space Space · Space f r	find a file in the project · a recent file
Space , · Space b b	the open buffers · back to the one before
Shift+H · Shift+L	previous · next buffer
Space b d · Space b o	close this buffer · close all the others
`:e path`	open a file by its path; Tab completes
Space - · Space |	split: a new window below · to the right
Ctrl+H · J · K · L	to the window on the left · below · above · on the right
Ctrl+↑ · ↓ · ← · →	make the window taller · shorter · narrower · wider
Space w d · Space w m	close the window (the file stays open) · zoom it to the whole screen and back
Ctrl+W `=`	make all windows the same size
Space Tab Tab · Space Tab d	a new tab, a whole new layout of windows · close it



Two windows side by side, two shell scripts, the file explorer on the left and the open buffers listed along the top.

Finding

Space Space opens the *picker*, LazyVim's search window: a line to type in, the matches under it, a preview of the file beside them. It matches loosely — `brc` finds `bashrc`. The same window serves every search in LazyVim (files, text, buffers, help, keys), with the same keys:



The file picker (Space Space): the typed letters, the matching files, a preview of the one selected.

Keys in a picker	Does
typing	filters the list
↑ ↓ or Ctrl+J · Ctrl+K	move in the list
Enter	open
Ctrl+V · Ctrl+S · Ctrl+T	open in a window to the right · below · in a new tab
Tab	mark several; Enter then opens them all
Alt+H · Alt+I	include hidden files (`.bashrc`, a dotfiles folder) · files git ignores
Ctrl+Q	put every result in the quickfix list (*Search and replace*, below)
Alt+P · Ctrl+F · Ctrl+B	preview on or off · scroll it down · up
Esc Esc, or Ctrl+C	close. The first Esc only stops the typing; `?` then lists every key

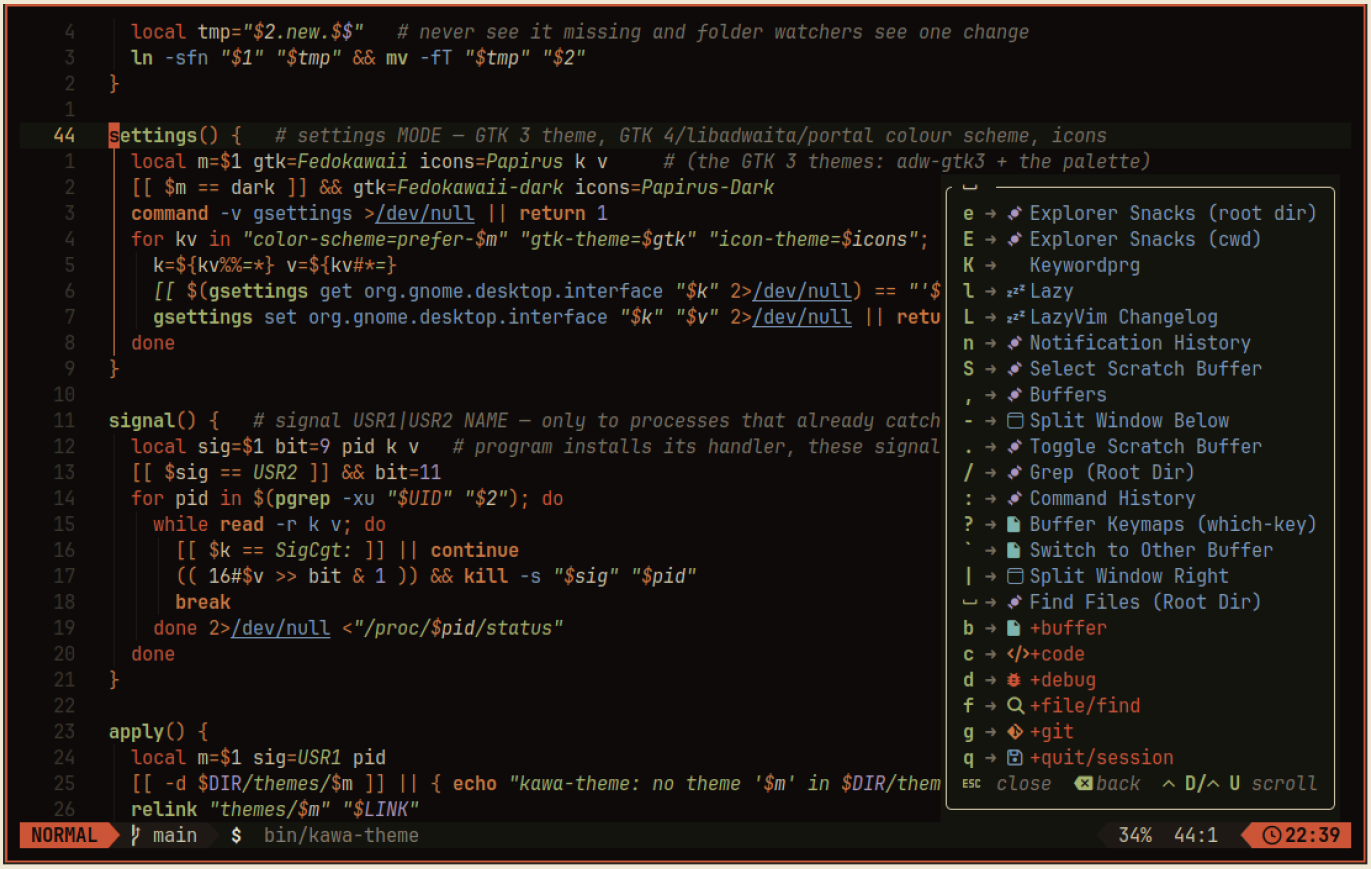
The "project" (LazyVim calls it the *root*) is the folder the language server works in, or else the nearest folder above the file that holds `.git` (a git repository) or `lua`, or else the folder neovim was started in.

The file tree

Space e opens the project's folders at the left; Enter (or `l`) on a file opens it and the tree stays, `h` closes a folder. In the tree, `a` makes a new file (end the name with `/` for a folder), `r` renames, `d` deletes — to the desktop trash, after asking — `c` and `m` copy and move, `H` shows hidden files and `I` the ones git ignores, `o` opens a file with the desktop's program for it, `q` closes the tree and `?` lists the rest.

The leader key

Most of LazyVim's own commands start with **Space**, the *leader*. Press it and wait a moment: a panel in the lower right lists what can follow, in groups — `f` files, `s` search, `g` git, `c` code, `b` buffers, `w` windows, `u` interface switches, `x` problem lists, `q` quitting and sessions. Press a letter to go down a level, Backspace to come back up, Esc to give up. The same panel helps after `g`, `z`, `[`, `]` and `"`, and Space s k searches every key by its description — type *split* or *comment*.

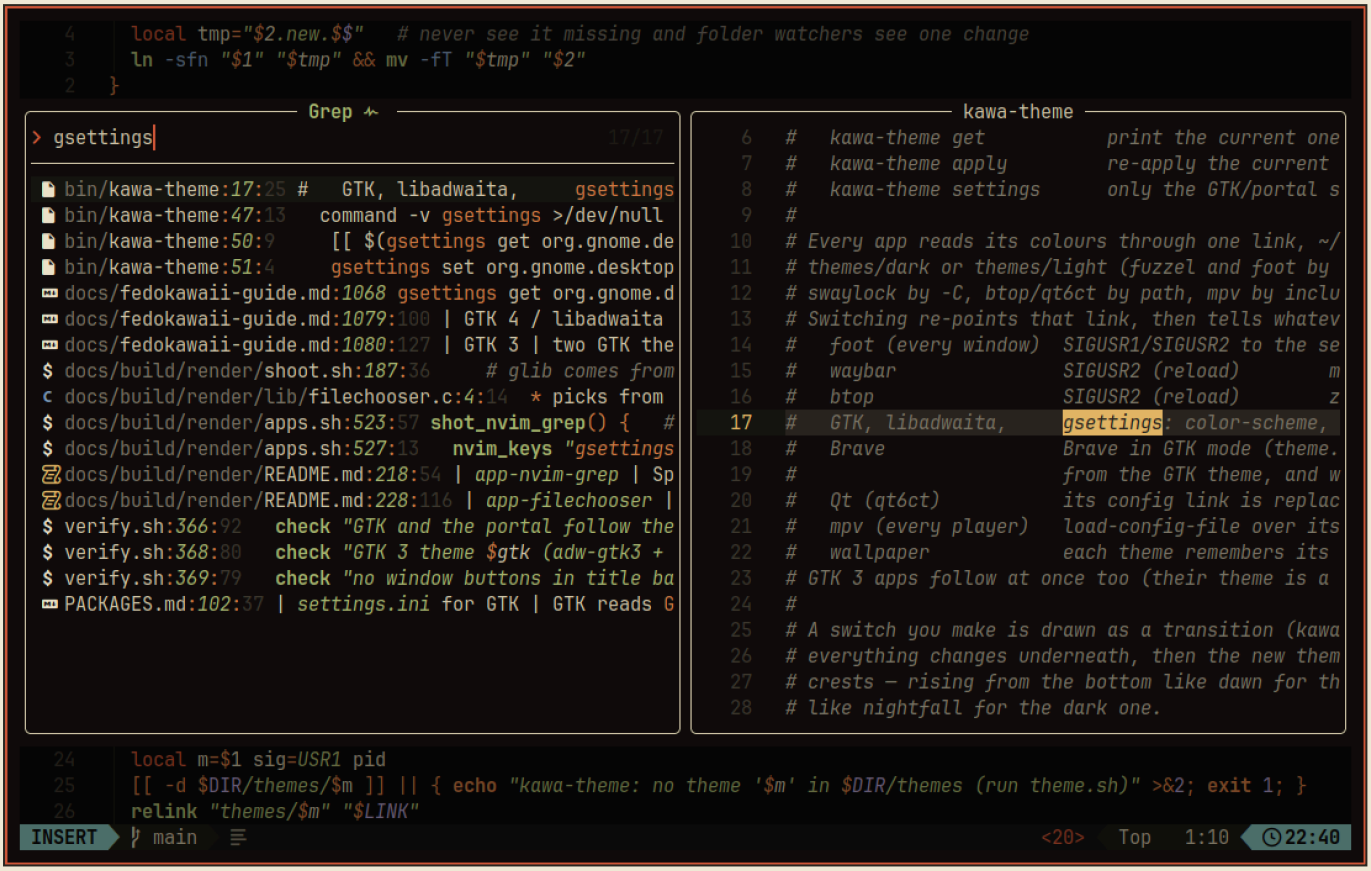


The key panel after Space: the groups that can follow, in the palette.

Keys	Does
Space Space · Space f f	find a file in the project
Space / · Space s w	search the project's text · the word under the cursor (or the selection) everywhere
Space , · Space f r	the open buffers · recent files
Space f c · Space f n	a file of neovim's own config · a new empty file
Space e	the file tree
Space g g	lazygit, in a floating window (§11.6)
Space g b · Space g f · Space g l	the history of this line · of this file · of the project
Space b d	close the buffer
Space - · Space | · Space w d	split below · split right · close the window
Space c f	format the file, or the selection
Space c a · Space c r	code actions · rename a symbol everywhere
Space c d · Space x x	the problems on this line · all problems, in a list
Space s r	search and replace across files
Space s k · Space s h	search the keys · the help pages
Space s " · Space s u	the registers · the undo history
Space u w · Space u s · Space u l	line wrap · spelling · line numbers, on or off
Space u f · Space u F	formatting on save, on or off: everywhere · this file
Space q s · Space q q	restore the last session in this folder · quit everything
Space l · Space c m	plugins (Lazy) · tools (Mason)
Space ?	the keys that belong to this buffer only

Search and replace across a project

Space / searches the text of every file in the project as you type (with ripgrep: what `.gitignore` excludes and hidden files are skipped — Alt+H and Alt+I bring them in). Each result shows the file, the line and the match; the preview shows the place around it, and Enter goes there.



Space /: every line in the project containing the typed text, the place previewed beside it.

To change text in many files, Space s r opens **grug-far**, a replace form in a window of its own:

1. Type what to find on the *Search* line. Matches from every file appear underneath as you type.
2. Move down to *Replace* (Esc, then `j`, or Tab in Normal mode) and type the new text. The results show each line as it will be — nothing is written yet.
3. *Files Filter* starts with the current file's extension (`*.sh` when you came from `setup.sh`; empty when you came from a script without one, such as those in `bin/`): clear it to search every file, or write your own (`*.md`). *Flags* takes ripgrep's options: `-i` to ignore case, `-w` for whole words, `--hidden` for hidden files.
4. In Normal mode, `\r` — backslash, then r; backslash is LazyVim's *local leader* — writes every replacement into the files. `g?` lists the other keys, `\c` closes the form.

The classic way works too: Space /, then Ctrl+Q puts every match in the *quickfix list* — neovim's list of places to step through — and `:cdo s/old/new/g | update` replaces in each listed line and saves each file. `]q` and `[q` step through the list; Space x q shows it.

Tip: run a project-wide replace in a git repository with everything committed first: lazygit then shows exactly what changed, and discarding takes it all back (§11.6).

Language servers

A *language server* is a helper program that understands one language: where a name is defined, where it is used, what is wrong. Neovim talks to it and shows the answers — a problem is underlined, with a short note at the end of the line and a sign in the margin. LazyVim brings one for Lua (the language of neovim's config) from the start; others come with *extras* and Mason (below). Space c l shows which servers serve the current file.



A language server at work: a problem underlined with its note, and the documentation that K shows.

Keys	Does
`gd` · `gr`	go to where the name under the cursor is defined · list every place it is used
`K`	the documentation of the name under the cursor (Ctrl+F · Ctrl+B scroll it)
`gK` · Ctrl+K in Insert mode	the parameters of the function being called
Ctrl+O	back to where you were before `gd`
Space c a	code actions: the fixes the server offers here
Space c r	rename the symbol everywhere it is used
`]d` · `[d`	next · previous problem (`]e` `[e` errors only, `]w` `[w` warnings)
Space c d	the full text of the problems on this line
Space x x · Space s d	every problem, in a list · in the picker
Space s s	jump to a function, heading or other symbol in this file
Space u h	the grey hints the server writes into the code (types, parameter names), on or off

For this desktop's shell scripts:

1. `:Mason`, type `/bash` and Enter to find **bash-language-server**, press `i`. (It is an npm package; `:checkhealth mason` says whether npm is found.)
2. Open a shell script. The server checks it with ShellCheck (installed with the desktop: the server finds it by itself), completes variable names and finds a function's definition with `gd`.
3. Try it: a line `echo $1` gets a note — *Double quote to prevent globbing and word splitting*, ShellCheck's SC2086, the quoting rule of §11.2; `echo "$1"` clears it. Every finding has a page that explains it: shellcheck.net/wiki/SC2086.

Completion

While you type, a menu offers what could come next: names from the language server, file paths (after `./`, `~/` or `/`), snippets (small templates with blanks to fill in, such as a whole `for` loop), and words from the open files. A box beside it shows the

documentation of the highlighted entry.

Keys in Insert mode	Does
↑ ↓ or Ctrl+P · Ctrl+N	move in the menu
Enter · Ctrl+Y	take the highlighted entry
Ctrl+E	close the menu and keep typing
Ctrl+Space	open the menu yourself; again for the documentation
Tab · Shift+Tab	in an inserted snippet: to its next · previous blank to fill in
Ctrl+B · Ctrl+F	scroll the documentation

On the command line (after `:`), Tab completes commands and file names.

Formatting

LazyVim tidies a file's layout — indentation, spacing, line breaks — **each time you save it**, when it has a formatter for the type (conform.nvim runs them; stylua, for Lua, comes installed). Shell scripts are the exception here: `after/ftplugin/sh.lua` turns formatting on save off for them, because shfmt would re-indent and re-align them, this desktop's own scripts among them.

Keys or command	Does
Space c f	format now: the file, or the selection — a shell script too, when you ask
Space u f	formatting on save on or off, everywhere
Space u F	the same for this file only (shell scripts start at off)
`:ConformInfo`	which formatters apply to this file, and whether they are installed

Extras and Mason: adding a language

Language support comes in *extras*: `:LazyExtras` (or `x` on the start screen) lists them; move to one, press `x` to switch it on or off, and restart neovim. The choice is kept in `~/.config/nvim/lazyvim.json`, which is yours.

Extra	Brings
`lang.markdown`	for notes: a language server for links and headings, Markdown checks, headings and tables drawn in place (Space u m), a live preview in the browser (Space c p)
`lang.python` · `lang.json` · `lang.yaml` · `lang.toml`	each language's server, and its formatter where it has one
`coding.mini-surround`	the `gs` keys for quotes and brackets (above)

The servers and tools then install through **Mason** (`:Mason`, or Space c m), which downloads them into neovim's data folder — binaries, PyPI and npm packages alike (Node.js is installed). In the Mason window: `i` install, `u` update, `U` update all, `X` remove, Ctrl+F show one language's tools only, `g?` help.

Keeping it updated and healthy

`update` already runs `Lazy! sync`, which brings every plugin to its newest version and records the versions in `~/.config/nvim/lazy-lock.json`. Inside neovim, `:Lazy` (Space l) shows them: `U` update, `S` sync, `R` restore the versions in the lock file — the way back when an update breaks something. Mason's tools are **not** part of `update`: open `:Mason` and press `U` now and then. Space L shows LazyVim's changelog. `:checkhealth` explains anything that does not work: every plugin and tool reports OK, WARNING or ERROR, with advice. `:LazyHealth` loads every plugin first, so none is skipped.

A daily workflow

1. **Super+Return**, the project's folder (`~/repos/notes` — a folder name alone changes into it, §11.2), then `nvim`.
2. On the start screen, `s` brings back the files you had open here last time; or Space Space for a file, `r` for a recent one.

3. Work. Ctrl+S saves; Space / finds anything in the project; Shift+H and Shift+L move between the open files; `gd` and `K` explain code.
4. Space g g: read the diff, stage, commit, push (§11.6); `q` returns to the editor.
5. Space q q quits. The session of this folder is saved by itself, ready for step 2 tomorrow.

A learning path

Fifteen minutes a day beats an afternoon a month: the keys move into your hands only by use.

When	Do	Why
day 1	`nvim --clean +Tutor`, about 30 minutes; then, in the same neovim, `:Tutor vim-02-beginner` (neovim 0.11.3 and later; Fedora 44 has 0.12). Inside LazyVim `:Tutor` is switched off	the official lessons, in plain neovim without LazyVim's changes, exactly as written (`vimtutor` belongs to Vim, which is not installed)
week 1	the first hour above, again each day: moving, typing, `x`, `dd`, `u`, `:w`, `:q`	until you stop reaching for the mouse and the arrows
week 2	the grammar: `dw`, `cw`, `ciw`, `ci"`, `dt)` — and `.`	every verb you know multiplies with every motion
week 3	Space Space, Space /, Space e, buffers and splits	moving between files as quickly as within one
week 4	visual block, `:%s`, one macro a day on real work	the keys that save hours
afterwards	one new key a week, from the Space panel or Space s k	small steps stay learnt

Pitfalls.

- LazyVim changes a few keys of plain Vim: `s` and `S` are flash (Vim's `s` is `cl`), Shift+H and Shift+L switch buffers (in Vim they move to the top and bottom of the screen), Ctrl+H/J/K/L move between windows, and `K` shows documentation. Tutorials written for Vim mean the plain keys; `nvim --clean` has them.
- With the completion menu open, **Enter takes the highlighted entry** instead of starting a new line. Ctrl+E closes the menu first.
- Markdown, plain text and commit messages wrap long lines — `j` and `k` then move by screen line — and check spelling: `]s` · `[s` jump between mistakes, `z=` offers corrections, `zg` adds the word to your list, Space u s turns the checking off.
- Hidden files do not show in Space Space or Space / — the dotfiles of a dotfiles repository included. Alt+H in the picker, `H` in the tree.
- `lang.markdown` also corrects Markdown files on save where its checker finds problems. For a file whose layout must stay as it is — this handbook's source, say — Space u F turns it off.
- Do not set `background` yourself, and leave Space u b alone: neovim learns light or dark from foot and changes with the theme.
- Your own plugins and settings go in `lua/plugins/*.lua` and `lua/config/*.lua`; only the three files named at the start of this section are managed by Theme (`lua/config/lazy.lua` among them, so leave that one to the repository). A setting of your own, for example:

```
-- ~/.config/nvim/lua/config/options.lua — yours; LazyVim reads it at start
vim.opt.relativenumber = false   -- the margin shows plain line numbers
vim.opt.wrap = true              -- long lines wrap in every file, not only in text
```

Cheat sheet.

To	Keys
get back to Normal mode	Esc
save · quit · both · quit without saving	Ctrl+S · `:q` · `:wq` · `:q!`
undo · redo · repeat the last change	`u` · Ctrl+R · `.`
move by character · word · to the line's start, end	`h` `j` `k` `l` · `w` `b` `e` · `0` `$`
top · bottom of the file · line 42	`gg` · `G` · `42G`

To	Keys
half a screen down · up	Ctrl+D · Ctrl+U
jump to anything on screen	`s`, a letter or two, the label
back · forward along your jumps	Ctrl+O · Ctrl+I
type before · after · at the line's start · end · on a new line	`i` · `a` · `I` · `A` · `o` `O`
delete a character · a line · to the line's end	`x` · `dd` · `D`
change a word · inside quotes · inside brackets	`ciw` · `ci"` · `ci(`
copy a line · a paragraph · paste	`yy` · `yip` · `p` `P`
indent · comment out a line	`>>` · `gcc`
move a line down · up	Alt+J · Alt+K
select characters · lines · a block	`v` · `V` · Ctrl+V
search · next · previous · the word under the cursor	`/text` · `n` · `N` · `*`
replace in the whole file	`:%s/old/new/g` (`gc` to confirm each)
record · stop · play a macro	`qa` · `q` · `@a`, `@@`, `10@a`
open the link under the cursor in Brave	`gx`
fold · unfold · open every fold	`zc` · `zo` · `zR`
find a file · text · an open buffer	Space Space · Space / · Space ,
the file tree · lazygit	Space e · Space g g
previous · next buffer · close it	Shift+H · Shift+L · Space b d
split below · right · move between · close	Space - · Space | · Ctrl+H/J/K/L · Space w d
definition · uses · documentation · rename	`gd` · `gr` · `K` · Space c r
next problem · all problems	`]d` · Space x x
format · code action	Space c f · Space c a
replace across the project	Space s r
every key	Space, then wait · Space s k
plugins · tools · diagnosis	`:Lazy` · `:Mason` · `:checkhealth`

Read more: inside neovim, `:help` — `:help usr_toc` is the user manual chapter by chapter, and `:help text-objects`, `:help registers`, `:help pattern` and `:help lsp` go straight to a topic (the same pages are at neovim.io/doc/user). LazyVim's keys, plugin by plugin: lazyvim.org/keymaps, with the extras under *Extras* on the same site. Mason: github.com/mason-org/mason.nvim.

11.6 lazygit — git



lazygit in a repository with changes: files, the diff, branches and commits.

What git is

git keeps the history of a folder. The folder becomes a *repository*: you change its files as usual, and whenever a set of changes is worth remembering, you record a *commit* — a snapshot of those changes with a one-line message saying why, your name and the time. Commits line up into a history you can read, compare and return to; nothing committed is lost to later edits. Before committing you choose what goes in: *staging* a change puts it on the tray for the next commit, so one commit can hold the alias you added and not the half-finished experiment in the same file. git keeps all of this in a hidden `.git` folder inside the repository.

A *branch* is a named line of commits. The usual one is called `main`; a second branch lets you try something — a new bar layout, a rewrite of a chapter — without disturbing it, then *merge* it in if it works or delete it if not. A *remote* is another copy of the same repository somewhere else: a private repository on GitHub or Codeberg, or another computer. *Push* sends your new commits there; *pull* brings in the commits made elsewhere — on your other machine, say. For dotfiles and notes, that means every change is recorded with its reason, any old version is a few keys away, and the same files live on every machine you use.

A small history, as `git log --oneline` prints it in a terminal — newest first, one commit per line:

```
a0044fc notes: fonts to try
cd74973 notes: a clock with seconds
7398591 notes: first ideas for the bar
```

The seven characters in front are the start of each commit's *hash*, its unique name; `git show a0044fc` prints that commit in full. lazygit's *Commits* panel is the same list.

Term	In one line
working tree	the files as they are on disk now, with edits not yet committed
staging area (index)	the tray: exactly what the next commit will hold

Term	In one line
hash	a commit's unique name, a long hexadecimal number; its first seven characters are enough
`HEAD`	"where you are now": normally the newest commit of the branch you are on
upstream	the remote branch yours follows, such as `origin/main`: push and pull go there
fetch	download the remote's new commits without touching your files or your branch
rebase	replay your commits on top of another branch, as new commits (*Branches*, below)
reflog	git's private record of every place `HEAD` has been, which makes undo possible (*Recipes*)

Read more: chapters 1–3 of the free book Pro Git.

Once, before the first commit

```
git config --global user.name "Your Name"        # written into every commit you make
git config --global user.email you@example.com   # likewise: the address your Git host knows
git config --global init.defaultBranch main      # new repositories start on a branch called main
git config --global pull.rebase false            # pull joins two diverged histories with a merge
```

The settings go to `~/.gitconfig`, and `git config --global --list` prints them back (git writes the names in lower case):

```
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
pull.rebase=false
```

Without the fourth line, a `pull` on a branch that has new commits on both sides stops with `fatal: Need to specify how to reconcile divergent branches.` With it, git joins the two lines with a merge commit, as drawn under *Branches* below.

Opening it

Type `lazygit` inside any git repository, press **Space g g** in neovim (it opens the repository of the current file), or use `:` then `lazygit` in yazi. Started in a folder that is not a repository, it asks *Create a new git repository?* — `y`, then Enter for git's default branch name: the way to start keeping a folder under git (`mkdir -p ~/repos/notes && cd ~/repos/notes && lazygit`). A repository that already exists on a website comes with `git clone URL ~/repos/NAME`. Everything lazygit does is an ordinary git command, listed in its command log, so `git`, the prompt (§4.6) and neovim see the same thing.

The screen

Five panels on the left; on the right the *main view*, showing whatever the selected line stands for — a file's changes, a commit, a branch's log.

Panel	Key	Lists	More tabs, with [ ]
Status	1	the repository and branch; ↑ commits not pushed yet, ↓ commits waiting to be pulled	—
Files	2	every changed file, as a folder tree	Worktrees, Submodules
Branches	3	the local branches; the one you are on is marked `*`	Remotes, Tags
Commits	4	the history of the current branch, newest at the top	Reflog
Stash	5	changes put aside	—

A file's status letters are git's own, the same `git status --short` prints. Each file has two columns: the left one for the staged part, the right one for the part not yet staged.

Letters	Means
`??`	new and not yet tracked: git has never stored it
`M` · `A` · `D` · `R`	modified · added · deleted · renamed
`UU`	in conflict (*Resolving a conflict*, below)

Four files, as `git status --short` shows them:

```
MM local.sh     left M: a change staged · right M: another change, not staged yet
 M bar.md       a change, not staged (the left column is blank)
A  ideas.md     a new file, staged whole
?? todo.md      new, untracked
```

Keys	Does
1 … 5 · 0	jump to a panel · to the main view
← → or h l (Tab, Shift+Tab)	previous · next panel
↑ ↓ or k j · `<` `>`	move in the panel · to its top, bottom
Enter · Esc	go into the selected item (a file's lines, a commit's files) · back out
/ · n N	search or filter the panel · next, previous match
?	every key of the panel you are in, with what it does
`+` · `_`	a bigger main view (half, then the whole screen) · back
:	run any command — `git log --stat`, say
q	quit

Read more: lazygit's key reference, the same list **?** shows, panel by panel.

The colours here

The repository's `config.yml` names the terminal's colours instead of fixing its own, so lazygit switches with the theme along with foot:

What	Colour
the frame of the panel you are in	vermilion, bold — like the frame of the focused window
the other frames and their titles	a quiet warm grey (🔲 `#877f6f`, one value that reads on ink and on paper)
the frame while you search with `/`	yamabuki
the key hints along the bottom	ai blue
the selected line	the terminal's "black": a shade off the ground, on either theme
unstaged changes	vermilion
a commit's hash	vermilion: only here, not pushed yet · yamabuki: pushed · matcha: already in `main`

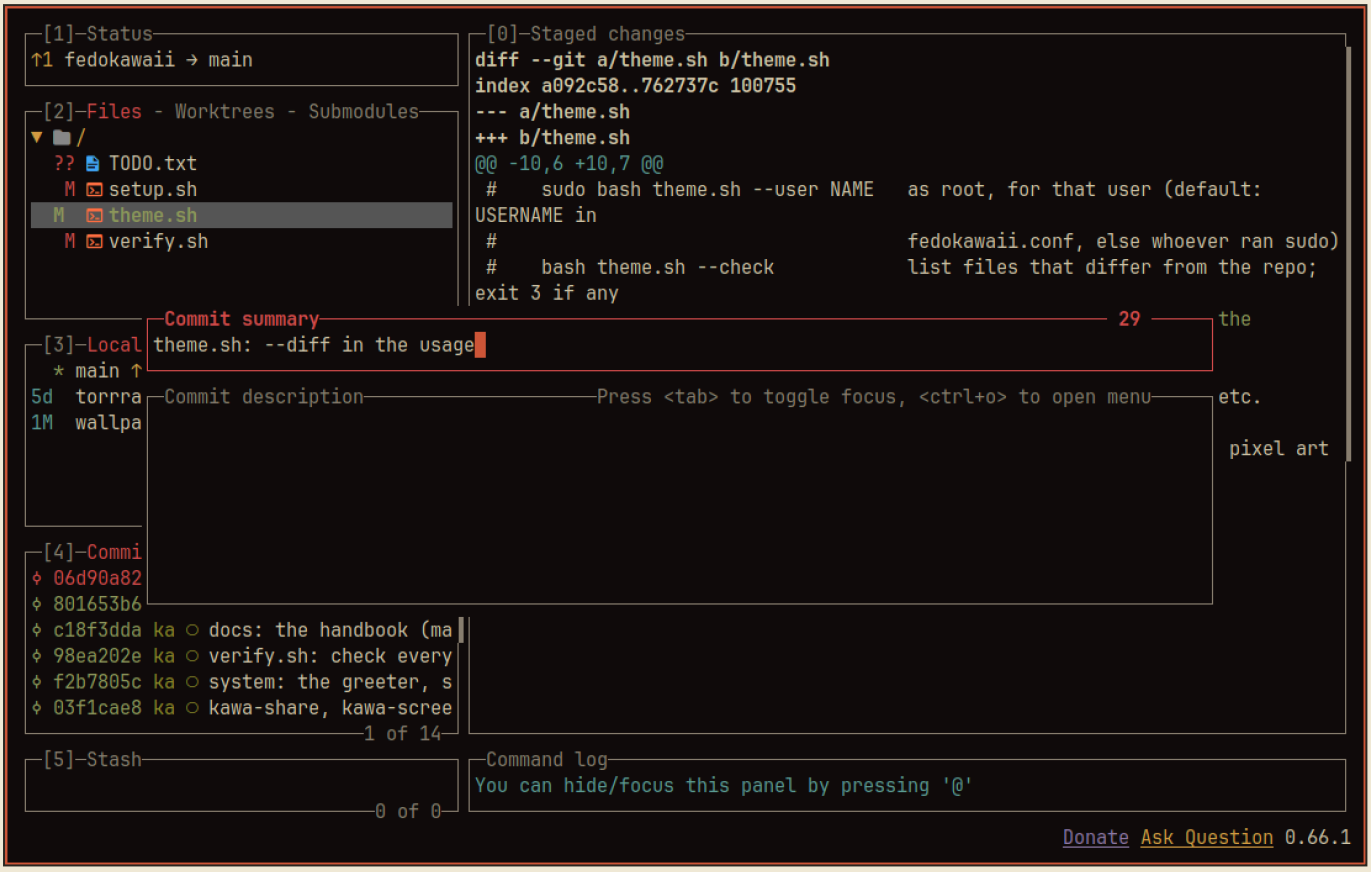
Frames have square corners (`border: single`), files carry Nerd Font icons, and `overrideGpg: true` keeps lazygit on screen while a signed commit is made, instead of handing the terminal to git — which suits a key whose passphrase gpg-agent already holds. Nothing here turns commit signing on; until you do, the setting has no effect. Inside neovim (Space g g), Snacks lays a colour file made from the editor's colours over this one, so the frames and the selected line follow the editor there.

A first commit, step by step

Say you keep notes in `~/repos/notes` and have just written `ideas.md`.

1. Start `lazygit` in the folder (or Space g g in neovim). *Files* lists `ideas.md` with `??`: git does not know the file yet.
2. With the file selected, **Space** stages it: its letters turn to `A`, in green. The main view shows what will go into the commit.

3. **c** opens the commit box. Type a short summary — what changed and why, in one line: `notes: first ideas for the bar` — and press **Enter**. For more, **Tab** moves to the description box below (Enter there starts a new line); Tab again goes back, and Enter in the summary commits both.
4. The file leaves *Files* — there is nothing left to commit — and *Commits* shows the new commit at the top, its hash in vermilion: so far it exists only on this machine.



The commit box: a one-line summary above, the description below, the staged file behind them.

If nothing is staged when you press **c**, lazygit asks whether to commit every change.

The same commit typed in a terminal — this is what lazygit runs for you:

```
git status --short                              # ?? ideas.md
git add ideas.md                                # stage it (Space)
git commit -m "notes: first ideas for the bar"  # commit what is staged (c)
git log --oneline -1                            # the new commit, at the top
```

Tip: before trying anything unfamiliar, commit or stash what you have. With nothing uncommitted, a step that goes wrong can be taken back with **z**, or from the reflog (*Recipes*, below).

Staging part of a file

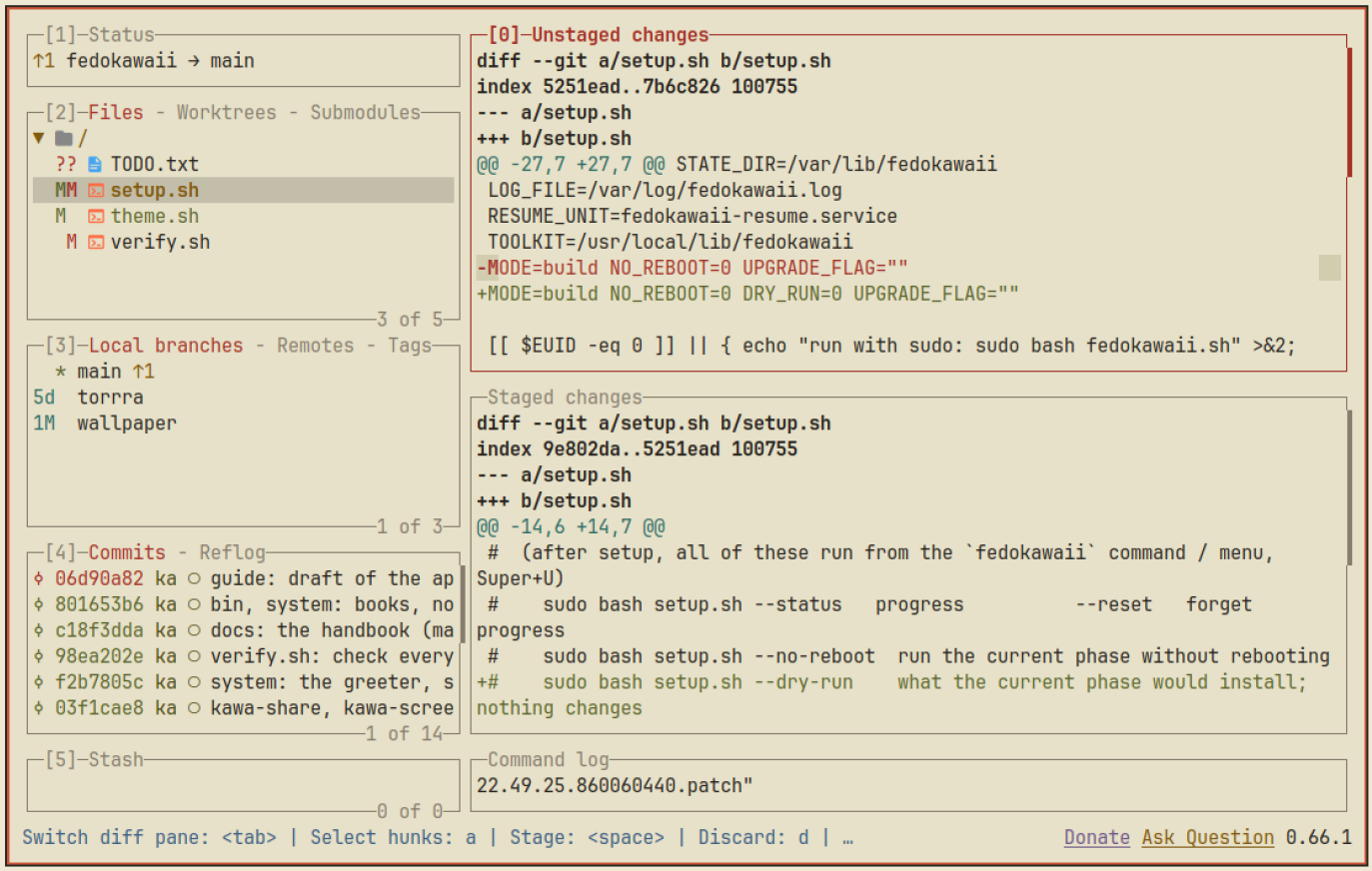
A commit should hold one change you can describe in one line. When a file holds two — a fixed alias and a half-written function — stage them apart:

1. In *Files*, select the file and press **Enter**. The main view takes over, showing the changes not yet staged, with one *hunk* (a block of changed lines) selected.
2. ↑ ↓ move from hunk to hunk; **a** switches to single lines and back; **v** starts a range.
3. **Space** stages the selection, which moves to the *Staged changes* pane beside it. **Tab** switches between the two panes; Space in the staged one unstages.
4. **Esc** returns to the files, and **c** commits only what is staged. The rest stays as it is, for the next commit.

In git's own notation that file — `local.sh`, say — has two hunks. Each starts at an `@@` line that says where it sits (`-3 +3`: line 3, before and after); `-` lines go, `+` lines come:

```
@@ -3 +3 @@
-alias ll='eza -la'
+alias ll='eza -la --git'
@@ -23,0 +24,3 @@
+backup() {
+  # half-written
+}
```

Stage the first hunk only, and *Files* shows `MM local.sh`: one change staged, one not.



Staging by hunk: the unstaged changes on one side, the staged ones on the other, the selected hunk highlighted.

In this view **d** removes the selected change **from the file** (in the staged pane, it unstages it); **E** opens the hunk in neovim to edit it before staging. Neovim can stage too, from inside the file: the coloured bars in the margin mark changed lines, `]h` · `[h` jump between hunks, Space g h s stages the hunk under the cursor, Space g h r resets it, and Space g h p shows what changed.

Commit, amend, undo

Keys (in Files)	Does
c	commit what is staged
C	commit, writing the message in neovim — save and quit (`:wq`) to finish
A	amend: add what is staged to the last commit (it asks first)
w	commit without running the repository's hooks — scripts a repository can run before each commit, a code formatter say; a new repository has none switched on
z · Z	undo · redo the last git action

z reads git's own record of actions (the *reflog*) and asks before acting. Undoing a commit takes the commit back and leaves its changes staged; a checkout, a reset or a rebase is undone too. It does not bring back changes discarded with **d** that were never committed, nor a stash entry, nor a branch you created; it cannot take back a push; and in the middle of a rebase it does nothing — **m** → abort is the way out there.

Tip: a typo in the last commit's message: select it in *Commits* and press **r**. A file forgotten from it: stage the file and press **A** in *Files*.

Stash

Stashing puts uncommitted changes aside and leaves the files as they were at the last commit — for when you need to switch branches or pull in the middle of something.

Keys	Does
s (Files)	stash the changes to tracked files; it asks for a name — Enter alone will do
S (Files)	stash options: *including untracked files*, only the staged changes, only the unstaged …
Space (Stash, 5)	apply the entry: the changes come back, the entry stays
g (Stash)	pop: the changes come back, the entry goes
d (Stash)	drop the entry

A typical use, in three steps — halfway through an edit, you want the commits from your other machine first:

1. *Files*: **s**, Enter. The files go back to the last commit; the edit waits in *Stash* (5).
2. **p** pulls.
3. *Stash*: select the entry, **g**. Your edit is back, on top of the pulled commits. If both changed the same lines it stops with a conflict (below), and the entry stays in the list until you drop it with **d**.

Read more: Pro Git, Stashing and Cleaning.

Branches

Keys (in Branches, 3)	Does
n	a new branch from the selected one, and switch to it
Space · `-`	switch to the selected branch · back to the previous one
M	merge the selected branch into the one you are on (then *Regular merge*)
r	rebase the branch you are on onto the selected one (then *Simple rebase*)
R · d	rename · delete (it asks: the local branch, the remote one, or both)
f	fast-forward the selected branch to its remote
]	the *Remotes* tab: the branches of other copies

Merge or rebase. Both bring the work of one branch into another; they differ in the history they leave. Say `main` gained one commit while the branch `bar-layout` gained two (`git log --oneline --graph --all`; lazygit draws the same graph):

```
* fb0d748 (bar-layout) bar: smaller clock
* 10519bf bar: try two rows
| * 985820e (HEAD → main) notes: ideas for the bar
|/
* 6c84175 notes: start
```

	Merge (M)	Rebase (r)
what it does	joins the two lines with a new *merge commit*	replays your branch's commits, one by one, on top of the other branch
the history after	both lines kept, with a fork and a join	one straight line
your commits	unchanged, same hashes	rewritten, with new hashes

	Merge (M)	Rebase (r)
on a branch others share	safe	only for commits nobody else has

After merging `bar-layout` into `main`, the join shows:

```
*   a2b2184 (HEAD → main) Merge branch 'bar-layout'
|\
| * fb0d748 (bar-layout) bar: smaller clock
| * 10519bf bar: try two rows
* | 985820e notes: ideas for the bar
|/
* 6c84175 notes: start
```

Rebasing `bar-layout` onto `main` instead gives one line — and its two commits have new hashes, because they now sit on a different parent:

```
* 124d160 (HEAD → bar-layout) bar: smaller clock
* 46ba52e bar: try two rows
* 985820e (main) notes: ideas for the bar
* 6c84175 notes: start
```

Merging that into `main` then needs no merge commit: `main` simply moves up to `124d160`, a *fast-forward*.

For an experiment that worked on its own branch: switch to `main` (Space), select the experiment's branch, **M**. To bring `main`'s new commits under a branch you are still working on: stay on that branch, select `main`, **r**.

Read more: Pro Git on branching and merging and on rebasing.

Pull and push

A repository needs a remote before it can push. Create an empty private repository on GitHub or Codeberg, then in *Branches* press **]** for *Remotes*, **n**, name it `origin` and paste the address the site shows (an SSH address needs your SSH key added to the site; an HTTPS one asks for a token instead of a password).

Keys	Does
P	push the branch. The first time, lazygit asks which remote branch to track: `origin main`
p	pull: fetch the remote's new commits and merge them
f (Files)	fetch only: see what is new (↓ in *Status*) without merging it yet

lazygit also fetches by itself every minute while it is open (`git.autoFetch`), so the counts in *Status* are current. Reading them, in a repository called `notes`:

Status shows	Means	Do
`✓ notes → main`	the same commits here and on the remote	nothing
`↑2 notes → main`	two commits here the remote does not have	**P**
`↓1 notes → main`	one commit on the remote, not here yet	**p**
`↓1↑1 notes → main`	both sides moved on: the branch has *diverged*	**p** (it merges), then **P**

After amending or rebasing commits that were already pushed, **P** says the branch has diverged and offers a *force push*: Enter replaces the remote's history with yours. lazygit pushes with `--force-with-lease`, which still refuses if the remote holds commits you have not fetched. Do it only on a branch nobody else uses — on a one-person dotfiles repository, that is every branch.

Resolving a conflict

A merge, rebase or pull stops with a *conflict* when both sides changed the same lines — the same alias edited on two machines, say. Nothing is lost; git waits for you to choose.

In the file itself a conflict looks like this — here `main` and the branch `laptop` both changed one alias:

```
<<<<<<< HEAD
alias ll='eza -lah'
=======
alias ll='eza -la --git'
>>>>>>> laptop
```

Above `=======` is your side (`HEAD`, the branch you are on), below it the incoming one. The fix is the file as it should be, with the three marker lines gone — here perhaps `alias ll='eza -lah --git'`.

1. lazygit reports the conflicts; *Files* lists the files concerned, marked `UU`.
2. Select one and press **Enter**. The main view shows each conflict: your side (`HEAD`), then the incoming side, between git's `<<<<<<<`, `=======` and `>>>>>>>` lines.
3. ↑ ↓ choose a side and **Space** keeps it; **b** keeps both; ← → go to the next or previous conflict; **z** takes a choice back. Or **e** edits the file in neovim: write the lines as they should be, delete the three marker lines, `:wq`.
4. When a file has no conflict left, lazygit stages it. When none remain, it asks *All merge conflicts resolved. Continue the merge?* (or the rebase) — Enter. A merge then commits; a rebase moves on to its next commit, which may stop again.

m offers *continue* and *abort* at any time; abort puts everything back as it was before the merge or rebase began.

Rewriting history: interactive rebase

Commits not pushed yet — vermilion hashes — can be tidied before anyone sees them: joined, reworded, reordered, dropped. Select a commit in *Commits*:

Keys	Does
s	squash it into the commit below; both messages are kept, joined
f	fixup: into the commit below, its own message dropped
r · R	reword its message · reword it in neovim
d	drop the commit, and its changes with it
Ctrl+J · Ctrl+K	move it down · up the list
A	amend it — any commit, not only the last — with what is staged
e	stop the history at this commit to change it: edit the files, stage, **A**, then **m** → continue
i	start a rebase of the branch's commits: mark several with s, f, d (or p, to keep), then **m** → continue runs them all

Each single key runs at once, after asking. While a rebase is stopped, **m** → abort puts everything back; once it has finished, **z** undoes it.

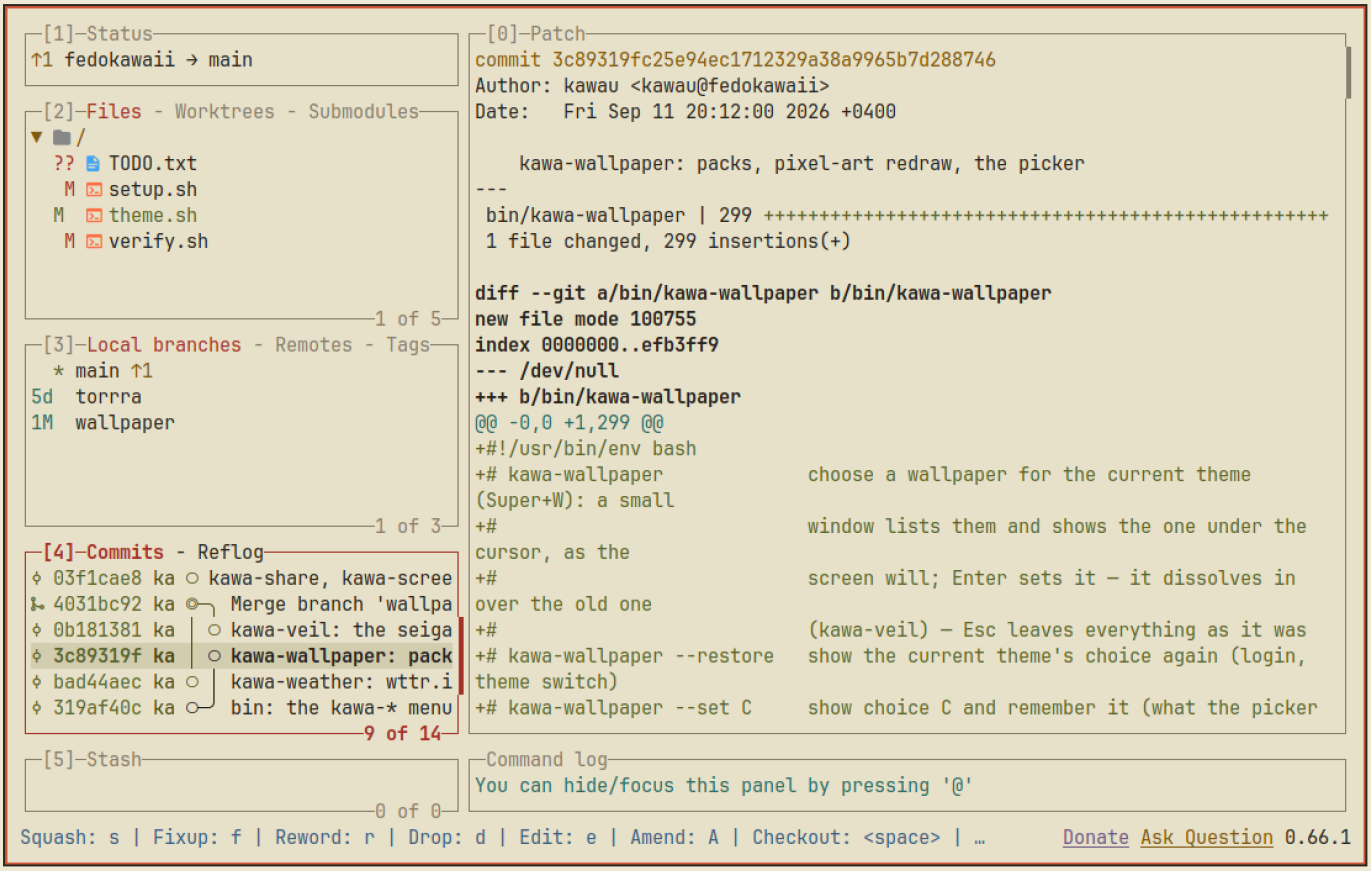
An example: four commits, newest at the top, where `typo` only fixed a slip in the commit below it. Select `typo` and press **f**:

Before	After
`606c48b bar: seconds in grey`	`17912ba bar: seconds in grey`
`7d19c63 typo`	—
`c40a45e bar: show seconds`	`4bf80c7 bar: show seconds`, now with the slip fixed
`bb03eba bar: smaller clock`	`bb03eba bar: smaller clock`

The commit below takes in the change and keeps its own message. Every commit from there up gets a new hash, because what it builds on changed; the oldest one is untouched.

Read more: Pro Git, Rewriting History.

History and diffs



The commit history: the branch graph beside the hashes, one commit's changes on the right.

Keys	Does
4, then ↑ ↓	the commits; the main view shows each one's message and changes
Enter · Enter · Esc	the files a commit changed · one file's diff · back
/	search the messages
Ctrl+S	filter the history: only the commits touching a path (a file's history), or by one author
W	compare: **W** → *Diff* the selected commit, move to another, and the main view shows the difference; **Esc** (or **W** → *Exit diff mode*) ends it
a (in Status, 1)	every branch, as a graph
Ctrl+L	log options: order, graph
] (in Commits)	the *Reflog* tab: every position the branch has been in, even ones no longer in the history

Cherry-pick

To copy a commit from one branch onto another without merging the whole branch: in *Branches*, press Enter on the branch that has it, to see its commits; select the commit and press **C** (for several: **C** on each, or a range with **v**); switch to the branch where it should land; in *Commits*, **V** pastes it there as a new commit. Esc or Ctrl+R forgets the copied set.

Recipes

I made a mess — go back.

The mess	The way back
one file's uncommitted changes are wrong	*Files*: select it, **d** → *Discard all changes*. Gone for good

The mess	The way back
everything uncommitted is wrong, but might be useful later	**s**: it waits in panel 5 until you **g** it back or **d** it
the last commit is wrong, not pushed	**z** (its changes come back as staged), or fix the files and **A**
an older commit is wrong, already pushed	*Commits*: select it, **t** (revert) — a new commit that undoes it, safe to push
a merge or rebase is going badly	**m** → abort
a rebase or reset has finished, and the result is wrong	**z**; or *Commits* → **]** *Reflog*, select the line from before, **g** → *Hard reset*
everything since the last commit must go	**D** → *Nuke working tree* — think twice: untracked files go too

A hard reset also throws away whatever is uncommitted, so stash first. git keeps commits that dropped out of the history in the reflog for about a month (`gc.reflogExpireUnreachable`, 30 days), so a commit you can still remember is almost never really lost.

What the reflog shows after the rebase drawn under *Branches* — newest first, one line for each move of `HEAD` (`git reflog` in a terminal, the *Reflog* tab here):

```
124d160 HEAD@{0}: rebase (finish): returning to refs/heads/bar-layout
124d160 HEAD@{1}: rebase (pick): bar: smaller clock
46ba52e HEAD@{2}: rebase (pick): bar: try two rows
985820e HEAD@{3}: rebase (start): checkout main
fb0d748 HEAD@{4}: checkout: moving from main to bar-layout
```

The line below `rebase (start)` is where the branch stood before: select `fb0d748`, **g** → *Hard reset*, and `bar-layout` is back as it was.

Commit only part of a file. Files, Enter on the file, select the hunk (**a** for single lines), **Space**, **Esc**, **c** — see *Staging part of a file*, above.

Move a commit to a new branch. You committed on `main` and meant to start a branch:

1. In *Branches*, with `main` selected, **n** and a name: the new branch starts here, with the commits, and you are on it.
2. Select `main` and press **Space** to go back to it.
3. In *Commits*, select the last commit that belongs on `main`, **g** → *Hard reset*. `main` is back where it was; the commits live on in the new branch.

N (*Move commits to new branch*) does the same in one step for commits not pushed yet, and asks where the new branch should start. For a single commit from the middle: cherry-pick it to the other branch (above), then **d** drops it here.

Recommended. From inside neovim (Space g g), lazygit is set up by LazyVim itself: it takes the editor's colours, and **e** opens the file in that same neovim instead of starting a second one. Started on its own, `e` runs neovim inside lazygit's window and returns when you quit it. Commit small and often, with a message that says why: `bar: show seconds` reads better in a year than `update`.

Pitfalls.

- **z** undoes git actions, not edits you have not committed — discarded changes (**d**) are gone.
- Commits are made with your git identity: set it once (above) if git complains.
- **Space** on a *commit* checks that commit out: the files show the old state, and lazygit reports a *detached HEAD*. Space on a branch in *Branches* (or **-**) brings you back.
- Rewriting commits that are already pushed — amend, squash, reword, drop — needs a force push and breaks other copies that have them. On a one-person repository that is harmless; anywhere shared, revert (**t**) instead.

Cheat sheet.

To	Keys
move between panels · jump to one	← → · 1 … 5

To	Keys
every key here	?
stage a file · all files · single lines	Space · a · Enter, then Space
commit · in neovim · amend the last	c · C · A
discard a file's changes · everything	d · D
stash · pop it back	s · g (in Stash)
undo · redo	z · Z
pull · push · fetch	p · P · f
new branch · switch · back to the previous	n · Space · `-`
merge into this branch · rebase onto	M · r
squash · fixup · reword · drop · move	s · f · r · d · Ctrl+J, Ctrl+K
copy commits · paste them here	C · V
revert a commit · reset to it	t · g
look inside a commit · back out	Enter · Esc
a file's history	Ctrl+S, by path
continue or abort a merge or rebase	m
quit	q

Read more: lazygit's own pages on configuration and undo; git's reference manual at https://git-scm.com/docs.

11.7 bat, ripgrep, fd, fzf, eza, zoxide and tldr

Small command-line tools, each for one job; ripgrep and fd also power neovim's finders and yazi's search, fzf the clipboard window, the shell's Ctrl+R / Ctrl+T / Alt+C and yazi's **z**, zoxide the shell's `cd` and yazi's **Z**. Their keys in the shell are in §11.2.

Command	Does
`bat file`	the file with line numbers, git changes in the margin and syntax colours from the terminal's palette (so it follows the theme)
`bat -p file` · `bat -A file`	plain, no decorations · show tabs, spaces and line ends
`rg text`	search below the current folder, skipping what `.gitignore` excludes
`rg -i text` · `rg -l text` · `rg -t py text`	ignore case · only file names · only Python files
`rg -uu text`	also hidden and ignored files
`fd name`	find files whose name contains *name*
`fd -e pdf` · `fd -t d conf`	by extension · folders only
`fd -u -S +1g . ~`	files over 1 GB anywhere in your home, hidden and ignored ones included (`-u`; `-H` alone still skips what a `.gitignore` names)
`fd -e jpg -x magick {} {.}.webp`	run a command on every match
`nvim "$(fzf)"`	pick any file below here by typing part of its name, and edit it
`ll` · `lt` · `eza -lT -L 3 --git`	the folder in detail · as a tree · a deeper tree with sizes and git state
`ll -s modified` · `ll -s size -r`	newest last · biggest first
`cd proj` · `cdi` · `zoxide query -l`	jump by a word · pick in fzf · every folder zoxide knows, best first

Command	Does
`zoxide remove ~/old/folder`	forget a folder
`tldr ffmpeg` · `tldr --update`	examples · refresh the pages now

On Fedora the fd package is called `fd-find`, but the command is `fd`.

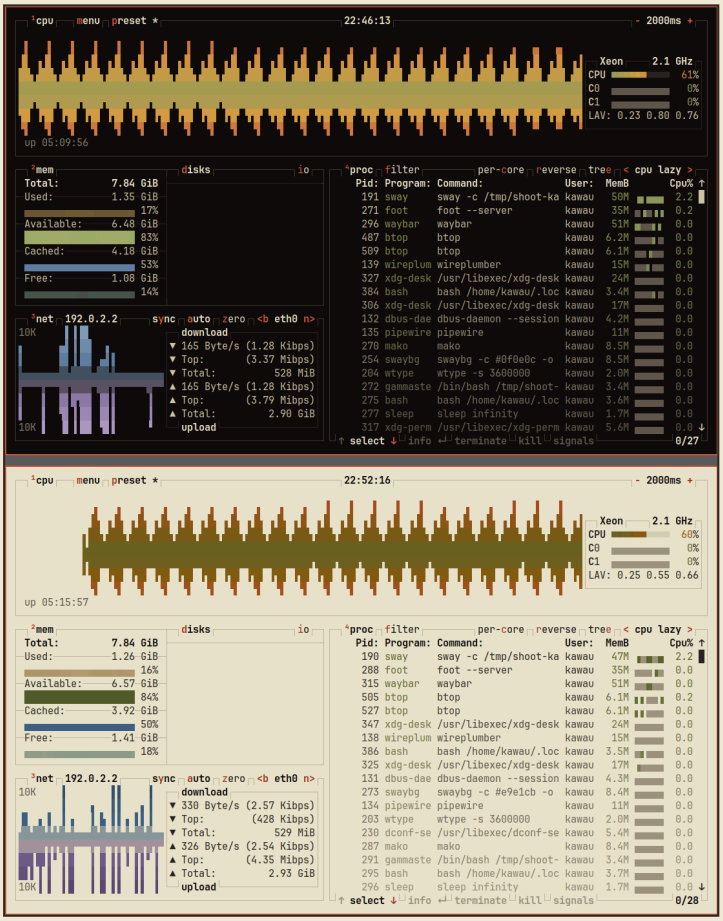
Tips. fzf matches *fuzzily*: the letters you type must appear in that order, not side by side. In Ctrl+R and every other fzf list, words separated by a space may come in any order, and a few marks narrow the search:

You type	Finds
`ssh 192`	lines holding both, in any order — `ssh admin@192.168.1.20`
`'word`	*word* exactly, its letters side by side
`^git` · `.md$`	lines that start with `git` · that end in `.md`
`!test`	everything that does **not** contain `test`
`.sh$ | .py$`	either one: `|` means *or*

zoxide ranks by *frecency* — a score of how often and how recently you went to a folder — so the folder you live in wins after a day or two; `zoxide query -ls` lists the scores.

Read more: `man rg`, `man fd` and `man fzf` (they open in neovim here), or `tldr` with any of these names.

11.8 btop — the system monitor



btop: CPU, memory, network and the process list, in the palette.

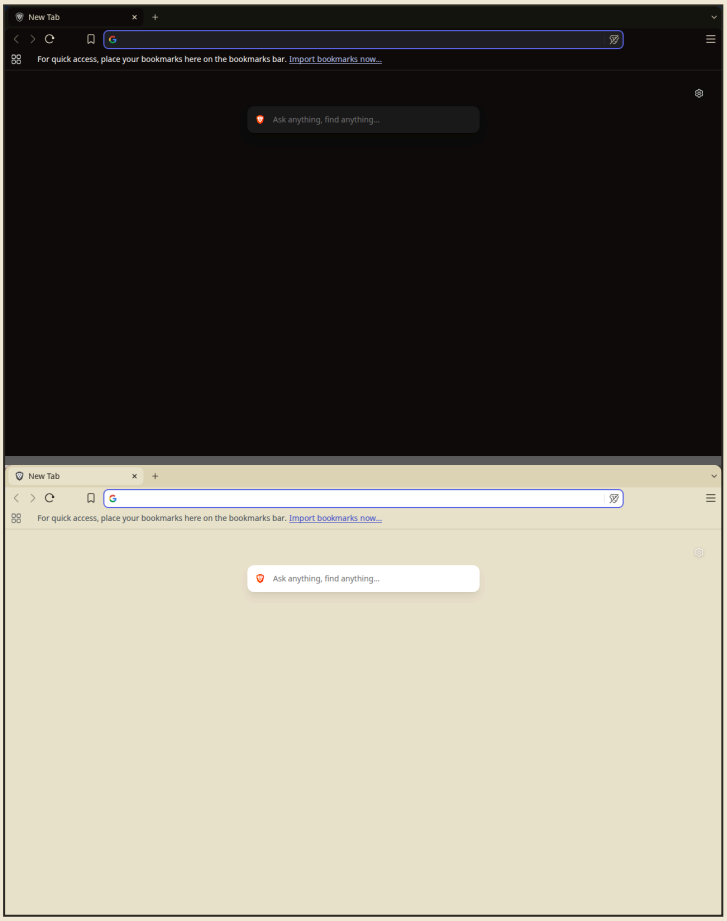
Click the **CPU** part of the bar: btop opens in a floating window. Or type `btop`.

Keys	Does
↑ ↓ · Enter	choose a process · details of it
f or /	filter the process list (start with `!` for a regular expression); Delete clears it
← → · r	sort by the previous / next column · reverse
e	process tree
t · k · s	end the chosen process (SIGTERM: a request to stop, so it can save and tidy up) · kill it (SIGKILL: stopped at once, nothing saved) · send another signal
1 2 3 4 · 5	show / hide CPU, memory, network, processes · the GPU box
d	disks in the memory box
p · m · o	cycle layout presets · menu · options
q	quit

Pitfalls. Changes made in btop's own options menu last until you close it: the config says `save_config_on_exit = False`, so `~/.config/btop/btop.conf` stays exactly as the repository has it. Make lasting changes in your working copy (`configs/home/.config/btop/btop.conf`) and apply (chapter 15). On AMD, btop reads the card through the ROCm SMI library, which is not installed: it finds no GPU, so **5** shows no GPU box at all. LACT (§11.19) shows the card instead.

Read more: btop's README; **F1** (or **?**) inside btop lists every key.

11.9 Brave Origin — the browser



Brave Origin's new-tab page on both themes: the tab strip and toolbar in the palette, no window buttons; the address bar, focused, keeps Brave's own grey (white on paper) and blue ring.

Super+B. Brave Origin is Brave with Rewards, Wallet, VPN, Leo, News and the Tor windows removed from the build — Talk, Playlist, Speedreader, the Web Discovery Project and the usage and crash reports go too; the Shields (ad and tracker blocking) remain. It is free on Linux and the only browser here (`verify` fails if another appears).

What this desktop sets up

Very little, on purpose:

- **The desktop's colours.** Brave runs in its GTK mode (Theme sets it in the profile while Brave is closed): the tab strip, toolbar, menus and dialogs take the palette from the GTK theme, the new-tab page is the theme's ground, and both follow a theme switch live. Web pages that offer a dark version follow the portal's dark / light. The address bar and Brave's own pages (settings, Shields) keep Brave's greys.
- **No window buttons** — the tab strip ends at the tab list; Super+Q closes the window.
- **File dialogs.** Upload buttons and *Save as* open the desktop's file dialog, served by the portal (§4.9).
- **Screen sharing** goes through the portal and the *share what?* menu; a tab's sound always travels, and *Also share system audio* is turned on for screens and windows (§4.10).
- **No flags file** — Chromium 140 and later choose Wayland by themselves. Whether videos are decoded by the GPU is Brave's own default; `brave://gpu` → *Video Decode* shows it.
- Links opened elsewhere — foot's URL mode, lazygit, spotify_player's sign-in — open in Brave, and magnet links clicked in Brave open in torrra (§12).

Worth checking once

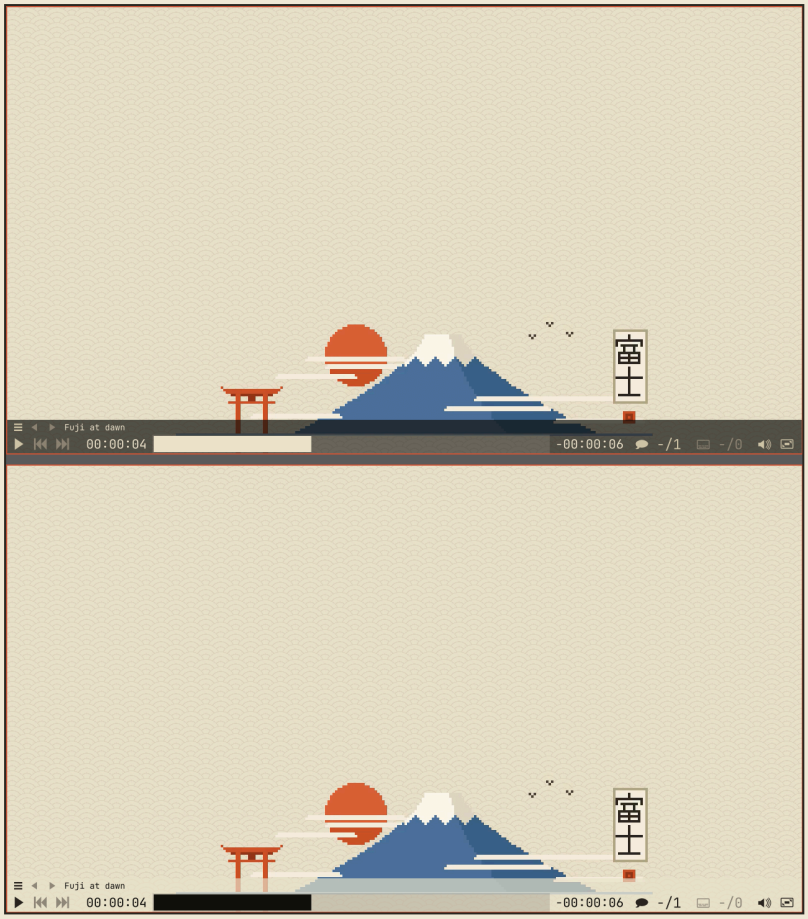
These live in Brave's own settings, are not managed here, and their names can move between versions:

- *Settings → Shields*: the defaults for blocking trackers and ads and for fingerprinting. The lion icon in the address bar changes them for one site — the first thing to try when a site misbehaves.
- *Settings → Privacy and security*: the data-collection switches. Origin removes most reporting; look rather than assume.
- *Settings → Search engine*, the password manager and sync, to taste.

Keys	Does
Ctrl+T · Ctrl+W · Ctrl+Shift+T	new tab · close tab · reopen the last closed tab
Ctrl+L · Ctrl+Tab · Ctrl+1…8 · Ctrl+9	address bar · next tab · that tab · the last tab
Ctrl+Shift+N	private window
Ctrl+J · Ctrl+H · Ctrl+F	downloads · history · find in page
F11 or Super+F	fullscreen

Pitfalls. Super+Q closes the whole window with all its tabs; Ctrl+W closes one, and Ctrl+Shift+T brings tabs back. Choosing a theme from the Chrome Web Store replaces GTK mode (Theme then leaves it be); *Settings → Appearance → Theme → GTK* brings the desktop's colours back.

11.10 mpv and yt-dlp — video



mpv with its on-screen controller in the palette.

Term	In one line
container format	the file type that holds the streams together: `.mkv`, `.mp4`, `.webm`
codec	how one stream is compressed: H.264, HEVC, VP9 or AV1 for video, AAC or Opus for sound. One container can hold different codecs
bitrate	how many bits one second of a stream takes: more is sharper and larger (**i** shows it)
hardware decoding (VA-API)	the graphics card unpacks the video instead of the CPU, which stays cool and quiet
MPRIS	the standard way, over D-Bus (the desktop's message bus), for a player to say what it plays and to take play and pause from the bar and the media keys

So a `.mkv` file (container) may hold HEVC video and Opus sound (codecs); **i** in mpv shows which, and at what bitrate.

Open a video from yazi or Brave's downloads (mpv is the default for video and sound), or type `mpv file`, `mpv folder/` or `mpv 'https://…'`. The config decodes on the GPU (VA-API) and leaves the output to mpv's own probing (Vulkan, falling back to OpenGL, so a broken driver still shows a picture); on a discrete AMD card (2 GB of video memory or more) it scales with the high-quality filters (`profile=high-quality`). It smooths motion to the monitor's refresh rate, starts at 80 % volume (up to 150 %), loads subtitles whose names resemble the video's, prefers English then Arabic for sound and subtitles, and saves screenshots as PNG in `~/Pictures/Screenshots`.

It **remembers where you stopped** — and a film watched to the end opens at the beginning next time, not on its last frame (`scripts/kawa-resume.lua`). Its on-screen text, controller and menus wear the theme's colours and switch live with it (`scripts/kawa-ipc.lua` lets `kawa-theme` reach every open player); subtitles keep paper on ink, which reads on any picture. It shows in the bar's *now playing* and answers the media keys (mpv-mpris).

Keys	Does
Space · p	pause
← → · ↑ ↓	back / forward 5 s · 1 min
Shift+← →	exactly 1 s
9 0 · m	quieter / louder · mute
f · Esc	fullscreen · leave it
v · j / J · #	subtitles on / off · next / previous subtitle track · next audio track
z · Z	subtitles 0.1 s earlier · later
[ ] · Backspace	slower / faster · normal speed
s · S · Ctrl+S	screenshot · without subtitles · of the window as you see it
g p · g s · g a · g c · g w	pick from the playlist · subtitles · audio tracks · chapters · the files with a saved position
Ctrl+P	menu of everything
i · ?	playback statistics · the key list
Ctrl+H	GPU decoding off / on
q · Q	quit (the position is kept either way)

yt-dlp

mpv plays web videos through yt-dlp, at up to 2160p. On its own, `yt-dlp URL` downloads into the current folder; `yt-dlp -F URL` lists the formats; `yt-dlp -x --audio-format mp3 URL` keeps only the sound (ffmpeg does the conversion). YouTube hides its video addresses behind JavaScript challenges: yt-dlp solves them in Node.js (`~/.config/yt-dlp/config`), which it starts with Node's `--permission` flag. On the Node 24 installed here that blocks file access, child programs and native add-ons — not the network, which Node restricts only from version 25. The code Node runs is yt-dlp's own solver (EJS).

How the config chooses what to play — `ytdl-format=bestvideo[height⩽?2160]+bestaudio/best`:

Part	Means
`bestvideo[height⩽?2160]`	the best video-only stream at most 2160 lines high (`?`: or one whose height is unknown)
`+bestaudio`	together with the best sound-only stream
`/best`	failing that, the best single file holding both

The same syntax works with `-f`: `yt-dlp -f 'bv*[height⩽1080]+ba/b' URL` downloads at most 1080p (`bv`, `ba` and `b` are short for best video, best audio, best). `yt-dlp -F URL` lists every stream with its container, codecs and bitrate.

Read more: the mpv manual (also `man mpv`) and yt-dlp's format selection.

Pitfalls. Japanese, Chinese and Korean subtitles come out in Noto Sans CJK, Arabic in Noto Naskh. If `i` shows dropped frames on a weak GPU, try `mpv --interpolation=no file` first.

11.11 imv — pictures

Open a picture from yazi or Brave's downloads (imv is the default for pictures), or type `imv picture.jpg`, `imv folder/` (every picture in it) or `imv -r folder/` (subfolders too). It opens **floating**, on the theme's ground colour.

Keys	Does
← → or [ ]	previous · next picture
gg · G	first · last
+ or i or ↑ · - or o or ↓ · a · r · c	zoom in · out · actual size · fit again · centre
h j k l	move around a zoomed picture
s	cycle the scaling mode

Keys	Does
f · d	fullscreen · name, size and position overlay
t · T	slideshow: one second more · less per picture (0 = off)
Space · .	play / pause an animation · next frame
x · q	take the picture off the list · quit

Pitfalls. x only removes the picture from imv's list; the file stays. imv cannot edit: to mark up a picture, `satty --filename picture.png --output-filename marked.png` (§11.13). Its colours are read at start, so an open imv keeps the old theme.

11.12 zathura — PDF, EPUB and comics



A PDF in zathura, in the palette.

Open a document from yazi or Brave's downloads (zathura is the default for PDF, EPUB and comic books), or type `zathura --fork file.pdf` — `--fork` gives the terminal back. zathura reads **PDF**, **EPUB** and **XPS** (through mupdf) and **CBZ/CBR** comic archives. Pages open fitted to the window, one per row; the status and command bars are hidden until needed.

Keys	Does
j k · h l	scroll down / up · left / right
J K · Space · Shift+Space	next / previous page · a screen down / up
gg · G · 42G	first page · last page · page 42
a · s	fit the page · fit the width
+ - =	zoom in · out · original size
d	two pages side by side
/ · ? · n · N	search forward · backward · next · previous result

Keys	Does
Tab	table of contents (Enter opens an entry)
f · F · c	follow a link by its label · show where it leads · copy its address
Ctrl+O · Ctrl+I	back · forward through your jumps
m*x* · '*x*	set mark *x* · go back to it
Ctrl+R	**recolour the page in the theme's colours** — paper text on ink, or ink on paper
r	reload the file (set here; zathura's own `r` would rotate)
p	print (set here)
Ctrl+N · Ctrl+M	show / hide the status bar (page number) · the command bar
F5 · F11	presentation · fullscreen
q	quit

Selecting text with the mouse copies it to the clipboard. Commands start with `:`, as in neovim:

```
:bmark ch3     save a bookmark called ch3 at this page
:blist         list the bookmarks
:bjump ch3     go back to it
:info          the document's details: title, author and so on
```

Read more: `man zathura` (keys and commands) and `man zathurarc` (every setting).

Pitfalls. Rotation has no key, because `r` reloads here. A theme switch makes open zathura windows re-read their config (§5.4), and the config says `recolor false`, so a recoloured page returns to normal: press Ctrl+R again. Printing to paper needs a print service; none is in the manifest, so the print dialog can save to a file but may list no printers.

11.13 satty — annotating screenshots

Super+Print: drag a region (the selection frame is in the theme's vermilion), and satty opens with it, the arrow tool ready (Print and Shift+Print in §4.3 save and copy without annotating).

Keys	Does
z · i · r · e	arrow · line · rectangle · ellipse
t	text: type, Shift+Enter for a new line, Enter to place it, Esc to drop it
m	numbered markers (1, 2, 3 … with each click)
g · u · b	highlighter (hold Ctrl for freehand) · blur · brush
c · p	crop · pointer (select and move)
1 … 5	colour: vermilion, yamabuki, matcha, indigo, paper
Ctrl+Z · Ctrl+Y	undo · redo
Enter	**copy to the clipboard and close**
Ctrl+S	**save to `~/Pictures/Screenshots/` and close**
Ctrl+Shift+S	save under a name you choose, and close
Esc	close without keeping anything

Pitfall. Satty closes after the first copy or save (`early-exit`). Enter copies but does **not** save a file — unlike Print and Shift+Print, which do both. Pick the one you need. The copy stays on the clipboard after satty closes (§4.8).

Tip: screenshots pile up; `fd -e png --changed-before 30d . ~/Pictures/Screenshots` lists those older than a month, and adding `-X rm` deletes them.

Read more: Satty's README, for version 0.22, the one installed here.

11.14 Sound — pavucontrol and EasyEffects

The audio menu (**Super+A**, §4.2) covers everyday needs: volume, mute, the output and the input. Two programs go further.

pavucontrol, the full mixer: audio menu → *mixer*…. It opens floating. Its tabs: *Playback* — a volume and an output device for **each program** (send a call to headphones and music to speakers, or a game to the **Share** output for a call, §4.10); *Recording* — which microphone each program listens to, shown while it records; *Output* and *Input Devices* — levels and ports of each device; *Configuration* — the card's profile (for example HDMI against analogue, or turning a card off).

EasyEffects (from the launcher; on Fedora 44 it is a Qt 6 program): effects in real time. *Output* effects change what you hear — an equaliser, bass, loudness. *Input* effects change your microphone — noise reduction, a gate (it mutes everything below a set level, so the room's hum drops out between words), a compressor (it evens out loud and quiet) — and every program that records hears the cleaned sound. Save a chain as a preset; `easyeffects -p` lists the presets, `easyeffects -l name` loads one, `easyeffects -b 1` / `-b 2` bypasses all effects / restores them.

Pitfalls. EasyEffects works **only while it runs**, and nothing starts it at login. Its *Autostart on login* switch writes an autostart file that only full desktops read; dwl does not. To keep it running, put one line in `~/.config/dwl/autostart.local.sh`, the login file that is yours alone (§15.1: the theme never writes it):

```
easyeffects --service-mode --hide-window &   # at every login, window hidden; & keeps it running
```

It takes effect at the next login. A program you have moved to a device in pavucontrol stays there; choosing another output in the audio menu no longer moves it.

11.15 spotify_player and cava — music

spotify_player is Spotify in the terminal: type `spotify_player`. It needs a Premium account. The first start opens Spotify's sign-in page in Brave, usually twice: once for the Web API (lists, search) and once for the built-in player. Approve each, and both are cached in `~/.cache/spotify-player` (`spotify_player authenticate` signs in again from scratch). It plays through its own built-in client, appears in the bar's *now playing*, and answers the media keys.

Keys	Does
Space · n · p	play / pause · next · previous
+ - · _	louder · quieter · mute
> · < · ^	forward · back · to the start of the track
Ctrl+S · Ctrl+R	shuffle · repeat mode
g s · g l · g y · z	search page · library · liked tracks · queue
/	search in the current page
Enter · a	play the selection · actions on the current track
Tab · Backspace	next panel · previous page
D	choose the playback device: this computer, or a phone or speaker signed in to the same account (Spotify Connect)
?	every key
q	quit

Its colours are the terminal's own with vermilion accents (the `sumi` theme in `theme.toml`), so it switches with the desktop.

cava draws the bar's music meter (§4.1) with its own config in `~/.config/fedokawaii`. Type `cava` in a terminal for the classic full-window bars: Up / Down sensitivity, Left / Right bar width, `f` / `b` colours, `r` reloads the config after an edit, `q` quits. That copy reads `~/.config/cava/config`, which it creates on first run and which is yours to change.

Read more: spotify_player's README (every key and command) and cava's.

11.16 presenterm — slides in the terminal

A presentation is a Markdown file. `presenterm talk.md` shows it full-window and **reloads it every time you save**, so you can edit in neovim beside it; `presenterm --present talk.md` turns that off for the real thing. Super+F makes the window fullscreen; Ctrl++ in foot makes the text larger. Colours follow the theme at start (`terminal-dark` or `terminal-light`).

```
---
title: Fedokawaii
author: me
---

First slide

<!-- pause -->
This line appears on the next key press.

<!-- end_slide -->

Second slide
<!-- speaker_note: say hello -->
```

Keys	Does
l j → ↓ PageDown Space	next
h k ← ↑ PageUp	previous
n · p	next · previous slide at once, skipping its pauses
gg · G · 7G	first · last · slide 7
Ctrl+P	slide index
Ctrl+E	run the code block on screen (only when started with `-x`)
? · q	keys · quit

Pictures show in foot. `presenterm --export-html talk.md` writes one self-contained HTML file; PDF export needs weasyprint, which is not installed. For speaker notes, run `presenterm talk.md --publish-speaker-notes` in one window and `presenterm talk.md --listen-speaker-notes` in another.

Read more: presenterm's documentation — pauses, columns, code and exports, for the version installed here (0.16).

11.17 qalc — the calculator

`qalc` in a terminal, or `qalc 'expression'` for one answer (`-t` prints only the result). It knows units, currencies, bases and functions.

```
qalc '2^64'
qalc '75 km/h to mph'
qalc '255 to hex'
qalc '3.5 h to min'
qalc -e '100 USD to AED'      # -e fetches current exchange rates first
```

Each answer repeats the question as qalc understood it; `-t` keeps only the result. `=` marks an exact answer, `≈` a rounded one:

```
$ qalc '3.5 h to min'
3.5 hours = 210 min
$ qalc -t '255 to hex'
0xFF
$ qalc '75 km/h to mph'
75 kilometers/hour ≈ 46.60283942 mph
```

Inside it: `ans` is the last result, `to unit` converts it, `exrates` updates the rates, `help`, `quit`. Up and Down recall earlier lines. For a quick sum without leaving your work, `footclient -a float qalc` opens it in a small floating window.

Read more: `man qalc`, and the project's site for the full list of units and functions.

11.18 Documents — Xournal++, AbiWord, Gnumeric and the PDF tools

Xournal++ (launcher) writes on PDFs: *File → Annotate PDF*, then the pen (Ctrl+Shift+P), highlighter (Ctrl+Shift+H), text (Ctrl+Shift+T) and eraser (Ctrl+Shift+E). It saves your marks in a separate `.xopp` file and never changes the PDF; *File → Export as PDF* writes a new PDF with the marks in it — that is the file to send.

AbiWord for letters and simple documents, **Gnumeric** for spreadsheets (both from the launcher, both the defaults for their file types). Both open Microsoft and OpenDocument files; *File → Save As* lists what each can write. The Liberation fonts stand in for Arial, Times New Roman and Courier New with the same widths, so pages break where their authors meant. From a terminal: `abiword --to=pdf letter.docx` and `ssconvert sheet.xlsx sheet.csv` convert without opening a window. Neither is a full office suite: a document with tracked changes or a complex layout is better opened in the web version of the program it came from.

PDF tools in a terminal: `qpdf --empty --pages a.pdf b.pdf -- both.pdf` joins files, `qpdf in.pdf --pages . 1-3 -- first-three.pdf` takes pages out, `pdftotext file.pdf -` prints the text, `pdfunite a.pdf b.pdf both.pdf` is the other way to join.

In `--pages`, each file name is followed by the pages to take (`.` stands for the input file):

Range	Takes
`1-3`	pages 1 to 3
`1,4,7`	pages 1, 4 and 7
`5-z`	page 5 to the end: `z` is the last page
`z-1`	every page, in reverse order

Read more: the qpdf manual and Xournal++'s guide.

11.19 Hardware — OpenRGB, Solaar, liquidctl, LACT

OpenRGB (launcher) sets the lighting of RGB devices from one window: find the devices, choose colours and effects, save a *profile*. Nothing starts it at login, so the lighting falls back to the devices' own defaults after a power cut; `openrgb --profile NAME` applies a saved profile, and that command can go in `~/.config/dwl/autostart.local.sh` (§15.1) as `openrgb --profile NAME &`, like the EasyEffects line in §11.14. Memory and motherboard lighting is reached over *SMBus*, a small bus on the motherboard that chips such as the RAM's lighting controllers hang on; if those devices do not appear, see OpenRGB's own SMBus notes — this desktop sets up nothing beyond loading `i2c-dev` (for monitor brightness, which uses the same kind of bus). Its window takes the Qt palette, so it follows the theme.

Solaar (launcher) manages Logitech receivers and their devices: pair a mouse or keyboard to a receiver, read battery levels, change buttons and pointer settings. `solaar show` lists everything in a terminal. Like EasyEffects, its autostart entry is for full desktops, so it runs only when you start it.

liquidctl controls AIO coolers, fan hubs and some power supplies from the terminal: `liquidctl list` finds them, `liquidctl initialize all` prepares them (needed after every boot on many devices), `liquidctl status` shows temperatures and speeds, `liquidctl set pump speed 70` and `liquidctl set fan speed 20 30 40 60 50 100` set them. The fan line is a curve, read in pairs of temperature and duty (the share of full speed):

Liquid at	Fans at
20 °C	30 %
40 °C	60 %
50 °C	100 %

On liquid coolers the temperature is the coolant's, from the cooler's own probe, not the CPU's. Most devices forget these settings when powered off, and nothing here applies them again at boot; liquidctl's README shows a small systemd service that does.

LACT (launcher; AMD only) shows the graphics card — clocks, temperatures, power — and changes it: fan curve, power limit, performance level, and, thanks to the `amdgpu.ppfeaturemask` kernel argument setup added, clocks and undervolting (running the chip at a lower voltage for the same clock: cooler and quieter, until it is too low to be stable). Its service, `lactd`, applies the settings in `/etc/lact/config.yaml` at every boot. Change one thing at a time and test it: an unstable clock or voltage can freeze the machine, and LACT's wiki has a page on recovering from a bad overclock.

11.20 podman — the Prowlarr container

Term	In one line
image	a read-only bundle of one program and everything it needs, downloaded once
container	a running copy of an image, walled off from the rest of the system
container or VM	a virtual machine boots a whole second operating system on pretend hardware; a container is only a fenced-off process on this kernel, so it starts in seconds and costs only the program's own memory
rootless	the container runs as you, not as root: a break-out would have only your rights
quadlet	a small file that tells systemd how to run a container as a service
health check	a command run inside the container from time to time, asking whether it still answers

podman runs one container here: **Prowlarr**, the indexer service torrra searches through (§12). It runs rootless, as you — you are its user inside too, so every file in `~/.config/prowlarr` is yours — holds no capabilities and can gain none, runs **only when wanted** (a torrra search starts it and stops it again; the command `prowlarr` starts it and opens its page — idle it would hold some 300 MB of memory), and listens only on this machine, at http://localhost:9696. systemd manages it from a *quadlet* file, `~/.config/containers/systemd/prowlarr.container`, which `update` keeps current. A health check asks it every five minutes whether it answers; three misses in a row restart it. The quadlet's main lines, and what each does:

Line in `prowlarr.container`	Does
`Image=lscr.io/linuxserver/prowlarr:latest`	the image to run
`UserNS=keep-id:uid=1000,gid=1000`	you are the same user inside as outside, so its files stay yours
`NoNewPrivileges=true`	it can never gain more rights than it starts with
`PublishPort=127.0.0.1:9696:9696`	its port 9696, reachable from this machine only
`Volume=%h/.config/prowlarr:/config:Z`	its settings live in `~/.config/prowlarr` (`%h` is your home; `:Z` labels the folder for SELinux)
`HealthInterval=5m` · `HealthRetries=3`	the health check: every five minutes, three misses allowed

Command	Does
`systemctl --user status prowlarr`	is it running, and its last lines of output
`journalctl --user -u prowlarr -e`	its full log
`systemctl --user restart prowlarr`	restart it
`prowlarr stop`	stop it when you are done with its page (`prowlarr` leaves it running; it is stopped anyway at the next restart of the computer)
`podman ps` · `podman logs -f prowlarr`	running containers (with their health) · follow the log
`podman image ls lscr.io/linuxserver/prowlarr`	the Prowlarr image you have, and when it was made (`update` fetches the newest)

Command	Does
`podman image prune`	remove image layers left behind by updates

Pitfalls.

- Never `sudo podman`: root has its own, separate — and here empty — list of containers.
- Do not `podman rm` or `podman run` it by hand: systemd recreates the container from the quadlet at every start. To change it, edit `system/prowlarr.container` in your working copy and apply (§15.2): a change under `system/` makes apply run repair, which puts the new file in place (so does `update`).

Read more: podman-systemd.unit(5), every quadlet key, and Podman's documentation.

11.21 snapper — snapshots

snapper keeps snapshots of `/` (and of `/home`, which is its own subvolume): one every hour, thinned out over time; the *fedokawaii baseline* taken after the build; and a **pre / post pair around every dnf transaction**, described by the command that ran (`dnf install htop`). A snapshot costs almost nothing until files change after it. `~/Downloads` is not in them (§6.6).

Term	In one line
snapshot	a frozen, read-only view of a whole subvolume at one moment
subvolume	a part of the Btrfs file system that can be snapshotted on its own: `root` (mounted at `/`) and `home` here
copy-on-write	Btrfs writes a changed file to new space and leaves the old blocks to the snapshot — why a snapshot is free until files change

How many are kept — setup sets the same limits for `/` and `/home`:

Kind	Kept
hourly	the last 2 to 8
daily	the last 2 to 7 — about a week
weekly · monthly · yearly	none · at most 2 · none
numbered: the dnf pairs, and the one each `update` takes first	4 to 20
the *fedokawaii baseline*, and any you make yourself	until you delete them

Where a range is given, the clean-up keeps up to the higher number and goes down towards the lower one when space is short: less than a fifth of the disk free (`FREE_LIMIT=0.2`), or snapshots holding more than half of it (snapper's default `SPACE_LIMIT`).

Command	Does
`snaps`	list the snapshots of `/` (`sudo snapper -c root list`)
`sudo snapper -c root list --type pre-post`	only the dnf pairs, one line each
`sudo snapper -c root status 41..42`	which files the transaction changed: `+` created, `-` deleted, `c` content, `p` permissions
`sudo snapper -c root diff 41..42 /etc/some.conf`	what changed inside one file
`sudo snapper -c root undochange 41..42`	**put back every file the transaction changed**
`sudo snapper -c root undochange 41..42 /etc/some.conf`	put back one file only
`sudo snapper -c root create -d "before trying X"`	a snapshot of your own. It has no clean-up rule, so it stays until you delete it; add `-c number` to let it age out with the dnf pairs
`sudo snapper -c root delete 30-40`	delete a range

Command	Does
`sudo snapper -c home list`	the snapshots of `/home`

What `status` prints, one line per file — after `sudo dnf install htop`, say:

```
+..... /usr/bin/htop
+..... /usr/share/man/man1/htop.1.gz
c..... /usr/lib/sysimage/rpm/rpmdb.sqlite
```

The first column is the kind of change: `+` created, `-` deleted, `c` content, `t` type. The next five are `p` permissions, `u` owner, `g` group, `x` extended attributes and `a` access lists, with a `.` where nothing changed.

Recommended. Before an experiment — a new repository, a driver, a big config change — take a snapshot of your own with a description. If it goes wrong, `status` from that number to the newest one shows everything that changed, and `undochange` puts it back. §13.9 walks through undoing a bad update.

Pitfalls.

- **`undochange`, not `rollback`.** `snapper rollback` works by changing Btrfs's *default* subvolume, but Fedora's boot entries name the subvolume outright (`rootflags=subvol=root` — check with `cat /proc/cmdline`), so after a rollback the same old `root` would boot again. Undo with `undochange`.
- `/boot` is a separate partition and is **not** in the snapshots. For a kernel that misbehaves, pick the previous one in the boot menu (hold Esc if it is hidden) — it stays installed beside the new one (§6.6).
- snapper's own manual warns that consistency after `undochange` is not guaranteed: it puts files back; it does not re-run package scripts. Reboot afterwards, then run `verify`.
- Deleting files does not free space while snapshots still hold them (§13.10).

Tip: `sudo snapper -c root status N..0` shows everything changed since snapshot N — the quickest way to see what an experiment really touched (§7.2).

Read more: `man snapper` and `man snapper-configs`, also at http://snapper.io/manpages/snapper.html.

11.22 Claude Code — an assistant in the terminal

Type `claude` in a terminal, inside the folder you want to work on. It reads and edits files there and runs commands, asking first. It came from Anthropic's signed dnf repository (stable channel), so it updates with `dnf upgrade` (and `update`) and does not update itself; `claude doctor` checks the installation.

Signing in

Either sign in with a Claude account (Pro, Max, Team or Enterprise): the first start opens the sign-in page in Brave. Or use an API key — and keep it only in `~/.config/fedokawaii/secrets.sh` as `export ANTHROPIC_API_KEY=…` (§11.2); Claude Code asks once whether to use the key it finds in the environment. Never put a key in the repository or in a project's settings files.

Keys or input	Does
Esc	stop Claude mid-answer, keeping what it has done
Esc Esc	with an empty prompt: go back to an earlier point (also `/rewind`)
Shift+Tab	switch mode: ask before edits, accept edits, **plan** (it only reads and proposes)
`\` then Enter · Ctrl+J	a new line in the prompt
Ctrl+V	paste a picture from the clipboard — a screenshot, say
Ctrl+G	write the prompt in neovim
`!command`	run a shell command and show Claude the output
`@path`	point at a file
`/init`	write a `CLAUDE.md` describing the project, read at every start

Keys or input	Does
`/clear` · `/compact`	start over · summarise to free room
`/model` · `/usage` · `/resume` · `/theme`	model · cost and limits · an earlier session · colours

`claude -c` continues the last conversation in this folder, `claude -r` picks an older one, and `claude -p "question"` answers once and exits, which makes it usable in pipes: `journalctl -b -p err | claude -p "what stands out here?"`.

Recommended. Use plan mode for anything touching this repository or the system, read the plan, then let it work. Run `claude` in `~/fedokawaii` to ask about the desktop itself — it can read the scripts and this handbook. For colours that follow the theme, `/theme` → the *ANSI colours only* variant of dark or light.

Pitfalls. It acts as you: it can read anything you can, `secrets.sh` included, and every command it runs has your rights. Read the permission prompts instead of approving them by habit. When it edits a managed config in `~/.config`, the next Theme run replaces the change — point it at the repository copy instead.

Read more: Claude Code's documentation; interactive mode lists every key, and commands every `/` command.

11.23 Steam — games

Start **Steam** from the launcher. The client's main window always goes to **workspace 9** (a rule in `config.h`); press **Super+9** to reach it. Games open on whatever workspace you are on. Steam runs through XWayland, and its own interface ignores the desktop's themes (§9).

Term	In one line
XWayland	the bridge that lets an older X11 program, such as Steam, show on a Wayland desktop
Proton	Valve's compatibility layer, built on Wine: it runs Windows games on Linux
GE-Proton	a community build of Proton that carries fixes and media patches ahead of Valve's
`%command%`	in launch options, the place where Steam puts the game's own command line

Each game's **launch options** (in Steam: the game's *Properties → General*) take the tools installed with it:

Launch options	Does
`gamemoderun %command%`	gamemode: the CPU and scheduler tuned for the game while it runs (`gamemoded -s` reports whether it is active)
`mangohud %command%`	the performance overlay, in the desktop's ink and Departure Mono; **Right Shift+F12** shows or hides it
`gamemoderun mangohud %command%`	both
`gamescope -W 2560 -H 1440 -r 120 -f -- %command%`	the game inside its own small compositor at a fixed size and refresh rate; add `-w 1920 -h 1080 -F fsr` to render lower and upscale
`gamescope --mangoapp -f -- %command%`	gamescope with the overlay — inside gamescope, use `--mangoapp` rather than `mangohud`

GE-Proton

Setup installs the newest GE-Proton, and `update` adds new releases, keeping the newest two so a game pinned to the older one still starts. Restart Steam to see a new one. Choose it per game in *Properties → Compatibility* (*Force the use of a specific Steam Play compatibility tool*), or for every game in Steam's *Settings → Compatibility*. GE-Proton uses the kernel's `ntsync` driver (loaded at boot) by itself: the waiting and signalling between a Windows game's threads is done by the kernel instead of being imitated, which is faster.

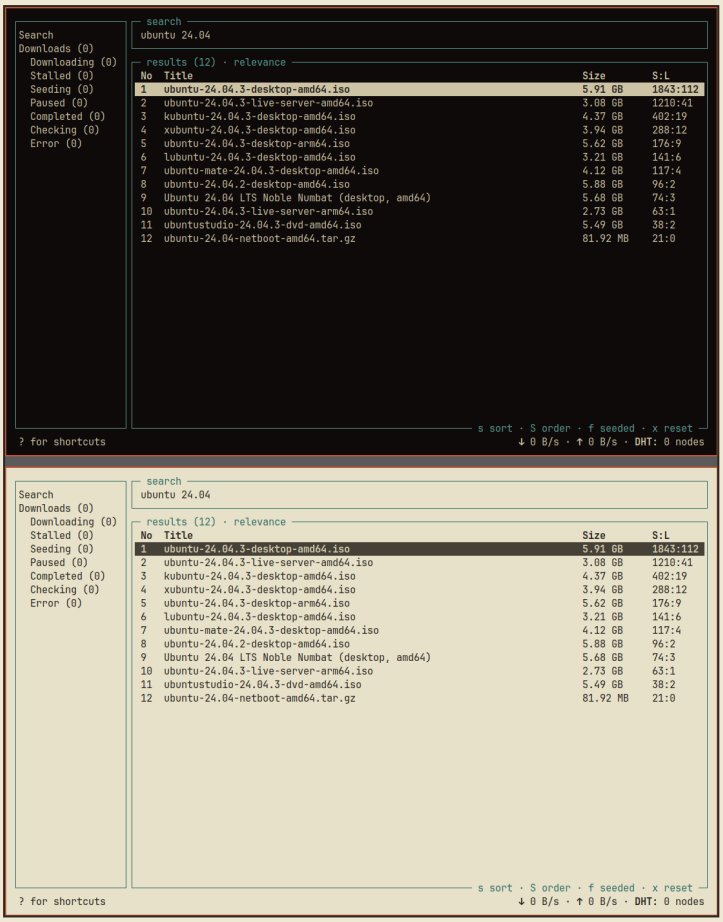
```
ls -l /dev/ntsync      # the device is there: the driver is loaded
```

Should one game misbehave with it, the launch option `PROTON_NO_NTSYNC=1 %command%` runs that game without it.

Pitfalls. The overlay stays off until a game's launch options ask for it, and outside gamescope `mangohud` is the switch; inside it, `--mangoapp`. MangoHud's settings are a managed file: change them in the repository's `MangoHud.conf`. `update` keeps only the newest two GE-Proton builds, so a game forced to an older one needs a new choice under *Compatibility*.

Read more: the READMEs of GE-Proton (its launch options), gamescope and MangoHud.

12. Torrents: searching and downloading



torrra after a search: twelve results with their sizes and seeders:leechers, in the terminal's colours.

Two pieces work together. **Prowlarr** (a small service in a container that runs only while it is needed, §11.20) knows how to search hundreds of torrent sites — *indexers* — and answers one search for all of them. **torrra** is the program you use: it sends your search to Prowlarr, shows the results, and downloads what you pick with libtorrent, the engine behind most torrent clients. One API key connects them. Setup made it, and its one master copy is `~/.config/fedokawaii/secrets.sh`: Prowlarr receives it as a podman *secret* (podman keeps that copy in `~/.local/share/containers/storage/secrets/`, readable by you alone, and `update` renews it from secrets.sh should you change the key there), torrra receives it from its wrapper each time it starts, and neither program writes it into its own settings.

Term	In one line
torrent	a set of files shared piece by piece among many computers; a small `.torrent` file describes it: names, sizes and a checksum for every piece
peer · seeder · leecher	any computer sharing the same torrent · one that has all of it · one still downloading
`S:L`	torrra's column of seeders:leechers for each result, such as `412:37`; more seeders means a faster, surer download
magnet link	a link that names a torrent by its fingerprint, the *info-hash*, instead of carrying the `.torrent` file; the rest comes from peers
indexer	a website that lists torrents and answers searches; Prowlarr asks all of yours at once
seeding · ratio	sharing a finished download · uploaded ÷ downloaded: a ratio of 2 means you gave back twice what you took
API key	a long password one program shows another; here 32 hexadecimal characters in `secrets.sh`

A magnet link, taken apart:

Part	Says
`magnet:?`	this is a magnet link
`xt=urn:btih:` and 40 hexadecimal characters	which torrent: its info-hash
`&dn=debian-13.1.0-amd64-netinst.iso`	a name to show while the rest arrives
`&tr=udp://…`	a *tracker*, a server that introduces peers to each other (optional)

Download only what you have the right to: Linux and BSD images, public-domain films and books, Creative Commons music, your own backups. Torrenting shares what you download with others while it runs, and your address is visible to every peer.

12.1 The first time: Prowlarr

1. Type `prowlarr` in a terminal: it starts Prowlarr and opens http://localhost:9696 in Brave. Prowlarr asks you to **create a login** — a user name and password for this page (it asks even on this computer, so no web page can reach it behind your back). Brave can remember it.
2. **Indexers → Add Indexer.** Prowlarr lists every site it knows, public and private; filter by language or type, pick the ones you use, and **Test** each one before saving. Sites marked as needing *FlareSolverr* (a browser that gets past Cloudflare checks) will not work here — none is installed — so choose others. In each indexer's settings, tick **Prefer Magnet URL** (an advanced setting: **Show Advanced** reveals it); torrra's own documentation asks for it.
3. Optional: **Settings → Apps** is for connecting other programs (Sonarr, Radarr); torrra needs nothing there — it already holds the key.
4. Done with the page: `prowlarr stop`. A search in torrra starts Prowlarr by itself and stops it again afterwards.

That is all. `verify` warns while Prowlarr has no indexers, and torrra says so plainly instead of showing an empty list.

12.2 Searching and downloading with torrra

Open **Torrents** from the launcher, or type `torrra` in a terminal (`torrra search 'debian 13'` searches straight away; `torrra downloads` shows only the downloads). Its colours are the terminal's, so it follows the theme. When Prowlarr is not running, a search starts it first — `torrra: starting Prowlarr…` — which takes up to a minute the first time after the computer starts.

Keys	Does
type, then Enter	search all your indexers
j k · ↑ ↓ · Ctrl+D / Ctrl+U · gg · G	move · a page · top · bottom
Enter (or l) · Esc	the torrent's details · close them
Enter, in the details	its file list, every file ticked; **Enter** there starts the download (Space unticks a file; **d** takes all without waiting for the list)
s · S · f · x	sort menu · reverse the order · hide results without seeders · back to your defaults
Tab	the next panel (search, results, downloads)
p	pause or resume a download
f (on a download)	choose which files of it to download (Space selects, Enter confirms)
o	its options: speed limits, ratio, sequential download (for watching while it loads)
d · D	remove the torrent · remove it **and delete its files**
t	"turtle mode": the slow speed limits on / off
Ctrl+T · ? · Ctrl+Q	another colour theme · every key · quit

Results are sorted by **seeders** — the number of people sharing the whole file — highest first; more seeders means faster and more certain. Downloads go to `~/Downloads`, which is its own Btrfs subvolume, so a film you delete gives its space back at once rather than living on in snapshots.

Tip: when you are done with a torrent, **D** removes it and deletes its files in one step. Move anything you keep out of `~/Downloads` — nothing there is in a snapshot.

Magnet links in Brave

The first magnet link you click makes Brave ask whether to open it with *xdg-open*; tick *Always allow* and say yes. From then on a magnet link — or a `.torrent` file opened from the downloads — starts torrra in its own window at that torrent's file list; **Enter** starts the download (pressed before the list has loaded, it takes every file). torrra runs one window at a time: if it is already open, a notice says so and the link waits on your clipboard — close torrra and click the link again (or, in a terminal, `torrra download 'PASTE'`).

12.3 How it behaves

- **Downloads run only while torrra's window is open.** Close it and they pause; open torrra again and they continue where they were. There is no background service.
- **Seeding.** A finished download keeps sharing while torrra is open, without a limit by default. To stop at a ratio or a time, set it per torrent (**o**), or for all; the torrent pauses when it gets there, and `0` removes a limit again:

`sh torrra config set seeding.default_max_ratio 2 # once it has shared twice its size torrra config set seeding.default_max_time 3d # or after 3 days (also 12h, 90m)` * **Speed.** Unlimited by default. **t** turns *turtle mode* on and off: the slow limits, 10 KB/s each way until you set your own, with `TURTLE` in the status bar while it is on. `torrra config set speed_limit.download_limit 5M` sets one (`500K`, `2M`; `unlimited` turns one off), and `speed_limit.upload_limit` the other. * **Ports.** libtorrent listens on **6881** (TCP and UDP, opened in the firewall by setup) and asks your router to forward it (UPnP / NAT-PMP). If 6881 is busy, it quietly uses a random port the firewall does not open, and fewer peers can reach you. * **One torrra at a time** — a second window would share the same downloads, so it says torrra is already open. * Settings live in `~/.config/torrra/config.toml` (yours); `torrra config list` shows them, `torrra config edit` opens the file in neovim.

12.4 When something is wrong

torrra itself exits quietly on every failure; its wrapper (`kawa-torrra`) checks Prowlarr first and says what is missing — in the terminal, or as a notice when torrra was opened from the launcher or a link:

Message	Do
Prowlarr is not answering on 127.0.0.1:9696	`systemctl --user status prowlarr`; `journalctl --user -u prowlarr -e`
Prowlarr refused the key in secrets.sh	`update` hands Prowlarr the key again
Prowlarr has no indexers yet	§12.1, step 2
no PROWLARR_API_KEY in secrets.sh	`update` makes one
secrets.sh must be yours and private	`chmod 600 ~/.config/fedokawaii/secrets.sh`
Prowlarr did not start	`journalctl --user -u prowlarr -e` says why
Prowlarr answered HTTP … (a number)	the same: `journalctl --user -u prowlarr -e`
not installed … · cannot start …	`update` reinstalls torrra
torrra is already open in another window	close that one first; a link you clicked waits on the clipboard
a search finds nothing although indexers are set	test them in Prowlarr (**Indexers → Test All**); one may be down or need FlareSolverr

Read more: torrra's usage guide (for version 2.5.1, the one installed), Prowlarr's indexer pages and the magnet link format.

13. Everyday recipes

13.1 Upload a file from Brave

1. In Brave, click the site's upload button. The file dialog opens: yazi, floating, in the folder you last uploaded from (§4.9).
2. Find the file: ↑ ↓ and → ← through the folders — or **z** and a few letters of its name to jump to it below this folder, **Z** and a word to jump to a folder you use often, **g d** for Downloads.
3. Press **Enter** on it. For several files (where the site takes several), **Space** on each, then Enter. **Esc** cancels.

13.2 Share your screen, with sound, in a call

1. In the call, choose to share. In Brave's picker:
2. a **tab** — its sound comes along when *Also share tab audio* is ticked;
3. a **window** or the **entire screen** — tick *Also share system audio* to send what you hear (it includes the call itself; see §4.10 for the quieter way).
4. For a window or a screen, the desktop's *share what?* menu appears; pick one. With only one choice it is taken without asking.
5. Notifications are held back while you share; critical ones still show.
6. Your microphone travels as usual; check it in the audio menu (Super+A → input).

13.3 Take, annotate and paste a screenshot

1. **Super+Print**, drag a region.
2. In satty, the arrow is ready: drag to draw. **t** for text, **m** for numbered markers, **1…5** for colours, Ctrl+Z to undo.
3. Press **Enter**: the picture is copied and satty closes. (Ctrl+S saves it to `~/Pictures/Screenshots/` instead.)
4. Paste with **Ctrl+V** in Brave, a chat, or Claude Code. **Super+V** brings back earlier copies, pictures included.

For a plain shot, **Print** (region) or **Shift+Print** (whole screen) both saves and copies.

13.4 Switch the theme and the wallpaper

1. **Super+Shift+W** switches dark ⇄ light (or quick settings → *theme*, or `kawa-theme light` in a terminal).
2. **Super+W** chooses a wallpaper **for the theme you are on**; each theme remembers its own. Pictures of a day / night pair set the other theme too.
3. For your own pictures, put them in `~/Pictures/Wallpapers`; they appear in the same menu, on both themes.

13.5 Choose which program opens a type of file

The defaults are in `~/.config/mimeapps.list`, which Theme manages (§4.9).

1. Find the type — its *MIME type*, the standard name for a kind of file: `xdg-mime query filetype book.pdf` → `application/pdf`.
2. See what is chosen now: `xdg-mime query default application/pdf` → `org.pwmt.zathura-pdf-mupdf.desktop` (zathura).
3. Find the candidates: `grep -l 'application/pdf' /usr/share/applications/*.desktop`.
4. For good: change the line in `configs/home/.config/mimeapps.list` in the repository and run Theme. To try one first, `xdg-mime default NAME.desktop application/pdf` changes it at once — until the next Theme run puts the repository's choice back.

13.6 Install a program and keep `verify` happy

`verify` fails on anything user-installed that is not in the manifest, so the manifest comes first.

1. Check that it is not on the *never present* list in `PACKAGES.md`, and that it is not a second tool for a job one already does.

2. **Super+Shift+U → packages**: `lib/manifest.sh` and `PACKAGES.md` open side by side in your working copy. Add the name to `PKG_APPS` with a short comment, for example `hyperfine # benchmarks a command (runs it many times)`, and a row to `PACKAGES.md`, so the list stays true.
3. Close neovim and choose *apply now*: apply sees the manifest changed and runs repair, which installs every package the manifest has gained, then runs `verify`.

A package from Fedora or RPM Fusion needs nothing more. One from a new COPR also needs its name in `COPRS` and `sudo dnf copr enable owner/project` by hand — `update` does not add repositories. Flatpak and Snap are refused outright.

13.7 Update everything

1. **Super+U** → *update* (its mark reads **due** once the season has come round), or type `update`. It snapshots, upgrades every package, installs what the manifest gained, checks the pinned tools and fonts, brings GE-Proton, torrra's libtorrent and Prowlarr's image up to date, syncs neovim's plugins, and finishes with `verify` (§6.2).
2. If it reports a new kernel, restart; the one before stays installed as the way back.
3. Not included: Mason's tools in neovim (`:Mason`, then `U`). The toolkit itself stays the version you installed (§6.1).

Tip: the update clears dependencies nothing needs any more by itself (`dnf autoremove`, the list in the log); `dnf repoquery --unneeded` shows any that are left (§7.3).

13.8 Move to a new Fedora

1. Wait for the menu: *upgrade to Fedora N* reads **ready** once the new release has settled (and a reminder says so). Before then it shows when it will be offered.
2. Have an hour, and 20 GB free (the status shows the space). Close what you are working on.
3. **Super+U** → *upgrade to Fedora N*, Enter, your password. It checks everything first and stops — nothing changed — if something is not ready yet, saying what.
4. If dnf reports conflicts it lists the packages it would remove and asks; *y* goes on (any the desktop needs come back afterwards), anything else stops with nothing changed.
5. The computer restarts into the installation: 20–40 minutes, then the login. A minute or two later a notice says the move is finished; the status lists anything to look at.

13.9 Undo a bad update

1. `sudo snapper -c root list --type pre-post` — find the pair for the update, by its date: a *pre* snapshot taken just before the dnf run and a *post* just after. Say they are 41 and 42.
2. `sudo snapper -c root status 41..42 | less` — read what it changed.
3. `sudo snapper -c root undochange 41..42` — every one of those files goes back.
4. Reboot, then `verify`.
5. The next `update` would bring the same packages back, because it upgrades everything. Until a fixed version arrives, upgrade with `sudo dnf upgrade --exclude=PACKAGE` instead.

If the problem is the new kernel, use the boot menu instead (§11.21). If the desktop does not start at all, log in on a text console (**Ctrl+Alt+F2**) and run the same commands there.

13.10 Free disk space

1. Look first: `sudo btrfs filesystem usage /` for the real picture (`df` is not reliable with snapshots), and `du -xh -d1 ~ | sort -h | tail` for the biggest folders in your home.
2. Snapshots hold deleted files: `snaps`, then delete old ranges you do not need, `sudo snapper -c root delete 30-60`. Keep the baseline (it is never cleaned up by itself).
3. The trash: **g t** in yazi, or `gio trash --empty`.

4. Big files: `fd -u -S +1g . ~` (`-u` also looks in hidden folders and in what a `.gitignore` leaves out, such as build output in a code folder; `-H` alone skips that).
5. Caches and logs: `podman image prune`, `sudo journalctl --vacuum-size=200M`, `sudo dnf clean all`, and Steam's own *Settings* → *Storage* for games.

Tip: chapter 7 explains each of these, and which caches are safe to clear (§7.4).

13.11 Plug in a second monitor

1. Plug it in. dwl places it beside the first, and the bar appears on it.
2. It starts at its preferred mode; for the best refresh rate up to `MONITOR_HZ`, run `kawa-display` (it runs by itself at the next login).
3. Run `kawa-wallpaper --restore`, so the wallpaper is drawn at the new screen's exact size.
4. **Super+,** and **Super+.** move the focus between screens; **Super+Shift+,** and **Super+Shift+.** send a window across.
5. Brightness keys reach the new screen after the next login, or at once after `rm "$XDG_RUNTIME_DIR/kawa-ddc-buses"` (the list of monitors is kept for the session).
6. To put it on the other side: `wlr-randr --output NAME --pos X,Y` (names from `wlr-randr`). This lasts until you log out.

If the new screen is now the one the machine starts on, re-run Theme so the boot menu matches its resolution (§4.7).

13.12 Watch or keep a video from the web

1. Copy the link in Brave.
2. In a terminal: `mpv 'LINK'` to watch it in mpv, or `yt-dlp 'LINK'` to download it into the current folder.
3. Keep the quotes: links often contain `&`, which the shell would otherwise act on.

13.13 Change a setting for good

1. Is the file managed? The table at the start of chapter 11 says; `fedokawaii drift` lists the managed files that differ from the toolkit.
2. Managed: **Super+Shift+U**, pick the part, change it, close neovim, *apply now* (chapter 15). It reloads what it can; the terminal's settings and dwl's keys take effect at the next login.
3. Yours: edit it where it is. Nothing will overwrite it.
4. Keybindings live in `src/dwl/config.h`; the *keys* part opens it beside `keybinds.txt`, so Super+/ stays complete.

14. Do it yourself: what websites charge for

Joining two PDFs, shrinking a video for WhatsApp, turning an iPhone photo into a JPEG: the usual answer is a website that wants the file uploaded, stops at a size or a page count, stamps a watermark on the result or asks for an account, and keeps a copy on a server you know nothing about. Every recipe in this chapter does the same job on this computer, with programs setup has already installed (`lib/manifest.sh`). Nothing leaves the machine; there are no size limits, no daily quotas, no watermarks and no sign-ups; a contract, a passport scan or a bank statement stays yours. Most recipes are one command and finish in a second.

Term	In one line
PDF · picture	a PDF page keeps text, drawings and pictures apart, so text stays sharp at any size; a picture (JPEG, PNG, HEIC) is only a grid of pixels
lossless · lossy	lossless keeps every pixel or sound exactly (PNG, zip); lossy drops detail you are unlikely to miss, for a much smaller file (JPEG, WebP, MP3, H.264)
resolution, dpi	dots (pixels) per inch of paper: a scan 2480 pixels wide is a sharp A4 page at 300 dpi
OCR	*optical character recognition*: a program reads the letters in a picture and turns them into text
metadata	facts about a file kept inside it: author, dates, and in a photo (its *EXIF*) the phone and the place
codec · container	how a stream is compressed (H.264, HEVC, AAC) · the file that holds the streams (`.mp4`, `.mov`); see §11.10
checksum	a fingerprint computed from every byte of a file: change one byte and it comes out completely different
encoding	how letters are stored as bytes: UTF-8 today, Windows-1256 in many older Arabic files
regular expression	a pattern for text: in `s/\r$//`, `\r$` means "a carriage return at the end of a line"

14.1 Before you start

- Open a terminal (**Super+Return**) and go to the folder with the files: `cd ~/Documents`, or, once you have been there, a word of its name (`cd docs`, §11.7). In yazi, **c d** copies the folder's path; paste it after `cd` with Ctrl+Shift+V.
- Names with spaces go in quotes, as in the examples: `"Tenancy contract.pdf"`. Tab completes a name and adds the quoting itself.
- Every recipe writes a **new** file and leaves the original alone, unless it says otherwise. Open the result first; only then send the original to the trash (§14.10).
- A password typed into a command is kept in the shell history unless the line starts with a **space** (§11.2). The code blocks below that hold a password start with one.

Tip: `tldr qpdf`, `tldr magick` and `tldr ffmpeg` show a page of further examples for each program (§11.7).

14.2 PDFs: join, split, pick and turn pages

qpdf moves whole pages around without touching what is on them, so nothing is lost, and it is instant even on a book.

Want	Command
join files, in this order	`qpdf --empty --pages cover.pdf "Annual report.pdf" annex.pdf -- complete.pdf`
join every PDF in a folder, by name	`qpdf --empty --pages scans/*.pdf -- scans.pdf`
pages from several files	`qpdf --empty --pages report.pdf 1-3 annex.pdf 2 -- summary.pdf`
pages 3 to 7	`qpdf report.pdf --pages . 3-7 -- part.pdf`
everything but pages 2 and 5	`qpdf report.pdf --pages . 1-z,x2,x5 -- trimmed.pdf`
a new order · back to front	`qpdf report.pdf --pages . 3,1,2,4-z -- reordered.pdf` · `--pages . z-1`
odd pages · even pages	`--pages . 1-z:odd` · `--pages . 1-z:even`

Want	Command
one file per page	`qpdf --split-pages report.pdf page.pdf` → `page-01.pdf`, `page-02.pdf` … (as many digits as the last page needs)
files of ten pages each	`qpdf --split-pages=10 book.pdf part.pdf` → `part-01-10.pdf`, `part-11-20.pdf` …
turn every page a quarter right	`qpdf scan.pdf --rotate=+90 scan-turned.pdf`
turn some pages (`-90` turns left)	`qpdf scan.pdf --rotate=180:2,4 scan-fixed.pdf`
turn page 1 in the file itself	`qpdf --replace-input --rotate=+90:1 scan.pdf`
how many pages	`qpdf --show-npages report.pdf`

How a page list reads, on a 12-page file:

Page list	Gives
`3-7`	pages 3 to 7
`z` · `z-1`	the last page · every page, last first
`1-z,x2,x5`	1, 3, 4, 6 … 12: `x` takes pages out of the range before it
`3,1,2,4-z`	3, 1, 2, then 4 to 12
`2-z:odd`	2, 4, 6 … 12: `:odd` and `:even` count positions in the list, not page numbers

`.` stands for the file named before `--pages`, and a file given without a list comes in whole. Turning a page only changes a flag, so it costs nothing either.

Pitfalls. A wildcard sorts by name, so `scan-10.pdf` comes before `scan-2.pdf`: name files `scan-02.pdf`, or list them in the order you want. A wildcard also matches the file you made last time — keep the output outside that folder, as above. `--replace-input` is the one recipe here that changes the original.

Read more: qpdf's manual, at `--pages`, `--split-pages` and `--rotate`.

14.3 PDFs and pictures

Pictures out of a PDF. `pdfimages -list brochure.pdf` shows what is inside, one line per picture:

```
page   num  type   width height color comp bpc  enc interp  object ID x-ppi y-ppi size ratio
--------------------------------------------------------------------------------------------
   1     0 image    2480  3508  rgb     3   8  jpeg   no         8  0   300   300 3493K  14%
```

`width` and `height` are its pixels, `enc` how it is stored (`jpeg`; `image` for one kept without loss) and `x-ppi` its resolution on the page. `-all` writes each picture in the format it is stored in, so a JPEG comes out byte for byte as it went in, at its full size — which no screenshot gives you:

```
mkdir brochure-images
pdfimages -all brochure.pdf brochure-images/img          # img-000.jpg, img-001.png …
pdfimages -all -f 3 -l 3 brochure.pdf brochure-images/p3  # page 3 only
```

Black-and-white scans can come out in fax formats (`.ccitt`, `.jb2e`) that few programs open; `-png` in place of `-all` turns everything into PNG.

Pages into pictures. `pdftoppm` draws each page; `-r` is the resolution — 150 for a screen or a message, 300 for printing or OCR.

Want	Command
every page as PNG	`pdftoppm -r 150 -png report.pdf page` → `page-1.png` … (`page-01.png` from ten pages up)

Want	Command
JPEG, smaller files	`pdftoppm -r 150 -jpeg -jpegopt quality=85 report.pdf page`
only page 2, one file	`pdftoppm -r 300 -png -f 2 -l 2 -singlefile report.pdf page2` → `page2.png`
a thumbnail, 300 px on its long side	`pdftoppm -png -scale-to 300 -f 1 -singlefile report.pdf cover` → `cover.png`

Pictures into a PDF. ImageMagick puts one picture on each page. `-density` sets the page size, because pixels ÷ density = inches. Left out, the picture's own value is used — 72 on a phone. For an A4-wide page, use the picture's width ÷ 8.3:

Picture	`-density`	Page width
phone photo, 4032 px wide	left out (72)	56 in — 1.42 m
phone photo, 4032 px wide	480	8.4 in — 21.3 cm, A4
scan, 1240 px wide	150	8.3 in — 21 cm, A4

```
magick receipt-1.jpg receipt-2.jpg receipt-3.jpg -units PixelsPerInch -density 480 receipts.pdf
magick id-front.jpg id-back.jpg -append +repage -units PixelsPerInch -density 300 "ID card.pdf"
```

The second puts both sides of a card on one page, front above back; without `+repage` the page keeps the size of the first picture. PNGs go in without loss. JPEGs are decoded and saved again at their own quality — hard to see, but a second generation. To put JPEGs in exactly as they are, let tesseract build the PDF: it copies JPEG data untouched, and makes the text searchable as a bonus (it reads every page, so allow a few seconds each):

```
ls receipt-*.jpg > list.txt
tesseract list.txt receipts --dpi 480 -l eng+ara pdf      # writes receipts.pdf
```

Read more: `man pdfimages`, `man pdftoppm` and `man tesseract`; every ImageMagick option is on its options page.

14.4 PDFs: smaller, locked, unlocked

Shrink. Ghostscript writes the file anew and scales down every picture above the target resolution, saving it as JPEG. Text and drawings stay sharp, because they are not pictures.

```
gs -q -sDEVICE=pdfwrite -dPDFSETTINGS=/ebook -o "Scanned bill-small.pdf" "Scanned bill.pdf"
```

Setting	Pictures scaled to	For
`/screen`	72 dpi	the smallest file; small print in scans becomes hard to read
`/ebook`	150 dpi	email and upload forms — the usual choice
`/printer` · `/prepress`	300 dpi	printing; little smaller than before

What is lost is detail in the pictures, and only there. A picture is scaled only when it is more than 1.5 times the target: `/ebook` takes a 300 dpi scan down to 150, but leaves a 200 dpi one as it is. Pure black-and-white scans (1 bit per pixel) are never taken below 300 dpi by `/screen` or `/ebook`, so small print stays readable. In a test, two pages of 300 dpi photographs (7.8 MB) came out at 390 KB with `/ebook` and 115 KB with `/screen`; a PDF that is mostly text hardly changes.

For a middle way, give a target of 200 and lower that 1.5 to 1 — otherwise a 300 dpi scan stays untouched, since 300 is not more than 1.5 × 200:

```
gs -q -sDEVICE=pdfwrite -dPDFSETTINGS=/ebook \
   -dColorImageResolution=200 -dGrayImageResolution=200 \
   -dColorImageDownsampleThreshold=1.0 -dGrayImageDownsampleThreshold=1.0 \
   -o "Scanned bill-200.pdf" "Scanned bill.pdf"
```

Resolution is counted against the page: a phone photo on a page 1.4 metres wide (§14.3) is only 72 dpi there, so no setting touches it — make such PDFs at A4 width first.

Smaller without any loss: `qpdf --recompress-flate --compression-level=9 --object-streams=generate report.pdf report-small.pdf` packs the file's structure tighter without touching a picture or a letter: anything from a few per cent to a third on text-heavy files, nothing on scans.

Lock with a password.

```
qpdf --encrypt --user-password='open sesame' --owner-password='something else' --bits=256 -- \
     contract.pdf contract-locked.pdf
```

The user password opens the file. The owner password governs printing, copying and changes; qpdf insists on one, and it should differ from the user password — if they are the same, whoever can open the file can also lift its limits. AES-256 opens in every current PDF reader, phones included. To let anyone open a file but ask readers not to print or copy it, give only the owner password, with the limits after `--bits` (qpdf refuses them before it):

```
qpdf --encrypt --owner-password='something else' --bits=256 --print=none --extract=n -- \
     contract.pdf contract-noprint.pdf
```

Not every reader obeys these limits.

Remove a password you know.

```
qpdf --password='open sesame' --decrypt contract-locked.pdf contract.pdf
```

A file that opens without a password but refuses printing or copying needs no `--password`: `qpdf --decrypt` alone lifts those limits.

Look before you open, clean before you send. `pdfinfo invoice.pdf` shows the page count and size, the program that made it, and whether it is encrypted, holds a form or carries JavaScript — which an invoice or a CV has no business carrying. The lines to read:

```
Form:           none
JavaScript:     no
Pages:          2
Encrypted:      no
```

`qpdf --list-attachments invoice.pdf` lists files tucked inside it, and `qpdf --show-attachment=NAME invoice.pdf > NAME` takes one out. Before passing a file on, `qpdf --remove-info --remove-metadata draft.pdf final.pdf` drops the author, title, creating program and creation date recorded in it (the date of the last change stays).

Pitfall. A PDF with a certificate-based digital signature — the kind a reader can verify — stops validating once any program here rewrites it. Keep and send it as you received it.

Read more: Ghostscript's PDF settings (*-dPDFSETTINGS* and the image parameters) and qpdf's encryption options.

14.5 PDFs: text, scans and OCR

Text out. `pdftotext -layout statement.pdf` writes `statement.txt`, columns where they were; `-f 2 -l 3` limits it to pages 2–3, and `-` in place of a file name prints it (`pdftotext -layout statement.pdf - | less`). If nothing comes out, the PDF is a scan — pictures of pages — and needs OCR. There is no faithful PDF-to-Word converter here: the text is what you get, ready to paste into AbiWord.

Make a scan searchable. Tesseract reads the pages and writes an invisible text layer; qpdf lays it over the original, whose pages stay exactly as they were:

```
mkdir ocr
# every page as a 300 dpi picture: ocr/page-01.png, ocr/page-02.png …
pdftoppm -r 300 -png "Tenancy contract.pdf" ocr/page
# their names, in page order (the padded numbers keep page-10 after page-09)
ls ocr/page-*.png > ocr/list.txt
# tesseract reads them and writes ocr/text.pdf: the text alone, invisible
tesseract ocr/list.txt ocr/text --dpi 300 -l eng+ara -c textonly_pdf=1 pdf
# qpdf lays that text over the original pages
qpdf "Tenancy contract.pdf" --overlay ocr/text.pdf -- "Tenancy contract (searchable).pdf"
```

Check it: `pdftotext "Tenancy contract (searchable).pdf" - | head` prints the first lines it recognised.

Afterwards zathura's `/` finds words in it and text can be selected and copied; the file grows only a little. Then `gio trash ocr`. `-l eng+ara` reads English and Arabic; name only the languages on the page, since each one adds time. `tesseract --list-langs` shows the ones installed; others are packages (`dnf list 'tesseract-langpack-*'`) and go into the manifest like any program (§13.6). Turn sideways pages upright first (§14.2): OCR reads upright text best.

A photo or a picture.

Want	Command
its text, printed in the terminal	`tesseract receipt.jpg - -l eng+ara`
its text, into `receipt.txt`	`tesseract receipt.jpg receipt -l eng+ara`
a searchable PDF, `receipt.pdf`	`tesseract receipt.jpg receipt -l eng+ara pdf`

Tip: to read the text of anything on screen, press **Print** and drag around it — the shot is copied (§4.3) — then `wl-paste -t image/png | tesseract - - -l eng` prints its text in the terminal, where selecting it copies it.

14.6 Fill in and sign a PDF

Xournal++ (§11.18) writes on top of a PDF and exports a new one — the job of the "sign a PDF" websites.

1. **Once: turn your signature into a picture.** Sign plain white paper with a dark pen, photograph it close and square-on, and cut it out with a transparent background:

`sh magick signature-photo.jpg -colorspace gray -level 25%,75% -fuzz 30% -transparent white \ -trim +repage signature.png`

If grey specks of paper remain, raise `-fuzz` to 40%; if the strokes break up, lower it. Keep `signature.png` somewhere private, such as `~/Documents`. 2. Start Xournal++ and choose **File → Annotate PDF**. 3. **Fill in:** the text tool (Ctrl+Shift+T); click where an answer goes and type. The toolbar changes the font and colour. The text sits on top of the page, not inside the form's own fields. 4. **Sign:** the image tool (Ctrl+Shift+I); click where the signature goes and choose `signature.png`. Drag its corners to size it; to move it later, select it with Select Rectangle (Ctrl+Shift+R). 5. **File → Export as PDF**, under a new name such as `contract-signed.pdf`: that is the file to send. Ctrl+S keeps your marks in a separate `.xopp` file, so you can change them and export again — from a terminal too: `xournalpp -p contract-signed.pdf contract.xopp`.

Pitfall. A picture of your signature is what most forms ask for, but it is not a certificate-based digital signature, which a reader can verify and which a government or bank signing service adds.

14.7 Pictures

ImageMagick's `magick` does what the picture websites do. The pattern is always `magick input [what to do] output`, and the output's extension chooses the format. Instructions on the web often write `convert`, ImageMagick 6's name; Fedora has ImageMagick 7, where the command is `magick` (and `magick identify`, `magick mogrify` for the others).

Want	Command
iPhone photo (HEIC) → JPEG	`magick IMG_2041.HEIC IMG_2041.jpg`

Want	Command
WebP → PNG or JPEG	`magick picture.webp picture.png`
PNG → WebP, without loss	`magick chart.png -define webp:lossless=true chart.webp`
JPEG → WebP	`magick photo.jpg -quality 80 photo.webp`
half the size	`magick photo.jpg -resize 50% photo-half.jpg`
fit inside 1920 × 1920, proportions kept	`magick photo.jpg -resize 1920x1920 photo-1920.jpg` (`'1920x1920>'` only ever shrinks)
a smaller JPEG file	`magick photo.jpg -quality 75 photo-q75.jpg`
under a size limit, for upload forms	`magick photo.jpg -resize 600x600 -define jpeg:extent=200kb photo-200k.jpg`
crop a box: width × height + left + top	`magick shot.png -crop 800x600+100+50 +repage part.png`
a square from the centre	`magick photo.jpg -gravity center -crop 1:1 +repage square.jpg`
a border	`magick chart.png -bordercolor white -border 40 framed.png`
side by side · one above the other	`magick a.jpg b.jpg +append +repage both.jpg` · `-append` in place of `+append`
a "copy for …" mark across an ID scan	`magick id-front.jpg -gravity center -pointsize 48 -fill 'rgba(200,0,0,0.45)' -annotate -25x-25+0+0 'COPY FOR VISA APPLICATION ONLY' id-front-marked.jpg`
an animated GIF	`magick -delay 50 -loop 0 frame-*.png slideshow.gif` (delay in hundredths of a second)

JPEG quality 90 looks like the original; 75 is about half the file; below 60, blocks start to show. `jpeg:extent` tries qualities until the file fits.

What a photo gives away. A phone photo records where it was taken (GPS, to a few metres), when, and on which phone, and the file carries all of it wherever it goes. To see the position:

```
$ magick identify -format '%[EXIF:GPS*]' photo.jpg
…
exif:GPSLatitude=25/1,12/1,432/25
exif:GPSLatitudeRef=N
exif:GPSLongitude=55/1,16/1,372/25
exif:GPSLongitudeRef=E
…
```

The fractions are degrees, minutes and seconds: 25° 12′ 17.28″ N, 55° 16′ 14.88″ E. No output means the photo holds no position. To send a photo without any of it:

```
magick photo.jpg -auto-orient -strip photo-clean.jpg
```

`-strip` removes everything that is not the picture. `-auto-orient` comes first because "which way up" is stored the same way: stripped alone, a portrait photo turns on its side.

Many at once. Write the results into a new folder, so the originals stay and a second run does not pick up the first one's output:

```
mkdir -p small
for f in *.jpg; do magick "$f" -auto-orient -resize '1600x1600>' -quality 82 -strip "small/$f"; done
fd -d 1 -e heic -x magick {} {.}.jpg      # every HEIC here → a JPEG beside it
```

fd matches `.HEIC` and `.heic` alike and converts several files at once; `-d 1` keeps it to this folder (without it, subfolders too). Avoid `magick mogrify` unless you add `-path other-folder`: on its own it overwrites the originals.

Tip: `magick identify photo.jpg` prints the format, the size in pixels and the file size — check before and after.

Read more: ImageMagick's options page explains each option used here; `tldr magick` gives short examples.

QR codes.

Want	Command
a link as a QR code, in the terminal	`qrencode -t utf8 'https://example.org/menu'` (`-t utf8i` on the light theme)
the same as a picture	`qrencode -s 10 -o menu-qr.png 'https://example.org/menu'`
your Wi-Fi: guests join by scanning	`qrencode -s 10 -o wifi.png 'WIFI:S:Home Net;T:WPA;P:correct horse 42;;'`

`-s` is the size of one square in pixels. In the Wi-Fi text, `S:` is the network name, `T:` the security (`WPA` for a network with a password, `nopass` for an open one) and `P:` the password; a `;`, `,`, `:`, `\` or `"` inside a name or password needs a `\` in front. Start that line with a space, since it holds a password (§14.1). In the terminal, `utf8` draws the light squares in the text colour, which is light only on the dark theme; `utf8i` swaps them for the light theme.

14.8 Video and sound

ffmpeg — the full RPM Fusion build — converts nearly anything to anything. `ffprobe -hide_banner clip.mkv` first shows what a file holds; its video and audio codecs decide which recipe is needed. The lines to read look like this, for an iPhone video:

```
Stream #0:0[0x1](und): Video: hevc (Main 10) (hvc1 / 0x31637668), …, 1920x1080, …
Stream #0:1[0x2](und): Audio: aac (LC) (mp4a / 0x6134706D), …, stereo, …
```

The word after `Video:` is the video codec (`hevc`), the word after `Audio:` the sound's (`aac`).

An MP4 that plays everywhere (H.264 video, AAC sound) — from an iPhone's HEVC `.MOV`, a `.webm` or a `.mkv`:

```
ffmpeg -i IMG_4471.MOV -c:v libx264 -crf 23 -pix_fmt yuv420p -c:a aac -b:a 160k \
  -movflags +faststart IMG_4471.mp4
```

If ffprobe already shows `h264` and `aac`, only the container changes — instant, and nothing is lost: `ffmpeg -i "Phone video.mkv" -c copy "Phone video.mp4"`.

Shrink for WhatsApp or email.

```
ffmpeg -i "Phone video.mp4" \
  -vf scale=1280:1280:force_original_aspect_ratio=decrease:force_divisible_by=2 \
  -c:v libx264 -crf 28 -preset slow -pix_fmt yuv420p -c:a aac -b:a 96k \
  -movflags +faststart "Phone video-small.mp4"
```

Each part, in order:

Part	Does
`scale=1280:1280:…`	fits the picture within 1280 pixels on its long side, upright or sideways; `force_divisible_by=2` keeps both sizes even, which `yuv420p` needs
`-crf 28`	the quality: *constant rate factor*, steady quality with the size following it (below)
`-preset slow`	spends more time for a smaller file
`-pix_fmt yuv420p`	the colour format every phone plays
`-b:a 96k`	the sound at 96 kbit/s
`-movflags +faststart`	lets it start playing before it has fully arrived

`-crf`	Gives
18	looks like the original

`-crf`	Gives
23	ffmpeg's default
28	small
32	smaller still

Each 6 more roughly halves the file; 0 is lossless and 51 the worst.

Want	Command
cut 1:30–2:10, no re-encoding	`ffmpeg -ss 00:01:30 -to 00:02:10 -i talk.mp4 -c copy talk-cut.mp4`
the same, cut to the frame	`ffmpeg -ss 00:01:30 -to 00:02:10 -i talk.mp4 -c:v libx264 -crf 18 -c:a aac talk-cut.mp4`
the sound alone, untouched	`ffmpeg -i talk.mp4 -vn -c:a copy talk.m4a` (`.opus` when ffprobe says `opus`)
the sound as MP3	`ffmpeg -i talk.mp4 -vn -c:a libmp3lame -q:a 2 talk.mp3`
a WhatsApp voice note as MP3	`ffmpeg -i PTT-20261015-WA0003.opus -c:a libmp3lame -q:a 4 voice.mp3`
without sound	`ffmpeg -i clip.mp4 -c copy -an clip-silent.mp4`
one frame as a picture	`ffmpeg -ss 00:00:12 -i clip.mp4 -frames:v 1 still.png`

The keyframe catch. A copied cut can only begin on a keyframe — a complete picture, which videos carry every few seconds. ffmpeg starts at the one before your time and tells the player to skip ahead; most players start cleanly, some show a still picture for a moment. When the cut must land on the exact frame, use the second row, which re-encodes.

A GIF from a clip — four seconds from 0:05, 12 frames a second, 480 pixels wide:

```
ffmpeg -ss 5 -t 4 -i clip.mp4 \
  -vf "fps=12,scale=480:-1:flags=lanczos,split[a][b];[a]palettegen[p];[b][p]paletteuse" \
  -loop 0 clip.gif
```

`palettegen` picks the GIF's 256 colours from the clip itself, so it does not come out grainy. GIFs are large: keep them short and narrow.

Join clips without re-encoding:

```
printf "file '%s'\n" "part 1.mp4" "part 2.mp4" "part 3.mp4" > list.txt
ffmpeg -f concat -safe 0 -i list.txt -c copy joined.mp4
```

The clips must share codec, picture size and frame rate, as clips from one phone or camera do; convert mixed clips with the MP4 recipe at one size first. `-safe 0` allows spaces and full paths in the list; rename files whose names contain an apostrophe.

Read more: FFmpeg's H.264 encoding guide — CRF, presets and compatibility.

14.9 Download and record

Video and sound from the web, with yt-dlp (§11.10) — only what you have the right to keep:

Want	Command
the best version, into this folder	`yt-dlp 'LINK'`
into `~/Videos`	`yt-dlp -P ~/Videos 'LINK'`
no larger than 1080p	`yt-dlp -S res:1080 'LINK'`
only the sound, as MP3	`yt-dlp -x --audio-format mp3 'LINK'`
only 1:30–2:10 of it	`yt-dlp --download-sections '*1:30-2:10' 'LINK'`

Want	Command
one video from a playlist link	`yt-dlp --no-playlist 'LINK'`
which formats exist	`yt-dlp -F 'LINK'`

Keep the link in single quotes (§13.12). When YouTube changes its site, downloads fail until a fixed yt-dlp reaches Fedora; `sudo dnf upgrade --refresh yt-dlp` fetches exactly that, without a full `update`.

Read more: yt-dlp's README, under *Usage and Options* and *Sorting Formats*.

Record the screen with wf-recorder. **Ctrl+C** in its terminal stops it and finishes the file; keep recordings in `~/Videos`:

```
wf-recorder -f ~/Videos/screen-$(date +%F-%H%M).mp4                    # the whole screen
wf-recorder -g "$(slurp)" -f ~/Videos/clip.mp4                         # drag a region first
wf-recorder -g "$(slurp)" -a -f ~/Videos/howto.mp4                     # … with your microphone
wf-recorder -g "$(slurp)" --audio=@DEFAULT_MONITOR@ -f ~/Videos/demo.mp4 # … with what you hear
```

With two monitors and no region, it asks which one to record; `-g "$(slurp -o)"` lets you click one instead. For a file meant for a phone or a website, add `-x yuv420p`. A recording of the whole screen includes the terminal you started it from: start it, switch workspace (Super+2) and work there. An existing file is not overwritten without asking — `Overwrite? Y/n`, where Enter alone means yes — which the date in the first name avoids.

Read more: `man wf-recorder`.

Tip: for a short animation of something on screen, record an MP4 and make the GIF from it (§14.8) — smaller and smoother than recording a GIF directly.

14.10 Files: archives, checksums, trash, renaming, text

Archives, with 7-Zip (`7z`):

Want	Command
zip a folder (opens anywhere)	`7z a "Tax 2026.zip" "Tax 2026"`
7z with a password, names hidden too	`7z a "Tax 2026.7z" "Tax 2026" -p -mhe=on`
zip with a password (AES-256)	`7z a -tzip "Tax 2026.zip" "Tax 2026" -p -mem=AES256`
look inside · test it	`7z l archive.7z` · `7z t archive.7z`
unpack here · into a folder	`7z x archive.zip` · `7z x archive.zip -o"Tax 2026"` (no space after `-o`)
a `.tar.gz` or `.tar.xz`	`tar xf archive.tar.gz` (7z would only peel off the outer layer)
a `.rar` · look inside it	`bsdtar -xf archive.rar` · `bsdtar -tf archive.rar`

`-p` with nothing after it asks for the password, so it never reaches the history. A password-protected zip shows its file names to anyone, and its AES encryption may need 7-Zip on the other side; a `.7z` with `-mhe=on` hides the names as well. 7z also opens ISO images and most other formats, but not RAR: Fedora's 7-Zip leaves RAR out for licence reasons, so RAR files open with `bsdtar` (libarchive, which the books window and zathura use for comics; it unpacks RAR, though it cannot make one). In yazi, `;` then `7z x %h` unpacks the file under the cursor (§11.4).

Checksums. To check a download against the checksum its website publishes:

```
sha256sum Fedora-Server-dvd-x86_64-44-1.7.iso                              # prints it
echo 'PASTED-CHECKSUM  Fedora-Server-dvd-x86_64-44-1.7.iso' | sha256sum -c  # OK or FAILED
sha256sum -c --ignore-missing Fedora-Server-44-1.7-x86_64-CHECKSUM          # a checksum file
```

Two spaces go between the checksum and the name. A signed checksum file adds a warning about "improperly formatted lines" — its signature — which is harmless; the `OK` is what counts:

```
Fedora-Server-dvd-x86_64-44-1.7.iso: OK
sha256sum: WARNING: … lines are improperly formatted
```

A checksum only proves the file matches the list it came with. Fedora also signs that list; its download page shows how to check the signature with `gpgv` (part of gnupg2, installed).

Delete safely. `gio trash old-draft.docx` (or **d** in yazi) moves a file to the trash instead of deleting it; **g t** in yazi shows the trash, restores from it and empties it. Only emptying gives the space back (§11.4).

What fills the disk. `fd -u -S +500m . ~ -X du -h | sort -h` lists every file over 500 MB in your home, the largest last (`-u` takes in hidden folders and what a `.gitignore` leaves out; `-H` alone would skip the latter); `du -xh -d1 ~ | sort -h | tail` shows the largest folders (§13.10).

Rename many files. In yazi, select them (Space, or v for a range) and press **r**: neovim opens with one name per line. Edit them as text, then `:wq` , and yazi asks before renaming. Keep the order and the number of lines.

Typed in neovim	Does
`:%s/IMG_/Holiday-/`	on every line (`%`), `IMG_2041.jpg` → `Holiday-2041.jpg`
`:%s/^/2026-10 /`	`IMG_2041.jpg` → `2026-10 IMG_2041.jpg` (`^` is the start of a line)
Ctrl+V, move down and across, then `d`	deletes the same block of characters from every line

Text files.

Want	Command
Arabic subtitles show as gibberish	`iconv -f WINDOWS-1256 -t UTF-8 film.ar.srt > film.ar.utf8.srt`
… or just play them right	`mpv --sub-codepage=cp1256 film.mkv`
which encoding is it	`file notes.txt`
Windows line endings (`^M`) out, in place	`sed -i 's/\r$//' notes.txt`
compare two texts side by side	`nvim -d old.txt new.txt`
compare two PDFs	`pdftotext -layout old.pdf; pdftotext -layout new.pdf; nvim -d old.txt new.txt`

`file` reports old Arabic files as "ISO-8859" or "Non-ISO extended-ASCII"; if the letters are garbled, Windows-1256 is almost always the answer.

14.11 Office documents and spreadsheets

AbiWord and Gnumeric (§11.18) convert without opening a window. AbiWord writes its result next to the original; `ssconvert` writes the name you give.

Want	Command
Word → PDF	`abiword --to=pdf "Offer letter.docx"` → `Offer letter.pdf`
Word → OpenDocument · and back	`abiword --to=odt "Offer letter.docx"` · `abiword --to=docx "Offer letter.odt"`
Word → plain text	`abiword --to=txt "Offer letter.docx"`
every Word file here → PDF	`for f in *.docx; do abiword --to=pdf "$f"; done`
Excel → CSV (the sheet that was open when it was saved)	`ssconvert "Expenses October.xlsx" "Expenses October.csv"`
one sheet, by its name	`ssconvert -O 'sheet=October' budget.xlsx october.csv`
every sheet to its own CSV	`ssconvert -S budget.xlsx 'budget-%s.csv'` (`%s` is the sheet's name)

Want	Command
CSV → Excel	`ssconvert "Expenses October.csv" "Expenses October.xlsx"`
Excel → PDF · → OpenDocument	`ssconvert budget.xlsx budget.pdf` · `ssconvert budget.xlsx budget.ods`

Pitfalls. Ordinary letters and sheets convert well; tracked changes, text boxes, macros and elaborate layouts do not survive. Gnumeric writes dates and numbers into a CSV the way it displays them, so `2026-10-01` can come back as `2026/10/01` and `120.50` as `120.5`: check a CSV before you feed it to something strict.

Read more: `man ssconvert` (every option) and `abiword --help`.

14.12 Quick conversions: units, money, time

qalc (§11.17) answers in one line, repeating the question (`75 kilometers/hour ≈ 46.60283942 mph`); `-t` prints the result alone.

Want	Command	Answer
speed	`qalc -t '75 km/h to mph'`	`46.60283942 mph`
height	`qalc -t '5 ft 11 in to cm'`	`180.34 cm`
temperature	`qalc -t '30 °C to °F'`	`86 °F`
area	`qalc -t '1500 sqft to m^2'`	`139.35456 m²`
20 % of a sum	`qalc -t '4500 * 20%'`	`900`
a sum plus 5 %	`qalc -t '4500 + 5%'`	`4725`
money, at today's rate	`qalc -e '100 USD to AED'`	about 367 AED (the dirham is pegged to the dollar)
days until a date	`qalc -t 'days(today; 2026-12-25)'`	`71`, asked on 15 October 2026
data sizes	`qalc -t '3 GiB to MB'`	`3221.225472 MB`

`-e` (long form `-exrates`) fetches current exchange rates first; without it, qalc uses the rates it fetched last, and offline it says so and carries on with those. Write "20 % of" as a multiplication: qalc misreads `of`.

Time zones. `date` knows every zone and its summer time:

```
TZ=Asia/Tokyo date                                        # the time in Tokyo now
date -d 'TZ="Europe/London" 2026-10-20 15:00'             # 15:00 in London, in your time
TZ=Europe/London date -d 'TZ="Asia/Dubai" 2026-10-20 15:00' # 15:00 in Dubai, in London's time
```

On a computer set to Dubai time the second line answers 18:00, and the third 12:00 BST: London keeps summer time until 25 October 2026, and `date` knows it.

Zone names come from `timedatectl list-timezones | rg -i york`.

Tip: the password-generator websites, offline: `tr -dc 'A-Za-z0-9' </dev/urandom | head -c 24; echo` prints 24 random letters and digits.

15. Changing it yourself

Everything this desktop is — every key, colour, menu, config and package — is a text file in one folder, and any of it can be changed. This chapter shows how to do that so it lasts: which files are yours to change where they are, how the desktop itself is changed so that every update keeps the change, where each part lives, and the rules the scripts count on. Chapter 16 is about learning the languages those files are written in.



The change menu (Super+Shift+U) on both themes: every part of the desktop by name, then what is yours alone, what changed, history, apply and learn.

15.1 Two kinds of change

Yours alone. The theme never writes these files. Change them where they are, and nothing will undo it.

File	For	Read
`~/.bashrc.d/local.sh` (any name but `fedokawaii.sh`)	your aliases, functions and `export`s. Fedora's `~/.bashrc` reads every file in `~/.bashrc.d` by name, so `local.sh` comes after `fedokawaii.sh`, and what you set there wins	by every new terminal
`~/.config/dwl/autostart.local.sh`	programs to start at every login. End each one that keeps running with `&`	at the next login (its output goes to `~/.cache/dwl.log`)
`~/.config/nvim/lua/plugins/*.lua`, all but `colorscheme.lua`	LazyVim plugins, and changes to the ones it has	at neovim's next start
`~/.local/bin/` — names the desktop does not use there (`kawa-…`, `fetch`, `brave-origin`, the release tools such as `yazi`, `torrra`)	your own scripts, on your `PATH`	at once

File	For	Read
the right-hand column of the table at the start of chapter 11	each program's own settings: `yazi.toml`, `mpv/input.conf`…	by that program

Super+Shift+U → yours alone opens the first three in neovim — the first time, it starts each file with a few lines saying what it is for — and `~/.local/bin` in yazi. Two small examples:

```
# ~/.bashrc.d/local.sh
alias gs='git status --short'           # gs: what changed in this repository
mkcd() { mkdir -p -- "$1" && cd -- "$1"; } # mkcd notes/2026: make a folder and go in
export LESS='-R --mouse'                 # less scrolls with the mouse wheel
```

```
# ~/.config/dwl/autostart.local.sh
syncthing serve --no-browser &           # keeps running: ends with &
sleep 5 && notify-send "logged in" &     # a one-off, also in the background
```

The desktop itself. Every file the toolkit ships belongs to the toolkit. The theme puts the toolkit's version back each time it runs (yours goes to `~/.local/state/fedokawaii/backup/`), and every update and repair runs from the toolkit. So the desktop is changed where the toolkit comes from: your **working copy**.

15.2 The working copy

The working copy is `~/repos/fedokawaii`: the toolkit as a folder of your own, kept as a git repository. You change it, and **apply** makes it the desktop's. (To keep the copy somewhere else, set `FEDOKAWAII_COPY=/that/folder` in `~/.bashrc.d/local.sh`. That reaches `fedokawaii` in a terminal only: **Super+Shift+U**, started by dwl, never reads `~/.bashrc.d` and keeps `~/repos/fedokawaii`.)

Step	Command, or Super+Shift+U	What it does
1. make it, once	`fedokawaii copy` (the menu makes it on first use)	copies the toolkit to `~/repos/fedokawaii` and commits it unchanged: a point you can always return to. git needs your name for that commit (§11.6)
2. change	a part in the menu, or `nvim ~/repos/fedokawaii/FILE`	the menu opens the part's file in neovim, in a window of its own, with the copy as the project (**Space f f** finds any file in it)
3. look	`fedokawaii diff` · *what changed*	every line the copy changes against the toolkit, in a pager; `q` closes
4. apply	`fedokawaii apply` · *apply*	see below
5. keep	*history* (lazygit in the copy)	commit with a line that says why; push it to your GitHub repository, so another machine or a reinstall starts from it

A whole change, start to finish — a wider gap between windows:

1. **Super+Shift+U → keys.** `src/dwl/config.h` and `keybinds.txt` open side by side.
2. Find `gappx` near the top (`/gappx` in neovim): its line ends in `= 8;`, before a comment. Change `8` to `12` and save with `:w`; close neovim with `:qa`.
3. The menu asks: *apply now*. Your password, then apply rebuilds dwl.
4. Log out and in (**Super+Shift+E → log out**): the gaps are 12 pixels.
5. *history*: lazygit shows `src/dwl/config.h` changed. Press **space** to stage it, **c** to commit (`dwl: gaps 12 px, wider for the 1440p screen`), **P** to push.

What apply does, in order:

1. ShellCheck reads the shell scripts you changed and prints its notes. These are advice: apply goes on.
2. Every shell script in the copy must parse (`bash -n`). If one does not, nothing is applied, and apply names the file.
3. The copy becomes the toolkit. It is copied into a folder only root can enter, refused if it holds a symbolic link, and its `fedokawaii.conf` is merged with what you saved through *settings* (§6.1).

4. What changed decides the rest:

You changed	Apply runs	Because
`configs/`, `bin/`, `src/` (dwl's `config.h`, the C helpers), `art/`, `theme.sh`	the theme	it installs the configs, rebuilds dwl when `config.h` changed, and reloads the running session
`lib/manifest.sh`, `fedokawaii.conf`, `setup.sh`, `lib/`, `system/`	repair	it installs what the manifest gained, puts the system's parts in place (login, session launcher, SSH, time zone, DNF), and runs the theme
`docs/`, `tools/`, `*.md`, `verify.sh`, `check.sh`, `fedokawaii.sh`	nothing more	they travel with the toolkit; `verify.sh` is used at the next check
nothing	nothing	it says so

Apply asks for your password once (sudo). Run from the copy with `sudo bash ~/repos/fedokawaii/fedokawaii.sh theme`, `update` or `repair`, the theme, update and repair also make the copy the toolkit first, in the same way. `fedokawaii theme` from the menu re-applies the toolkit as it is.

Trying something live first. Edit the file where the program reads it — say `~/.config/waybar/style.css` — and watch the result (15.3 says when each part re-reads its file). When you like it:

```
fedokawaii keep ~/.config/waybar/style.css  # → the copy: configs/home/.config/waybar/style.css
fedokawaii diff                             # the change, as the toolkit will see it
fedokawaii apply
```

`keep` does not copy two kinds of file:

- files a tool draws — the colours, the icon font, the pointers. It names the source to change instead;
- files the theme fills in from `fedokawaii.conf` — they contain `@HOME@`, `@WEATHER_CITY@` or a similar token. Make the same change in the copy by hand.

Until you keep or apply, `fedokawaii drift` lists the file, and the next theme run puts the toolkit's version back (yours goes to the backup folder).

Undoing.

- *In the copy*: lazygit → *Files* → **d** discards a change not yet committed. For a commit you regret, select it in *Commits* and press **t** to revert it (§11.6). Then apply again.
- *A config, as it was before a theme run*: `~/.local/state/fedokawaii/backup/<time>/`.
- *The system*: a snapshot (§7.2).

The copy and GitHub. The project lives on GitHub as the private repository `MoeDia/fedokawaii`. Clone it to `~/repos/fedokawaii` instead of running `fedokawaii copy`:

```
git clone https://github.com/MoeDia/fedokawaii ~/repos/fedokawaii
```

A private repository asks for your GitHub user name, and for a *personal access token* as the password. You make the token on github.com: *Settings* → *Developer settings* → *Personal access tokens*. The clone's `fedokawaii.conf` is GitHub's version. Apply merges your saved settings into it (§6.1): a setting you changed through *settings* stays, unless GitHub's file changes that same setting itself since you last applied — then GitHub's value is taken, and apply names it.

15.3 Where each thing lives

Every path is inside your working copy. "When" is when the change shows after apply.

To change	Edit	Written in	When	In the menu
keys	`src/dwl/config.h` (`keys[]`), and a line in `configs/home/.config/fedokawaii/keybinds.txt`	C: one `{ … }` row per key	next login	keys
gaps, borders, which windows float	`src/dwl/config.h`: `gappx`, `borderpx`, `rules[]`	C	next login	keys
frame and border colours	`src/dwl/config.h`: `bordercolor`, `focuscolor`, from the names `SUMI`, `SHU` … defined above them	C	next login	keys
the bar: modules, their order	`configs/home/.config/waybar/config.jsonc`	JSON with comments	at once	bar
the bar: sizes, spacing	`configs/home/.config/waybar/style.css` (its colours come from the palette)	CSS, as GTK reads it	at once	bar look
what a bar module shows	`bin/kawa-cpu`, `kawa-nowplaying`, `kawa-dwlstatus`, `kawa-drives` … (the weather above quick settings: `kawa-weather`)	bash	at once	—
the menus	`bin/kawa-quick`, `kawa-power`, `kawa-audio`, `kawa-config` …; their look: `bin/kawa-menu` and `configs/home/.config/fuzzel/fuzzel.ini`	bash · INI	the next menu that opens	launcher
every colour, both themes	`tools/build-themes.py` (the palettes near its top) — the menu runs it after you save	Python: a table of hex colours	at once; the terminal's at the next login	colours
the terminal	`configs/home/.config/foot/foot.ini`	INI	next login (the terminal server)	terminal
the shell: prompt, aliases, the tools' settings	`configs/home/.bashrc.d/fedokawaii.sh`	bash	every new terminal	shell
notifications	`configs/home/.config/mako/config`	INI-like	at once	notifications
files (yazi)	`configs/home/.config/yazi/init.lua` (the panes), `theme.toml`, `keymap.toml` (keys); the file dialog's keys: `configs/home/.config/fedokawaii/yazi-pick/keymap.toml`	Lua · TOML	the next time yazi starts	files
drives	`configs/home/.config/udiskie/config.yml` (what opens by itself, the hook); `bin/kawa-drives` (the notices, the mark, the menu, eject); `configs/home/.config/yazi/plugins/drives.yazi/main.lua` (M in yazi)	YAML · bash · Lua	`kawa-drives` at once; udiskie's file at the next login; M the next time yazi starts	drives
autostart, idle and lock times	`configs/home/.config/dwl/autostart.sh` (the `swayidle` line: seconds)	POSIX sh	next login	autostart
the lock screen	`bin/kawa-lock`; its picture: `configs/home/.local/share/fedokawaii/{dark,light}/lock.xpm`	bash · XPM	the next lock	—
pixel art	`configs/home/.local/share/fedokawaii/{dark,light}/*.xpm`	XPM: a text grid, one letter per pixel	at once	—
icons	`art/icons.txt` — `#` is a pixel. The menu draws them into the font with `tools/build-icons.py`, fetching fontTools once	text grids	next login	icons
pointers	`art/cursors.txt`, then `python3 tools/build-cursors.py` in the copy	text grids	next login	—

To change	Edit	Written in	When	In the menu
packages	`lib/manifest.sh`, and a row in `PACKAGES.md`	bash lists	apply installs them (repair)	packages
settings	`fedokawaii.conf` (§3.3)	`KEY="value"` lines	apply puts each in place (repair)	settings
the system's own steps	`setup.sh` (one function per step: `s_…`, `r_…` for repair); `verify.sh` (its checks)	bash	the next update or repair	—
what this hardware gets	`lib/hardware.sh` (what it looks for: firmware, microcode, Wi-Fi, sound)	bash	the next update or repair	—
what counts as doubled or fighting	`lib/manifest.sh` (`FORBIDDEN_PKGS`), `lib/hygiene.sh` (services that fight)	bash lists	the next update or repair	—
the control (Super+U)	`system/fedokawaii`	bash	after apply	—
this handbook	`docs/fedokawaii-guide.md`; the PDF is built by `docs/build/`	Markdown	—	—

An example of each language, as it appears in these files:

```
/* C — src/dwl/config.h: one row per key: modifiers, key, what it does, its argument */
{ MODKEY,          XKB_KEY_b,   spawn,   {.v = browsercmd} },   /* Super+B → Brave */
```

```
// JSON — waybar/config.jsonc: which modules sit in the middle of the bar
"modules-center": ["custom/nowplaying"],
```

```
/* CSS — waybar/style.css: one rule for one module (the time), in the palette's colours */
#clock.time { color: @strong; padding: 0 10px 0 0; }
```

```
# INI — foot/foot.ini: a [section], then key=value lines
[main]
font=JetBrainsMono Nerd Font:size=11.5,Sumi Icons Term:pixelsize=11
```

```
-- Lua — a table describes a plugin to LazyVim
return { { "folke/zen-mode.nvim", cmd = "ZenMode" } }
```

```
# bash — lib/manifest.sh: a list (an array) of package names
PKG_APPS=(
  hyperfine            # benchmarks a command
)
```

15.4 Rules the scripts count on

1. **A file under `configs/home/` lands at the same place in your home.** `configs/home/.config/foot/foot.ini` becomes `~/.config/foot/foot.ini`, and a new file there is installed by the next apply. A file you *remove* from the copy is not deleted from your home; delete it there yourself.
2. **Drawn files are made, not edited**: `configs/home/.config/fedokawaii/themes/`, the three `gtk.css` files (GTK 4's and the two GTK 3 themes'), the colour block of `foot.ini`, `nvim/lua/plugins/colorscheme.lua`, `icons.sh`, the `SumiIcons` fonts and the pointers. Change their source — the palette, `art/icons.txt`, `art/cursors.txt` — and run the tool. The next run of the tool replaces any hand edit.

3. **Tokens are filled in at apply**: `@HOME@`, `@MONITOR_HZ@`, `@WEATHER_CITY@`, `@NIGHT_LIGHT@`, `@MPV_SCALING@`, `@ICON_CHEVRON_RIGHT@`. Leave them as they are. (`@DEFAULT_AUDIO_SINK@` is no token: it is wpctl's own name for the default output.)
4. **A script's first line decides its mode.** The theme makes a file executable when it starts with `#!`, so a new script in `bin/` begins with `#!/usr/bin/env bash`.
5. **Names**: the desktop's helpers are `kawa-…`, beside `fetch`, `brave-origin` and the release tools in `~/.local/bin`. Give your own scripts other names.
6. **A key in `config.h` gets its line in `keybinds.txt`**, so **Super+/** stays complete.
7. **A package goes into the manifest.** Never only `dnf install`: `verify` would list it (§7.3).
8. **Afterwards, `fedokawaii verify`** proves the rest still holds.

When something goes wrong:

What you see	Why	Do
apply: *these do not parse*	a shell script has a syntax error	`bash -n FILE` shows the line; often a missing `fi`, `done` or quote
apply: *dwl failed to build with src/dwl/config.h*	`config.h` has a C mistake	`fedokawaii log` shows the compiler's message and line; the dwl you have keeps running
the bar is gone after apply	`config.jsonc` is not valid JSON	`waybar -l debug` in a terminal names the line; fix it and apply
a menu looks wrong	`fuzzel.ini` has a key fuzzel does not know	`fuzzel --check-config`
the terminal will not start at login	`foot.ini` has a mistake	`foot --check-config`

16. Learning: languages, ideas, and where to read

This desktop is a good place to learn programming. Every part of it is a short, readable file you can change, apply and undo, and the files cover most of what working programmers write: shell scripts, a little C, Python, Lua, CSS, JSON and TOML. This chapter covers what to learn, in what order, the ideas behind it in plain words with an example each, where the reliable documentation is — and, in §16.7, where Linux and the trade stand in October 2026.

16.1 How to learn here

Habit	Why it works	For example
change something real	a change you can see sticks better than an exercise	the gaps between windows (§15.2), a new bar module, a shell alias
one small step, then check	each step shows whether you understood the last	`bash -n script` · `fedokawaii diff` · `fedokawaii verify`
type examples, never paste them	your hands learn the shapes: brackets, quotes, `fi`	the examples in this chapter
read the manual first, then search	the manual is right for your version; a search result may be for another	`man 5 foot.ini` before a web search
write down what you learnt	you will need it again in a month	`~/Notes/notes.md` (**Super+N**): one line per thing learnt
ask an AI to *explain*, not only to *write*	you learn from the explanation; you only get used to the code	§16.4
one language at a time	two new syntaxes at once blur together	finish §16.2's step 1 before step 2

Three tools on this desktop answer most small questions:

```
tldr tar          # a few common examples of a command
man 1 tar         # its whole manual (/word searches, n the next match, q quits)
tar --help | less # the short list of its options
```

16.2 Languages worth learning now

The surveys agree on a short list. In the 2026 Stack Overflow Developer Survey, the most used languages were JavaScript (62 %), then SQL, HTML/CSS and Python (58 % each). Rust rose to 12th place. In the 2025 survey, Python grew by 7 percentage points in a year, "the go-to language for AI, data science, and back-end development", and Rust was again the most admired language (72 %). GitHub's Octoverse 2025 found TypeScript the most used language on GitHub for the first time (August 2025): typed languages make AI-assisted code more reliable, because the type checker catches the AI's mistakes before they run. Shell scripts were the fastest-growing language in AI-related projects (+324 %). IEEE Spectrum's 2025 ranking put Python first, overall and for jobs.

The order below suits this desktop: each step uses what you learnt in the one before.

Step	Language	What it is for	Why now	Start here, on this desktop
1	**bash** (the shell)	gluing programs together; most of this desktop	half of all developers use it (48.7 %, 2025 survey); every server and container speaks it	`~/.bashrc.d/local.sh`, then `bin/kawa-weather` in your working copy
2	**Python**	data, AI and machine learning, automation, web back ends	first in IEEE Spectrum, overall and for jobs; the language of AI libraries	`tools/build-themes.py` — the palettes are a Python table

Step	Language	What it is for	Why now	Start here, on this desktop
3	**SQL**	asking questions of data kept in tables	second most used in 2026 (58 %); every company keeps its data in tables	Python's built-in `sqlite3`, below
4	**TypeScript** (and JavaScript)	web pages, web apps, tools that run in Node.js	the most used on GitHub (2025); types keep AI-written code honest	Node.js 24 is installed: it runs a `.ts` file directly
5	one of **Rust** or **Go**	fast, safe programs; command-line tools; servers	Rust: the most admired language for years. Go: the language of containers and cloud tools	add `rust cargo` or `golang` to the manifest when you start (chapter 15)
beside	**C**, only to read	dwl, the kernel, most of Linux	`config.h` is C; reading it is enough at first	`src/dwl/config.h`
beside	**Lua**	neovim, yazi, mpv scripts	small: learnt in an afternoon	`~/.config/nvim/lua/plugins/local.lua`
beside	**Markdown, JSON, TOML, YAML**	notes, configs, data files	every tool reads one of them	`~/Notes/notes.md`, `waybar/config.jsonc`

Each language's first program, runnable here:

```
# 1. bash — save as ~/hello.sh, then: bash ~/hello.sh Mohamed
name=${1:-world}                  # the first argument, or "world"
echo "hello, $name — it is $(date +%H:%M)"
```

```
# 2. Python — save as ~/hello.py, then: python3 ~/hello.py
import json, urllib.request       # parts of Python itself: nothing to install
with urllib.request.urlopen("https://wttr.in/Sharjah?format=j1") as r:
    now = json.load(r)["current_condition"][0]
print(f"{now['temp_C']} °C, {now['weatherDesc'][0]['value']}")   # e.g. 31 °C, Sunny
```

```
# 3. SQL, inside Python — save as ~/books.py, then: python3 ~/books.py
import sqlite3
db = sqlite3.connect(":memory:")  # a database that lives only while this runs
db.execute("CREATE TABLE books (title TEXT, pages INTEGER)")
db.executemany("INSERT INTO books VALUES (?, ?)", [("Kokoro", 248), ("Snow Country", 175)])
for title, pages in db.execute("SELECT title, pages FROM books WHERE pages > 200"):
    print(title, pages)           # Kokoro 248
```

```
// 4. TypeScript — save as ~/hello.ts, then: node ~/hello.ts
// (Node.js removes the types and runs the rest; it may print a one-line "experimental" note)
function area(width: number, height: number): number {
  return width * height;
}
console.log(area(1920, 1080));    // 2073600
// area("1920", 1080) would be refused by a type checker: a string is not a number
```

What AI changes, and what it does not. AI assistants now write a large share of everyday code. In the 2026 survey, 73 % of developers who use them use them daily. That makes some things matter more than before:

- **Reading code** you did not write, quickly and critically.
- **Fundamentals**: data structures, algorithms, how a computer and a network work. IEEE Spectrum expects architecture and the choice of algorithm to stay vital skills.
- **Types and tests**, which catch an assistant's mistakes before they run.
- **Version control**, so that any change, yours or an assistant's, can be undone (§11.6).
- **Explaining a problem precisely**: to a person, to an assistant, or in a commit message.

16.3 Ideas, defined, with an example each

Idea	In one line	See it here
process	a program while it runs, with its own number (PID)	`pgrep -a waybar` → `1234 waybar`
signal	a short message to a process: stop, reload, …	`pkill -USR2 waybar` makes the bar re-read its files
standard input / output / error	the three streams every program has: what it reads, what it prints, where its errors go	`ls /nope 2>/dev/null` hides the error (stream 2)
pipe `|`	one program's output becomes the next one's input	`ls ~/Books | wc -l` counts your books
redirection `>` `>>`	output into a file, replacing it or adding to it	`date >> ~/log.txt` adds a line
exit status	a number each program ends with: 0 means success	`grep -q x file; echo $?` → `0` or `1`
variable · **environment variable**	a named value; an *exported* one is passed to every program started after it	`export EDITOR=nvim` → git opens neovim
PATH	the folders the shell searches for a command, in order	`echo $PATH` · `command -v kawa-menu` → `~/.local/bin/kawa-menu`
permissions	who may read (r), write (w) and run (x) a file	`ls -l secrets.sh` → `-rw-------`: only you, never run
root · **sudo**	the user allowed everything · run one command as root	`sudo dnf install X` asks for your password
package · **repository** · **dependency**	software as one file · a server that offers packages · a package another needs	`dnf repoquery --requires mpv`
service (systemd unit)	a program systemd starts, watches and restarts	`systemctl status bluetooth`
configuration file	settings a program reads when it starts or reloads	`~/.config/foot/foot.ini`
regular expression	a pattern that matches text	`rg 'kawa-[a-z]+'` finds `kawa-menu`, `kawa-lock`…
version control · **commit** · **branch**	a history of every change · one saved change · a line of work	§11.6
container	a program with its own files, sharing the computer's kernel	Prowlarr runs in one (§11.20)
compositor	the program that draws every window on the screen and handles input	dwl, on Wayland
API · **HTTP** · **JSON**	a program's way in for other programs · how the web asks and answers · text that holds data	the Python example in §16.2 asks wttr.in's API
type	what kind of value something is: a number, text, a list	`area(1920, 1080)` in §16.2
compile · **interpret**	turn the whole program into machine code first · run it line by line	dwl is compiled (C); bash and Python are interpreted
LLM · **token** · **context** · **prompt**	a large language model · the pieces of text it reads and writes · everything it can see at once · what you ask it	§16.4

Two of these ideas in one line of shell — the pipe and the exit status:

```
pgrep -x waybar >/dev/null && echo "the bar runs" || echo "the bar is down"
# pgrep ends with 0 when it finds the bar: && runs only on 0, || only on anything else
```

16.4 Learning with an AI assistant

Claude Code is installed (§11.22). It reads the files in the folder you start it in, so start it in your working copy: `cd ~/repos/fedokawaii && claude`.

Ask	You learn
"Explain line 120 of `bin/kawa-quick`, word by word."	what each piece of the syntax does
"Why does ShellCheck say SC2086 on line 40? Show the fix and why it is safer."	the rule behind the warning
"I want the bar's CPU module to turn red above 90 %. Which file, and what would you change? Do not edit yet."	where things live, and the plan before the change
"Write three tests for this function, then explain what each one checks."	how a function can fail
"Here is the error from `fedokawaii log`. What does it mean, step by step?"	how to read an error

Rules that keep it safe:

1. **Commit first** (lazygit, **c**). Whatever it changes is then one key away from undone.
2. **Read every change before you apply it.** `fedokawaii diff` shows them all.
3. **Never paste a secret** — `secrets.sh`, a token or a password — into a prompt.
4. **Ask for sources** when it states a fact about a program, and check them in the manual. In the 2026 survey, 93 % of developers said they need sources to trust an AI's answer.
5. **Type the fix yourself** now and then, instead of accepting it. It is slower, and it is how the syntax becomes yours.

16.5 Where to read

Reliable, free and kept up to date. Official documentation comes first.

Subject	Read	Where
the shell and Linux, from zero	*The Linux Command Line*, William Shotts (a free book)	linuxcommand.org/tlcl.php
the missing basics: shell, editors, git, data	*The Missing Semester of Your CS Education* (MIT)	missing.csail.mit.edu
programming from the start	*CS50* (Harvard, free)	cs50.harvard.edu/x
bash, the reference	the GNU Bash manual · `man bash`	gnu.org/software/bash/manual
shell mistakes, explained	the ShellCheck wiki: one page per finding	shellcheck.net/wiki
git	*Pro Git* (free) · `man git-commit`	git-scm.com/book
Python	the official tutorial	docs.python.org/3/tutorial
regular expressions	Python's *Regular Expression HOWTO*	docs.python.org/3/howto/regex.html
SQL	SQLite's language reference · PostgreSQL's tutorial	sqlite.org/lang.html · postgresql.org/docs/current/tutorial.html
TypeScript	the TypeScript Handbook	typescriptlang.org/docs/handbook
JavaScript and the web	MDN Web Docs	developer.mozilla.org
Rust	*The Rust Programming Language*	doc.rust-lang.org/book
Go	*A Tour of Go*	go.dev/tour
Lua	*Programming in Lua* · neovim's `:help lua-guide`	lua.org/pil
C	the dwl wiki and its `config.def.h` · `man 3 printf` and friends	codeberg.org/dwl/dwl/wiki
machine learning	Google's Machine Learning Crash Course · fast.ai's *Practical Deep Learning*	developers.google.com/machine-learning/crash-course · course.fast.ai
prompting an assistant	Anthropic's prompt engineering guide	docs.claude.com
Fedora	the Fedora documentation · `man dnf5`	docs.fedoraproject.org

Subject	Read	Where
any Linux program	its man page · the Arch Wiki (written for Arch, mostly true here)	`man NAME` · wiki.archlinux.org

16.6 A ladder of first projects

Each step is a real change to this desktop, made the chapter 15 way, and each teaches one more thing.

Step	Project	You learn
1	three aliases and a function in `~/.bashrc.d/local.sh`	variables, quoting, functions
2	a script in `~/.local/bin` that tells you the free space on `/` and `~`	arguments, `if`, pipes, exit status
3	a new key in `config.h`: **Super+Shift+B** opens Brave in a private window	reading C, rebuild and apply
4	a bar module that shows how many updates wait (`dnf check-update -q | wc -l`, every hour)	waybar's JSON, signals, a helper script
5	a Python script that keeps each day's weather in an SQLite file and prints the week	Python, JSON from an API, SQL
6	a small TypeScript tool that turns your notes' headings into a table of contents	types, Node.js, reading files
7	the same disk-space tool from step 2, in Rust or Go	compiling, a type system, a package manager

Sources for the numbers in this chapter: the Stack Overflow Developer Survey 2025 and its 2026 results; GitHub Octoverse 2025; IEEE Spectrum, The Top Programming Languages 2025.

16.7 Where things stand, October 2026

A snapshot taken on 15 October 2026: the state of the parts of Linux this desktop is built from, and of the trade you are learning. Each fact carries the date it was true. A fact marked † rests on a secondary source only (a press report or an aggregator), because the primary one could not be read; treat it as likely, not settled. The survey figures in §16.2 are not repeated here. The last table says where to look when this section is out of date.

The Linux kernel

The kernel is the core of Linux: it drives the hardware and runs every program. `uname -r` shows the one you are running.

Version	Released	Worth knowing
6.18	30 November 2025	the newest longterm (LTS) kernel, projected to be kept until December 2028
6.19	8 February 2026	the last 6.x; the next was named 7.0
7.0	12 April 2026	Rust no longer marked experimental; lazy preemption on by default in the scheduler
7.1	14 June 2026	a record 530 first-time contributors
7.2	16 August 2026	**the current release**: 16,418 changes, the second-busiest cycle ever, from a record 2,652 developers
7.3	expected 18 October 2026 †	7.3-rc7 came out on 11 October

How a release is made (kernel.org; Torvalds's mails on LWN):

Stage	What happens	For example
merge window	two weeks in which new features go in; it closes with the first release candidate	7.2-rc1, 28 June 2026
release candidates	one a week, fixes only, usually up to rc7 or rc8	7.2-rc7, 9 August 2026

Stage	What happens	For example
mainline release	a new kernel every 9–10 weeks	7.2, 16 August 2026
stable updates	fixes for the newest release, "usually once a week", until the next mainline one	7.2.2, 28 August 2026
longterm (LTS)	a release kept for years with "only important bugfixes"; it starts with a two-year projected end, which can be extended	6.18

Six LTS series are kept (kernel.org, February 2026): 6.18 and 6.12 until December 2028, 6.6 and 6.1 until December 2027, 5.15 and 5.10 until December 2026. kernel.org says LTS kernels "are picked based on various factors"; every one in its table came out between October and December, so the next is likely near the end of 2026 — a pattern, not a promise.

Rust is no longer an experiment. At the Maintainers Summit (December 2025) the kernel's maintainers judged the Rust experiment a success, and 7.0 (April 2026) dropped the "experimental" label. Rust code in the kernel grew five-fold in the year to December 2025. What has landed: Android's binder driver (6.18); a Rust ashmem module that ships on Android 16 devices with 6.12 kernels; parts of Nova, the new driver for NVIDIA GPUs, whose 7.3 work is on starting the GPU's firmware processor (August 2026 †); and the zerocopy crate (7.2). The code that runs this desktop — amdgpu, Btrfs — is still C. DRM's maintainer, Dave Airlie, said in December 2025 that the graphics subsystem is "about a year away" from refusing new drivers written in C: an expectation, not yet a rule.

Fedora

Release	Out	Supported until
42	15 April 2025	ended 27 May 2026 †
43	28 October 2025	four weeks after Fedora 45 comes out; the schedule's date is 9 December 2026 †
44 — this desktop	28 April 2026	2 June 2027
45	**not out yet**: beta 15 September 2026, release due 20 October 2026 (27 October and 3 November held in reserve †)	about thirteen months

A release is supported until four weeks after the one two later comes out (§6.4). `grep SUPPORT_END /etc/os-release` shows the date your own system was given.

What changed from Fedora 41 to 45, and what it means here:

Release	What changed	On this desktop
41 (October 2024)	DNF 5 became `dnf`	its settings live in `/etc/dnf/libdnf5.conf.d/` (§6.6); it does not show the old dnf's history
42 (April 2025)	`/usr/sbin` became a link to `bin`, and left `$PATH`	`ls -ld /usr/sbin` → `/usr/sbin → bin`; call a command by its name, not its path
43 (October 2025)	GNOME's X11 session removed from Fedora; RPM 6.0, with enforced signatures put off	nothing for dwl, which is Wayland; Steam still runs through XWayland (§11.23)
44 (April 2026)	the `ntsync` module offered to Wine and Steam through a package recommendation	the driver GE-Proton uses (§11.23)
45	**kmscon** draws the text consoles instead of the kernel	**Ctrl+Alt+F2…F6** keep their own colours and font (§4.7)
45	RPM refuses an unsigned package by default	a downloaded `.rpm` without a valid signature is refused; `rpm --nosignature` overrides it for one command
45	`ptrace` restricted (`kernel.yama.ptrace_scope=1`) unless debugging tools are installed	`gdb -p PID` or `strace -p PID` on a running program needs `yama-ptrace-enable`, which Fedora's gdb and strace are meant to bring

Release	What changed	On this desktop
45	oo7 replaces GNOME Keyring as the desktops' password store	the manifest lists no keyring: nothing changes
45	a kernel panic shows a QR code that opens an explanation page	scan it with a phone; the trace travels in the link, not to Fedora's server
45	Fedora's own repository files move from `/etc/yum.repos.d` to `/usr/share/dnf5/repos.d` (accepted)	a file you edited in `/etc` stays, and wins

Fedora 45 also brings Python 3.15, GCC 16.2, glibc 2.44 and Go 1.27 (beta, September 2026).

The Linux desktop

Wayland is now the only session on the two big desktops. X11 programs still run on both, through XWayland, as Steam does here.

Desktop	The X11 session	When
GNOME 49	switched off by default	17 September 2025
Fedora 43	GNOME's X11 packages removed	28 October 2025
GNOME 50	removed from the code	18 March 2026
KDE Plasma 6.8	removed; Plasma 6.7 keeps it, supported "into early 2027"	14 October 2026

Xorg. The X.Org server's releases are now security fixes in its 21.1 series: 21.1.22 (April 2026), 21.1.23 (June 2026) and 21.1.24 (July 2026). **XLibre** is a fork of it, started in June 2025 by a contributor whose freedesktop.org GitLab account was blocked at the time. A proposal to make it Fedora 43's X server was withdrawn (26 June 2025); for Fedora it exists only in COPR repositories, outside Fedora's own. Neither matters to dwl, which needs no X server.

wlroots and dwl. dwl 0.9 (20 September 2026) moved to wlroots 0.20 (March 2026 †) and added screen sharing through the ext-image-copy-capture protocols, input methods, mouse-axis bindings and snapping windows to screen edges. This desktop is built on exactly these two (§3.3). The release before, dwl 0.8 (28 December 2025), was for wlroots 0.19.

NVIDIA. This desktop supports AMD and Intel graphics only (§3.3), so this matters only if you change cards. NVIDIA's own driver uses its open-source kernel modules by default from the R560 series on Turing (RTX 20, GTX 16) and newer cards (announced 17 July 2024); the 580 series is the last for the older Maxwell, Pascal and Volta (1 July 2025). The fully open alternative in Mesa, NVK, is a conformant Vulkan 1.4 driver (December 2024); from Mesa 25.1 (March 2025) OpenGL on Turing and newer runs through Zink on top of it, and Mesa 25.2 (August 2025) added Blackwell, the RTX 50 series.

HDR and colour. Wayland's colour-management protocol landed on 13 February 2025, after twelve years of work, and shipped in wayland-protocols 1.41. KDE Plasma has had partial HDR since 6.0 (28 February 2024); GNOME 48 added a High Dynamic Range switch to the display settings (19 March 2025); wlroots 0.20 supports the protocol and Sway's 1.12 test release added HDR10 (March 2026 †). dwl 0.9's release notes mention neither, so treat this desktop as standard range.

How many people use Linux. Two measures, counting different things:

Measure	Linux	Month	What it counts
StatCounter, desktop systems, worldwide	**4.54 %** (Windows about 76 %)	September 2026	3 billion page views a month on more than 1 million websites
Steam Hardware & Software Survey	**3.05 %**, down from 3.90 % †	September 2026	Valve's monthly sample of Steam users; the highest yet was 5.33 %, March 2026 †

A single month of either moves easily: the Steam share often falls when more of the sample is set to Simplified Chinese (GamingOnLinux, September 2026). Read the trend over a year.

Gaming

Proton 11.0 moved Valve's compatibility layer onto Wine 11.0 (11.0-1, 7 July 2026 †); 11.0-2 (21 August 2026) is the current release. GE-Proton follows it (§11.23). **SteamOS 3.8** became the stable release in June 2026, with first support for the Steam Machine and wider support for other makers' handhelds †.

Valve hardware	Status, October 2026
Steam Deck	OLED model only: the LCD model was discontinued and prices rose on 27 May 2026, to $789 (512 GB) and $949 (1 TB); Valve gave component costs as the reason †
Steam Machine	announced 12 November 2025 with the Steam Frame headset and a new Steam Controller †; a small PC running SteamOS on a semi-custom AMD Zen 4 / RDNA 3 chip †; sold through a waitlist, with the first sales set for 29–30 June 2026, from $1,049 (512 GB) †

Anti-cheat decides whether many online games run at all. Easy Anti-Cheat has supported Proton since 23 September 2021, but each game's developer has to switch it on. Some systems do not run on Linux: EA's Javelin (no Linux support, March 2026 †; Battlefield and EA Sports FC do not work) and Activision's Ricochet (every Call of Duty title on GamingOnLinux's list broken †). Check a game on that list before you buy it.

The software industry and AI

How developers use coding agents now. The tools have moved from suggesting a line to carrying out a task: an *agent* reads a repository, edits files, runs the tests and opens a pull request, often several at once. GitHub's Copilot app (technical preview, 2 June 2026) runs each agent session in its own git worktree, an isolated copy of the branch, and its cloud agents ask before every change by default. The same post counted 1.4 billion commits a month on GitHub, nearly double a year before.

Open-source projects have written rules for it:

Project	Rule	Since
the Linux kernel	only a human may add `Signed-off-by`; the human takes full responsibility; credit the tool as `Assisted-by: AGENT_NAME:MODEL_VERSION`	in the 7.0 documentation (April 2026)
Fedora	the contributor is accountable for every line; significant AI help should be disclosed, for example with `Assisted-by:`; AI is never the final judge in review	22 October 2025

The effect is visible: 299 commits in 7.1 carried `Assisted-by:`, and LWN ties the record number of newcomers largely to LLM-based tools (June 2026). AI also finds bugs, in numbers: by May 2026 the reports had made the kernel's security list "almost entirely unmanageable" (Torvalds), and the kernel now treats a bug found with AI as public (May 2026).

The Model Context Protocol (MCP) is an open standard for connecting an AI assistant to tools and data — files, a database, a web service — through small programs called *servers*. Anthropic gave it to the **Agentic AI Foundation**, a fund under the **Linux Foundation** co-founded by Anthropic, Block and OpenAI (9 December 2025), beside Block's goose and OpenAI's AGENTS.md. At that point it had more than 10,000 public servers and 97 million SDK downloads a month, and ChatGPT, Cursor, Gemini, Microsoft Copilot and Visual Studio Code used it (Anthropic's figures). The foundation added 97 members (24 February 2026). The current specification, 2026-07-28, dropped sessions: every request now stands on its own.

Open-weight models are published as files anyone may download and run on their own computer, so the code you show them stays on it. Two with a plain Apache 2.0 licence: OpenAI's gpt-oss-20b and gpt-oss-120b (5 August 2025; the smaller needs 16 GB of memory) and Google's Gemma 4 (2 April 2026; up to 31 billion parameters).

The job market. Figures for the United States, where the official series are fullest.

Measure	Figure	Date
software developers, QA analysts and testers (BLS)	1,905,400 jobs; median pay $134,040 a year	2025
the BLS projection for them	+10 % from 2025 to 2035, "much faster than average"	page of 27 August 2026
software-development job postings on Indeed, February 2020 = 100	**78.3**: about a fifth below early 2020	2 October 2026

Measure	Figure	Date
share of all postings that mention AI (Indeed)	5.9 %, above the 2022 peak of 3.3 %	June 2026
employment of 22–25-year-olds in AI-exposed jobs (Stanford)	about 16 % lower, on the stricter measure	to October 2025

Read together: the trade is large and projected to grow, while postings remain well below 2020 and the fall is sharpest for the youngest workers. Indeed's economists say AI roles may be much of the recent rebound in software postings (June 2026). The Stanford authors say AI is not the only cause and that the clearest change dates from 2024, not 2022 (February 2026).

What employers ask for. The World Economic Forum's *Future of Jobs Report* (7 January 2025) put AI and big data first among the fastest-growing skills, then networks and cybersecurity, then technological literacy, with creative thinking, resilience, and curiosity and lifelong learning rising too; it lists software and applications developers among the fastest-growing jobs. The Linux Foundation's *State of Tech Talent* (10 June 2026; 400 hiring and training leaders, surveyed in February 2026) found 57 % short of skills in AI security and risk and in running AI systems, security and privacy the first barrier to adopting AI (48 %, up from 17 % in 2024), and training their own staff preferred (57 %) to hiring (49 %).

Security

The xz backdoor (2024). On 29 March 2024 a report titled "backdoor in upstream xz/liblzma leading to ssh server compromise" reached the oss-security list (CVE-2024-3094): xz 5.6.0 and 5.6.1 carried hidden code. A contributor calling himself "Jia Tan" had spent a long time earning the trust of xz's one unpaid maintainer, helped by accounts, widely thought to be sock puppets, that criticised the maintainer for being slow (LWN, 2 April 2024). Fedora Rawhide and Fedora 40's pre-release received the bad versions; Fedora 40 itself never did (Fedora Magazine, 29 March 2024).

Lesson	On this desktop
a release tarball can hold code that is not in the source repository	dwl is built from one pinned git commit, and setup refuses any other (§6.3)
check every download against a known checksum	each release binary in `lib/upstream.sh` has one; a changed download is refused (§6.5)
a signature says who built a package, not only that it arrived whole	Fedora 45's RPM refuses unsigned packages; `fedokawaii verify` checks the machine against the manifest (§6.7)
one tired volunteer can be the weak point of thousands of systems	every program here comes from such projects: report bugs, test new versions

Secure Boot and the 2011 certificates. Secure Boot lets the firmware start only signed boot code. Microsoft signs Linux's first-stage loader, **shim**, so that firmware which trusts Microsoft's keys will start it; shim then checks GRUB and the kernel with the distribution's own key.

Microsoft certificate	Expires	Replaced by	Signs
Microsoft UEFI CA 2011	26–27 June 2026	Microsoft UEFI CA 2023	third-party boot loaders: shim
Microsoft Corporation KEK CA 2011	June 2026	Microsoft Corporation KEK 2K CA 2023	updates to the firmware's lists of trusted and banned keys
Microsoft Windows Production PCA 2011	October 2026	Windows UEFI CA 2023	Windows's own boot loader

What it means: a machine keeps booting with the shim it has. From October 2025 until the expiry Microsoft signed each shim with both keys; from now on a new shim carries only the 2023 signature, so a machine whose firmware lacks the 2023 certificate could fail to boot a future shim (LWN, 1 July 2026). Fedora's advice (Fedora Magazine, 11 June 2026): no need to panic, but act. Fedora 45's shim is signed with both keys; update the firmware's key list through fwupd when your maker offers it (§6.9); never remove or revoke the 2011 key, which also signs *option ROMs*, the firmware on cards such as a graphics card. To look:

```
mokutil --sb-state                    # is Secure Boot on?
mokutil --db | grep 'Subject:'        # the trusted keys: look for "Microsoft UEFI CA 2023"
sudo fwupdmgr update                  # firmware and key-list updates your maker offers (§6.9)
```

(`mokutil` may not be installed: `sudo dnf install mokutil` brings it — keep it in the manifest if you do, §13.6.)

Memory-safe languages (Rust, for one) build in checks — on the bounds of a buffer, on memory that has been freed — that prevent whole classes of bug by design. C, in which dwl and most of the kernel are written, has no such checks.

Who	What	When
CISA and FBI (US)	list starting a new product for critical infrastructure in a memory-unsafe language such as C or C++ among "product security bad practices"; existing products should publish a memory-safety roadmap by 1 January 2026	October 2024
NSA and CISA (US)	software makers should "plan for and begin using" memory-safe languages, step by step rather than by rewriting	24 June 2025
Google (Android)	memory-safety bugs fell below 20 % of Android's vulnerabilities for the first time; an estimated 0.2 per million lines of Rust against close to 1,000 in C and C++	13 November 2025
the Linux kernel	Rust no longer experimental	April 2026
the EU's Cyber Resilience Act	makers must send an early warning of an actively exploited vulnerability within 24 hours, from 11 September 2026; open-source stewards from 11 December 2027	2026

How to keep up

Source	Where	Good for
LWN.net	lwn.net	careful news and analysis of the kernel and free software; the weekly *Kernel release status*
kernel.org	kernel.org/releases.html	the current mainline, stable and LTS versions, with their projected ends
Torvalds's release mails	lore.kernel.org/lkml	what each release candidate and release brought, in his words
KernelNewbies	kernelnewbies.org/LinuxChanges	each kernel's changes in plain language (written by volunteers)
Fedora Magazine	fedoramagazine.org	releases, how-tos and alerts, such as the Secure Boot one
Fedora's change lists	fedoraproject.org/wiki/Releases/45/ChangeSet	every change in a coming release, with its owner and reasons
This Week in GNOME · This Week in Plasma	thisweek.gnome.org · blogs.kde.org	desktop work as it lands
dwl's releases	codeberg.org/dwl/dwl/releases	the compositor here, and the wlroots each version needs
Steam Hardware & Software Survey	store.steampowered.com/hwsurvey	Linux among Steam users, monthly; read the trend, not one month
StatCounter	gs.statcounter.com	desktop systems' share of web traffic
GitHub blog	github.blog	Copilot and agent features; the yearly Octoverse report
Stack Overflow Developer Survey	survey.stackoverflow.co	what developers use and think, once a year
BLS Occupational Outlook Handbook	bls.gov/ooh	US jobs, pay and projections
Phoronix	phoronix.com	fast news and benchmarks; confirm a claim at its source before relying on it

Sources:

- Kernel: kernel.org, Releases · LWN, Development statistics for 6.19 · 7.0 · 7.1 · 7.2 · LWN, Kernel release status, 15 April 2026 · 12 August 2026 · LWN, Linux 7.2-rc1 · LWN, Linux 7.2.2 · Phoronix, Linux 7.2 released · Phoronix, Linux 7.3-rc7 · Phoronix, Nova in 7.3
- Rust: LWN, The state of the kernel Rust experiment · rust: conclude the Rust experiment (patch)
- Fedora: Announcing Fedora Linux 44 · Announcing Fedora Linux 45 Beta · Fedora docs: Fedora 45 release milestone · Accepted changes for Fedora 45 · F43 · F44 · SwitchToDnf5 · Unify bin and sbin · WaylandOnlyGNOME · RPM 6.0 · UseKmsconVTConsole · Enforcing signature checking · Restrict ptrace · oo7 · DrmPanicFrontend · RelocateRpmRepoConfigsToUsr · Fedora schedule issue #218 (the EOL rule) · endoflife.date, Fedora · Warp2Search, Fedora 45 Final Freeze
- Desktop: release.gnome.org (feed) · GNOME 48 · The X11 GNOME session removal · The Register, GNOME 50 · KDE, Going all-in on a Wayland future · LWN, KDE Plasma 6.8 released · KDE's 6th Megarelease · LWN, Fedora xorg-x11-server 21.1.23 · FreeBSD VuXML, xorg-server 21.1.24 · The Register, XLibre · F43 X11Libre proposal, withdrawn · dwl releases · Phoronix, wlroots 0.20 and Sway 1.12-rc1 · NVIDIA, open-source GPU kernel modules · NVIDIA, deprecation schedule · nouveau and NVK news · Collabora, 12 years of Wayland colour management · StatCounter, desktop OS share · StatCounter, about · GamingOnLinux, Steam survey September 2026 · KitGuru, March 2026 record
- Gaming: Proton 11.0-2 (Valve, GitHub) · Phoronix, Proton 11.0-1 · Digital Citizen, SteamOS 3.8 · Game Informer, Steam Deck prices · Phoronix, Steam Machine announced · Notebookcheck, Steam Machine price and date · Epic, Easy Anti-Cheat for Linux · GamingOnLinux, EA Javelin · GamingOnLinux, anti-cheat list
- AI and industry: GitHub, the Copilot app · Linux kernel, AI coding assistants · Fedora Council, AI-assisted contributions policy · LWN, Dirk and Linus discuss AI · Anthropic, donating MCP · Agentic AI Foundation, news · MCP 2026-07-28 changelog · OpenAI, gpt-oss · Google, Gemma 4 · BLS, Software developers · FRED, Software development postings on Indeed · Indeed Hiring Lab, June 2026 · Stanford Digital Economy Lab, February 2026 · WEF, Future of Jobs Report 2025 · Linux Foundation, 2026 State of Tech Talent
- Security: oss-security, xz backdoor · LWN, Free software's not-so-eXZellent adventure · Fedora Magazine, CVE-2024-3094 · Microsoft, Secure Boot certificate expiration · LWN, Linux and Secure Boot certificate expiration · LWN, Secure Boot certificate expiration is here · Fedora Magazine, Don't Panic · Fedora test day, multi-signed shim · CISA and FBI, Product Security Bad Practices · NSA, memory-safe languages · Google, Rust in Android · European Commission, CRA reporting

17. Linux, from the ground up

The other chapters tell you what to press and which file to change. This one explains what happens underneath: how the machine starts, where its files live, who may do what, how programs run and talk to each other, and how the packages fit together into this desktop. Each idea comes with something you can type on this machine to see it for yourself.

How to read the examples.

- A line that starts with `$` is a command you type; the lines under it are what it prints. Lines without `$` in a `sh` block are commands too, with a `#` comment after them.
- Output marked *for example* varies from machine to machine: process numbers, sizes, versions, dates. The shape is what matters.
- `…` marks lines or columns left out.
- `ls` on this desktop is eza (§11.2). Where an example needs the classic `ls` output, it uses `command ls`, which skips the alias and runs coreutils' own `ls`.
- `sudo` asks for your password; everything else here only reads, unless it says so.

Terms. The words this chapter leans on most; the full glossary is §17.13.

Term	Plain meaning
kernel	the core program that drives the hardware and shares it out between all other programs
userspace	everything that is not the kernel: systemd, the shell, the desktop, your programs
distribution	a kernel plus thousands of userspace programs, built, tested and updated together: Fedora is one
package	one piece of a distribution in a single file (`.rpm`): programs, libraries, settings, and what it needs
process	a program while it runs, with its own number (PID)
unit	something systemd manages: a service, a timer, a mount, a target
upstream	the project that writes a piece of software; Fedora is *downstream* of it

17.1 What Linux is

Strictly, **Linux** is only the kernel: the program that starts first, talks to the hardware and decides which program runs when. What people call "Linux" is a whole system built around it.

Layer	What it is	On this machine	See it
kernel	Linux itself, plus its drivers (modules)	the Fedora kernel package	`uname -r` → `7.2.8-200.fc44.x86_64` (for example)
distribution	the kernel and everything around it, as one tested release	Fedora Linux 44 or 45	`cat /etc/fedora-release` → `Fedora release 44 (Forty Four)`
desktop	the programs that draw windows and take input	dwl, waybar, mako, fuzzel, foot	`dwl -v` → `dwl v0.9-dirty` (*dirty*: built with this desktop's patch)

The same kernel runs phones, servers and supercomputers. What makes this machine a desktop is the userspace on top, and Fedokawaii chooses that userspace piece by piece.

Fedora

Fedora is a community distribution sponsored by Red Hat. It comes in **editions**, each a different starting set of packages for one purpose: Workstation (GNOME), KDE Plasma, Server, Cloud, CoreOS and IoT, with Atomic desktops, Spins and Labs beside them. This desktop starts from **Server** with *Minimal Install* (§3.1): the smallest set that boots, logs in and connects to a network — Fedora's `@core` package group — and adds everything else on purpose.

Fact	Value
a new release	about every six months, aimed at April and October

Fact	Value
support for each release	until four weeks after the release two later: about thirteen months
recent releases	Fedora 43 on 28 October 2025, Fedora 44 on 28 April 2026; Fedora 45 is in beta in October 2026
where the date is written	`grep SUPPORT_END /etc/os-release`

Upstream first. Fedora tries not to differ from the projects it packages: a fix goes to the project that wrote the program, not into a Fedora-only patch, so everyone gets it and Fedora carries less of its own. The effect for you: Fedora is usually among the first to ship new versions of the kernel, Mesa, GNOME, systemd and PipeWire, close to how their authors made them, and the upstream documentation fits what you have.

Much of what this chapter describes started as a *Change* in one Fedora release and has been the default since. The ones that matter here:

Fedora	Became the default
17	the `/usr` merge: `/bin`, `/sbin` and `/lib` became links into `/usr` (§17.3)
18	`/tmp` in memory (tmpfs)
30	dbus-broker as the D-Bus message bus (§17.10) · boot entries as Boot Loader Specification files (§17.2)
31	cgroups version 2 (§17.5)
33	Btrfs for the desktop editions (Server keeps XFS on LVM, hence §3.1's custom layout) · swap on zram · systemd-resolved for DNS (§17.9)
34	PipeWire for all sound, in place of PulseAudio and JACK (§17.9) · zram as large as memory, at most 8 GiB
35	WirePlumber as PipeWire's session manager
41	DNF 5 as `dnf` (§17.7)
42	`/usr/sbin` merged into `/usr/bin` (§17.3)
43	RPM 6.0 (§17.7)
45	kmscon for the text consoles (§17.6) · RPM refuses unsigned packages by default (§17.11)

The layers of this desktop

From the hardware up to the programs you use. Each layer uses only the one below it:

```
your programs      Brave Origin · foot + bash · mpv · yazi · neovim · Steam
the session        dwl (on wlroots) · waybar · mako · fuzzel · swayidle · swaylock · kawa-*
desktop services   PipeWire + WirePlumber · xdg-desktop-portal · D-Bus · polkit agent · udiskie
system services    systemd · greetd · NetworkManager · resolved · BlueZ · firewalld · tuned
libraries          glibc · Mesa · libva · ffmpeg · GTK · Qt · fontconfig · FreeType · HarfBuzz
kernel             Linux: amdgpu or i915/xe · Btrfs · zram · SELinux · cgroups · the network stack
firmware           UEFI · shim and GRUB2 (loaded by it) · CPU microcode · GPU firmware
hardware           CPU · GPU · NVMe disk · network · sound · USB
```

The rest of this chapter walks these layers: first in time, as the machine starts (§17.2), then one subject at a time.

Read more: Fedora's editions · the release life cycle · Fedora Changes.

17.2 From power button to desktop

Starting a computer is a relay: each stage finds, checks and starts the next, then steps back.

#	Stage	What it does here	Where it lives
1	UEFI firmware	tests the hardware, reads its boot entry, loads shim from the EFI system partition	the motherboard; `/boot/efi`

#	Stage	What it does here	Where it lives
2	shim	checks GRUB's signature (Secure Boot), then starts it	`/boot/efi/EFI/fedora/shimx64.efi`
3	GRUB2	shows the boot menu, loads the kernel and the initramfs, hands over the kernel arguments	`/boot/grub2/`, `/boot/loader/entries/`
4	the kernel	starts the CPU's cores, applies early microcode, starts its drivers, unpacks the initramfs	`/boot/vmlinuz-VERSION`
5	the initramfs	a small system in memory, made by dracut: finds the disk, mounts the Btrfs subvolume `root`, switches to it	`/boot/initramfs-VERSION.img`
6	systemd	process 1: mounts the rest, starts every service, reaches `graphical.target`	`/usr/lib/systemd/systemd`
7	greetd	the login manager on tty1, configured by `/etc/greetd/fedokawaii.toml`	`greetd.service`
8	agreety	greetd's plain greeter, started through `kawa-greeter`: user name, password	`/usr/local/bin/kawa-greeter`
9	your session	PAM checks the password; logind registers a session; `start-dwl` runs as you	`/usr/local/bin/start-dwl`
10	dwl	takes the screens and input, opens the Wayland socket, runs your autostart	`/usr/local/bin/dwl`
11	autostart	starts the bar, notifications, the terminal server, idle and lock, the polkit agent, automount, the clipboard history and the bar's status reader	`~/.config/dwl/autostart.sh`

The whole relay, timed by systemd:

```
$ systemd-analyze
Startup finished in 7.912s (firmware) + 2.017s (loader) + 1.104s (kernel) + 2.876s (initrd) + 3.402s
(userspace) = 17.312s
graphical.target reached after 3.389s in userspace.
```

(for example) *firmware* is stage 1, *loader* stages 2–3, then the kernel, the initramfs and everything systemd did. `systemd-analyze blame | head` lists the slowest units, and `systemd-analyze critical-chain graphical.target` the chain of units the target waited on.

1–2. UEFI, shim and Secure Boot

The firmware keeps a list of boot entries in its own memory (NVRAM); Fedora's points at **shim** on the *EFI system partition*, a small FAT partition mounted at `/boot/efi`:

```
$ sudo ls /boot/efi/EFI/fedora
BOOTX64.CSV  grub.cfg  grubx64.efi  mmx64.efi  shim.efi  shimx64.efi
```

(for example) **Secure Boot** lets the firmware start only programs signed with a key it trusts. Firmware trusts Microsoft's key, so every distribution has a small first-stage loader, shim, signed by Microsoft. Shim carries Fedora's own certificate and uses it to check GRUB; GRUB and shim check the kernel the same way. A kernel started under Secure Boot also *locks itself down*: it refuses unsigned kernel modules, among other things. `check.sh` reads the firmware's own variable to say whether Secure Boot is on — a file, like nearly everything here (§17.3):

```
$ od -An -tu1 -j4 -N1 /sys/firmware/efi/efivars/SecureBoot-*
   1
```

`1` is on, `0` off. (The first four bytes of the file are the variable's attributes; `-j4` skips them and `-N1` reads the one byte of value.) Both work here: the drivers this desktop uses are part of Fedora's signed kernel.

3. GRUB2 and the boot entries

GRUB reads one small file per installed kernel, in the **Boot Loader Specification** format, from `/boot/loader/entries/`. With two kernels kept (§6.6) there are two:

```
$ sudo ls /boot/loader/entries
3f9c…e1-7.2.7-200.fc44.x86_64.conf
3f9c…e1-7.2.8-200.fc44.x86_64.conf
```

The long prefix is the machine's ID (`/etc/machine-id`). One entry, shortened:

```
title Fedora Linux (7.2.8-200.fc44.x86_64) 44 (…)
version 7.2.8-200.fc44.x86_64
linux /vmlinuz-7.2.8-200.fc44.x86_64
initrd /initramfs-7.2.8-200.fc44.x86_64.img
options root=UUID=… ro rootflags=subvol=root rhgb quiet amdgpu.ppfeaturemask=0xffffffff
vt.default_red=…
```

`linux` and `initrd` are the kernel and the initramfs, as paths on the `/boot` partition; `options` holds the kernel's arguments (below). Nobody edits these files by hand: `grubby` changes every entry at once, and also `/etc/default/grub`, so kernels installed later inherit the change — setup adds the AMD argument that way, and the theme the console colours:

```
sudo grubby --info=ALL | grep -E '^(kernel|args)='     # every entry's kernel and arguments
sudo grubby --update-kernel=ALL --args=quiet           # how an argument is added (for example)
```

The menu itself is hidden while the last boot succeeded (`menu_auto_hide` in GRUB's environment, `grub2-editenv - list`); a user timer, `grub-boot-success.timer`, marks a boot as successful two minutes after you log in. With the theme, the menu shows for `BOOT_MENU_SECONDS` instead (§4.7).

4. The kernel and its arguments

```
$ cat /proc/cmdline
BOOT_IMAGE=(hd0,gpt2)/vmlinuz-7.2.8-200.fc44.x86_64 root=UUID=… ro rootflags=subvol=root rhgb quiet
amdgpu.ppfeaturemask=0xffffffff vt.default_red=15,… vt.default_grn=… vt.default_blu=…
```

(for example)

Argument	Means	Set by
`root=UUID=…`	the file system to start from, by its unique ID	the installer
`ro`	mount it read-only at first; systemd remounts it writable once it is checked	the installer
`rootflags=subvol=root`	on Btrfs, the subvolume called `root`	the installer
`rhgb quiet`	a graphical boot screen if one is installed; fewer kernel messages	the installer
`amdgpu.ppfeaturemask=0xffffffff`	unlocks the AMD power controls LACT uses	setup (AMD only)
`vt.default_red=…` and its two partners	the text console's 16 colours	Theme (§4.7)

5. The initramfs and early microcode

The kernel cannot mount the root file system before it has the drivers for the disk and for Btrfs, and those live *on* that file system. The way out is the **initramfs**: a small archive, loaded by GRUB beside the kernel, holding just enough of a system to find and mount the real one. **dracut** builds it for each kernel when the kernel is installed, from what is on this machine.

At the front of the same file sits a tiny uncompressed archive with the CPU's **microcode** — the manufacturer's fixes to the processor itself — so the kernel can apply them before anything else runs (`early_microcode`, on by default). Setup installs the microcode

package for your CPU (`amd-ucode-firmware` or `microcode_ctl`, `lib/hardware.sh`) and has dracut rebuild the images.

```
$ sudo lsinitrd | grep microcode
drwxr-xr-x   2 root root    0 … kernel/x86/microcode
-rw-r--r--   1 root root  … … kernel/x86/microcode/AuthenticAMD.bin
$ journalctl -k -b -g microcode
… kernel: microcode: Current revision: 0x…
```

(for example; `GenuineIntel.bin` on an Intel CPU)

6. systemd

The kernel starts one program, process 1, and that is **systemd**. It reads its *units* — small text files describing services, mounts, timers and more (§17.5) — and starts what the *default target* needs, as much in parallel as the dependencies allow. A **target** is a named point in the start-up, a group of units:

Target	Means	Here
`multi-user.target`	the full system with text logins and network	Fedora Server's default
`graphical.target`	the same, plus a graphical login	this desktop's default: setup sets it

```
$ systemctl get-default
graphical.target
```

7–9. greetd, the login, the session

`greetd.service` is Fedora's unit with a drop-in of this desktop's (§10.3) that points it at its own file and gives it tty1 alone:

```
# /etc/greetd/fedokawaii.toml
[terminal]
vt = 1

[default_session]
command = "/usr/local/bin/kawa-greeter"
user = "greetd"
```

greetd runs that *greeter* as the unprivileged system user `greetd`. `kawa-greeter` clears the screen and replaces itself (`exec`, §17.5) with `agreety --cmd /usr/local/bin/start-dwl`. agreety asks for the name and password and passes them to greetd, which checks them through **PAM** (the library every login uses) and starts the command as you. PAM also tells **logind**, systemd's login manager, that a session has begun:

```
$ loginctl
SESSION  UID USER SEAT  LEADER CLASS   TTY  IDLE SINCE
      2 1000 you  seat0 1290   user    tty1 no   -
      3 1000 you  -     1012   manager -    no   -

2 sessions listed.
```

(for example; the columns vary a little between systemd versions) Session 2 is your desktop on tty1. Session 3, class *manager*, is your own systemd — the *user manager*, `user@1000.service` — which runs your user services (PipeWire, the portals, Prowlarr). It keeps running with nobody logged in, because setup turned **linger** on for you (§17.5).

`start-dwl` sets the environment every program in the session inherits, then starts dwl:

Variable	Value	For
`XDG_SESSION_TYPE`, `XDG_CURRENT_DESKTOP`	`wayland`, `wlroots`	programs and portals: which kind of desktop this is (§17.10)

Variable	Value	For
`XKB_DEFAULT_LAYOUT`, `XKB_DEFAULT_OPTIONS`	`us,ara`, `grp:alt_shift_toggle`	the keyboard, from `fedokawaii.conf`
`QT_QPA_PLATFORM`, `GDK_BACKEND`	`wayland;xcb`, `wayland,x11`	Qt and GTK: Wayland first, X11 as the fallback (§17.8)
`GTK_USE_PORTAL`	`1`	GTK 3 programs' file dialogs through the portal (yazi)
`EDITOR`, `TERMINAL`, `BROWSER`	`nvim`, `foot`, `brave-origin`	programs that open one of these
`XCURSOR_THEME`	`Fedokawaii`	the pixel pointers
`PATH`	`~/.local/bin` first	the `kawa-*` helpers

Should dwl be missing, or unable to start because a library is gone, `start-dwl` tries the build before it (`dwl.prev`) and otherwise shows a rescue screen with a shell, instead of a login that loops (§6.5).

10-11. dwl and the autostart

dwl opens the GPU through the kernel (§17.8), the keyboard and mouse through libinput, and a Wayland socket in `$XDG_RUNTIME_DIR` (`/run/user/1000/wayland-0`) for programs to connect to. Then it runs `~/.config/dwl/autostart.sh`, which first hands the session's variables to your user manager and D-Bus (so services they start later can open windows here), starts `dwl-session.target` (which binds systemd's `graphical-session.target`, the signal the portals wait for), and then starts the helpers:

Started	Job	Restarted if it crashes
`kawa-display`, `kawa-theme apply`	resolution and refresh rate; the theme's settings and wallpaper	— (run once)
`waybar`	the bar	yes (`keep`)
`mako`	notifications	D-Bus starts it again with the next notification
`foot --server`	every Super+Return terminal	yes
`lxqt-policykit-agent`	the password dialog for administrative actions (§17.4)	yes
`udiskie`	mounts USB drives as you plug them in, and hands each change to `kawa-drives` (§4.13, §17.10)	yes
`kawa-clipkeep` and `wl-paste --watch cliphist store`	copies that outlive their program; the clipboard history	yes
`gammastep`	night light	—
`swayidle`	the notice, the lock and the screens off when you are away	yes
`kawa-dwlstatus`	reads dwl's status for the bar's workspaces, layout, title and keyboard	yes, in the foreground

`keep` is a small shell function in the file: it runs the program again when it exits with an error, and gives up with a notice after five crashes in a minute.

The result, as the process tree shows it — reduced to the lines that matter (the full view is `ps -ef --forest`):

```
greetd --config /etc/greetd/fedokawaii.toml
 └─ greetd                                (its worker for your session)
     └─ /bin/sh /usr/local/bin/start-dwl
         └─ /usr/local/bin/dwl -s /home/you/.config/dwl/autostart.sh
             ├─ /bin/sh /home/you/.config/dwl/autostart.sh
             │   ├─ sh ── waybar            (sh: a keep loop)
             │   ├─ mako
             │   ├─ sh ── foot --server ── bash ── …   (your terminals)
             │   ├─ sh ── lxqt-policykit-agent
             │   ├─ sh ── udiskie
             │   ├─ sh ── swayidle -w …
             │   └─ kawa-dwlstatus
             └─ Xwayland …                  (only once an X11 program asks for it)
```

systemd sees the same programs grouped differently, by *control group*: everything dwl started sits in your session's scope, beside your user services (§17.5, cgroups).

Reading the whole story afterwards. Every stage from the kernel on writes to the **journal** (§17.5): `journalctl -b` is this boot from the start, `journalctl -b -1 -p err` the errors of the boot before, `journalctl -b -u greetd` the login. dwl and what it starts write to `~/.cache/dwl.log` instead.

Read more: Boot Loader Specification · systemd-analyze(1) · bootup(7) · the kernel's microcode loader · Secure Boot on Fedora.

17.3 Files and the file system

Everything is a file — nearly

Unix's oldest idea: anything you might read or write is reached through a path and the same few operations — open, read, write, close. Programs that only know how to read text can therefore read the kernel's state, a device or a setting:

```
$ cat /proc/sys/vm/swappiness           # a kernel setting (tuned sets it, §6.6)
100
$ cat /sys/class/drm/card1/device/vendor # the GPU's maker: 0x1002 is AMD, 0x8086 Intel
0x1002
$ cat /proc/asound/cards                # the sound cards the kernel found
 0 [Generic        ]: HDA-Intel - HD-Audio Generic
 …
```

(for example; the card number in `/sys/class/drm` varies) The idea has limits, and knowing them saves confusion:

Not a file, or not only a file	How it is reached instead
network cards	by name through the kernel's network interface (`ip link`); there is no `/dev/eth0`
the GPU's real work	`/dev/dri/*` is opened like a file, but drawing goes through special calls (`ioctl`) made by Mesa, not `read` and `write`
messages between programs	sockets (`/run/user/1000/bus` is one) and D-Bus (§17.10): a path to connect to, then a conversation
a process	it appears as a folder, `/proc/PID`, but you start and stop it with `fork`, `exec` and signals (§17.5)

The hierarchy

Every file hangs from one tree whose top is `/`. Each top folder has one job:

Path	Holds	Here, for example
`/usr`	the installed software: programs in `/usr/bin`, libraries in `/usr/lib64`, data in `/usr/share`. Owned by packages; never edited by hand	`/usr/bin/foot`

Path	Holds	Here, for example
`/usr/local`	software installed by this machine's owner, outside any package	`/usr/local/bin/dwl`, `/usr/local/lib/fedokawaii` (the toolkit)
`/etc`	system settings, as text	`/etc/greetd/fedokawaii.toml`, `/etc/dnf/libdnf5.conf.d/`
`/var`	data that changes as the system runs: logs, caches, databases	`/var/log/journal`, `/var/lib/fedokawaii`, `/var/cache/libdnf5`
`/home`	each user's own folder	`/home/you` (`~`)
`/boot`	the kernels, initramfs images and GRUB; `/boot/efi` is the EFI partition	`/boot/vmlinuz-7.2.8-200.fc44.x86_64`
`/run`	runtime state, in memory, gone at shutdown	`/run/user/1000` (your `$XDG_RUNTIME_DIR`: the Wayland and D-Bus sockets)
`/tmp`	temporary files, in memory (tmpfs), emptied at each start	yazi's previews
`/proc`	the kernel's view of processes and settings, made up as you read it	`/proc/cmdline`, `/proc/1234/status`
`/sys`	the kernel's view of devices and drivers	`/sys/class/drm`, `/sys/firmware/efi`
`/dev`	device files, made by the kernel and udev (§17.9)	`/dev/nvme0n1`, `/dev/dri/renderD128`, `/dev/zram0`
`/opt`	large self-contained programs from outside the distribution	Brave's package puts the browser under `/opt/brave.com/`

The `/usr` merge. Long ago `/bin` and `/sbin` held the programs needed before `/usr` could be mounted. Fedora 17 moved everything into `/usr` and left links behind, and Fedora 42 went one step further: `/usr/sbin` (programs meant for the administrator) merged into `/usr/bin`. On a system installed with Fedora 42 or later all four names lead to one folder:

```
$ command ls -ld /bin /sbin /usr/sbin
lrwxrwxrwx. 1 root root 7 … /bin → usr/bin
lrwxrwxrwx. 1 root root 8 … /sbin → usr/sbin
lrwxrwxrwx. 1 root root 3 … /usr/sbin → bin
```

Old paths such as `/usr/sbin/sshd` keep working; they simply end in `/usr/bin`.

Mounts

A disk partition, or a Btrfs subvolume, is attached to the tree at a folder, its **mount point**. `findmnt` draws the tree of mounts:

```
$ findmnt -t btrfs,ext4,vfat -o TARGET,SOURCE,FSTYPE
TARGET          SOURCE                FSTYPE
/               /dev/nvme0n1p3[/root]  btrfs
├─/boot         /dev/nvme0n1p2         ext4
│ └─/boot/efi   /dev/nvme0n1p1         vfat
└─/home         /dev/nvme0n1p3[/home]  btrfs
```

(for example) Both `/` and `/home` come from the same partition, `nvme0n1p3`: they are two Btrfs *subvolumes* of one file system, shown in brackets. `findmnt -no OPTIONS /` lists the options, among them `compress=zstd:1` (every file is compressed as it is written) and `subvol=/root`. `/proc`, `/sys`, `/run` and `/tmp` are mounts too, of file systems that live in memory: `findmnt /tmp` shows `tmpfs`.

Btrfs: subvolumes, snapshots and copy-on-write

Most file systems overwrite a changed block where it lies. Btrfs writes the new version somewhere free and then points the file at it: **copy-on-write**. Three things follow:

Because of copy-on-write	You get	Here
old blocks stay valid until nothing points at them	a **snapshot** is a second set of pointers to the same blocks: instant, and free until something changes	snapper's snapshots of `/` and `/home` (§7.2)
a file can share blocks with another	a copy that costs nothing until you edit it: `cp` makes these by itself on Btrfs (GNU coreutils 9 and later)	`cp big.iso copy.iso` returns at once
every block is checksummed	a disk that returns wrong data is noticed	the monthly scrub (§7.4)

A **subvolume** is a folder with its own tree, snapshotted as a whole; a snapshot never reaches into a subvolume nested inside it. That is why `~/Downloads` is one (§7.2): it is left out of `/home`'s snapshots, so a deleted download frees its space at once. The top folder of every subvolume has inode number 256 (inodes below):

```
sudo btrfs subvolume list /    # root, home, home/you/Downloads, and the snapshots
stat -c %i ~/Downloads         # 256
```

Inodes and links

A file's name is not part of the file. The file is an **inode** — a numbered record holding its type, permissions, owner, size, times and where its data lies — and a folder is a list of names, each pointing at an inode number. That makes two kinds of link possible:

```
$ cd "$(mktemp -d)"
$ echo hello > a.txt
$ ln a.txt b.txt          # a hard link: a second name for the same inode
$ ln -s a.txt c.txt       # a symbolic link: a tiny file holding the path "a.txt"
$ stat -c '%i %h %F %N' a.txt b.txt c.txt
1052114 2 regular file 'a.txt'
1052114 2 regular file 'b.txt'
1052131 1 symbolic link 'c.txt' → 'a.txt'
```

(for example: the inode numbers vary)

	Hard link	Symbolic link
is	another name for the same inode (`%h`, the link count, says how many)	a small file that holds a path
when the original name is deleted	the data stays until the last name goes	the link dangles: it points at nothing
across file systems or subvolumes	no	yes
to a folder	no	yes

Symbolic links hold this desktop together in several places:

Link	Points at	Why
`~/.config/fedokawaii/current`	`themes/dark` or `themes/light`	every program reads its colours through it; switching the theme re-points one link (§4.4)
`/etc/resolv.conf`	`../run/systemd/resolve/stub-resolv.conf`	DNS goes through systemd-resolved (§17.9)
`/usr/local/bin/vi`	`/usr/bin/nvim`	`vi` opens neovim
`/etc/systemd/system/display-manager.service`	greetd's unit	which login manager starts (`fedokawaii repair` checks it)

Permissions

Each file has an owner, a group, and three sets of three permissions:

```
$ command ls -l ~/.config/fedokawaii/secrets.sh /usr/local/bin/start-dwl
-rw-------. 1 you  you   120 … /home/you/.config/fedokawaii/secrets.sh
-rwxr-xr-x. 1 root root 2875 … /usr/local/bin/start-dwl
```

(for example)

```
-  rw-  ---  ---  .
│  │    │    │    └─ "." an SELinux label (below); "+" would mean an ACL as well
│  │    │    └────── others: no access
│  │    └─────────── the group: no access
│  └──────────────── the owner (you): read and write
└─────────────────── the type: - file, d folder, l link, c character device, s socket
```

Letter	On a file	On a folder	Octal
`r`	read its contents	list the names in it	4
`w`	change its contents	create, rename and delete names in it	2
`x`	run it as a program	enter it and reach what is inside	1

Add the numbers for each set: `rw-` is 6, `r-x` 5, `---` 0. So `600` is "the owner reads and writes, nobody else anything" — what the shell demands of `secrets.sh` before it reads it — and `755` is "everyone may run it, only the owner may change it":

```
chmod 600 ~/.config/fedokawaii/secrets.sh   # private
chmod +x ~/.local/bin/myscript              # add x for everyone
stat -c '%a %U:%G %n' /usr/local/bin/dwl    # 755 root:root /usr/local/bin/dwl
```

umask. New files start from 666 (folders 777) with the bits in your *umask* taken away:

```
$ umask
0002
```

(for example) `0002` takes write away from others only: new files come out `664`, folders `775`. Fedora's `/etc/bashrc` gives that to a user whose primary group has the user's own name (the usual case, and that group holds only you); otherwise the mask is `0022`, from `/etc/login.defs`, and files come out `644`.

Three special bits sit in front of the usual nine:

Bit	Octal	Shows as	Does	Example
set-user-ID	4000	`s` in the owner's `x`	the program runs as its owner, not as you	`/usr/bin/sudo` is `---s--x--x`, owned by root: that is how it can become root
set-group-ID	2000	`s` in the group's `x`	on a folder: new files inherit the folder's group	shared project folders
sticky	1000	`t` in the others' `x`	on a folder: only a file's owner may delete or rename it	`/tmp` is `drwxrwxrwt`: everyone writes there, nobody deletes another's file

ACLs (access control lists) add rules for further users or groups to one file; `ls -l` shows a `+` after the permissions. logind uses them to give the person at the keyboard the GPU and the sound devices: `command ls -l /dev/dri/card1` shows `crw-rw----+`.

SELinux labels

On Fedora every file and process also carries an **SELinux label**, and the kernel's policy says which labels of process may touch which labels of file — on top of the permissions above, even for root (§17.11). `-Z` shows them:

```
$ command ls -Z ~/.bashrc /usr/local/bin/dwl
unconfined_u:object_r:user_home_t:s0 /home/you/.bashrc
unconfined_u:object_r:bin_t:s0 /usr/local/bin/dwl
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

(for example) The part that matters is the third, the **type**: `user_home_t` for files in a home folder, `bin_t` for programs. Your own shell runs as `unconfined_t`, which the policy lets do what the permissions allow; services run in confined types. Files moved into place keep the label of where they came from, which is why setup runs `restorecon` after installing a file: it sets the label the policy expects for that path. Prowlarr's folder is mounted into its container with `:Z`, which relabels it `container_file_t`, the one type a container may use.

Read more: file-hierarchy(7) · inode(7) · Btrfs subvolumes · Unify /usr/bin and /usr/sbin.

17.4 Users, groups and privilege

The kernel knows users and groups only as numbers: a **UID** and a **GID**. Names are looked up in two text files:

```
$ getent passwd you
you:x:1000:1000:You:/home/you:/bin/bash
```

Field	Value	Means
1	`you`	the user name
2	`x`	the password lives elsewhere: hashed, in `/etc/shadow`, readable by root only
3, 4	`1000`, `1000`	the UID and the primary GID. Fedora numbers people from 1000; lower numbers are system users
5	`You`	a description, usually the full name
6, 7	`/home/you`, `/bin/bash`	the home folder and the login shell

root is UID 0: the kernel lets it past every permission check (SELinux still applies). On this machine root has no password and cannot log in (§3.1); administrators borrow its power one command at a time. **System users** such as `greetd` own a service's files and run it with less power than root.

```
$ id -nG
you wheel video input render gamemode
```

(for example)

Group	Gives	Added by
`wheel`	administrator: `sudo`, and polkit's password prompts accept your password	the installer (*administrator* ticked)
`video`, `render`	the GPU's device files, also outside a graphical session	setup
`input`	input devices directly	setup
`gamemode`	gamemode's CPU and GPU boosts	setup, with Steam

Group changes reach a session only at the next login.

sudo

`sudo COMMAND` runs one command as root after asking for *your* password. Fedora's `/etc/sudoers` allows it to the `wheel` group with one line:

```
%wheel  ALL=(ALL)       ALL
```

(*members of wheel, on any host, as any user, may run any command*.) Change sudo's rules only with `sudo visudo`, which checks the file before it saves it, so a slip cannot lock you out; for any other system file, `sudoedit FILE` opens a copy in neovim as you and puts it in place as root. Fedora still ships the original sudo; a proposal to switch to sudo-rs, a Rust rewrite, was drafted for Fedora 44 but has not been accepted.

polkit

`sudo` is for commands. **polkit** decides for *services*: when a program asks a system service over D-Bus (§17.10) to do something privileged — restart a unit, mount a disk, change a network — the service asks polkit whether this user may. polkit's rules answer *yes*, *no* or *only after authentication*, and Fedora's rules treat members of `wheel` as administrators. For the password it needs an **authentication agent** in your session: here `lxqt-policykit-agent`, started by the autostart. Try it:

```
systemctl restart bluetooth.service   # no sudo: polkit's dialog asks for your password
```

On a text console there is no graphical agent, and `systemctl` asks on the terminal itself. Removable disks need no password at all: polkit allows mounting them to whoever sits at the machine, which is how `udiskie` mounts a USB stick for you.

Why the desktop never runs as root

Run as root, and	Because
every program it starts is root: a bad web page in the browser could change the system	privilege is inherited by every child process
files you save in your home belong to root, and your own programs cannot change them	new files take their creator's UID; `verify` checks for root-owned files in your home
the session's sockets, settings and services are in the wrong place	the session lives in `/run/user/1000` and your user manager, not root's
a typing slip in a terminal can delete the system	nothing stands between root and `rm`

Use `sudo` for the one command that needs it, and nothing else.

Read more: sudoers(5) · polkit(8).

17.5 Processes, signals and services

How a program starts: fork and exec

A new process is always made by an existing one, in two steps:

1. **fork** — the process makes an exact copy of itself, the *child*, with a new number, the **PID**. The child records who made it: its parent's number, the **PPID**.
2. **exec** — the child replaces its own program with another one, keeping its PID, its open files and its environment.

When you type `ls` in bash, bash forks, the child execs `eza`, and bash waits for it to end. `start-dwl` does the same to run dwl: it stays behind as dwl's parent, so it can show the rescue screen if dwl ends at once. `kawa-greeter` instead ends with `exec agreety …`: no fork, the shell *becomes* agreety. Every process descends this way from process 1:

```
$ ps -o pid,ppid,user,args -p 1,$$,$PPID
    PID    PPID USER     COMMAND
      1       0 root     /usr/lib/systemd/systemd rhgb --switched-root --system --deserialize=…
   1352    1349 you      foot --server
   2310    1352 you      bash
```

(for example: `$$` is this shell's PID, `$PPID` its parent's; the order is by PID)

Exit status. A process ends with a number for its parent: 0 is success, anything else a failure (§11.2). A process killed by a signal reports 128 plus the signal's number:

```
$ sleep 60          # press Ctrl+C
^C
$ echo $?
130
```

130 = 128 + 2, and 2 is SIGINT.

Signals

A **signal** is a short message the kernel delivers to a process. A program may *catch* most signals and decide what to do; if it does not, each signal has a default action, which for most is to end the process.

No.	Name	Default	Sent by, or used for
1	`SIGHUP`	end	the terminal closed; many services treat it as *re-read your settings*
2	`SIGINT`	end	**Ctrl+C** in a terminal
9	`SIGKILL`	end, at once	the last resort: it cannot be caught, so the program cannot tidy up
10	`SIGUSR1`	end	free for programs: waybar shows or hides the bar; foot's server switches to the dark palette
12	`SIGUSR2`	end	waybar and btop reload their settings; foot's server switches to the light palette
15	`SIGTERM`	end	the polite stop: `kill` sends it by default, and so does systemd to stop a service
18	`SIGCONT`	continue	`fg` and `bg`
19	`SIGSTOP`	pause	pauses; cannot be caught
20	`SIGTSTP`	pause	**Ctrl+Z** in a terminal
34+n	`SIGRTMIN+n`	end	real-time signals, free for programs: waybar's custom modules (`"signal": 8` means `SIGRTMIN+8`)

(the numbers are those of x86-64; `kill -l` lists them all)

This desktop uses signals in plain sight. Switching the theme (§4.4) amounts to these, among other things (the script sends each one only after the check below):

```
pkill -USR2 waybar      # the bar reads its files again
pkill -USR2 btop        # so does btop
pkill -USR1 foot        # foot: every window to the dark palette (USR2: light)
```

and `kawa-dwlstatus` updates one part of the bar by sending `SIGRTMIN+8` (workspaces), `+9` (layout), `+10` (title) or `+11` (keyboard layout) to waybar, which re-runs only the module with that `"signal"`; others poke through it — quick settings and the screen-share portal `+12` (the mode seal), `kawa-drives` `+13` (the drive mark). Because the default action of these signals is to *end* the process, both scripts first check that the program has installed its handler. The kernel publishes that in a bit mask:

```
grep SigCgt /proc/"$(pgrep -nx waybar)"/status    # SigCgt: the signals this process catches
```

Signal *n* is bit *n − 1* of that hexadecimal number: `kawa-theme` tests bit 11 for `SIGUSR2` (12), and `kawa-dwlstatus` bit 33 + *n* for `SIGRTMIN+n`.

Jobs: the shell's own processes

`command &` starts a program in the background, and the prompt comes back at once. **Ctrl+Z** pauses the one in the foreground (`SIGTSTP`); `bg` lets it carry on in the background, `fg` brings it back to the front, `jobs` lists them, and **Ctrl+C** ends the one in front:

```
$ sleep 300
^Z
[1]+  Stopped                 sleep 300
$ bg
[1]+ sleep 300 &
$ jobs
[1]+  Running                 sleep 300 &
```

Background jobs end with the terminal. Something that must outlive it belongs in a systemd unit (below), or at least in `systemd-run --user COMMAND`, which starts it as a unit of your user manager.

Priority: nice

Each process has a **nice** value from −20 (first in line for the CPU) to 19 (last). You may raise your own programs' niceness; lowering it needs root:

```
nice -n 19 tar -cf big.tar ~/Videos      # start at the lowest priority
renice -n 10 -p 2310                     # lower the priority of process 2310
ps -o pid,ni,args -C btop                # the NI column
```

The monthly scrub (§7.4) is an example: its unit runs `btrfs scrub start -B -c 3 /` with `Nice=19`. Nice orders the CPU; `-c 3` puts its disk reads in the *idle* class, served only when nothing else wants the disk.

systemd and its units

systemd starts, watches, restarts and stops everything that runs without you starting it by hand. Each thing it manages is a **unit**, described by a small text file; the suffix says what kind:

Type	Is	Examples on this machine
`.service`	a program to start and watch	`greetd.service`, `NetworkManager.service`, `tuned.service`; yours: `pipewire.service`, `prowlarr.service`
`.socket`	a socket that starts its service when the first client connects	yours: `pipewire.socket` — PipeWire starts when something first plays a sound
`.timer`	a time at which to start a service	`fedokawaii-scrub.timer`, `snapper-timeline.timer`, `fstrim.timer`; yours: `fedokawaii-remind.timer`, `grub-boot-success.timer`
`.target`	a named group of units, a point in the start-up	`graphical.target`, `multi-user.target`; yours: `dwl-session.target`, `graphical-session.target`
`.mount` · `.swap`	a mounted file system · a swap device	`home.mount`, `boot.mount`, `tmp.mount` · `dev-zram0.swap`
`.path`	a path to watch; a change starts its service	`systemd-ask-password-console.path`
`.slice` · `.scope`	groups of processes for resources (cgroups, below)	`user.slice` · `session-2.scope`

Units come from three places, the later overriding the earlier: packages (`/usr/lib/systemd/system/`), generated at boot (`/run/systemd/generator/`), and the administrator (`/etc/systemd/system/`, with drop-in folders such as greetd's, §10.3). `systemctl status greetd` shows a unit's state, processes and last log lines; `systemctl cat greetd` the file and its drop-ins (the rest is in §17.12).

Enable and *start* are different: `start` runs a unit now, `enable` makes a link so it starts at every boot, from the `[Install]` section of its file (`enable --now` does both). A unit without one (Prowlarr's) runs only when asked.

The system manager and yours

There are two kinds of systemd: the **system manager** (process 1, root's) and one **user manager** per user (`user@1000.service`), which runs your own units as you. Add `--user` to talk to yours — no `sudo`:

```
systemctl --user status pipewire.service wireplumber.service
systemctl --user list-timers              # fedokawaii-remind.timer, grub-boot-success.timer
journalctl --user -u prowlarr -n 20       # your units' logs
```

Your units live in `~/.config/systemd/user/` (this desktop's: `dwl-session.target` and the reminder's timer and service) and in `/usr/lib/systemd/user/` (the packages'). Normally a user manager starts with your first login and stops after your last logout. Setup turns on **linger** for you (`loginctl enable-linger`): your manager then starts at boot and stays, so `update` , `repair` and `verify` can reach Prowlarr and the portals even from a text console. `ls /var/lib/systemd/linger/` lists the users who have it.

Timers, not cron

A timer is a unit that starts a service on a schedule. The monthly scrub, as setup writes it (its comment lines and `[Unit]` left out):

```
# /etc/systemd/system/fedokawaii-scrub.timer
[Timer]
OnCalendar=monthly
RandomizedDelaySec=1d
Persistent=true

[Install]
WantedBy=timers.target
```

Line	Means
`OnCalendar=monthly`	at midnight on the first of each month
`RandomizedDelaySec=1d`	plus a random delay of up to a day
`Persistent=true`	if the machine was off then, run at the next start

and it starts the service of the same name, `fedokawaii-scrub.service` . The reminder's timer counts from events instead: `OnActiveSec=2min` (two minutes after the session starts it) and `OnUnitActiveSec=6h` (six hours after each run). The older **cron** does the same job from a table of times. Fedora's minimal install brings no cron daemon and this desktop adds none: timers do the job better here, because each run is logged in the journal, a missed run catches up, and the job gets the same limits and security settings as any service. `systemctl list-timers fedokawaii-scrub.timer` shows when it runs next and when it last ran.

A container as a unit: the Prowlarr quadlet

Prowlarr runs in a rootless podman container (§11.20, §17.11), and systemd runs the container. The file `~/.config/containers/systemd/prowlarr.container` is a **quadlet**: a description in systemd's own format, which podman's *generator* — a small program systemd runs at boot and at every `daemon-reload` — turns into an ordinary service:

```
$ systemctl --user cat prowlarr.service | head -n 3
# /run/user/1000/systemd/generator/prowlarr.service
# Automatically generated by /usr/lib/systemd/user-generators/podman-user-generator
#
```

(for example) The quadlet names the image, the port (`127.0.0.1:9696` , this machine only), the folder (`%h/.config/prowlarr`), the key (a podman secret) and a health check; systemd adds what it adds to any service: restart on failure, logs in the journal, start and stop on demand. The file has no `[Install]` section, so Prowlarr runs only when a torrra search or the `prowlarr` shell function starts it.

The journal

systemd's **journald** collects the log lines of the kernel, every unit and every program that logs, with the time, the unit, the process and a priority, into binary files in `/var/log/journal` . `journalctl` reads them. Members of `wheel` may read the whole system journal without `sudo` .

Filter	Shows	Example
`-b` · `-b -1`	this boot · the one before	`journalctl -b -1 -e` (the end of the last boot)
`-u UNIT`	one unit	`journalctl -u NetworkManager -b`
`--user -u UNIT`	one of your units	`journalctl --user -u wireplumber`
`-k`	the kernel's messages	`journalctl -k -b -g amdgpu`
`-p PRIORITY`	that priority and worse: `err`, `warning`, `info`	`journalctl -b -p err`
`-t TAG`	one program's tag	`journalctl -t kawa-snap` (the DNF snapshots)
`--since` · `--until`	a time window	`journalctl --since "1 hour ago"`
`-g PATTERN`	lines matching a regular expression	`journalctl -b -g 'firmware'`
`-f`	follow new lines as they come; Ctrl+C stops	`journalctl -f -u greetd`
`-n N` · `-e`	the last N lines · jump to the end	`journalctl -n 50`

cgroups

A **control group** (cgroup) is a set of processes the kernel counts and limits together: CPU time, memory, disk and process count. systemd puts every unit in one, which is how it knows exactly which processes belong to a service — a service that forks a hundred helpers is still stopped cleanly. Fedora uses **cgroups version 2**, one tree for everything (since Fedora 31). Any process can tell you where it sits:

```
$ cat /proc/self/cgroup
0::/user.slice/user-1000.slice/session-2.scope
```

The single `0::` line is the mark of version 2. This shell was started by foot's server, which the autostart started, which dwl started: all in session 2's scope. `systemd-cgls` draws the whole tree:

```
$ systemd-cgls --no-pager
…
├─user.slice
│ └─user-1000.slice
│   ├─user@1000.service
│   │ ├─session.slice
│   │ │ ├─pipewire.service
│   │ │ │ └─1402 /usr/bin/pipewire
│   │ │ └─wireplumber.service
│   │ │   └─1404 /usr/bin/wireplumber
│   │ └─…
│   └─session-2.scope
│     ├─1301 /bin/sh /usr/local/bin/start-dwl
│     ├─1310 /usr/local/bin/dwl -s /home/you/.config/dwl/autostart.sh
│     ├─1340 waybar
│     └─…
└─system.slice
  ├─greetd.service
  │ └─1102 /usr/bin/greetd --config /etc/greetd/fedokawaii.toml
  └─…
```

(for example, shortened) Your session's programs sit in `session-2.scope`; your user services under `user@1000.service`; system services in `system.slice`. `systemd-cgtop` shows the groups using the most CPU and memory, live (`q` quits).

Read more: signal(7) · systemd.unit(5) · systemd.timer(5) · journalctl(1) · loginctl(1) · podman-systemd.unit(5).

17.6 The terminal and the shell

Five words are easy to mix up:

Word	Is	Here
console	the screen and keyboard attached to the machine, in text mode, before any desktop	tty1 (the login) and tty2–tty6 (**Ctrl+Alt+F2…F6**)
TTY	a terminal device in the kernel: `/dev/tty1` is the first console	`tty` prints the one you are on
PTY (pseudo-terminal)	a terminal device made for a program that draws a terminal in a window; it appears as `/dev/pts/N`	every foot window
terminal emulator	the program that draws the text, sends your keys and owns the PTY	foot (§11.1)
shell	the program in the terminal that reads commands and runs them	bash (§11.2)

```
$ tty            # in a foot window
/dev/pts/3
```

(for example; on the console it prints `/dev/tty2`)

On Fedora 44 the consoles are drawn by the kernel itself. Fedora 45 moves them to **kmscon**, a console that runs as an ordinary program and draws through KMS (§17.8), with its own font and colours; greetd keeps tty1 for itself (its drop-in declares a conflict with `kmsconvt@tty1.service`), so the login looks the same on both.

Three streams, redirection and pipes

Every process starts with three open **file descriptors**:

Number	Name	Normally
0	standard input	the keyboard (or a pipe)
1	standard output	the terminal (or a pipe, or a file)
2	standard error	the terminal, even when output is redirected

```
ls ~/Books > books.txt          # 1 into a file
ls /nope 2> /dev/null           # 2 thrown away
journalctl -b 2>&1 | wc -l      # 2 joined to 1, then counted
```

The autostart uses the same tools: `waybar <&- >~/.cache/waybar.log 2>&1` closes the bar's input (`<&-`) and sends both its outputs to its log. §11.2 has the full tour, by example.

Environment and PATH

Each process carries **environment variables**, a copy of its parent's; `export` marks a shell variable to be copied. The session's variables come from `start-dwl` (§17.2), so every program you start inherits `XDG_CURRENT_DESKTOP=wlroots` and the rest. **PATH** lists the folders searched for a command typed by its name:

```
$ echo "$PATH" | tr : '\n'
/home/you/.local/bin
/home/you/bin
…
/usr/local/bin
/usr/bin
$ type -a vi
vi is /usr/local/bin/vi
```

(for example) Services started by your user manager do not get your shell's PATH. That is why the screen-sharing portal's config names the helpers by their full paths (`/home/you/.local/bin/kawa-share`, written `@HOME@/…` in the toolkit): it runs with the manager's environment, not yours.

Quoting, globs and regular expressions

	A glob	A regular expression
used by	bash, before the command runs; `fd -g`	`rg`, `grep`, `sed`, `awk`, `journalctl -g`
matches	file names	text
any text	`*`	`.*`
one character	`?`	`.`
example	`*.pdf` — every PDF here	`kawa-[a-z]+` — `kawa-` followed by letters

Quote a regular expression (`rg 'kawa-[a-z]+'`), or bash may try to read it as a glob first. Quote every `"$variable"` that holds a file name (§11.2).

Scripts and the shebang

A script is a text file of commands with `x` permission. Its first line, the **shebang**, names the program that runs it:

First line	Runs with	Here
`#!/bin/sh`	the POSIX shell (bash in its plain mode, on Fedora)	`start-dwl`, `autostart.sh`
`#!/usr/bin/env bash`	the first `bash` on PATH	`kawa-theme`, `kawa-nowplaying`, `setup.sh`

```
bash -n ~/.local/bin/myscript     # check the syntax without running it
shellcheck ~/.local/bin/myscript  # explain likely mistakes (§11.5)
```

17.7 Packages, and how this desktop's pieces connect

What an RPM holds

A package is one file, `NAME-VERSION-RELEASE.ARCH.rpm`. The name, version, release and architecture together (the **NEVRA**, with an *epoch* in front when one is set) identify it exactly: `mpv-0.41.0-5.fc44.x86_64` is mpv 0.41.0, Fedora's fifth build of it, for Fedora 44, for 64-bit PCs.

Part	Holds	See it with
header (metadata)	name, version, summary, licence, what it **provides** and what it **requires**	`rpm -qi mpv` · `rpm -q --provides mpv` · `rpm -q --requires mpv`
payload	the files, in a compressed archive	`rpm -ql mpv`
scriptlets	small shell scripts run before or after installing or removing it: enabling a service, adding a user	`rpm -q --scripts greetd`
signatures	the packager's OpenPGP signature over the header and payload	`rpmkeys --checksig FILE.rpm`

`rpm` itself works on single packages and the database of what is installed (`/usr/lib/sysimage/rpm`); it does not download, and it does not solve dependencies. That is dnf's job. Asking rpm is always safe:

Question	Command	Answer, for example
which package owns this file?	`rpm -qf /usr/bin/foot`	`foot-…fc44.x86_64`
which files are in this package?	`rpm -ql greetd`	`/usr/bin/agreety`, `/usr/bin/greetd`, `/usr/lib/systemd/system/greetd.service`, …
what is it, and from whom?	`rpm -qi ffmpeg-libs`	`Name : ffmpeg-libs` … `Vendor : RPM Fusion` …

Question	Command	Answer, for example
have its files changed since?	`rpm -V greetd`	nothing when they are as packaged; a line per changed file otherwise
which keys does rpm trust?	`rpmkeys --list`	one line per key: its fingerprint and its name — Fedora's, RPM Fusion's, Brave's, Claude Code's, each COPR's

Fedora 43 brought **RPM 6.0**, which identifies keys by their full fingerprint and makes `rpmkeys` the tool for them. From Fedora 45 rpm also refuses, by default, any package whose signature it cannot check (§17.11).

dnf5

dnf (DNF 5 since Fedora 41) works with *repositories*. A **repository** is a server holding packages and a **metadata** index of them: every package's provides and requires. dnf fetches the index over HTTPS — for Fedora's own repositories it also checks it against checksums from Fedora's mirror list — and checks every package against its signature. Every install or update goes the same way:

1. **refresh** — download each enabled repository's metadata (`--refresh` forces it; this desktop fetches it only when you update, §6.8).
2. **solve** — work out a set of packages that satisfies every *requires*, with nothing in conflict. This is why `dnf install mpv` also brings mpv's libraries.
3. **show and ask** — the list of what will be installed, upgraded and removed.
4. **download and check** — each package's signature against the keys of its repository.
5. **transaction** — install everything as one unit, run the scriptlets, record it.

Around step 5 this desktop adds a snapper *pre* and *post* snapshot (§7.2), so a transaction can be undone from the disk as well. dnf keeps its own record:

```
$ dnf history list | head -n 3
ID Command line            Date and time       Action(s) Altered
58 dnf upgrade --refresh   2026-10-15 08:52:11               95
57 dnf install hyperfine   2026-10-02 19:40:03                1
```

(for example)

```
dnf history info 57          # what transaction 57 changed, package by package
sudo dnf history undo 57     # the reverse transaction: remove what it installed, and so on
```

Weak dependencies, install reasons and groups

Idea	Means	See it
Requires	must be there, or the package is not installed	`dnf repoquery --requires mpv`
Recommends (a weak dependency)	installed by default too, but may be left out or removed	`dnf repoquery --recommends tree-sitter-cli` → `nodejs` — the reason setup installs it with `--exclude='nodejs22*'` : this desktop has Node 24 under its own name, and keeps one Node.js
install reason	why a package is here: *User* (asked for by name), *Dependency*, *Weak Dependency*, *Group*	`dnf repoquery --installed --qf '%{name} %{reason}\n' mpv ffmpeg-libs` → `mpv User` and `ffmpeg-libs Dependency`
group (comps)	a named set of packages Fedora defines; the installer's *Minimal Install* is the `core` group	`dnf group list --installed`

The install reason is how `dnf autoremove` knows what may go: dependencies that nothing user-installed needs any more. Setup marks every manifest package *User*, so the clean-up can never take one (§7.3). Packages a group brought in carry the reason *Group*: a new release may add some (kmscon arrives with Fedora 45's `core` group), and `verify` counts them as Fedora's.

Keys and gpgcheck

Every repository file in `/etc/yum.repos.d/` names the key its packages must be signed with, and turns the check on:

```
# /etc/yum.repos.d/claude-code.repo, as setup writes it
[claude-code]
name=Claude Code
baseurl=https://downloads.claude.ai/claude-code/rpm/stable
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-claude-code
includepkgs=claude-code
```

`gpgcheck` (DNF 5's preferred name is `pkg_gpgcheck`) makes dnf refuse a package whose signature does not match. `includepkgs` limits what this repository may install to the one package. Setup checks the key's fingerprint against the one in `lib/manifest.sh` before trusting it; Fedora's own keys come with Fedora, in `/etc/pki/rpm-gpg/`.

The repositories this desktop uses

```
dnf repolist          # the enabled ones, by id and name
```

Repository id	Holds	Why it is here
`fedora` · `updates`	the release as it shipped · every fix since	nearly everything
`fedora-cisco-openh264`	Cisco's H.264 codec, built and shipped by Cisco for licensing reasons	enabled by Fedora by default
`rpmfusion-free` (+ `-updates`)	free software Fedora leaves out, mostly for patent reasons	the full **ffmpeg**, Mesa's **freeworld** VA-API and Vulkan drivers (H.264 and HEVC), `libheif-freeworld`
`rpmfusion-nonfree` (+ `-updates`)	software that is free to use but not open source	**Steam**; Intel's full VA-API driver
`brave-browser`	Brave's own, signed	`brave-origin`
`claude-code`	Anthropic's own, signed, fingerprint checked	`claude-code`
`copr:copr.fedorainfracloud.org:ilyaz:LACT`	a COPR (§3.2)	LACT, the AMD GPU control
`copr:copr.fedorainfracloud.org:dejan:lazygit`	a COPR	lazygit

A worked chain: a video, from mpv down to the GPU

What happens, in packages, when mpv plays a video with hardware decoding (`hwdec=vaapi` in `mpv.conf`)? Every link below can be checked on this machine. AMD is shown; §17.8 has the Intel equivalents.

```
mpv                                  the player                        Fedora
 └─ mpv-libs                         its library                       Fedora
    └─ libavcodec.so.62 (ffmpeg-libs)  decoding, the full build        RPM Fusion free
       └─ libva.so.2 (libva)         the VA-API library                Fedora
          └─ radeonsi_drv_video.so   the VA-API driver, freeworld      RPM Fusion free
             └─ /dev/dri/renderD128  the kernel's DRM interface
                └─ amdgpu.ko         the kernel driver (kernel-modules)   Fedora
                   └─ amdgpu/*.bin.xz  the GPU's firmware (amd-gpu-firmware)  Fedora
```

1. The program, and its package.

```
$ command -v mpv
/usr/bin/mpv
$ rpm -qf /usr/bin/mpv
mpv-0.41.0-5.fc44.x86_64
```

2. What it needs. mpv's work is done in `mpv-libs`, which needs ffmpeg's decoding library. A library dependency is written as the library's *soname*, not a package name:

```
$ rpm -q --requires mpv-libs | grep -E '^lib(avcodec|va)\.'
libavcodec.so.62()(64bit)
libva.so.2()(64bit)
```

(shortened)

3. Who provides it. Fedora's own build of that library is `libavcodec-free`, without the patent-encumbered codecs. Setup swapped it for RPM Fusion's full one (`dnf swap ffmpeg-free ffmpeg`), which provides the same soname, so mpv uses it without being rebuilt:

```
$ rpm -q --whatprovides 'libavcodec.so.62()(64bit)'
ffmpeg-libs-8.1.1-1.fc44.x86_64
$ dnf repoquery --installed --qf '%{name}  %{from_repo}\n' mpv ffmpeg-libs
ffmpeg-libs  rpmfusion-free-updates
mpv  updates
```

(for example)

4. Down to VA-API. ffmpeg hands decoding to the GPU through **libva**:

```
rpm -q --requires ffmpeg-libs | grep '^libva'    # libva.so.2, libva-drm.so.2, …
rpm -qf /usr/lib64/libva.so.2                     # libva-…fc44.x86_64
```

5. The driver: a runtime link, not a dependency. libva loads its driver *when it runs*, by name, from the kernel driver in use: amdgpu means `radeonsi`. No package requires the driver, so `--requires` cannot show this link; `vainfo` can:

```
$ vainfo --display drm 2>&1 | head -n 6
libva info: VA-API version 1.23.0
libva info: Trying to open /usr/lib64/dri-freeworld/radeonsi_drv_video.so
libva info: Found init function __vaDriverInit_1_23
libva info: va_openDriver() returns 0
vainfo: VA-API version: 1.23 (libva 2.23.0)
vainfo: Driver version: Mesa Gallium driver 26.0.6 for AMD Radeon RX 6700 XT (radeonsi, navi22, …)
$ rpm -qf /usr/lib64/dri-freeworld/radeonsi_drv_video.so
mesa-va-drivers-freeworld-26.0.6-1.fc44.x86_64
```

(for example) Since Fedora 44, Fedora's own VA-API drivers sit inside `mesa-dri-drivers`, in `/usr/lib64/dri/`, with H.264 and HEVC switched off. RPM Fusion's freeworld build installs beside them into `/usr/lib64/dri-freeworld/`, a folder Fedora's libva provides for exactly this, and libva opens it first. Further down `vainfo` lists `VAProfileH264High`, `VAProfileHEVCMain` and, on newer cards, `VAProfileAV1Profile0`: what `verify` checks.

6. The kernel's side. The driver talks to the kernel through a *render node*, and the kernel's `amdgpu` driver does the rest:

```
$ lspci -k -d ::0300
03:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Navi 22 [Radeon RX 6700/6700 XT/6750 XT / 6800M/6850M XT] (rev c1)
    Subsystem: …
    Kernel driver in use: amdgpu
    Kernel modules: amdgpu
$ rpm -q --whatprovides 'kmod(amdgpu.ko)'
kernel-modules-7.2.7-200.fc44.x86_64
kernel-modules-7.2.8-200.fc44.x86_64
```

(for example; `-d ::0300` picks VGA controllers; one line per kernel kept)

7. The firmware. The GPU runs small programs of its own, which the driver loads from files at start-up:

```
$ rpm -qf /usr/lib/firmware/amdgpu/aldebaran_mec.bin.xz
amd-gpu-firmware-20260916-1.fc44.noarch
```

(`aldebaran_*` is simply the first file in the folder; each GPU loads its own, named after its chip.) The package is part of Fedora's `linux-firmware` and is updated with it.

The lesson of the chain: three kinds of link join the pieces — *package dependencies* (`--requires`, solved by dnf), *runtime loading by name* (libva and its drivers, which no dependency records), and *the kernel's interfaces* (device files and firmware files). Only the first is visible to dnf, which is why setup installs the drivers explicitly and `verify` tests the result with `vainfo`.

A second chain: the music in the bar

The bar's middle shows what plays. Here the links are a config line, a script and a D-Bus name — no package requires another:

```
waybar                        the bar
 └─ "custom/nowplaying"       a module in ~/.config/waybar/config.jsonc: "exec": "kawa-nowplaying"
     └─ kawa-nowplaying       a bash script: prints one line of JSON per change
         └─ playerctl --follow  watches every MPRIS player
             └─ D-Bus, your session bus     the names org.mpris.MediaPlayer2.*
                 └─ mpv-mpris   a plugin inside mpv that registers org.mpris.MediaPlayer2.mpv
```

Link	Joined by	Check it
waybar → script	`"exec": "kawa-nowplaying", "return-type": "json"`: waybar runs it and reads each line	`pgrep -af kawa-nowplaying`
script → playerctl	it runs `playerctl --follow metadata --format …` and reads its lines	`playerctl metadata --format '{{status}} {{artist}} – {{title}}'`
playerctl → players	D-Bus: every name starting with `org.mpris.MediaPlayer2.` (§17.10)	`playerctl -l` → `mpv`
mpv → D-Bus	`mpv-mpris`, a plugin mpv loads from `/etc/mpv/scripts/` at start	`rpm -qf /etc/mpv/scripts/mpris.so` → `mpv-mpris-…`
a click → the player	`"on-click": "playerctl play-pause"`: a D-Bus call to the player	the button in the bar

The script prints lines such as `{"text":"… artist – title …","class":"playing"}`; waybar puts the text in the bar and the class in its stylesheet. Spotify's player and Brave join the same way, because they register MPRIS names too.

The manifest, setup and verify

The whole desktop is one list, applied, then proved:

```
lib/manifest.sh           every package, by job; what is forbidden; which services
     │
     ▼
setup.sh · update · repair   dnf installs the lists, marks them User, enables the services
     │
     ▼
verify.sh                 every listed package present · nothing forbidden · nothing
                          user-installed beyond the list · services running · the chains
                          above working (vainfo, mpv-mpris, the portals)
```

A package you want to keep goes into the manifest (§13.6); one installed by hand shows in `verify` as *installed outside the manifest* until it is removed or listed.

Why no Flatpak or Snap. Both are second package systems, with their own runtimes, updates and sandbox. Here they would mean a second place for software to live, out of dnf's view: `verify` could no longer prove the machine matches the list, and an update would be two updates. Everything this desktop needs is packaged for Fedora or pinned from its upstream release (§6.3), so both are forbidden in the manifest, as are PackageKit and GNOME Software. The desktop portals (§17.10), first made for Flatpak's sandbox, serve the native programs here just as well.

Read more: rpm.org documentation · rpmkeys(8) · dnf5 repoquery · dnf5 history · dnf5.conf(5) · RPM 6.0 in Fedora.

17.8 The graphics stack

From a program's drawing to light on the screen:

```
a program (Brave, foot, mpv)   draws its window into a buffer, through its toolkit (GTK, Qt):
  │                            on the GPU with Mesa (radeonsi · RADV · ANV) and the kernel's DRM,
  │                            or on the CPU
  ▼
the Wayland protocol           the program hands the finished buffer to the compositor
  ▼
dwl (on wlroots)               places the buffers and composes the screen, also on the GPU
  │  KMS
  ▼
the kernel: amdgpu, i915 or xe sends the finished picture to the monitor
```

DRM and KMS

DRM (Direct Rendering Manager) is the kernel's GPU subsystem; **KMS** (kernel mode setting) is its part that sets resolutions and refresh rates and puts pictures on the screens. Both are reached through files in `/dev/dri`:

```
$ command ls /dev/dri
by-path  card1  renderD128
```

(for example)

Node	Opened by	For
`card1`	the compositor alone, through logind	KMS: screens and modes, plus drawing
`renderD128`	any program	drawing and video decoding only: no control of the screens

The drivers

GPU	Kernel driver	OpenGL (Mesa)	Vulkan (Mesa)	Video decoding (VA-API)
AMD Radeon	`amdgpu`	radeonsi	**RADV**	radeonsi, RPM Fusion's freeworld build

GPU	Kernel driver	OpenGL (Mesa)	Vulkan (Mesa)	Video decoding (VA-API)
Intel, up to Meteor Lake and Arc Alchemist	`i915`	iris	**ANV**	iHD (`intel-media-driver`)
Intel Lunar Lake, Arc Battlemage, Panther Lake	`xe`	iris	ANV	iHD

Facts worth knowing for 2025–2026:

- **RADV** is the Vulkan driver for AMD. AMD ended its own open driver, AMDVLK, in September 2025 and now backs RADV.
- Intel's new kernel driver **xe** is the default for Lunar Lake and Battlemage since Linux 6.12, and for Panther Lake since 6.17. Meteor Lake was the last generation to use `i915` by default; older chips stay on it.
- The kernel driver also needs the GPU's **firmware** files (`amd-gpu-firmware` on AMD); setup installs whatever the kernel asked for and did not find (`lib/hardware.sh`).

Wayland, wlroots and dwl

Under X11, a separate *X server* owned the screen, and the window manager was one more client of it. Under **Wayland** the compositor *is* the display server: dwl owns the screens (KMS) and the input devices, and each program talks to it directly over a Unix socket using the Wayland protocol. Programs draw their own windows into buffers and hand them over; the compositor only arranges and composes them, and sends input to the window it belongs to.

wlroots is a library that does the heavy parts of a compositor — DRM and KMS, input through libinput, the protocols, XWayland. **dwl** is about three thousand lines of C on top of it: the layouts, the keys and the bar's status. Setup builds it against its own copy of wlroots 0.20 (§6.5).

Layer shell: the bar, notifications and menus

Ordinary windows are tiled by dwl. The desktop's own pieces use the **layer-shell** protocol (`wlr-layer-shell-unstable-v1`) instead: a surface placed on one of four layers, anchored to edges, outside the tiling.

Layer	Holds here
background	the wallpaper (swaybg)
bottom	—
top	the bar (waybar, `"layer": "top"`), which also reserves its strip of the screen (an *exclusive zone*) so windows tile below it
overlay	notifications and the level display (mako, `layer=overlay`)

fuzzel's menus are layer surfaces too. The lock screen uses its own protocol (`ext-session-lock-v1`), which guarantees that nothing else shows while it is up.

XWayland

Programs written only for X11 — Steam, many games — run through **XWayland**, an X server that is itself a Wayland client. dwl is built with it switched on (`XWAYLAND=-DXWAYLAND`) and starts it only when an X11 program first connects; copy and paste work in both directions.

Toolkits

Most programs do not speak Wayland directly; their **toolkit** does — GTK for most programs here, Qt for a few. `start-dwl` tells both to prefer Wayland and fall back to X11 through XWayland (`GDK_BACKEND=wayland,x11`, `QT_QPA_PLATFORM="wayland;xcb"`, §17.2), and Qt to take its colours and fonts from qt6ct (§5.4).

Fonts

Three libraries put text on the screen, in this order:

Library	Job	Ask it
fontconfig	which font file to use for a name, a language and a missing character	`fc-match monospace` · `fc-list :lang=ar family`
HarfBuzz	*shaping*: which glyphs, in which order and position — essential for Arabic, which joins its letters	(inside every toolkit)
FreeType	turns each glyph's outline into pixels	(inside every toolkit)

```
$ fc-match monospace
JetBrainsMonoNerdFont-Regular.ttf: "JetBrainsMono Nerd Font" "Regular"
```

(for example) The rules are in `~/.config/fontconfig/fonts.conf` (§5.1); after changing them, `fc-cache -f` rebuilds the index.

Read more: Wayland's architecture · wlr-layer-shell · the kernel's DRM documentation · Intel's xe GPUs · Mesa.

17.9 Sound, devices and network

Sound: ALSA, PipeWire, WirePlumber

Layer	Is	Here
ALSA	the kernel's sound drivers, and the oldest way for a program to reach a sound card	`/proc/asound/cards` ; `/dev/snd/`
PipeWire	one sound server for every program: it mixes streams, moves them between devices and passes sound between programs (and video, for screen sharing)	`pipewire.service` , started by `pipewire.socket`
pipewire-pulse	PipeWire speaking PulseAudio's protocol, so programs written for PulseAudio (Brave, Steam) work unchanged	`pipewire-pulse.service` (package `pipewire-pulseaudio`)
pipewire-alsa	sends programs that open ALSA directly (spotify_player) through PipeWire	a plugin, no process of its own
WirePlumber	PipeWire's *session manager*: the policy — which device is the default, where a new stream goes, what is remembered	`wireplumber.service`

PipeWire replaced PulseAudio (desktop sound) and JACK (low-latency studio sound) as Fedora's default in Fedora 34; WirePlumber became its session manager in Fedora 35. All of it runs as your own user services, not as root:

```
$ wpctl status
…
Audio
 ├─ Devices:
 │      45. Navi 21/23 HDMI/DP Audio Controller [alsa]
 │      46. Family 17h/19h HD Audio Controller  [alsa]
 ├─ Sinks:
 │  *   58. Family 17h/19h HD Audio Controller Analog Stereo [vol: 0.40]
 ├─ Sources:
 │  *   59. Family 17h/19h HD Audio Controller Analog Stereo [vol: 1.00]
…
```

(for example, shortened) A **sink** is an output, a **source** an input; `*` marks the default. The list also holds *Share* and *Share (as microphone)*, this desktop's own pair for calls (`~/.config/pipewire/pipewire.conf.d/60-share.conf` , §4.10).

```
wpctl get-volume @DEFAULT_AUDIO_SINK@       # Volume: 0.40 — what the bar shows as 40
wpctl set-mute @DEFAULT_AUDIO_SINK@ toggle  # what right-clicking the volume does
pw-top                                      # every stream live, with its load (q quits)
```

Devices: the kernel, udev and /dev

When a device appears — at boot, or a USB stick plugged in — the kernel's driver announces it, and **udev** (`systemd-udevd`) reacts: it creates the file in `/dev`, gives it its owner, permissions and stable names, and tells anyone listening. Its rules live in `/usr/lib/udev/rules.d/` (the packages') and `/etc/udev/rules.d/` (yours).

```
command ls -l /dev/disk/by-label/                  # stable names, by label (also by-uuid, by-id)
udevadm info /dev/dri/card1 | head                 # what udev knows about one device
udevadm monitor --udev                             # watch devices come and go; Ctrl+C stops
```

Kernel modules that no device asks for are loaded from `/etc/modules-load.d/`: setup puts `i2c-dev` there, for monitor brightness over DDC/CI (§4.2).

libinput reads keyboards, mice and touchpads for the compositor: wlroots uses it, so dwl gets pointer acceleration, tapping and scrolling the same way every Wayland desktop does. The keyboard layout itself comes from **XKB** (`XKB_DEFAULT_LAYOUT`, §17.2).

Bluetooth: BlueZ

BlueZ is the Bluetooth stack: the kernel part, and the daemon `bluetoothd` (`bluetooth.service`). Quick settings and Blueman use it; so can you:

```
bluetoothctl show          # the adapter: powered, discoverable
bluetoothctl devices       # paired and known devices
```

Bluetooth headphones appear in PipeWire as one more sink once connected.

Network: NetworkManager

NetworkManager brings up the network cards, keeps Wi-Fi and wired connections, and hands out DNS settings. `nmcli` is its command-line face (and `nmtui`, in quick settings, a menu):

```
$ nmcli device
DEVICE  TYPE      STATE                   CONNECTION
enp5s0  ethernet  connected               Wired connection 1
lo      loopback  connected (externally)  lo
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp5s0           UP             192.168.1.23/24 fe80::…/64
```

(for example) Its settings are in `/etc/NetworkManager/`; this desktop adds one drop-in (`conf.d/90-fedokawaii.conf`) that turns Wi-Fi power saving off.

Names: systemd-resolved

Turning `fedoraproject.org` into an address is the job of **systemd-resolved**, Fedora's default since Fedora 33. Programs ask a local *stub* at `127.0.0.53`; resolved asks the DNS servers NetworkManager gave it, and caches the answers:

```
$ grep -v '^#' /etc/resolv.conf
nameserver 127.0.0.53
options edns0 trust-ad
search .
$ resolvectl status
Global
  resolv.conf mode: stub
…
Link 2 (enp5s0)
…
       DNS Servers: 192.168.1.1
…
```

(for example) `resolvectl query NAME` looks a name up through it, and `resolvectl flush-caches` empties the cache.

The firewall: firewalld

firewalld holds the kernel's packet filter. It sorts networks into **zones**, each a list of what may come in; a network card belongs to one zone. On Fedora Server the default zone is normally `FedoraServer`. Setup takes `ssh` and `cockpit` out of it (with `SSH_SERVER="no"`) and opens torrra's port:

```
$ sudo firewall-cmd --get-default-zone
FedoraServer
$ sudo firewall-cmd --list-services
dhcpv6-client
$ sudo firewall-cmd --list-ports
6881/tcp 6881/udp
```

(for example) Port 6881 is open on purpose, so other peers can reach torrra while it downloads. Prowlarr listens on this machine only (`127.0.0.1:9696`), out of the network's reach whatever the firewall says. `ss -tln` shows everything that listens, and where.

Read more: PipeWire · pipewire-pulse(1) · WirePlumber · udev(7) · nmcli(1) · systemd-resolved(8) · firewalld.

17.10 The buses that tie a desktop together

D-Bus

Programs on a desktop need to ask each other things: *show this notice*, *pause*, *mount this disk*, *open a file dialog*. **D-Bus** is the message bus they use. Two buses run:

Bus	Run by	Carries	Examples
the **system bus**	the system's `dbus-broker.service`	messages to system services	NetworkManager, logind, systemd, polkit, udisks2, BlueZ, resolved
your **session bus**	your user manager's `dbus-broker.service`, socket `/run/user/1000/bus`	messages between your programs	mako, the portals, MPRIS players

Fedora uses **dbus-broker**, a newer implementation of the same protocol, since Fedora 30. Every service on a bus has a **name**, publishes **objects** at paths, and each object offers **interfaces** with methods, properties and signals:

```
name       org.mpris.MediaPlayer2.mpv
object     /org/mpris/MediaPlayer2
interface  org.mpris.MediaPlayer2.Player
  method     PlayPause()
  property   PlaybackStatus  "Playing"
```

`busctl` reads all of it:

```
busctl list | head                    # the system bus: every name
busctl --user list | grep -E 'mpris|Notifications|portal'
busctl --user tree org.mpris.MediaPlayer2.mpv
busctl --user get-property org.mpris.MediaPlayer2.mpv /org/mpris/MediaPlayer2 \
  org.mpris.MediaPlayer2.Player PlaybackStatus      # s "Playing"
busctl --user monitor org.freedesktop.Notifications # watch messages go by; Ctrl+C stops
```

A service need not be running to be reached: D-Bus can **activate** it — start it when the first message for its name arrives. That is how mako comes back after it was closed, and how the portals start the first time a program asks for a file dialog.

The desktop portals

A **portal** is a D-Bus service through which a program asks the desktop for something it should not, or cannot, do alone: open a file dialog, share the screen, read the dark or light setting. The front end, `xdg-desktop-portal`, takes every request and hands it to a **backend**, chosen by a file named after `XDG_CURRENT_DESKTOP` — here `wlroots`, so `~/.config/xdg-desktop-portal/wlroots-portals.conf` (§4.9):

Interface	Backend	Does
`FileChooser`	`termfilechooser`, else `gtk`	yazi in a floating terminal, through `kawa-pick`
`ScreenCast` · `Screenshot`	`wlr`	screen sharing and screenshots for Brave and others
`Settings`	`gtk`	the colour scheme GTK 4 and Brave follow
everything else	`gtk`	

An upload in Brave, link by link:

1. Brave calls the portal's `OpenFile` method on your session bus.
2. `xdg-desktop-portal` reads `wlroots-portals.conf` and passes the request to the termfilechooser backend (D-Bus starts it if it is not yet running).
3. The backend runs `~/.local/bin/kawa-pick`, which opens yazi in a floating foot window.
4. You choose a file; the path travels back the same way, and Brave reads that file.

The portals start only inside a graphical session, which is why the autostart starts `dwl-session.target` (it binds systemd's `graphical-session.target`) and `start-dwl` stops it at logout.

Notifications

The **Desktop Notifications** specification defines one name on the session bus, `org.freedesktop.Notifications`, with a `Notify` method. Whatever program owns the name shows the notices; here that is **mako**. Any program can call it — `notify-send` from a shell:

```
notify-send "Backup" "Copied 1 240 files"   # a notice, as any program would send it
makoctl dismiss --all                       # what Super+Shift+N does
makoctl mode                                # the current modes: do-not-disturb, sharing
```

MPRIS

MPRIS is the specification for media players on D-Bus: each player takes a name starting with `org.mpris.MediaPlayer2.`, at the object `/org/mpris/MediaPlayer2`, with the interfaces `org.mpris.MediaPlayer2` and `org.mpris.MediaPlayer2.Player` (play, pause, next, what is playing). The bar, the media keys and `playerctl` all speak it, so any player that does works with all three (§17.7, the second chain).

polkit and udisks

polkit (§17.4) is itself a D-Bus service on the system bus, `org.freedesktop.PolicyKit1`; services ask it, and it asks your agent for a password when the rules want one. **udisks2** (`org.freedesktop.UDisks2`) manages disks for ordinary users: listing, mounting, unmounting, powering off. **udiskie**, started by the autostart, listens to udisks2 and mounts each new removable drive under `/run/media/you/LABEL` (the machine's own disks it leaves closed: udisks marks them *HintSystem*, and udiskie's config says not to open those by itself); every change it sees, it hands to `kawa-drives`, which shows the notice, lights the bar's drive mark and ejects (§4.13). The same calls, by hand:

```
udisksctl status                        # the drives udisks knows: model, serial, device
busctl --system tree org.freedesktop.UDisks2 | head   # its objects: one per drive and block device
kawa-drives eject /dev/sdb1             # write out, close everything on that drive, switch it off
```

Read more: D-Bus specification · busctl(1) · portals.conf · Desktop Notifications · MPRIS · udisks.

17.11 The security model, briefly

Each layer limits a different kind of mistake or attack:

Layer	Stops	Here
permissions (§17.3)	one user's programs touching another's files	your home is yours alone; `secrets.sh` is 600
users and privilege (§17.4)	ordinary programs changing the system	root only through `sudo` and polkit, after your password
SELinux	a program doing what its type may not, even as root	`enforcing`, the *targeted* policy
firewalld	connections from outside that nothing asked for	`FedoraServer` zone, no SSH unless chosen
Secure Boot	a changed boot loader or kernel	shim, GRUB and the kernel checked at every start (§17.2)
signed packages	a package changed on its way, or from someone else	every repository with `gpgcheck`; keys checked by fingerprint (§17.7)
rootless containers	a containerised program reaching the system	Prowlarr as you, in its own user namespace

SELinux

Fedora's SELinux policy is **targeted**: the services most exposed — network daemons, the login, containers — run in confined types with exactly the access their policy grants, while your own programs run `unconfined_t`. **Enforcing** means a refused access is refused, not just logged:

```
$ getenforce
Enforcing
```

(`sestatus` adds the policy's name, `targeted`, and more.)

Reading a denial. A refusal is logged as an *AVC* message:

```
$ sudo ausearch -m avc -ts recent
----
time→Thu Oct 15 20:41:07 2026
type=AVC msg=audit(1792082467.512:812): avc:  denied  { read } for  pid=4120 comm="myservice"
name="notes.md" dev="nvme0n1p3" ino=93321 scontext=system_u:system_r:myservice_t:s0
tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0
```

(an invented service, for example)

Field	Reads
`denied { read }`	the access that was refused
`comm="myservice"`, `pid=4120`	the program that asked
`name="notes.md"`, `ino=93321`	the file it wanted
`scontext=…:myservice_t:…`	the asker's type
`tcontext=…:user_home_t:…`	the file's type
`tclass=file`	the kind of object

Read it as a sentence: *a process of type* `myservice_t` *was refused* `read` *on a* `file` *of type* `user_home_t`. The usual cause on a desktop is a file with the wrong label — made elsewhere and moved into place — and the usual fix is `restorecon -v FILE`, which gives it the label its path should have. Never switch SELinux off to make a problem go away (§9).

Rootless containers

Prowlarr runs in podman *without* root. Its container gets its own **user namespace**: inside, it sees users from 0 upwards; outside, those numbers are a range of IDs set aside for you in `/etc/subuid` and `/etc/subgid`, none of which is a real user on the machine. The quadlet maps your own UID through as 1000 (`UserNS=keep-id`), so the files it writes stay yours:

```
$ grep "^$USER:" /etc/subuid
you:524288:65536
$ podman unshare cat /proc/self/uid_map
         0       1000          1
         1     524288      65536
```

(for example) Inside, 0 is you; 1 and up are borrowed IDs. Add `NoNewPrivileges=true`, a port bound to `127.0.0.1` only, the SELinux type `container_t` for its processes and `container_file_t` for its folder, and Prowlarr can reach very little even if it misbehaves.

Read more: SELinux on Fedora · Troubleshooting SELinux · Enforcing signature checking by default · podman(1).

17.12 Commands by job

Every command here is installed on this desktop: Fedora's minimal set (coreutils, util-linux, procps-ng, iproute, iputils, findutils, grep, sed, gawk, tar, systemd, dnf5, rpm, and from the same `@core` group `less`, `man`, `curl`, firewalld, SELinux's tools and the audit tools) plus what the manifest adds. `tldr NAME` gives more examples; `man NAME` the full manual. Rows whose *Does* changes something — copy, move, delete, install — act at once: read the example before you run it.

Files and folders

Command	Does	Example
`ls` · `ll` · `lt`	list a folder (eza here): plain · everything with sizes and git state · as a tree	`ll ~/Documents`
`command ls -l`	coreutils' own long listing, with permissions and link counts	`command ls -l /usr/local/bin`
`cd` · `cdi`	change folder; zoxide jumps by a remembered word · pick one in fzf	`cd repos`
`pwd`	print the current folder	`pwd`
`mkdir -p`	make a folder and any parents	`mkdir -p ~/Documents/2026/tax`
`cp` · `cp -a`	copy · copy keeping owner, times and links	`cp -a ~/Notes /mnt/backup/`
`mv`	move or rename	`mv report.pdf ~/Documents/`
`rm` · `rmdir`	delete files · delete an empty folder (no undo; prefer the trash)	`rm -i old.txt`
`gio trash`	move to the trash instead	`gio trash old.txt` · `gio trash --empty`
`ln -s`	make a symbolic link	`ln -s ~/Books/manual.pdf ~/Desktop/`
`touch`	create an empty file, or update its time	`touch notes.md`
`stat`	every detail of one file: inode, links, permissions, times	`stat -c '%a %U %s %n' ~/.bashrc`
`realpath` · `readlink -f`	the full path, links resolved	`realpath ~/.config/fedokawaii/current`
`yazi`	the file manager (Super+E)	`yazi ~/Downloads`

Viewing

Command	Does	Example
`cat`	print a file	`cat /etc/fedora-release`
`bat`	print with line numbers and colour	`bat ~/.config/foot/foot.ini`
`less`	page through: Space, `b`, `/word`, `n`, `q`	`less ~/.cache/dwl.log`
`head` · `tail`	the first · last lines	`tail -n 40 ~/.cache/dwl.log`
`tail -f`	follow a growing file; Ctrl+C stops	`sudo tail -f /var/log/fedokawaii.log`
`file`	what kind of file it really is	`file ~/Downloads/*`
`wc`	count lines, words, bytes	`wc -l ~/Notes/notes.md`
`nvim -R`	open read-only in neovim	`nvim -R /etc/greetd/fedokawaii.toml`

Searching

Command	Does	Example
`fd`	find files by name (regex; `-g` for a glob), fast, skipping hidden and ignored ones	`fd -e pdf . ~/Documents`
`find`	find files by any property	`find ~ -xdev -size +1G`
`rg`	search inside files (ripgrep)	`rg -n 'kawa-[a-z]+' ~/.config/waybar`
`grep`	search lines in a file or a pipe	`grep -i error ~/.cache/waybar.log`
`fzf`	pick from any list by typing a few letters	`nvim "$(fd -e md | fzf)"`
`command -v` · `type -a`	where a command comes from: program, alias or function	`type -a cd`
`rpm -qf`	which package a file belongs to	`rpm -qf "$(command -v waybar)"`

Text processing

Command	Does	Example
`sort`	sort lines (`-n` numbers, `-h` sizes like 2G, `-r` reverse)	`du -sh ~/* | sort -h`
`uniq -c`	collapse repeated lines, counting them (sort first)	`journalctl -b -o cat -u NetworkManager | sort | uniq -c | sort -rn | head`
`cut`	take fields by a separator	`cut -d: -f1,7 /etc/passwd`
`awk`	fields and small programs	`ps -eo rss,comm --sort=-rss | awk 'NR>1 && NR<7 {printf "%6.0f MB %s\n", $1/1024, $2}'`
`sed`	replace or delete by pattern	`sed -n '/^\[preferred\]/,$p' ~/.config/xdg-desktop-portal/wlroots-portals.conf`
`tr`	translate or delete characters	`echo "$PATH" | tr : '\n'`
`jq`	query and reshape JSON	`pw-dump | jq -r '.[] | .info.props["node.description"] // empty' | sort -u`
`column -t`	line text up in columns	`mount | column -t | head`
`tee`	write to a file and the screen at once	`dnf repoquery --userinstalled | tee ~/pkgs.txt`
`xargs`	turn lines into arguments	`fd -0 -e tmp | xargs -0 -r rm` (deletes them, spaces in names and all)

Command	Does	Example
`git diff --no-index`	compare two files or folders	`git diff --no-index old.conf new.conf`

Archives

Command	Does	Example
`tar -czf` · `tar -xzf`	pack into · unpack from a gzip-compressed tar	`tar -czf notes.tar.gz ~/Notes`
`tar -tf`	list what is inside	`tar -tf notes.tar.gz | head`
`7z a` · `7z x` · `7z l`	make · extract · list 7z, zip and most other formats	`7z x ~/Downloads/photos.zip -o"$HOME/Pictures/trip"`
`bsdtar -xf`	extract almost any archive, comic books included	`bsdtar -xf book.cbz -C pages/`
`sha256sum`	the fingerprint of a file, to compare with a published one	`sha256sum fedora.iso`

Processes

Command	Does	Example
`ps`	list processes (`-ef` all; `--forest` as a tree)	`ps -ef --forest | less`
`pgrep -a`	find processes by name	`pgrep -a waybar`
`kill`	send a signal (TERM by default) to a PID	`kill 4120` · `kill -USR2 4120`
`pkill`	the same, by name	`pkill -USR2 waybar` (reloads the bar)
`top` · `btop`	live view of CPU, memory and processes	`btop`
`nice` · `renice`	start at · change to a lower priority	`nice -n 19 7z a big.7z ~/Videos`
`jobs` · `fg` · `bg`	this shell's jobs	after **Ctrl+Z**: `bg`
`timeout`	stop a command after a time	`timeout 10 ping fedoraproject.org`
`watch`	run a command again every two seconds	`watch -n 5 free -h`
`cat /proc/PID/status`	everything the kernel knows of one process	`grep -E 'State|VmRSS' /proc/"$(pgrep -nx waybar)"/status`

Services and logs

Command	Does	Example
`systemctl status`	a unit's state, processes and last log lines	`systemctl status NetworkManager`
`systemctl --user status`	one of your units	`systemctl --user status wireplumber`
`systemctl start` · `stop` · `restart`	now	`sudo systemctl restart bluetooth`
`systemctl enable` · `disable`	at every boot	`sudo systemctl enable --now fstrim.timer`
`systemctl list-timers`	every timer, next and last run	`systemctl --user list-timers`
`systemctl cat`	a unit with its drop-ins	`systemctl cat greetd`
`systemctl list-units --failed`	what failed	`systemctl --user list-units --failed`
`journalctl`	read the journal (filters in §17.5)	`journalctl -b -p err`

Command	Does	Example
`systemd-analyze`	boot time, the slow units, the chain	`systemd-analyze blame | head`
`systemd-cgls` · `systemd-cgtop`	the cgroup tree · live use per group	`systemd-cgtop`
`loginctl`	sessions, users, linger	`loginctl session-status`
`busctl`	D-Bus: names, objects, calls	`busctl --user list | grep mpris`
`systemd-run --user`	run a command as a unit of its own	`systemd-run --user --scope 7z a big.7z ~/Videos`

Packages

Command	Does	Example
`dnf search` · `dnf info`	find a package · describe one	`dnf info btop`
`dnf install` · `dnf remove`	add · take away (and what only it needed)	`sudo dnf install hyperfine`
`dnf upgrade --refresh`	update everything (the season's update does more, §6.2)	`sudo dnf upgrade --refresh brave-origin`
`dnf provides`	which package has a file or command	`dnf provides '*/bin/7z'`
`dnf repoquery --requires` · `--recommends`	what a package needs · would like	`dnf repoquery --requires waybar`
`dnf repoquery --installed --whatrequires`	what installed packages need this one	`dnf repoquery --installed --whatrequires libva`
`dnf repoquery --userinstalled`	what was asked for by name	`dnf repoquery --userinstalled | wc -l`
`dnf repolist`	the enabled repositories	`dnf repolist`
`dnf history`	past transactions; undo one	`dnf history list | head`
`dnf group list`	the comps groups	`dnf group list --installed`
`rpm -qi` · `-ql` · `-qf`	one package's details · its files · a file's package	`rpm -ql mpv-mpris`
`rpm -q --changelog`	what changed in recent builds	`rpm -q --changelog greetd | head`
`rpm -V`	check a package's files against how it was installed	`rpm -V greetd`
`rpmkeys --list`	the trusted signing keys	`rpmkeys --list`

Users and permissions

Command	Does	Example
`id` · `id -nG`	your UID, groups and SELinux context · group names	`id -nG`
`whoami` · `groups`	your name · your groups	`whoami`
`getent passwd` · `getent group`	a user's · a group's entry	`getent group wheel`
`sudo`	run one command as root	`sudo dnf upgrade --refresh`
`sudoedit`	edit a system file safely, in neovim	`sudoedit /etc/systemd/journald.conf.d/50-size.conf`
`passwd`	change your password	`passwd`
`chmod`	change permissions	`chmod 600 ~/.config/fedokawaii/secrets.sh`
`chown`	change the owner (root only)	`sudo chown you:you ~/somefile`

Command	Does	Example
`umask`	the mask for new files	`umask`
`getenforce` · `sestatus`	SELinux mode · full status	`getenforce`
`ls -Z` · `id -Z` · `ps -eZ`	SELinux labels of files · of you · of processes	`command ls -Z ~`
`restorecon -v`	put back a file's proper SELinux label	`restorecon -v ~/.local/bin/myscript`
`ausearch -m avc`	SELinux refusals	`sudo ausearch -m avc -ts recent`

Disks and file systems

Command	Does	Example
`lsblk`	disks and partitions as a tree	`lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS`
`findmnt`	what is mounted where, as a tree	`findmnt -no OPTIONS /`
`df -h`	space per file system (on Btrfs, ask Btrfs: §7.4)	`df -h / /boot`
`du -sh`	space a folder's files use	`du -sh ~/Videos`
`btrfs filesystem usage`	the real space on Btrfs	`sudo btrfs filesystem usage /`
`btrfs subvolume list`	the subvolumes and snapshots	`sudo btrfs subvolume list /`
`snapper list` · `snaps`	the snapshots of `/`	`snaps`
`snapper status`	what changed between two snapshots	`sudo snapper -c root status 41..42`
`swapon --show` · `zramctl`	swap in use · the zram device	`swapon --show` → `/dev/zram0 partition 8G 0B 100`
`blkid`	file system types, labels and UUIDs	`sudo blkid /dev/nvme0n1p3`
`mount` · `umount`	attach · detach a file system	`sudo mount /dev/sdb1 /mnt`
`fstrim`	tell the SSD which blocks are free (a weekly timer does it)	`sudo fstrim -v /`

Network

Command	Does	Example
`ip -br addr` · `ip route`	addresses per card · the routes, with the default gateway	`ip route show default`
`nmcli`	NetworkManager: devices, connections, Wi-Fi	`nmcli device wifi list`
`nmtui`	the same, as a menu	`nmtui`
`ss -tlnp`	what listens on which port (your own processes named)	`ss -tlnp`
`ping -c 3`	is a host reachable	`ping -c 3 1.1.1.1`
`tracepath`	the route to a host, hop by hop	`tracepath fedoraproject.org`
`curl`	fetch a URL	`curl -sS -o /dev/null -w '%{http_code}\n' https://fedoraproject.org/`
`resolvectl`	DNS: status, look-ups, cache	`resolvectl query fedoraproject.org`
`firewall-cmd`	the firewall's zones, services and ports	`sudo firewall-cmd --list-all`
`hostnamectl`	the machine's name and identity	`hostnamectl`

Hardware

Command	Does	Example
`lspci -k`	PCI devices and the driver holding each	`lspci -k -d ::0300` (the GPU)
`lsusb`	USB devices	`lsusb`
`lscpu`	the processor: model, cores, caches	`lscpu | head`
`free -h`	memory and swap in use	`free -h`
`uptime`	how long since the start, and the load	`uptime`
`uname -r`	the running kernel	`uname -r`
`cat /etc/fedora-release`	the release	`cat /etc/fedora-release`
`vainfo`	the video-decoding driver and its codecs	`vainfo --display drm`
`wlr-randr`	monitors, modes and refresh rates (in the session)	`wlr-randr`
`ddcutil`	the monitor's own settings over DDC/CI	`ddcutil getvcp 10`
`fwupdmgr`	firmware inside devices: list, check, update	`fwupdmgr get-updates`
`lsmod` · `modinfo`	loaded kernel modules · one module's details	`modinfo -F description amdgpu`
`journalctl -k`	the kernel's messages	`journalctl -k -b -g firmware`

Help

Command	Does	Example
`man`	the manual (opens in neovim here; `gO` lists sections, `q` closes)	`man 5 systemd.timer`
`man -k` · `apropos`	search the manuals' one-line descriptions	`apropos -s 1 snapshot`
`whatis`	one command's one-line description	`whatis findmnt`
`tldr`	a few common examples	`tldr ss`
`--help`	a program's own short list of options	`nmcli --help`
`help`	bash's own commands	`help jobs`

17.13 Glossary, A–Z

Term	Meaning
ACL	extra per-user or per-group permissions on a file; `+` in `ls -l`
agreety	greetd's plain text greeter: user name, password
ALSA	the kernel's sound drivers and their programming interface
AMDVLK	AMD's own open Vulkan driver, ended in 2025 in favour of RADV
ANV	Mesa's Vulkan driver for Intel GPUs
AVC	an SELinux decision; a *denied* AVC is a refusal in the audit log
backend (portal)	the program that carries out a portal request: wlr, gtk, termfilechooser
BLS	Boot Loader Specification: one small file per boot entry in `/boot/loader/entries`
BlueZ	the Bluetooth stack: kernel part and `bluetoothd`
Btrfs	the copy-on-write file system, with subvolumes and snapshots, that `/` and `/home` use
bus (D-Bus)	the message channel programs talk over: the system bus or your session bus
cgroup	a group of processes the kernel counts and limits together; version 2 is one tree
comps	Fedora's package groups, such as `core`

Term	Meaning
compositor	the program that composes windows on screen and handles input; under Wayland, the display server
console	the text screen of the machine itself, tty1–tty6
COPR	Fedora's service for small personal repositories
copy-on-write	writing a change to a new place, leaving the old blocks for snapshots and shared copies
cron	the older way to run jobs on a schedule; timers do it here
D-Bus	the message system desktop programs and services use to talk
dbus-broker	the D-Bus implementation Fedora uses
dependency	a capability one package needs another to provide
distribution	a kernel plus the userspace around it, built and released together: Fedora
dracut	the tool that builds the initramfs
DRM	the kernel's GPU subsystem (Direct Rendering Manager)
drop-in	a file in a `NAME.d/` folder that adds to or overrides a config without editing it
dwl	the tiling Wayland compositor of this desktop, built on wlroots
edition	one of Fedora's starting sets for a purpose: Workstation, Server, …
EFI system partition	the small FAT partition, `/boot/efi`, from which the firmware loads shim
environment variable	a named value copied to every process a shell or session starts
exec	replacing a process's program with another, keeping its PID
exit status	the number a process ends with: 0 success, 128 + *n* killed by signal *n*
ffmpeg	the library and tool that decodes and encodes nearly every video and sound format
file descriptor	a numbered open file of a process: 0 input, 1 output, 2 errors
fingerprint	the long, unique number of a signing key, checked before a key is trusted
firewalld	the service that holds the packet filter, by zones
firmware	software that runs on a device itself, or the motherboard's UEFI
fontconfig	picks the font file for a name, a language and a missing character
fork	a process copying itself to make a child
FreeType	turns font outlines into pixels
freeworld	RPM Fusion's builds of Mesa and libheif with the codecs Fedora leaves out
generator	a small program systemd runs at boot to write units, such as podman's for quadlets
GID	a group's number
glob	a file-name pattern bash expands: `*.pdf`
greetd	the login manager on tty1
GRUB2	the boot loader that shows the menu and loads the kernel
hard link	a second name for the same inode
HarfBuzz	shapes text: which glyphs, where; joins Arabic letters
initramfs	the small in-memory system that mounts the real root file system
inode	a file's record: type, permissions, owner, size, times, data location; not its name
install reason	why dnf has a package: User, Dependency, Weak Dependency, Group
ioctl	a special call on a device file, beyond read and write: how Mesa drives the GPU
job	a program a shell started, in the foreground or background
journal	systemd's log, read with `journalctl`

Term	Meaning
kernel	the core program that drives the hardware and shares it out: Linux
kernel argument	an option handed to the kernel at boot; `cat /proc/cmdline`
KMS	the kernel's control of screens and modes (kernel mode setting)
kmscon	the userspace text console Fedora 45 uses on the virtual consoles
layer shell	the Wayland protocol for bars, notifications, menus and wallpapers
libinput	the library that reads keyboards, mice and touchpads for the compositor
libva	the VA-API library: programs' way to the GPU's video decoder
linger	a user's systemd manager kept running without a login
logind	systemd's login manager: sessions, seats, device access
Mesa	the open OpenGL, Vulkan and VA-API drivers
metadata (repository)	a repository's index of its packages and their dependencies
microcode	the CPU maker's fixes, loaded into the processor at every start
mount	attaching a file system to a folder of the tree
MPRIS	the D-Bus specification for media players
NetworkManager	the service that runs network connections; `nmcli`
NEVRA	name, epoch, version, release, architecture: a package's exact identity
nice	a process's CPU priority, from −20 to 19
OpenGL	the older GPU programming interface; Vulkan is the newer
package	one piece of the distribution in one `.rpm` file
PAM	the library every login uses to check passwords and open sessions
PATH	the folders searched for a command typed by name
PID · PPID	a process's number · its parent's
PipeWire	the sound and video server
pipewire-pulse	PipeWire's PulseAudio-compatible server
polkit	decides which users may ask system services for privileged actions
polkit agent	the session program that asks for your password for polkit (here lxqt-policykit-agent)
portal	a D-Bus service through which a program asks the desktop for a file dialog, a screen share, a setting
process	a program while it runs
PTY	a pseudo-terminal, `/dev/pts/N`, owned by a terminal emulator
quadlet	a container described in systemd's format, turned into a service
radeonsi	Mesa's OpenGL and VA-API driver for AMD GPUs
RADV	Mesa's Vulkan driver for AMD GPUs
regular expression	a pattern that matches text: `kawa-[a-z]+`
render node	`/dev/dri/renderD128`: the GPU for drawing and decoding, without screen control
repository	a server offering packages and their metadata
resolved	systemd-resolved: DNS look-ups through the stub at `127.0.0.53`
root	the user with UID 0, past every permission check
rootless container	a container run by an ordinary user, in a user namespace
RPM	Fedora's package format, and the tool that installs single packages

Term	Meaning
RPM Fusion	the community repositories for what Fedora leaves out
scope · slice	systemd's groups of processes: one session or service · a branch of the tree
scriptlet	a small script inside a package, run when it is installed or removed
Secure Boot	the firmware starting only signed boot loaders and kernels
SELinux	the kernel's mandatory access control by labels; *enforcing*, *targeted* on Fedora
session (logind)	one login of one user, with its processes in `session-N.scope`
setuid	a permission bit that runs a program as its owner
shebang	the `#!` first line of a script, naming its interpreter
shell	the program that reads and runs your commands: bash
shim	the small Microsoft-signed first boot loader that checks GRUB
signal	a short message from the kernel to a process: TERM, KILL, USR2…
signature	proof, made with a private key, that a package comes unchanged from its packager
sink · source	in PipeWire, an output · an input
snapshot	a frozen copy of a Btrfs subvolume, sharing its unchanged blocks
socket activation	starting a service when the first client connects to its socket
soname	a shared library's name with its major version: `libavcodec.so.62`
sticky bit	on a folder: only a file's owner may delete it (`/tmp`)
subvolume	a Btrfs folder with its own tree, snapshotted as a whole
sudo	runs one command as root after your password
symbolic link	a small file holding a path to another
systemd	process 1: starts and supervises everything; also each user's manager
target	a named group of units, a point in the start-up
terminal emulator	the program that draws a terminal in a window: foot
timer	a unit that starts a service on a schedule
tmpfs	a file system in memory: `/tmp` , `/run`
transaction	one dnf run that installs, updates or removes a set of packages together
TTY	a terminal device; `/dev/tty1` is the first console
udev	creates and names device files and announces devices
udisks2 · udiskie	the disk service on the system bus · its automounter in your session
UEFI	the motherboard's firmware, which starts the boot loader
UID	a user's number; people start at 1000
umask	the permission bits taken away from new files
unit	anything systemd manages, described by a small file
upstream	the project that writes a program; Fedora is downstream
user manager	your own systemd, `user@1000.service`
user namespace	a private range of user numbers for a container
userspace	everything that runs outside the kernel
VA-API	the interface for hardware video decoding and encoding
Vulkan	the modern GPU programming interface

Term	Meaning
Wayland	the protocol between programs and the compositor
weak dependency	a *Recommends*: installed by default, but optional
wheel	the administrators' group
WirePlumber	PipeWire's session manager: defaults, routing, memory
wlroots	the library dwl is built on
XDG_RUNTIME_DIR	your per-session folder in memory, `/run/user/1000`
xe · i915	Intel's new and older kernel GPU drivers
XKB	the keyboard layouts and options: `us,ara`, `grp:alt_shift_toggle`
XWayland	an X server inside Wayland, for X11-only programs
zone (firewalld)	a set of rules for one kind of network
zram	swap kept compressed in memory

17.14 Sources

The official documentation this chapter was checked against, by subject.

Subject	Source
Fedora's editions, releases and changes	fedoraproject.org · Release life cycle · Staying close to upstream
Fedora change pages	dbus-broker · BLS · cgroups v2 · Btrfs · zram · zram size · systemd-resolved · PipeWire · WirePlumber · DNF 5 · bin and sbin · RPM 6.0 · kmscon · signature checking · sudo-rs (proposal)
boot	Boot Loader Specification · Secure Boot on Fedora · microcode loader · dracut.conf(5)
systemd	systemd man pages · systemd-analyze(1) · loginctl(1) · desktop environments and systemd
files and processes	file-hierarchy(7) · inode(7) · signal(7) · GNU coreutils 9.0 news · Btrfs documentation
packages	rpm.org · rpmkeys(8) · RPM 6.0 release notes · dnf5.conf(5) · repoquery · history · mpv-libs in Fedora 44 · mesa-dri-drivers in Fedora 44 · amd-gpu-firmware · RPM Fusion's mesa-freeworld
graphics	Wayland architecture · wlr-layer-shell · Intel xe GPUs · AMDVLK discontinued · Mesa
sound, devices, network	PipeWire · pipewire-pulse(1) · WirePlumber · NetworkManager · firewalld
buses and specifications	D-Bus · portals.conf · Desktop Notifications · MPRIS · XDG Base Directory
security and containers	SELinux on Fedora · podman-systemd.unit(5)
secondary	the Arch Wiki — written for Arch, mostly true here
this desktop	`setup.sh`, `lib/manifest.sh`, `lib/hardware.sh`, `system/`, `configs/home/` in the toolkit, `/usr/local/lib/fedokawaii`